

Chapter 1 — DSA in idiomatic Python (40 Questions)

Python Interview Prep — 2026 Edition · Learn & Excel · File 07 Backend Track Library

This is the first chapter of the Backend Track Library and the one Indian backend, SDE, and platform interviewers open with in round one. Forty questions on the data-structures-and-algorithms surface — but framed in idiomatic Python, not Java-translated-into-Python. The distinction matters: a backend candidate who reaches for `dict[int, list[int]]` instead of writing a `Node` class for graphs, who calls `heapq.nlargest` instead of hand-coding a sort, who knows `collections.deque` is the right BFS frontier and not `list.pop(0)`, signals to the panel that they've shipped Python services and not just done LeetCode in Python syntax. Each question follows the same shape — tags, the interviewer's verbatim phrasing, a 30-second answer, the step-by-step "what to say out loud" thought process, edge cases to surface, what NOT to say, the common follow-up, and a stand-out line you can drop in the room. Answers assume Python 3.12 unless a version is called out explicitly.

Table of Sections

- Section A — Arrays, lists, deques, slicing (Q1–Q5)
- Section B — Hashmaps and sets (Q6–Q15)
- Section C — Heaps and `heapq` (Q16–Q20)
- Section D — Trees (Q21–Q30)
- Section E — Graphs (Q31–Q35)
- Section F — DP and recursion (Q36–Q40)

Section A — Arrays, Lists, Deques, Slicing

Five questions on the linear-structure choices that show up in every backend DSA round. `list`, `collections.deque`, `array.array`, and the slicing patterns that Python gives you for free. Indian product-co panels — Razorpay, Flipkart, Swiggy, Postman — open with one of these because they're cheap to ask and quickly separate the "I've written Python at work" candidate from the "I've done a LeetCode template" candidate.

Q1 · When do you reach for `list` vs `collections.deque` vs `array.array` ?

Tags: Difficulty: Easy · Asked at: Razorpay, Flipkart, Swiggy, Postman, Microsoft India · Topic: Container choice

The question (interviewer's verbatim phrasing):

Python gives you `list`, `collections.deque`, and `array.array`. Walk me through when you'd pick each one in a backend service.

The 30-second answer: `list` is the default — a dynamic array of `PyObject*` pointers. $O(1)$ append, $O(1)$ index, $O(n)$ prepend. `collections.deque` is a doubly-linked block list — $O(1)$ at both ends, including prepend and popleft, which makes it the right BFS frontier and the right sliding window. `array.array` is a typed C array of homogeneous primitives — far smaller in memory than a list of ints, used when you're packing megabytes of numeric data or interoperating with C code. In production Python services the split is roughly 90% list, 9% deque, 1% array.

Step-by-step thought process: 1. Anchor on the implementation. `list` is a pointer array, so prepend slides every pointer right — $O(n)$. `deque` is a doubly-linked chain of fixed-size blocks, so prepend just moves the head pointer inside the first block — $O(1)$. `array` is a contiguous C buffer of primitives — no `PyObject` per element. 2. Map access patterns to choices. Iterate-or-append: list. Queue, BFS, sliding window: deque. Numeric packing where you'd otherwise reach for NumPy but don't want the dependency: array. 3. Mention `deque(maxlen=N)`. That single argument turns a deque into a fixed-capacity ring buffer that auto-evicts the oldest — useful for rolling logs, recent-events caches, and sliding windows in stream processing. 4. Call out the indexing trade-off. Random access on a deque is $O(n)$ in the worst case because you walk block links; if your access pattern is "BFS frontier + middle-index reads", you're forcing two wrong choices. Pick the dominant pattern and live with the secondary cost.

Edge cases to mention: - `list.pop()` with no argument is $O(1)$; only `pop(0)` and middle pops are $O(n)$. - `deque` slicing is not supported with the `lst[a:b]` syntax — you have to `itertools.islice` it. Surprise for candidates who treat deque as a drop-in. - `array.array('i', ...)` rejects floats and strings at append time; that's a feature, not a bug — it's the type discipline you want for binary protocols.

What NOT to say: - "Use deque whenever you can — it's faster" — wrong; deque loses $O(1)$ random indexing, which 90% of code wants. - "Use array.array because it's faster than list" — only true for homogeneous numeric data; for general objects the overhead of the wrapping makes it slower.

Common follow-up: "You're implementing a BFS over a 100k-node graph. Which container is the frontier and why?" — `deque`, because BFS pops from the front and appends to the back; using `list.pop(0)` would push the whole BFS to $O(n^2)$ and the candidate would not finish on a large graph in interview time.

Stand-out line: *"In a Python service, the question isn't 'which is fastest' — it's 'which one matches the access pattern, because the wrong one turns an $O(n)$ algorithm into $O(n^2)$ without anything looking wrong in code review.'"*

```
from collections import deque
from array import array

events = []                # iterate-or-append → list
frontier = deque([root])   # BFS → deque,  $O(1)$  both ends
recent = deque(maxlen=100) # ring buffer → deque(maxlen=N)
pcm = array('h', [0] * 44_100) # 16-bit audio samples → array.array
```

Q2 · Why is slicing in Python idiomatic for sub-array problems and what's the cost?

Tags: Difficulty: Easy · Asked at: Razorpay, Postman, Flipkart, Goldman, JP Morgan · Topic: Slicing patterns

The question (interviewer's verbatim phrasing):

In Python you can write `arr[i:j]` instead of looping. Is that always the right call in an interview? What's the time and space cost?

The 30-second answer: `arr[i:j]` is $O(j - i)$ in both time and space — it allocates a new list and copies pointer references across, even if you only read it. So for read-only operations on a sub-range, slicing is wasteful; prefer index-based loops or `itertools.islice`. For write operations like reversing a sub-range or extracting a window for transformation, slicing is idiomatic and clear, and the copy cost is unavoidable anyway. The general rule: slice when you need a separate object, index when you're just reading.

Step-by-step thought process: 1. State the cost up front. Slicing is not a view; it's a copy. `arr[1_000_000:2_000_000]` allocates a million-pointer list and copies those references. The references are shared (no deep copy), but the list structure is new. 2. Contrast with NumPy. `np_arr[1_000_000:2_000_000]` is a view — same backing memory, new shape metadata. Candidates who confuse list-slicing semantics with NumPy-slicing semantics ship the wrong optimization. 3. Show the idiomatic alternatives. For read-only iteration over a sub-range, `for x in itertools.islice(arr, i, j)` doesn't copy. For two-pointer-style algorithms, you keep `left` and `right` indices and never slice. 4. Name the cases where slicing IS the right call. Reversing a sub-range (`arr[i:j] = arr[i:j][::-1]`), extracting a sub-array to pass to a function that needs a list, and the `arr[:]` shallow-copy idiom that the next question is about.

Edge cases to mention: - Negative slicing — `arr[-3:]` is $O(3)$, not $O(n)$; the slice length determines the cost. - `arr[::-1]` is a full $O(n)$ copy that reverses; for in-place reverse use `arr.reverse()`. - Slice assignment — `arr[i:j] = iterable` — can change the list length and is $O(n)$ because it shifts the tail.

What NOT to say: - "Slicing is free / a view" — wrong for lists; you've conflated it with NumPy. - "Slicing copies the objects" — wrong; it copies the references, not the underlying objects.

Common follow-up: "You're implementing a sliding-window maximum. Why is `nums[i:i+k]` inside the loop a bug?" — Each slice is $O(k)$ and you do n of them, so the loop is $O(nk)$. The correct pattern uses a deque to maintain candidate indices in $O(n)$ total.

Stand-out line: "Slicing is idiomatic for clarity, not for performance — when the loop is hot, indices beat slices every time."

```
import itertools

arr = list(range(1_000_000))

# wasteful: allocates a 500k-element list just to iterate read-only
for x in arr[250_000:750_000]:
    process(x)

# idiomatic: no copy
for x in itertools.islice(arr, 250_000, 750_000):
    process(x)
```

Q3 · Implement the classic two-pointer pattern in Python — pair sum on a sorted array.

Tags: Difficulty: Easy · Asked at: Flipkart, Swiggy, PhonePe, Razorpay, JP Morgan · Topic: Two pointers

The question (interviewer's verbatim phrasing):

Given a sorted list of integers and a target sum, return the indices of two numbers that add up to the target. Walk me through your approach and write the Python.

The 30-second answer: Two pointers. `left = 0`, `right = len(nums) - 1`. Each iteration, compute `nums[left] + nums[right]`. If it equals target, return the pair. If it's too small, move `left` right (need a larger value). If too large, move `right` left. $O(n)$ time, $O(1)$ space, one pass. Works only because the input is sorted — that's the unlock the pattern exploits.

Step-by-step thought process: 1. State the pre-condition out loud: "the array is sorted, that's what makes two-pointer the right answer." If the array were unsorted, the right answer would be a hash-set pass — that's the next question. 2. Walk through the invariant. At every step, the answer (if it exists) lies in the window `nums[left:right+1]`. Each pointer move shrinks the window by one in the direction that the sum needs to change. 3. Write the code with named pointers, not `i` and `j`. Naming `left` and `right` makes the intent obvious in code review. 4. Mention the edge cases inline: empty list, single element, no valid pair (return None or empty tuple — clarify with the interviewer which contract they want).

Edge cases to mention: - Duplicates in the input: if `nums = [3, 3, 5]` and target is 6, you return (0, 1). The two-pointer pattern handles this correctly — the duplicates are at distinct indices. - Floating-point inputs: pair-sum on floats is fine but exact equality is risky; if the interviewer slips in floats, ask "within an epsilon?" before answering. - `left < right` is the right termination condition, not `left <= right` — equal pointers would re-use the same index.

What NOT to say: - "I'd sort the array first" — the question said sorted; assume that, don't re-sort and add $O(n \log n)$. - "I'd use `itertools.combinations`" — that's the $O(n^2)$ brute force and you'll be cut for proposing it on a 1M-element list.

Common follow-up: "What if the array isn't sorted?" — Then two-pointer doesn't apply; the right answer is a hash-set single pass. Iterate, for each element check if `target - x` is in a `seen` set, otherwise add `x` to seen. $O(n)$ time, $O(n)$ space.

Stand-out line: "Two-pointer is the pattern for sorted; hash-set is the pattern for unsorted — and recognising which one the interviewer set up is half the answer."

```
def pair_sum(nums: list[int], target: int) -> tuple[int, int] | None:
    left, right = 0, len(nums) - 1
    while left < right:
        s = nums[left] + nums[right]
        if s == target:
            return (left, right)
        if s < target:
            left += 1
        else:
            right -= 1
    return None
```

Q4 · Sliding-window maximum — why deque is the idiomatic Python answer.

Tags: Difficulty: Hard · Asked at: Razorpay, Microsoft India, Goldman, Postman, Flipkart · Topic: Sliding window

The question (interviewer's verbatim phrasing):

Given an array of integers and a window size `k`, return the maximum of every window of size `k`. The array can be a million elements long.

The 30-second answer: Use a `collections.deque` that holds candidate *indices*, maintained in decreasing order of `nums[index]`. As the window slides right, pop indices from the right while their values are less than the new element (they can never be the max again), then append the new index. Pop from the left if the leftmost index has fallen out of the window. The deque's front is always the index of the current window's max. $O(n)$ total, because each index is pushed and popped at most once.

Step-by-step thought process: 1. Reject the brute force out loud. `max(nums[i:i+k])` per window is $O(nk)$ — for $n=1M$ and $k=1k$ that's a billion ops; the interview clock will run out. 2. Reject the heap. A heap gives you $O(n \log k)$ but the "remove element that fell out of window" operation is $O(k)$ on a vanilla heap because Python's `heapq` has no key-remove. Workable with lazy deletion but more code than the deque solution. 3. Walk through the deque invariant. The deque holds indices, not values. Reading values is `nums[deque[0]]`. The deque is monotonically decreasing in value. Adding a new value pops smaller values off the right (they're shadowed). Sliding the window forward pops the front if it's stale. 4. Show the amortized analysis. Each index is appended once and popped at most once. So total work across all windows is $O(n)$, not $O(nk)$.

Edge cases to mention: - `k == 1` returns the array unchanged. `k == len(nums)` returns a single-element list with `max(nums)`. - Empty array — return `[]`. Negotiate with the interviewer; some panels want a `ValueError`. - Equal values in the window — the deque holds the *first* index of equal values, which is correct because it stays the max longer.

What NOT to say: - "Use `max()` inside the loop" — $O(nk)$; you'll be cut. - "Use a heap" — workable but more code; deque is the canonical answer and showing it is what the panel is checking.

Common follow-up: "Modify your solution to return the sliding-window minimum." — Flip the comparison: pop indices from the right while their values are *greater* than the new element, and the deque becomes monotonically increasing.

Stand-out line: "The trick is to realize the deque doesn't need to hold all candidates — only the ones that could still become the max as the window slides."

```

from collections import deque

def sliding_window_max(nums: list[int], k: int) -> list[int]:
    if not nums or k <= 0:
        return []
    dq: deque[int] = deque()
    out: list[int] = []
    for i, x in enumerate(nums):
        while dq and dq[0] <= i - k:
            dq.popleft()
        while dq and nums[dq[-1]] < x:
            dq.pop()
        dq.append(i)
        if i >= k - 1:
            out.append(nums[dq[0]])
    return out

```

Q5 · Kadane's algorithm — maximum subarray sum, the idiomatic Python version.

Tags: Difficulty: Medium · Asked at: TCS, Infosys, JP Morgan, Goldman, Flipkart · Topic: Classic DSA

The question (interviewer's verbatim phrasing):

Given a list of integers including negatives, find the contiguous subarray with the largest sum. Return the sum. Tell me your approach first, then the Python.

The 30-second answer: Kadane's algorithm. Walk the array once, keeping two running values:

`current_sum` (the best sum ending at the current index) and `best_sum` (the best sum seen anywhere). At each index, `current_sum = max(nums[i], current_sum + nums[i])` — either extend the running window or restart at the current element. Update `best_sum = max(best_sum, current_sum)`. $O(n)$ time, $O(1)$ space, one pass. The intuition: a negative running sum can never improve a future window, so you reset.

Step-by-step thought process: 1. State the algorithm name. Indian panels love that you can name "Kadane" — it's the shibboleth for "I've seen this pattern." 2. Walk the invariant. `current_sum` is the maximum sum of any subarray ending exactly at index `i`. `best_sum` is the max of all `current_sum` values seen so far. The answer is `best_sum` at the end. 3. Handle the all-negative case. If every element is negative, the answer is the single largest (least negative) element. Kadane handles this naturally because `current_sum = max(nums[i], ...)` starts fresh each time. 4. Mention the $O(n)$ space follow-up — when the panel asks "now return the subarray itself, not just the sum", you track the start and end indices alongside the running sums. Same time, still $O(1)$ extra space.

Edge cases to mention: - Empty list — depending on contract, return 0 or raise. Ask up front; the canonical Kadane assumes non-empty. - Single element — the answer is that element. - All zeros — the answer is 0; Kadane returns it correctly without special-casing.

What NOT to say: - "I'd try all $O(n^2)$ subarrays" — wrong tier; brute force on a million-element list will not finish. - "Use `itertools.accumulate` to find the max" — that's the $O(n)$ prefix-sum approach which is *also* valid, but it's less clean than Kadane and harder to extend to "return the subarray itself."

Common follow-up: "What's the time complexity if the array is on disk and you can only stream it one element at a time?" — Kadane is already the streaming algorithm. It needs only the current element and two scalars, so it works on an iterator of arbitrary size. That's the answer the panel wants — Kadane is the streaming algorithm.

Stand-out line: "The elegance of Kadane is that it's both the optimal-time algorithm and the streaming algorithm — same code works on a list, an iterator, or a Kafka stream."

```
def max_subarray_sum(nums: list[int]) -> int:
    if not nums:
        raise ValueError("empty input")
    current = best = nums[0]
    for x in nums[1:]:
        current = max(x, current + x)
        best = max(best, current)
    return best
```

Section B — Hashmaps and Sets

Ten questions on `dict`, `set`, `collections.Counter`, and `collections.defaultdict`. These are the structures backend DSA rounds lean on hardest, because almost every "find duplicates," "group by," "count frequency," "two-sum," "anagram" problem in the interview catalog reduces to a one-pass dict or set operation. Idiomatic Python here is what separates the candidate who writes nine lines of imperative code from the one who writes three lines using `Counter`.

Q6 · When do you reach for `dict` vs `collections.Counter` vs `collections.defaultdict`?

Tags: Difficulty: Easy · Asked at: Razorpay, Flipkart, Swiggy, Postman, Microsoft India · Topic: Container choice

The question (interviewer's verbatim phrasing):

Walk me through `dict`, `Counter`, and `defaultdict`. When would you pick each one?

The 30-second answer: `dict` is the base hash table — use it when keys map to arbitrary values. `Counter` is `dict` specialized for frequency counts — `Counter(iterable)` gives you `{element: count}` in one line, plus `.most_common(k)` for top-K and arithmetic operators (`+`, `-`) for combining counts. `defaultdict(factory)` is `dict` that auto-initializes missing keys via the factory function — use it when you'd otherwise write `d.setdefault(k, []).append(x)` or `if k not in d: d[k] = 0` boilerplate. The decision tree: counting frequencies → `Counter`; grouping into lists/sets → `defaultdict`; everything else → `dict`.

Step-by-step thought process: 1. Anchor on the common interview tells. The panel asks "find the most frequent element" — say `Counter`. They ask "group these tuples by first element" — say `defaultdict(list)`. They ask "build a lookup table" — say `dict`. 2. `Counter` is the high-leverage import that under-prepared candidates forget exists. `Counter("mississippi")` returns `Counter({'i': 4, 's': 4, 'p': 2, 'm': 1})` — counting, sorting by frequency, all in $O(n)$. 3. `defaultdict` avoids the two-line `setdefault` dance. `groups = defaultdict(list); for x in items: groups[key(x)].append(x)` is the idiomatic group-by. The first time the key is accessed, the factory runs and an empty list is created. 4. Mention the gotcha: `defaultdict` creates the key on read. `d[unknown_key]` triggers the factory and adds the key to the dict. If you only meant to check membership, use `if k in d` first or fall back to `d.get(k, default)`.

Edge cases to mention: - `Counter` arithmetic — `c1 + c2` adds counts, `c1 - c2` subtracts and drops zeros, `c1 & c2` is min (intersection), `c1 | c2` is max (union). All return a new `Counter`. - `defaultdict(int)` is the integer-counter idiom; `defaultdict(list)` is the group-by idiom; `defaultdict(set)` is the dedupe-group idiom. Three patterns cover 80% of usage. - `Counter.most_common(k)` returns the top-K by count, sorted descending. Equivalent to `heapq.nlargest(k, c.items(), key=lambda kv: kv[1])` but cleaner.

What NOT to say: - "Counter and dict are the same" — wrong; `Counter` has methods (`most_common`, arithmetic) that plain `dict` doesn't. - "defaultdict is just dict with a default value" — close but the wording matters; it's a *factory function*, not a default value. The factory runs lazily.

Common follow-up: "What does `Counter([1, 2, 2, 3]) - Counter([2, 3])` return?" — `Counter({1: 1, 2: 1})`. The subtraction drops keys that go to zero or negative — that's a `Counter`-specific behaviour, not regular `dict` semantics.

Stand-out line: " `Counter` and `defaultdict` are the two imports that turn five-line loops into one-line answers — knowing when to reach for them is what tells the panel you've shipped Python."

```

from collections import Counter, defaultdict

# frequency count
freq = Counter("mississippi")          # Counter({'i': 4, 's': 4, 'p': 2, 'm': 1})
top2 = freq.most_common(2)             # [('i', 4), ('s', 4)]

# group-by
events = [("payment", 100), ("refund", 50), ("payment", 200)]
by_type = defaultdict(list)
for kind, amount in events:
    by_type[kind].append(amount)
# by_type = {'payment': [100, 200], 'refund': [50]}

```

Q7 · Solve two-sum on an unsorted array — the hash-map one-pass pattern.

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Razorpay, Postman · Topic: Hashmap patterns

The question (interviewer's verbatim phrasing):

Given an unsorted list of integers and a target, return the indices of two numbers that add up to the target. Each input has exactly one solution.

The 30-second answer: Single-pass hash map. Walk the list with `enumerate`. For each `(i, x)`, check if `target - x` is already in a `seen` dict mapping value $\rightarrow$ index. If yes, return `(seen[target - x], i)`. Otherwise, store `seen[x] = i` and continue. $O(n)$ time, $O(n)$ space, one pass. The trick is that you don't need to pre-populate the map — at the moment you see the second number, the first is already in the map.

Step-by-step thought process: 1. Reject the brute force. The nested-loop $O(n^2)$ is the failing answer; for a million-element list it's a trillion ops. Even if you write it out, you'll be cut for proposing it. 2. Reject the sort-then-two-pointer. That works but adds $O(n \log n)$ and returns sorted indices, not original ones — you'd have to remember the original indices, which means an extra structure anyway. Hash-map is cleaner. 3. Walk the invariant. After processing index `i`, the `seen` dict contains every value at indices `0..i` mapped to their index. When `target - x` is in `seen`, you've found the pair. 4. Handle duplicates. If `nums = [3, 3]` and target is 6, the second 3 looks up `target - 3 = 3`, which is in `seen` at index 0. Returns (0, 1). The pattern handles duplicates correctly because we check before we write.

Edge cases to mention: - Target sum requires the same element twice — handled correctly because the second occurrence sees the first in `seen`. - No solution — the problem statement says "exactly one solution" but real code should return None or raise. Ask the panel which contract they want. - Negative numbers — work fine; hash table doesn't care about sign.

What NOT to say: - "I'd sort first" — adds $O(n \log n)$ without need; you've also lost the original indices. - "Use `itertools.combinations`" — $O(n^2)$; shows you haven't thought about complexity.

Common follow-up: "How would you extend this to three-sum?" — Sort the array, then for each fixed `i`, run two-pointer on the rest of the array looking for `-nums[i]`. $O(n^2)$ total, $O(1)$ extra space if you can mutate. The sort is justified here because three-sum needs to dedupe triplets, and sorting makes dedupe trivial.

Stand-out line: "The two-sum pattern is the canonical hash-map one-pass — once you've seen it, you start spotting it everywhere: anagram pairs, complement-set problems, even some flavour of substring matching."

```
def two_sum(nums: list[int], target: int) -> tuple[int, int] | None:
    seen: dict[int, int] = {}
    for i, x in enumerate(nums):
        if (complement := target - x) in seen:
            return (seen[complement], i)
        seen[x] = i
    return None
```

Q8 · Group anagrams — sorted-string-key vs tuple-counter-key.

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Flipkart, JP Morgan, Microsoft India · Topic: Hashmap patterns

The question (interviewer's verbatim phrasing):

Given a list of strings, group them so that anagrams are in the same bucket. Return the list of groups.

The 30-second answer: Use a `defaultdict(list)` keyed by the canonical form of each string. Two anagram-canonical forms work: `"".join(sorted(s))` (clear, $O(k \log k)$ per string for length- k string), or `tuple(Counter(s).items())` after sorting ($O(k)$ per string but more code). For interview, the sorted-string key is the cleaner answer. Total time $O(n k \log k)$ for n strings of average length k .

Step-by-step thought process: 1. State the canonicalization idea. Two strings are anagrams iff their sorted forms are equal. So if we key a dict by sorted form, anagrams group automatically. That's the algorithm. 2. Walk the implementation. `groups = defaultdict(list); for s in strs: groups["".join(sorted(s))].append(s); return list(groups.values())`. Three lines plus the import. 3. Mention the alternative key. A 26-tuple of letter counts (`tuple(Counter(s).get(c, 0) for c in "abcdefghijklmnopqrstuvwxyz")`) is $O(k)$ instead of $O(k \log k)$ per string, so asymptotically faster — but only matters if k is huge. For typical

interview inputs the sorted-string key wins on clarity. 4. Tag the order question. The problem usually doesn't care about the order of groups or the order within groups. If the panel cares about preserving input order within groups, `defaultdict(list)` already does that — items are appended in input order.

Edge cases to mention: - Empty strings — they all canonicalize to `""` and group together. Correct behaviour. - Strings of different lengths — they have different sorted-form lengths and never collide. Correct behaviour. - Unicode — `sorted(s)` works on the Unicode code points, which is the correct anagram definition for non-ASCII. Be careful with combining characters (NFD vs NFC normalization) if the panel mentions Unicode.

What NOT to say: - "I'd compare each pair of strings" — $O(n^2)$ is the failing answer. - "Use a hash of the string" — wrong because two anagrams hash differently.

Common follow-up: "What if the strings are 1 million chars long? Sorted-string key is too slow." — Switch to the 26-tuple letter-count key. $O(k)$ per string instead of $O(k \log k)$. Or for arbitrary alphabets, use `tuple(sorted(Counter(s).items()))`.

Stand-out line: "The trick to group-by problems in Python is to ask 'what's the canonical form?' — once you have that, `defaultdict(list)` does the rest in three lines."

```
from collections import defaultdict

def group_anagrams(strs: list[str]) -> list[list[str]]:
    groups: defaultdict[str, list[str]] = defaultdict(list)
    for s in strs:
        groups["".join(sorted(s))].append(s)
    return list(groups.values())
```

Q9 · Longest substring without repeating characters — the sliding-window-with-dict pattern.

Tags: Difficulty: Medium · Asked at: Flipkart, Razorpay, Goldman, JP Morgan, Microsoft India · Topic: Sliding window + dict

The question (interviewer's verbatim phrasing):

Find the length of the longest substring of a string that contains no repeating characters.

The 30-second answer: Sliding window with a dict tracking the *last seen index* of each character. Two pointers `left` and `right`. Walk `right` forward. If the current char `s[right]` is in `seen` and `seen[s[right]] >= left`, the window has a duplicate — move `left` to `seen[s[right]] + 1`. Update `seen[s[right]] = right`. Track `best = max(best, right - left + 1)`. $O(n)$ time, $O(\min(n, \text{alphabet}))$ space. The unlock is tracking last-seen-index, not just last-seen-bool.

Step-by-step thought process: 1. Reject the $O(n^2)$ brute force — every-substring-with-set-check. That's the failing tier. 2. Anchor on the invariant: `s[left:right+1]` is always a window with no duplicates. As `right` moves forward by one, the window grows. If the new char is a duplicate of one in the window, `left` jumps past the old position to maintain the invariant. 3. The dict tracks the *most recent* index of each character. That's why we can skip `left` forward by one — we know exactly where the conflict was. 4. Edge-guard the `seen[char] >= left` check. Without it, characters outside the current window (from a previous duplicate that was already left behind) would falsely trigger a jump.

Edge cases to mention: - Empty string — answer is 0. Loop runs zero times; `best` stays initialized to 0. - All same char ("aaaa") — answer is 1. After the first char, every iteration jumps `left` to `right`. - All unique ("abcde") — answer is `len(s)`. `left` never moves; `right - left + 1` reaches `len(s)`.

What NOT to say: - "Use a set to track current window" — works but you have to remove from the set when `left` advances, which is bookkeeping. The dict-of-indices is cleaner. - "Try every substring" — $O(n^3)$ with the inner uniqueness check; instant fail.

Common follow-up: "What if the question asked for the substring itself, not just the length?" — Track `best_start` alongside `best`. When you update `best`, also update `best_start = left`. Return `s[best_start:best_start + best]`. Same time, $O(1)$ extra state.

Stand-out line: "Sliding window with a dict of last-seen-index is the pattern for 'longest [thing] with constraint' on a string — once you see it, half the medium-difficulty string problems collapse to it."

```
def longest_unique(s: str) -> int:
    seen: dict[str, int] = {}
    left = 0
    best = 0
    for right, c in enumerate(s):
        if c in seen and seen[c] >= left:
            left = seen[c] + 1
        seen[c] = right
        best = max(best, right - left + 1)
    return best
```

Q10 · Set operations — intersection, union, difference — and when you'd use each in an interview.

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Razorpay, Postman · Topic: Set operations

The question (interviewer's verbatim phrasing):

What's the difference between `a & b`, `a | b`, `a - b`, and `a ^ b` on sets? Give me an interview problem for each.

The 30-second answer: `a & b` is intersection (in both) — used for "common elements" problems. `a | b` is union (in either) — used for "all unique elements" problems. `a - b` is difference (in `a` not `b`) — used for "removed since last snapshot" problems. `a ^ b` is symmetric difference (in exactly one) — used for "what changed" or the find-the-single-different-element problems. All are $O(\min(|a|, |b|))$ average and far faster than the equivalent loops, plus they read like English.

Step-by-step thought process: 1. Match each operation to its interview tell. "Common friends between two users" → intersection. "All distinct skills across two resumes" → union. "Users who churned this month" → difference. "Edges that exist in only one of two graphs" → symmetric difference. 2. State the complexity. All four are roughly $O(\min(|a|, |b|))$ on average — they iterate the smaller set and look up in the larger. The method form (`a.intersection(b)`) accepts any iterable, but the operator form (`a & b`) requires both sides to be sets. 3. Mention the chainable forms. `set.intersection(a, b, c)` works on three or more sets; same for union, difference, symmetric difference. Useful when the problem grows to N sets. 4. Call out the in-place variants. `a &= b` updates `a` to the intersection; `a |= b` to the union. Avoids allocating a new set when the result is going to replace the original.

Edge cases to mention: - Sets aren't ordered — if the problem cares about order, you need to dedupe with `dict.fromkeys(iter)` (insertion-order preserving) and then convert, not a set. - Frozen sets work the same way, but are hashable and can themselves be set elements. - Sets vs lists: `a in some_set` is $O(1)$ average vs `a in some_list` $O(n)$; for membership-heavy code this is the single biggest perf win.

What NOT to say: - "I'd loop through and check membership" — never; set ops are both faster and clearer. - "Sets are slow because of hashing" — wrong; the hash is computed once per element and lookups are amortized $O(1)$.

Common follow-up: "How would you find the elements that appear in exactly two of three sets?" — `(a & b) ^ (b & c) ^ (a & c)` doesn't quite work — the elements in all three cancel. The cleaner form is `(a & b - c) | (b & c - a) | (a & c - b)`. Or you can `Counter` the disjoint union of the three sets and pick elements with `count == 2`.

Stand-out line: "Set ops are the highest-leverage Python idiom for the panel — they signal you've internalized that 'membership test' belongs in a set, not in a loop."

```
team_a = {"alice", "bob", "charlie"}
team_b = {"bob", "charlie", "dave"}

team_a & team_b      # {'bob', 'charlie'}    - both teams
team_a | team_b      # {'alice', 'bob', 'charlie', 'dave'} - either
team_a - team_b      # {'alice'}          - only team_a
team_a ^ team_b      # {'alice', 'dave'}   - exactly one team
```

Q11 · Find the duplicate in an array of $n+1$ integers in range $[1..n]$ — the constraint that unlocks $O(n)/O(1)$.

Tags: Difficulty: Medium · Asked at: Microsoft India, Goldman, JP Morgan, Razorpay, Postman · Topic: Hashmap tricks

The question (interviewer's verbatim phrasing):

You have an array of $n+1$ integers, each in the range $[1..n]$, and exactly one value appears more than once. Find it. Bonus: do it in $O(1)$ space.

The 30-second answer: Two answers, depending on the space constraint. With $O(n)$ space, use a `set` — walk the array, add to the set, return the first element already in the set. With $O(1)$ space, the trick is Floyd's cycle detection on the implicit linked list formed by `nums[i]` (since values are valid indices and one is duplicated, there's a cycle). The hash-set answer is the one to write; the Floyd answer is the one to mention.

Step-by-step thought process: 1. Lead with the hash-set solution — it's the cleanest and what most panels actually want. `seen = set()`; iterate; `if x in seen: return x; seen.add(x)`. $O(n)$ time, $O(n)$ space. 2. Mention the sort alternative — $O(n \log n)$ but $O(1)$ extra space if you can mutate. Lower tier than the hash-set unless the constraint specifically rules out auxiliary space. 3. Drop the Floyd hint. The problem reframed: treat `nums` as a function `i → nums[i]`. Because one value is duplicated, two indices map to the same value, which means the functional graph has a cycle. Floyd's tortoise-and-hare finds the cycle entry in $O(n)$ time and $O(1)$ space. 4. State the trade-off explicitly. The hash-set is more readable and works on any input. Floyd's is brilliant but only works because the constraint "values in $[1..n]$, length $n+1$ " guarantees a cycle. If the panel wants the brilliant version, they're fishing for Floyd.

Edge cases to mention: - Multiple duplicates — the hash-set returns the first one encountered, which depends on input order. Floyd's finds the one whose cycle is detected first. Both correct interpretations; clarify with the panel. - Values outside $[1..n]$ — Floyd's breaks (could index out of range); hash-set still works. - Empty array or no duplicate — depends on contract; default behaviour is to fall off the loop and return `None`.

What NOT to say: - "Sort and look for adjacent duplicates" — works but $O(n \log n)$; mention as an alternative, not the primary. - "Use XOR" — XOR finds the *single* element in a duplicates-paired array, not the duplicate in a near-unique array. Wrong tool.

Common follow-up: "Walk me through Floyd's on the array `[1, 3, 4, 2, 2]` ." — Treat as a graph: $0 \rightarrow 1, 1 \rightarrow 3, 2 \rightarrow 4, 3 \rightarrow 2, 4 \rightarrow 2$. The cycle is $2 \rightarrow 4 \rightarrow 2$. Tortoise and hare both start at index 0, advance by `nums[i]`; when they meet, reset tortoise to 0 and advance both by 1 step; they meet again at the cycle entry, which is the duplicate. The pointer indices, not the values, drive the algorithm.

Stand-out line: *"The hash-set is the right answer; Floyd's is the answer you mention to show you've seen the trick — and you'd reach for it only when $O(1)$ space is genuinely a hard constraint."*

```
def find_duplicate(nums: list[int]) -> int | None:
    seen: set[int] = set()
    for x in nums:
        if x in seen:
            return x
        seen.add(x)
    return None
```

Q12 · What is a hash collision and how does CPython's `dict` handle them?

Tags: Difficulty: Medium · Asked at: Microsoft India, Goldman, JP Morgan, Razorpay, Postman · Topic: Hash internals

The question (interviewer's verbatim phrasing):

What's a hash collision and how does Python's dict handle them under the hood?

The 30-second answer: A hash collision is when two distinct keys hash to the same table slot.

CPython's `dict` uses open addressing with perturbation-based probing — on a collision, it computes a next slot via `j = (5*j + 1 + perturb) & mask; perturb >>= 5` and re-checks. This is different from Java's `HashMap`, which uses separate chaining (each slot holds a linked list). Open addressing is what makes Python's compact dict layout possible. Average lookup stays $O(1)$ because CPython resizes the table once it hits $\sim 2/3$ load.

Step-by-step thought process: 1. Distinguish open addressing from separate chaining out loud. Open addressing: one entry per slot, collisions resolved by probing. Separate chaining: each slot holds a list of entries that hashed there. CPython does the former; Java's `HashMap` does the latter. 2. Walk through the lookup with a collision. Compute hash mod table size to get slot A. Slot A holds key X, which doesn't match the lookup key. Perturb to slot B. Slot B is empty → key is absent. Or: slot B holds the lookup key → return its value. 3. Mention deletion's quirk. Deletion can't just empty the slot, or probing for a later key would terminate early. Instead, deleted slots get a "dummy" sentinel that probing skips over but treats as occupied for termination. The dummies are compacted away on resize. 4. State the load-factor consequence. CPython resizes to grow when occupied slots exceed roughly $2/3$ of the table. This keeps the expected probe length below ~ 2 , which is why " $O(1)$ average lookup" is defensible.

Edge cases to mention: - The hash is mixed with the slot index to spread keys that hash to nearby values (`hash(key) & mask`). Without that, sequentially-hashing keys would cluster. - Two distinct keys with the same `hash()` value are still distinguishable — the slot stores both the hash and the key, and the lookup does an equality check. - An adversarial input can force worst-case $O(n)$ lookup (every

key collides). Python mitigates this with hash randomization for strings since 3.3 — see PEP 456 /

`PYTHONHASHSEED` .

What NOT to say: - "Python's dict uses chaining" — wrong; that's Java. - "Collisions never happen" — wrong; they happen all the time once the table fills; probing handles them.

Common follow-up: "Why does Python randomize string hashes by default?" — Without randomization, an attacker who knows the hash function can craft inputs that all collide, degrading dict to O(n) per op. By randomizing per-process, Python makes hash-flood DoS attacks impractical against running services. Disable via `PYTHONHASHSEED=0` for reproducibility — but only in tests, never in prod.

Stand-out line: "Open addressing is what makes the compact-and-ordered dict layout possible — once you understand that, every dict-internal interview question becomes the same question."

```
# probing in action – pseudocode of dict lookup
def lookup(table, key):
    h = hash(key)
    perturb = h
    slot = h & (len(table) - 1)
    while table[slot] is not EMPTY:
        if table[slot].key == key:
            return table[slot].value
        perturb >>= 5
        slot = (5 * slot + 1 + perturb) & (len(table) - 1)
    raise KeyError(key)
```

Q13 · `dict.setdefault` vs `defaultdict` — which is more idiomatic and why?

Tags: Difficulty: Easy · Asked at: Razorpay, Flipkart, Postman, Swiggy, Microsoft India · Topic: dict idioms

The question (interviewer's verbatim phrasing):

I see two ways to build a group-by dict — `d.setdefault(k, []).append(x)` and `defaultdict(list)` . Which is more Pythonic and why?

The 30-second answer: `defaultdict(list)` is more idiomatic when you know up front that all the values will be lists (or whatever the factory produces). `setdefault` is more idiomatic when you mix patterns — some keys get a default, others get a custom initial value, and you don't want the implicit auto-creation behaviour everywhere. The deeper rule: `defaultdict` commits the whole dict to one default; `setdefault` commits one call. `defaultdict` is slightly faster (no per-call lookup), but the real reason to pick it is intent — it says "every value in this dict is a list" at the top of the function.

Step-by-step thought process: 1. Write both versions side by side. `for x in items:`
`groups.setdefault(key(x), []).append(x)` vs `for x in items:`

`groups[key(x)].append(x)` where `groups = defaultdict(list)`. The `defaultdict` version is shorter and reads better. 2. Mention the gotcha with `defaultdict`. `d[unseen_key]` creates `unseen_key` in the dict. If you only meant to test, use `d.get(k)` or `k in d`. This is the trap where `defaultdict` silently grows the dict during what looks like read-only code. 3. Compare to dict comprehension. `{k: list(v) for k, v in itertools.groupby(sorted(items), key=key)}` is the no-mutation alternative — works if you can sort, doesn't if you can't or the input is huge. 4. Mention the perf difference if pressed. `defaultdict` is roughly 30% faster in tight loops because `setdefault` has to look up the method, call it, and handle the default arg per iteration. Doesn't matter unless the loop is hot.

Edge cases to mention: - `setdefault` returns the value at that key (existing or just-inserted), so the chain `setdefault(k, []).append(x)` works. `defaultdict`'s `d[k]` does the same. - Mutating the default value matters. `setdefault(k, [])` creates a new list per call, but `setdefault(k, default_list)` would alias every value to the same list — same bug as mutable-default-argument. - `defaultdict` can take any zero-arg callable. `defaultdict(lambda: 0)` is the same as `defaultdict(int)`. `defaultdict(lambda: {"count": 0, "total": 0})` lets you initialize compound structures.

What NOT to say: - "Use `if k not in d: d[k] = []; d[k].append(x)`" — verbose and slower than both alternatives. The panel will mentally tag you as "writes Python like Java." - "setdefault is always wrong" — not wrong, just more verbose; defensible when you genuinely want a mixed default policy.

Common follow-up: "What does `dict.fromkeys(keys, default)` do, and how is it different from `defaultdict`?" — `fromkeys(keys, default)` creates a dict pre-populated with every key in `keys` mapping to the *same* `default` value. No factory, no lazy init — eager creation, shared reference. Footgun: `dict.fromkeys(["a", "b"], [])` gives you both keys pointing to the *same* list; appending to one appends to both.

Stand-out line: *"defaultdict signals intent at the top of the function — setdefault signals intent at the call site — pick the one that matches your scope."*

```

from collections import defaultdict

# verbose
groups: dict[str, list[int]] = {}
for k, v in pairs:
    if k not in groups:
        groups[k] = []
    groups[k].append(v)

# setdefault
groups: dict[str, list[int]] = {}
for k, v in pairs:
    groups.setdefault(k, []).append(v)

# defaultdict — idiomatic when the whole dict is one shape
groups: defaultdict[str, list[int]] = defaultdict(list)
for k, v in pairs:
    groups[k].append(v)

```

Q14 · First non-repeating character — three solutions and which to write.

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Razorpay, Goldman · Topic: Hashmap patterns

The question (interviewer's verbatim phrasing):

Find the first character in a string that doesn't repeat. Return its index, or -1 if every character repeats.

The 30-second answer: Two-pass approach with `Counter`. First pass: build `freq = Counter(s)`. Second pass: walk `s` with `enumerate`, return the first index where `freq[c] == 1`. Total $O(n)$ time, $O(\text{alphabet})$ space. Why two passes? Because in one pass you don't yet know what comes later, so you can't decide "non-repeating." The two-pass is cleaner than the single-pass-with-ordered-dict trick.

Step-by-step thought process: 1. Reject the $O(n^2)$ brute force — `for c in s: if s.count(c) == 1: return s.index(c)` is short but quadratic. The interviewer is checking whether you'd write it. 2. Walk through the two-pass. `Counter(s)` is one pass. The second loop walks `s` again, in order, and returns the first character with count 1. Both passes are $O(n)$; total $O(n)$. 3. Mention the single-pass-with-ordered-dict alternative. Insert each character into an ordered dict that tracks count; at the end, the first key with count 1 in iteration order is the answer. Same asymptotic complexity, more code, no clear win — but worth mentioning if the panel pushes. 4. Edge-guard the return. If no character has count 1, return -1 (or None or raise — confirm contract). The default behaviour of falling off the loop should explicitly return -1.

Edge cases to mention: - Empty string — answer is -1. The Counter is empty; the second loop runs zero times. - All characters unique — return 0; the very first character has count 1. - All characters repeating — return -1.

What NOT to say: - "Use `s.index(c)` inside a loop" — that's $O(n^2)$ and shows you haven't internalized the cost. - "Sort the string and look for singletons" — $O(n \log n)$ and loses the original index, so it doesn't even answer the question.

Common follow-up: "What if the string is a stream and you can only see each character once?" — Single-pass with an ordered dict that tracks (count, index). At any point, the first key with count 1 is the answer. As you see repeats, you don't remove keys (you'd lose the original order); you just bump the count. Time $O(n)$, space $O(\text{alphabet})$.

Stand-out line: "Two-pass with `Counter` is the cleanest Python answer — it's three lines, it's $O(n)$, and it reads like English."

```
from collections import Counter

def first_unique(s: str) -> int:
    freq = Counter(s)
    for i, c in enumerate(s):
        if freq[c] == 1:
            return i
    return -1
```

Q15 · Longest consecutive sequence in an unsorted array — the set-membership trick.

Tags: Difficulty: Medium · Asked at: Microsoft India, Goldman, JP Morgan, Razorpay, Postman · Topic: Hashmap tricks

The question (interviewer's verbatim phrasing):

Given an unsorted array of integers, find the length of the longest sequence of consecutive integers — for example `[100, 4, 200, 1, 3, 2]` returns 4 because `[1, 2, 3, 4]` is the longest run.

The 30-second answer: Put all values in a `set` for $O(1)$ membership. Walk the array once; for each value `x`, check if `x - 1` is also in the set. If yes, skip — `x` isn't the start of a sequence. If no, `x` is a sequence start; walk forward (`x+1`, `x+2`, ...) counting matches until you fall off. Track the best run length. $O(n)$ time, $O(n)$ space. The trick is the "only start counting from sequence starts" guard; otherwise the walk-forward step would re-traverse the same runs and you'd be $O(n^2)$.

Step-by-step thought process: 1. Reject the sort-then-walk. $O(n \log n)$; the interviewer is asking for $O(n)$ specifically by phrasing it this way. 2. State the set-membership idea. Membership is $O(1)$ average, so we can ask "is $x-1$ here? is $x+1$ here?" in constant time. That's the foundation. 3. Walk the "sequence start" guard. Without it, for `[1, 2, 3, 4]` you'd start counting from 1 (run 4), from 2 (run 3), from 3 (run 2), from 4 (run 1) — $O(n)$ work per starting point, $O(n^2)$ total. With the guard, only $x = 1$ triggers a walk because $x-1 = 0$ isn't in the set. Total work is bounded by $2n$. 4. Edge-guard: empty input returns 0.

Edge cases to mention: - Duplicates in input — the set deduplicates automatically. Walking from a sequence start doesn't re-count duplicates. - Negative numbers — work fine; the set doesn't care about sign. - Single element — answer is 1.

What NOT to say: - "Sort the array first" — $O(n \log n)$; doesn't meet the $O(n)$ ask. - "Use a counter" — `Counter` doesn't help here; you need set-membership, not counts.

Common follow-up: "What's the worst case if I picked the wrong guard?" — Without the `x-1 not in seen` guard, the inner walk would re-traverse the same sequences from every start, making the algorithm $O(n^2)$ in the worst case. The guard is what makes the amortized analysis $O(n)$.

Stand-out line: "The trick is the 'only-start-from-sequence-starts' guard — without it, the walk-forward step is redundant and the algorithm collapses to $O(n^2)$."

```
def longest_consecutive(nums: list[int]) -> int:
    seen = set(nums)
    best = 0
    for x in seen:
        if x - 1 not in seen:          # only start from sequence starts
            length = 1
            while x + length in seen:
                length += 1
            best = max(best, length)
    return best
```

Section C — Heaps and `heapq`

Five questions on `heapq`, the priority-queue module that's a min-heap by default and the answer to roughly half of all "top-K" and "median-of-stream" interview problems. Indian product-co panels lean on heap problems in round two because they separate candidates who know the data structure from candidates who reach for `sorted()` and end up with $O(n \log n)$ where $O(n \log k)$ was on the table.

Q16 · When do you reach for `heapq` instead of `sorted()` ?

Tags: Difficulty: Easy · Asked at: Razorpay, Flipkart, Microsoft India, Goldman, JP Morgan · Topic: Heap fundamentals

The question (interviewer's verbatim phrasing):

When would you pick `heapq` over `sorted()` ?

The 30-second answer: Three patterns. (1) Top-K from a large stream — `heapq.nlargest(k, iter)` is $O(n \log k)$ vs `sorted(iter)[:k]` which is $O(n \log n)$. (2) Continuous merging of multiple sorted streams — `heapq.merge` is the streaming merge of k sorted iterables, $O(n \log k)$. (3) Anything you'd build a priority queue for — task scheduling, Dijkstra, Huffman, A. *The rule: if you need the smallest* (or largest) element repeatedly while new elements stream in, heap; if you need the whole thing sorted once, sort.*

Step-by-step thought process: 1. Anchor on the asymptotic. `sorted(iter)[:k]` builds the whole sorted list ($O(n \log n)$) and discards most of it. `heapq.nlargest(k, iter)` maintains a heap of size k as it walks ($O(n \log k)$). For $k=10$ and $n=10M$, that's a $10\times$ win. 2. Walk the heap operations. `heappush(h, x)` is $O(\log n)$. `heappop(h)` is $O(\log n)$ and pops the smallest. `heappushpop(h, x)` and `heapreplace(h, x)` combine the two ops in one $O(\log n)$ call. `heapify(lst)` turns an arbitrary list into a heap in $O(n)$ — not $O(n \log n)$. 3. Mention the min-heap default. `heapq` is a min-heap; the smallest element is always at `h[0]`. For max-heap, negate the keys (`heappush(h, -x)`) or pair with a sign field. There's no `MaxHeap` class. 4. Tag the stability question. Heaps aren't stable across equal keys — if order-of-insertion matters for ties, push tuples `(key, counter, value)` where `counter` is a monotonically increasing integer that breaks ties.

Edge cases to mention: - `heapq` operates on a plain `list` — the heap *is* the list, with the heap invariant maintained by the module functions. No `Heap` class. - Mixing heap-mutating functions (`heappush` , `heappop`) with non-heap-mutating functions (`list.append` , `list.pop`) breaks the invariant silently. Pick one mode. - `heapq.heapify` is $O(n)$, not $O(n \log n)$ — many candidates state it as $O(n \log n)$; it's a stand-out moment to know the linear-time bottom-up heapify proof.

What NOT to say: - "Use `sorted()` for everything" — wrong tier for top-K and merging streams. - "heapq has a MaxHeap" — wrong; there's only min-heap; negate keys for max.

Common follow-up: "What's the time complexity of `heapq.heapify(lst)` ?" — $O(n)$, not $O(n \log n)$. The proof is the geometric series of subtree depths: levels near the bottom have many nodes but small subtrees, levels near the top have few nodes with big subtrees, and the total work is bounded by $2n$.

Stand-out line: "The interview tell is that the panel asked for 'top K' or 'merge sorted' — both are heap-shaped questions, and reaching for `sorted()` is the answer that gets you marked as 'hasn't internalized data-structure cost.'"

```
import heapq

# top-K largest in a stream of 10M numbers
top10 = heapq.nlargest(10, stream)          # O(n log k)

# merge sorted streams without buffering
for x in heapq.merge(stream_a, stream_b, stream_c):
    process(x)                             # constant memory

# linear-time heapify
h = [5, 1, 3, 8, 2]
heapq.heapify(h)                           # O(n)
```

Q17 · Top-K elements — the heap-of-size-K pattern in idiomatic Python.

Tags: Difficulty: Medium · Asked at: Razorpay, Flipkart, Microsoft India, Goldman, Postman · Topic: Top-K

The question (interviewer's verbatim phrasing):

Given a stream of integers, return the K largest. Walk through your algorithm and write the Python.

The 30-second answer: Maintain a min-heap of size K. For each new element, if the heap has fewer than K items, push. Otherwise, if the element is larger than `heap[0]` (the current min of the top-K), `heapreplace` it. At the end, the heap holds the K largest, retrievable in any order or sorted via `sorted(heap, reverse=True)`. $O(n \log k)$ time, $O(k)$ space. The min-heap is the right shape because we keep ejecting the smallest of the top-K when a bigger candidate shows up.

Step-by-step thought process: 1. Reject `sorted(stream, reverse=True)[:K]` — $O(n \log n)$ time and $O(n)$ memory, both worse than the heap solution. On a stream of a billion this doesn't fit in memory. 2. Anchor on "min-heap of size K." Counter-intuitive — the K largest live in a *min*-heap because we want $O(1)$ access to the *smallest of the K* to compare against new candidates. 3. Walk the algorithm. While heap size is $< K$, just push. Once at K, peek at `heap[0]` (the min of the current top-K). If the new element is $> \text{heap}[0]$, it belongs in the top-K and the old min gets ejected. Use `heapreplace` to do that in one $O(\log k)$ call. 4. Mention the built-in `heapq.nlargest(k, iter)`. It implements exactly this pattern. The interview is asking you to show you'd write it; in production you'd call the built-in.

Edge cases to mention: - K larger than the input — return everything sorted descending. - $K = 0$ — return empty list; guard at the top. - Stream of length 0 — return empty list.

What NOT to say: - "Sort the whole stream and take the top K" — fails on infinite streams or memory-bounded environments. - "Use a max-heap" — wrong; with a max-heap you'd need $O(n)$ work per insertion to find and reject the smallest of the top-K. Min-heap is the correct shape.

Common follow-up: "What if the elements are tuples and you need top-K by a key?" — Push `(key(x), counter, x)` where `counter` is a monotonically-increasing tie-breaker (heaps aren't stable; without `counter`, equal keys would try to compare the original objects, which may not be comparable). Built-in `heapq.nlargest(k, iter, key=lambda x: ...)` handles this cleanly.

Stand-out line: "The min-heap of size K is the canonical top-K pattern — once you've seen why min and not max, you stop reaching for `sorted()` on every top-K question."

```
import heapq

def top_k(stream, k: int) -> list[int]:
    if k <= 0:
        return []
    heap: list[int] = []
    for x in stream:
        if len(heap) < k:
            heapq.heappush(heap, x)
        elif x > heap[0]:
            heapq.heapreplace(heap, x)
    return sorted(heap, reverse=True)
```

Q18 · Median of stream — the two-heap pattern.

Tags: Difficulty: Hard · Asked at: Microsoft India, Goldman, JP Morgan, Razorpay, Flipkart · Topic: Heap tricks

The question (interviewer's verbatim phrasing):

Numbers arrive one at a time. After each insertion, return the median of everything seen so far. Walk me through the data structure and the algorithm.

The 30-second answer: Two heaps. A max-heap (`low`) holds the lower half; a min-heap (`high`) holds the upper half. Invariants: `len(low) - len(high) ∈ {0, 1}` and every element in `low` ≤ every element in `high`. On insert, push to one heap, possibly rebalance by popping from the other. Median is `low[0]` if `low` is larger, or `(low[0] + high[0]) / 2` if equal. Insertion is $O(\log n)$; median lookup is $O(1)$.

Step-by-step thought process: 1. Anchor on the invariant. The median is the middle element of a sorted sequence. We split that sequence at the middle, store the lower half in a max-heap (so the largest

of the lower half is at the top, $O(1)$ to read), and the upper half in a min-heap (so the smallest of the upper half is at the top, $O(1)$). The median is one of those two tops, or their average. 2. Walk the insertion logic. If the new value is $\leq$ low's max, push to low. Otherwise push to high. Then rebalance: if low has 2 more elements than high, move low's max to high (`heappush(high, -heappop(low))`). Symmetric if high has more than low. 3. Note the max-heap trick. Python's `heapq` is min-only, so the max-heap is implemented by pushing negated values. `low[0]` is the most-negative, which is the most-positive when you negate it back. 4. State the API. `add(x)` is $O(\log n)$, `median()` is $O(1)$ by reading the tops. Total memory $O(n)$.

Edge cases to mention: - Even count — median is the average of the two tops; even Python's `int / int` gives a float, which is what you want. - Odd count — median is the top of the larger heap. Convention: when sizes are equal, the next element goes to `low`, making `low` the "larger by one" heap; then `low[0]` is the median for odd counts. - Stream of duplicates — equal values go to either heap consistently; the invariant still holds.

What NOT to say: - "Use `sorted()` after each insertion" — $O(n \log n)$ per insert, $O(n^2 \log n)$ total. Fails on long streams. - "Use a single sorted list with `bisect.insort`" — `bisect.insort` is $O(n)$ per insertion because of the shift; still $O(n^2)$ total. The two-heap pattern is the correct answer.

Common follow-up: "What's the memory cost if the stream has a billion elements?" — $O(n)$ for the heaps, which is unbounded in a true stream. For bounded memory, use a sliding-window variant or sketch-based approximate median like t-digest.

Stand-out line: "Two heaps is the elegant pattern because the median is always one $O(1)$ read from the tops — the maintenance is $O(\log n)$, and you've decoupled insert cost from query cost."

```
import heapq

class MedianStream:
    def __init__(self) -> None:
        self.low: list[int] = []    # max-heap (negated)
        self.high: list[int] = []   # min-heap

    def add(self, x: int) -> None:
        if not self.low or x <= -self.low[0]:
            heapq.heappush(self.low, -x)
        else:
            heapq.heappush(self.high, x)
        # rebalance: low has at most 1 more than high
        if len(self.low) > len(self.high) + 1:
            heapq.heappush(self.high, -heapq.heappop(self.low))
        elif len(self.high) > len(self.low):
            heapq.heappush(self.low, -heapq.heappop(self.high))

    def median(self) -> float:
        if len(self.low) > len(self.high):
            return -self.low[0]
        return (-self.low[0] + self.high[0]) / 2
```

Q19 · Merge K sorted lists — `heapq.merge` and the heap-of-heads pattern.

Tags: Difficulty: Medium · Asked at: Microsoft India, Goldman, JP Morgan, Postman, Razorpay · Topic: Heap merging

The question (interviewer's verbatim phrasing):

You have K sorted lists (or iterators). Merge them into a single sorted output. K can be in the thousands.

The 30-second answer: `heapq.merge(*iterables)` is the one-line idiomatic answer — it's a streaming K-way merge that returns an iterator and uses $O(K)$ memory regardless of total length. Underneath it's a heap of head elements: pop the smallest, advance that source, push the new head. $O(n \log K)$ total time for n elements across K lists. For the interview, you should also be able to write the heap-of-heads version by hand.

Step-by-step thought process: 1. Lead with `heapq.merge`. Show that you know the built-in exists. `for x in heapq.merge(*lists): yield x` is three words for the whole problem. 2. Walk through the manual version. Push the head of each list as a tuple `(value, list_index, iterator)` into a heap. Pop the smallest tuple, emit its value, advance that iterator, push the next value if any. Continue until the heap is empty. 3. The `list_index` is a tiebreaker — without it, two equal values would compare the iterators, which can't be compared. The tuple-comparison falls back to the index field, which is unique per list. 4. State the complexity. Each element is pushed and popped once. Heap operations on a K-sized heap are $O(\log K)$. Total: $O(n \log K)$ time, $O(K)$ space — independent of n.

Edge cases to mention: - One of the lists is empty — its iterator raises `StopIteration` on the first call; just skip pushing initially. - All lists empty — return empty iterator. - K equals 1 — the answer is just the one list; both `heapq.merge` and the manual version handle this trivially.

What NOT to say: - "Concatenate all lists and sort" — $O(n \log n)$, and you've lost the K-way structure that makes this faster. Also breaks on truly streaming inputs. - "Pairwise merge" — $O(nK)$ in the bad case; the heap-of-heads is strictly better.

Common follow-up: "What's the difference between `heapq.merge` and `sorted(chain(*lists))`?" — `heapq.merge` is $O(n \log K)$ and streaming (constant memory in K); `sorted(chain(*lists))` is $O(n \log n)$ and materializes the whole output in memory. For $K=10$ and $n=10^9$, that's the difference between a working pipeline and OOM.

Stand-out line: "`heapq.merge` is the one-liner that turns the merge-K-sorted question into a non-question — knowing it exists is half the answer; being able to write the heap-of-heads by hand is the other half."

```
import heapq
import itertools

# idiomatic — streaming, O(K) memory
for x in heapq.merge(list_a, list_b, list_c):
    process(x)

# manual — what to write if asked
def merge_k(lists: list[list[int]]):
    heap = []
    for i, lst in enumerate(lists):
        it = iter(lst)
        try:
            heapq.append((next(it), i, it))
        except StopIteration:
            pass
    heapq.heapify(heap)
    while heap:
        val, i, it = heapq.heappop(heap)
        yield val
        try:
            heapq.heappush(heap, (next(it), i, it))
        except StopIteration:
            pass
```

Q20 · Priority queue with tiebreakers — the (priority, counter, payload) tuple pattern.

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Microsoft India, Goldman, JP Morgan · Topic: Heap stability

The question (interviewer's verbatim phrasing):

You're building a task queue using `heapq`. Tasks have priorities, but two tasks can have the same priority. How do you ensure FIFO order for equal priorities, and what's the gotcha?

The 30-second answer: Push tuples of `(priority, counter, payload)` where `counter` is a monotonically increasing integer. The heap compares tuples lexicographically, so it sorts by priority first, then by counter — and the counter guarantees FIFO order for ties. The gotcha is twofold: without the counter, the heap would try to compare the payloads themselves to break ties (and may not be able to), and a `next(counter)` from `itertools.count()` is the right primitive — never use `len(heap)` because pops break monotonicity.

Step-by-step thought process: 1. Walk through what happens without a tiebreaker. Push `(priority, task)`. If two tasks share priority, Python compares the tasks; if `task` is a dict or a custom object without `__lt__`, you get `TypeError: '<' not supported`. If they happen to be comparable, the order is whatever the comparison gives — not the insertion order you wanted. 2. The fix: a unique, monotonically increasing counter as the second tuple field. `counter = itertools.count(); heappush(heap, (p, next(counter), task))`. Now tuples compare on priority first, counter second; the payload is never compared. 3. Explain why `len(heap)` is wrong. After a pop, `len(heap)` decreases, and the next push gets a *smaller* counter than the previous one. Two tasks with the same priority can then re-order based on insertion timing — the bug is silent until tested. 4. Mention `dataclasses` with `field(compare=False)`. The modern alternative for typed payloads: define a dataclass with priority + counter as comparable fields and the payload marked non-comparable. Cleaner than tuples for long-lived code.

Edge cases to mention: - A natural priority that's already unique (timestamps with sub-microsecond resolution) skips the need for a counter. But "naturally unique" is dangerous to rely on — add the counter anyway as defence. - For max-heap behaviour, negate the priority (`heappush(h, (-p, c, task))`). The counter stays positive — only the priority field flips. - For a tie-break that's *opposite* (LIFO instead of FIFO), use a decreasing counter (`-next(counter)`). Rarely useful but trivially possible.

What NOT to say: - "Use `(priority, task)` and hope ties don't happen" — they will, and you'll get `TypeError` at runtime in production. - "Use the system time as the tiebreaker" — `time.time()` has limited resolution and can go backwards under NTP adjustment; counters are the safer primitive.

Common follow-up: "*How would you support task cancellation?*" — Mark the task as cancelled in a separate dict; when popping, check and skip cancelled entries (re-pop). This is the "lazy deletion" pattern — `heapq` has no direct delete-by-key. For heavy cancellation, switch to `sortedcontainers.SortedList` or rebuild the heap periodically.

Stand-out line: "*The counter is the trick — without it, the heap silently tries to compare your payloads, and 'works on my machine' becomes 'crashes on dict tasks at peak load.'*"

```
import heapq
import itertools
from dataclasses import dataclass, field
from typing import Any

# tuple form
counter = itertools.count()
heap: list[tuple[int, int, Any]] = []
heapq.heappush(heap, (priority, next(counter), task))

# dataclass form — cleaner for long-lived code
@dataclass(order=True)
class PrioritizedTask:
    priority: int
    seq: int
    task: Any = field(compare=False)

heap2: list[PrioritizedTask] = []
heapq.heappush(heap2, PrioritizedTask(priority=3, seq=next(counter), task=my_task))
```

Section D — Trees

Ten questions on tree data structures — node representation, BFS, DFS in both flavours, depth, paths, lowest common ancestor, and the trie. Indian backend interviews lean heavily on trees in round two because they're the test of "do you understand recursion and the iterative equivalent" without requiring graph-theory background. The idiomatic Python angle: `dict`-based tree representations beat hand-rolled `Node` classes for half the problems, and `collections.deque` is the only correct BFS frontier.

Q21 · How would you represent a binary tree in idiomatic Python?

Tags: Difficulty: Easy · Asked at: Razorpay, Flipkart, Swiggy, Postman, JP Morgan · Topic: Tree representation

The question (interviewer's verbatim phrasing):

Show me how you'd define a binary-tree node in Python for an interview problem.

The 30-second answer: Two clean options. A dataclass — `@dataclass; class TreeNode: val: int; left: "TreeNode | None" = None; right: "TreeNode | None" = None` — which is what most interview panels expect and what LeetCode-style problems use. Or a plain class with

`__init__` if you're not in 3.10+. For competitive-DSA-style problems where the tree is implicit (e.g., heaps), an array indexed by position works (parent at $i//2$, children at $2i$ and $2i+1$), no class needed. The dataclass is what you write in the interview; the array form is what you write when the tree is a heap.

Step-by-step thought process: 1. State the dataclass version. It's three lines, types are explicit, equality is auto-generated. Use forward-reference strings for the recursive type (`"TreeNode | None"`). 2. Mention the plain-class fallback for codebases on Python 3.9 or older where 3.10's PEP 604 union syntax isn't available. `class TreeNode: def __init__(self, val, left=None, right=None): ...`. 3. For non-binary trees, use `children: list["TreeNode"] = field(default_factory=list)`. The factory pattern avoids the mutable-default-argument trap. 4. For tries, prefer nested `dict` — `{}` for the root, each child key maps to a sub-dict, the terminal marker is a sentinel like `"#"`. Lighter than a class hierarchy and far faster to iterate.

Edge cases to mention: - Mutable default arg trap. `def __init__(self, val, children=[]):` aliases the default list across instances. Use `children=None; self.children = [] if children is None else children` or the dataclass `default_factory=list`. - `__slots__` for memory savings on million-node trees: add `__slots__ = ("val", "left", "right")` to the class. Cuts ~50% of per-node memory by skipping the per-instance `__dict__`. - For BSTs you typically add nothing structural — same `TreeNode` class, the invariant is enforced by insertion code, not the type.

What NOT to say: - "I'd use `namedtuple`" — works but `namedtuples` are immutable, which makes it awkward to update children in place. Use dataclass. - "I'd write a `Tree` class wrapping a root" — overkill for interview; the panel wants nodes, not a tree object.

Common follow-up: "What if the tree has a million nodes? Would your representation matter?" — Yes; add `__slots__` to skip the per-instance dict, saving ~32 bytes per node. For really tight memory, switch to a parallel-array representation (`vals: list[int]; left_idx: list[int]; right_idx: list[int]`) where the "node" is an index, not an object. Less Pythonic but ~70% less memory.

Stand-out line: "The dataclass with forward-reference is the modern idiom — three lines, types, equality for free, and the panel sees you're on Python 3.10+ syntax without you saying so."

```

from dataclasses import dataclass, field

@dataclass
class TreeNode:
    val: int
    left: "TreeNode | None" = None
    right: "TreeNode | None" = None

# non-binary
@dataclass
class GenericNode:
    val: int
    children: list["GenericNode"] = field(default_factory=list)

# trie - nested dict, not a class
trie: dict = {}
def insert(word: str) -> None:
    node = trie
    for c in word:
        node = node.setdefault(c, {})
    node["#"] = True # terminal marker

```

Q22 · BFS on a binary tree — why `deque` and not `list` .

Tags: Difficulty: Easy · Asked at: Razorpay, Flipkart, Swiggy, Postman, JP Morgan · Topic: BFS

The question (interviewer's verbatim phrasing):

Walk through level-order traversal of a binary tree. Why does the container choice matter?

The 30-second answer: Use `collections.deque` as the frontier. Initialize `q = deque([root])` . While `q` is not empty, `popleft()` a node, visit it, append its children. The `deque` gives $O(1)$ at both ends; using a `list` with `pop(0)` would be $O(n)$ per pop and turn the traversal into $O(n^2)$ on a balanced tree of size n . The pattern: BFS == deque == FIFO; DFS == list (or recursion) == LIFO.

Step-by-step thought process: 1. State the frontier choice up front. BFS needs FIFO; FIFO needs $O(1)$ at both ends; that's deque, not list. 2. Walk the algorithm. `popleft` gives the next node to visit (oldest in the queue). Append its left and right children (if not None). Continue until the queue is empty. 3. Mention the level-tracking variant. If you need to know which level each node is on, push tuples `(node, level)` into the queue. Or process the queue in batches of `len(q)` snapshots — each batch is one level. 4. Tag the early-termination flavour. For "find shortest path to a target" or "minimum depth," BFS naturally finds the shallowest match because it explores by level. DFS would explore deep before broad and may miss the optimum.

Edge cases to mention: - Empty tree (`root is None`) — return empty result immediately; don't push None into the queue. - Single node — the loop runs once, no children to push. - Skewed tree (all-left chain) — BFS still works but offers no speedup over DFS; both are $O(n)$.

What NOT to say: - "I'd use a list and call `pop(0)`" — $O(n^2)$ traversal; the panel will mentally tag you as "doesn't know container costs." - "I'd use recursion" — that's DFS, not BFS. BFS without an explicit queue is awkward (need to track depth and dispatch); say deque.

Common follow-up: "How would you do level-order traversal where the output is a list of lists (one per level)?" — Snapshot `level_size = len(q)` at the start of each level. Pop exactly that many nodes into a `level` list, while appending children to the queue. Outer loop continues until queue is empty.

Stand-out line: "BFS + deque is one pattern, DFS + recursion-or-list is the other — once you've matched container to traversal, half the tree questions write themselves."

```
from collections import deque

def level_order(root: TreeNode | None) -> list[list[int]]:
    if root is None:
        return []
    out: list[list[int]] = []
    q: deque[TreeNode] = deque([root])
    while q:
        level_size = len(q)
        level: list[int] = []
        for _ in range(level_size):
            node = q.popleft()
            level.append(node.val)
            if node.left:
                q.append(node.left)
            if node.right:
                q.append(node.right)
        out.append(level)
    return out
```

Q23 · DFS — recursive vs iterative — when to switch and how.

Tags: Difficulty: Medium · Asked at: Microsoft India, Goldman, JP Morgan, Razorpay, Flipkart · Topic: DFS

The question (interviewer's verbatim phrasing):

Write both the recursive and iterative versions of inorder traversal. When would you pick iterative over recursive?

The 30-second answer: Recursive DFS is the natural Python expression — three lines, reads like the definition. Iterative DFS uses an explicit stack (`list`), pushes nodes and possibly state markers, and pops in LIFO order. Switch to iterative when (a) the tree is deep enough to hit Python's default recursion limit of 1000 (`sys.getrecursionlimit()`), or (b) you need to pause/resume the traversal mid-stream. Bumping the recursion limit via `sys.setrecursionlimit` works but doesn't change the underlying stack-overflow risk in the C stack.

Step-by-step thought process: 1. Show the recursive version first — it's three lines and demonstrably correct. `def inorder(node): if node: yield from inorder(node.left); yield node.val; yield from inorder(node.right)`. Note the use of `yield from` so it returns a generator. 2. Show the iterative version using a stack. The trick for inorder is to walk left as far as possible, pushing nodes, then pop, visit, switch to right, repeat. Pre-order is simpler — push root, loop, pop, visit, push right then left. Post-order is the trickiest iteratively — either two stacks or a visited marker. 3. State the recursion-limit constraint. Python's default cap is ~1000 frames. A balanced tree of 10M nodes has depth ~23, no problem. A skewed (linked-list-shaped) tree of 1000 nodes hits the wall. 4. Tag `sys.setrecursionlimit`. Raising it works for moderate cases (10k-50k) but the actual C-stack limit is platform-dependent (typically ~10MB, ~50k Python frames). Iterative is the bulletproof answer for trees of arbitrary depth.

Edge cases to mention: - Generators help. Recursive `yield from` traversal pauses between yields, but each suspended generator still has a frame; deep trees still hit the limit. `threading.Thread` with a bigger stack is one workaround but rarely worth it. - Iterative inorder is the trickiest of the three. If the panel asks for "DFS iteratively," they often mean pre-order. Clarify which order. - Tail-call optimization doesn't exist in CPython (deliberately). Don't rely on TCO when porting algorithms from Scheme or Haskell.

What NOT to say: - "Always use recursion in Python" — fails on deep trees. - "Always use iteration" — recursion is the more idiomatic Python for balanced trees; switching is a context-dependent call.

Common follow-up: "How would you do post-order iteratively?" — Two approaches: (1) push tuples `(node, visited)`; on first pop, push it back with `visited=True`, then push children; on the visited pop, emit the value. (2) Do reverse-pre-order (root, right, left) on a stack and reverse the output. Both are $O(n)$ time and $O(h)$ space.

Stand-out line: "Recursive DFS is the natural Python; iterative DFS is the defensive Python — pick recursive for clarity and switch to iterative when the depth might be adversarial."

```

import sys
from typing import Iterator

# recursive – three lines, idiomatic
def inorder_recursive(node: TreeNode | None) -> Iterator[int]:
    if node:
        yield from inorder_recursive(node.left)
        yield node.val
        yield from inorder_recursive(node.right)

# iterative – bulletproof for any depth
def inorder_iterative(root: TreeNode | None) -> Iterator[int]:
    stack: list[TreeNode] = []
    node = root
    while stack or node:
        while node:
            stack.append(node)
            node = node.left
        node = stack.pop()
        yield node.val
        node = node.right

# raise the limit if you must – but iterative is safer
# sys.setrecursionlimit(10_000)

```

Q24 · Maximum depth of a binary tree — recursion and the iterative variant.

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Postman, JP Morgan · Topic: Tree depth

The question (interviewer's verbatim phrasing):

Given the root of a binary tree, return its maximum depth.

The 30-second answer: Recursive: `def depth(node): return 0 if node is None else 1 + max(depth(node.left), depth(node.right))`. Three lines, $O(n)$ time visiting every node, $O(h)$ space for the recursion stack where h is the height. Iterative version uses BFS with a `deque`, counting levels: each loop iteration is one level deeper. Same time and space.

Step-by-step thought process: 1. State the recursive recurrence: $\text{depth}(\text{node}) = 1 + \max(\text{depth}(\text{left}), \text{depth}(\text{right}))$, with base case $\text{depth}(\text{None}) = 0$. The whole algorithm is one expression. 2. Mention the implicit invariant: $\max(0, 0)$ when both children are `None` gives 1, which is correct for a leaf. The base case at `None` returns 0 and the +1 at each level walks back up to the root. 3. The iterative BFS variant. Walk the tree level by level with a deque. Each outer loop iteration processes a complete level (snapshot `len(q)`) and increments the depth counter. Returns the final counter. 4. Tag the deep-tree concern.

For a skewed tree of 100k nodes, recursion hits the limit. The BFS variant is depth-independent — it only stacks a level at a time.

Edge cases to mention: - Empty tree (root None) — returns 0. The base case handles it directly. - Single node — returns 1; recursion gives $\max(0, 0) + 1 = 1$. - Heavily skewed tree — recursion depth equals node count; iterative is mandatory beyond ~1000 nodes.

What NOT to say: - "Depth of a leaf is 0" — convention is depth of a leaf = 1, depth of None = 0. The interview answer is "1" for a single-node tree. - "Use breadth-first to find max depth" — works but is more code than the recursive one-liner; use it only when depth might be adversarial.

Common follow-up: "What's the difference between max depth and balanced-tree check?" — Max depth returns the depth; balanced check returns whether $|\text{depth}(\text{left}) - \text{depth}(\text{right})| \leq 1$ at every node. The naive balanced check is $O(n^2)$ (computing depth at every node); the linear-time variant returns -1 from the depth call to signal "unbalanced" and short-circuits.

Stand-out line: "For balanced trees the recursive one-liner is the answer; for skewed trees the iterative BFS is the answer — knowing both signals you've thought about input shapes."

```
from collections import deque

def max_depth_recursive(node: TreeNode | None) -> int:
    return 0 if node is None else 1 + max(
        max_depth_recursive(node.left),
        max_depth_recursive(node.right),
    )

def max_depth_iterative(root: TreeNode | None) -> int:
    if root is None:
        return 0
    q: deque[TreeNode] = deque([root])
    depth = 0
    while q:
        depth += 1
        for _ in range(len(q)):
            node = q.popleft()
            if node.left:
                q.append(node.left)
            if node.right:
                q.append(node.right)
    return depth
```

Q25 · Lowest common ancestor in a binary tree — the recursive single-pass solution.

Tags: Difficulty: Medium · Asked at: Microsoft India, Goldman, JP Morgan, Razorpay, Flipkart · Topic: LCA

The question (interviewer's verbatim phrasing):

Given the root of a binary tree and two distinct node values `p` and `q`, return the lowest common ancestor.

The 30-second answer: Single-pass recursion. `lca(node)` returns either `p`, `q`, or the LCA. Base case: if `node` is `None` or matches `p` or `q`, return `node`. Recurse on left and right. If both recursive calls returned non-`None`, the current node is the LCA. Otherwise return whichever one returned non-`None`. $O(n)$ time, $O(h)$ space for recursion. The unlock: the LCA is the deepest node where both `p` and `q` are found in different subtrees (or one is the current node itself).

Step-by-step thought process: 1. Anchor on the recurrence. `lca(node, p, q)` returns "the LCA if it's in this subtree, else `p` or `q` if exactly one is in this subtree, else `None`." 2. Base cases: `node is None` returns `None`; `node.val == p or node.val == q` returns `node` itself (we've found one of the targets). 3. Recursive step: get `left = lca(node.left, p, q)` and `right = lca(node.right, p, q)`. If both are non-`None`, the current node is the split point — return it. If exactly one is non-`None`, propagate it up. If both are `None`, return `None`. 4. Why this works in a single pass. The function never explicitly "finds" both nodes — it just propagates one or both up, and the node that sees both at once becomes the answer. Elegantly recursive.

Edge cases to mention: - `p` or `q` is the LCA itself (one is an ancestor of the other) — the base case returns the ancestor node, the other branch returns `None`, and the parent sees one non-`None` and propagates it. Correct behaviour. - `p` or `q` doesn't exist in the tree — the function returns whatever ancestor is reachable, which is incorrect for that contract. Usual assumption is both exist; if not, do a pre-check. - Pointers vs values — the problem usually compares by value; the algorithm is identical with pointer comparison.

What NOT to say: - "Find the path to `p` and the path to `q`, then find the divergence point" — works but two passes plus path storage; the single-pass recursion is the cleaner answer. - "Use parent pointers" — only works if the tree has them; the canonical interview tree doesn't.

Common follow-up: "How would you find LCA in a BST?" — Simpler than general LCA. Walk from the root; if both `p` and `q` are less than the current node, go left; if both greater, go right; otherwise the current node is the LCA (one is left, one is right, or one matches). $O(h)$ time, no recursion needed.

Stand-out line: "The single-pass LCA is the cleanest recursion in tree DSA — it propagates partial information up without ever explicitly searching for the nodes."

```
def lca(node: TreeNode | None, p: int, q: int) -> TreeNode | None:
    if node is None or node.val in (p, q):
        return node
    left = lca(node.left, p, q)
    right = lca(node.right, p, q)
    if left and right:
        return node          # split point — current is LCA
    return left or right     # propagate the one that found something
```

Q26 · Validate a binary search tree — the bounds-passing recursion.

Tags: Difficulty: Medium · Asked at: Razorpay, Microsoft India, Goldman, JP Morgan, Flipkart · Topic: BST

The question (interviewer's verbatim phrasing):

Given a binary tree, return True if it's a valid binary search tree.

The 30-second answer: Recurse with bounds. `validate(node, low, high)` : if node is None, return True. If `node.val <= low` or `node.val >= high`, return False. Recurse with `validate(node.left, low, node.val)` and `validate(node.right, node.val, high)`. Initial call: `validate(root, -inf, +inf)`. O(n) time, O(h) space. The trick is that a per-node check ("left child less than node, right child greater than node") is insufficient — you need ancestor-range bounds, which the recursion threads through naturally.

Step-by-step thought process: 1. Reject the naive per-node check. `if node.left and node.left.val < node.val and node.right and node.right.val > node.val` is the answer that fails on a tree like `[5, 1, 6, None, None, 3, 7]` where 3 is in the right subtree of 5 but should be on the left — locally the check passes, globally the tree is invalid. 2. State the correct invariant: every value in the left subtree must be strictly less than every ancestor that has it on its right, and vice versa. That's a range constraint, not a pairwise constraint. 3. Walk the recursion. Pass `low` and `high` down. As you go left, the upper bound tightens to the current node's value. As you go right, the lower bound tightens. The check is `low < node.val < high`. 4. Mention the inorder-traversal alternative. An inorder traversal of a valid BST produces a strictly increasing sequence. Walk the tree inorder, track the previous value, return False at the first non-increase. Same complexity, sometimes cleaner.

Edge cases to mention: - Duplicate values — the standard BST definition is strict inequality; allow `<=` only if the problem explicitly says so. - Empty tree — return True. Vacuously valid. - Integer-bounds gotcha — using `float('-inf')` and `float('inf')` covers any integer input. If the problem uses bigints, this still works because Python ints can compare with floats.

What NOT to say: - "Check left child < node, right child > node at each node" — wrong; misses non-adjacent violations. - "Sort the tree's values" — meaningless; you can't sort a tree, you can only traverse it.

Common follow-up: "What's the time-complexity difference between the bounds-passing recursion and the inorder-traversal approach?" — Both are $O(n)$ time and $O(h)$ space. The bounds-passing approach can short-circuit on the first violation; the inorder approach also short-circuits as soon as it finds a non-increase. In practice, comparable.

Stand-out line: "The bounds-passing recursion is the trick — it threads global range information through local recursive calls, which is what a naive per-node check can't do."

```
def is_valid_bst(
    node: TreeNode | None,
    low: float = float("-inf"),
    high: float = float("inf"),
) -> bool:
    if node is None:
        return True
    if not (low < node.val < high):
        return False
    return is_valid_bst(node.left, low, node.val) and is_valid_bst(node.right, node.val, high)
```

Q27 · Serialize and deserialize a binary tree — the level-order JSON approach.

Tags: Difficulty: Hard · Asked at: Microsoft India, Goldman, JP Morgan, Razorpay, Postman · Topic: Serialization

The question (interviewer's verbatim phrasing):

Design `serialize(root) -> str` and `deserialize(s) -> root` for a binary tree.
Both must be $O(n)$.

The 30-second answer: Level-order with None placeholders. Serialize: BFS the tree, emit each node's value (or a `null` marker), separated by commas. Deserialize: split the string, build the root from the first token, then BFS-rebuild by popping a node from the queue and consuming the next two tokens as its left and right children. Both are $O(n)$. The level-order layout is also the LeetCode-standard tree format, so the panel recognizes it immediately.

Step-by-step thought process: 1. Lead with the format. Level-order BFS with `null` for missing nodes gives a compact, unambiguous string. Example: `[1,2,3,null,4,5]` represents the tree where 1's left is 2, 1's right is 3, 2's right is 4, 3's left is 5. 2. Walk serialize. Use a deque, BFS the tree. For each node popped, append its value or "null". For each non-null node, push its children (even if

they're None — we want to record the missing slots). 3. Walk deserialize. Split the string on `,`. The first token is the root. Create a queue containing the root. For each token consumed, peek the front of the queue, consume the next two tokens as left and right. If left is not "null", create the node and push it. Same for right. 4. Mention the pre-order recursive alternative. `serialize` returns `f"{val},{serialize(left)},{serialize(right)}"` with `"#"` for None. `deserialize` consumes tokens recursively. Simpler code but the tree gets visited via DFS, and the format is less human-readable than the level-order one.

Edge cases to mention: - Empty tree — `serialize` returns `"` or `"null"`; `deserialize` on that returns None. - Trailing nulls — the level-order format may include trailing nulls if the last level isn't full; strip them in `serialize` to save bytes. - Negative numbers and overflows — use string repr (`str(val)`); `int(token)` on `deserialize` handles arbitrary-precision Python ints.

What NOT to say: - "Use Python's `pickle` " — works but isn't a valid answer in the interview; the question is about the algorithm, not the library. - "Use `repr(tree)` " — there's no built-in tree repr; you'd have to define one anyway.

Common follow-up: *"What about a more compact format that drops trailing nulls?"* — Track the deepest non-null position during BFS and truncate the output there. Saves bytes for skewed trees but adds a pass. For interview purposes, the simpler version is fine.

Stand-out line: *"Level-order with null placeholders is the LeetCode-standard tree format — knowing it lets you parse and emit trees in the panel's native dialect."*

```
from collections import deque

def serialize(root: TreeNode | None) -> str:
    if root is None:
        return ""
    out: list[str] = []
    q: deque[TreeNode | None] = deque([root])
    while q:
        node = q.popleft()
        if node is None:
            out.append("null")
        else:
            out.append(str(node.val))
            q.append(node.left)
            q.append(node.right)
    return ",".join(out)

def deserialize(s: str) -> TreeNode | None:
    if not s:
        return None
    tokens = s.split(",")
    it = iter(tokens)
    root = TreeNode(int(next(it)))
    q: deque[TreeNode] = deque([root])
    for left_tok in it:
        node = q.popleft()
        if left_tok != "null":
            node.left = TreeNode(int(left_tok))
            q.append(node.left)
        right_tok = next(it)
        if right_tok != "null":
            node.right = TreeNode(int(right_tok))
            q.append(node.right)
    return root
```

Q28 · Path sum — all root-to-leaf paths matching a target.

Tags: Difficulty: Medium · Asked at: Razorpay, Flipkart, Swiggy, Microsoft India, JP Morgan · Topic: Tree paths

The question (interviewer's verbatim phrasing):

Return all root-to-leaf paths in a binary tree whose values sum to a target.

The 30-second answer: DFS with backtracking. Recurse down, push the current node's value onto a `path` list, subtract from the remaining target. At a leaf where remaining == 0, snapshot a copy of the

path into the results. Pop the value off the path on the way back up. $O(n)$ time, $O(h)$ recursion. The trick is the *copy* into results (`results.append(path[:])`) — appending `path` directly would store aliases that mutate as backtracking proceeds.

Step-by-step thought process: 1. State the algorithm. DFS the tree, maintaining the current path as a list and the remaining target as an int. Base case: at a leaf (no left, no right), check if the remaining equals zero. 2. Walk the backtracking. Push value, recurse left, recurse right, pop value. This is the canonical undo-on-return pattern — without it, the path leaks across sibling subtrees. 3. The copy gotcha. Appending `path` to `results` stores a reference; after backtracking pops, the path in results is empty. Always copy with `path[:]` or `list(path)` or `tuple(path)`. 4. Tag the variant: if you only need the count, not the paths, drop the path list and just return the count of matching leaves. Memory drops to $O(h)$.

Edge cases to mention: - Empty tree — return empty list. Don't enter the recursion. - Negative numbers and zero — Kadane-style negatives complicate "stop early." For root-to-leaf, you must walk to a leaf regardless; can't short-circuit. - Single-node tree where the node value equals the target — return `[[target]]`. Test this; many implementations miss the leaf-equals-target case at depth 1.

What NOT to say: - "Use BFS to find paths" — BFS is for shortest path; for enumerating all paths, DFS with backtracking is the cleaner fit. - "Use recursion without backtracking" — paths cross subtrees and you'd accumulate values that don't belong. The pop on return is required.

Common follow-up: "What if I want all paths (not just root-to-leaf) whose sum equals the target?" — Use a running prefix sum with a hash map of prefix-sum frequencies (like the array subarray-sum problem). At each node, look up `prefix - target` in the map. Update on entry, decrement on backtrack. $O(n)$ time.

Stand-out line: "The two traps are the snapshot copy on save and the backtracking pop on return — get either wrong and the function silently returns the wrong answer with no exception."

```
def path_sum(root: TreeNode | None, target: int) -> list[list[int]]:
    results: list[list[int]] = []
    path: list[int] = []

    def dfs(node: TreeNode | None, remaining: int) -> None:
        if node is None:
            return
        path.append(node.val)
        if node.left is None and node.right is None and remaining == node.val:
            results.append(path[:])      # copy, not reference
        else:
            dfs(node.left, remaining - node.val)
            dfs(node.right, remaining - node.val)
        path.pop()                      # backtrack

    dfs(root, target)
    return results
```

Q29 · Trie via nested dict — when this is more Pythonic than a `TrieNode` class.

Tags: Difficulty: Medium · Asked at: Microsoft India, Goldman, JP Morgan, Razorpay, Postman · Topic: Trie

The question (interviewer's verbatim phrasing):

Implement a trie that supports `insert(word)`, `search(word)`, and `starts_with(prefix)`. Show me the Python.

The 30-second answer: Nested dict. Root is `{}`. Each character maps to a sub-dict. A sentinel key (`"#"` or `True`) marks word terminals. `insert`, `search`, `starts_with` are all loops over the input string that walk into sub-dicts. $O(\text{len}(\text{word}))$ per operation, $O(\text{total_chars})$ memory. The nested-dict version is shorter than a `TrieNode` class, has zero per-node object overhead, and reads exactly like the recursive structure.

Step-by-step thought process: 1. State the layout. The trie is a dict of dicts of dicts. Words don't live anywhere explicitly — they're paths from root to a terminal-marked node. 2. Walk insert. Start at root. For each character, `node = node.setdefault(c, {})`. After the loop, mark the terminal with `node["#"] = True`. Three lines. 3. Walk search. Start at root. For each character, return False if not in node; else descend. After the loop, return `"#" in node`. Distinguishes "the word `"cat"` is here" from "`"cat"` is just a prefix of `"catalog"`." 4. Walk starts_with. Same as search but skip the terminal check at the end. If you reach the end of the prefix without falling off, return True.

Edge cases to mention: - Empty string insertion — `insert("")` marks the root as terminal. Searching `""` returns True. Reasonable behaviour. - The sentinel must be a value that can't be a character. `"#"` works for ASCII; for arbitrary-Unicode words, use a non-string sentinel like `True` keyed by a non-Unicode-character sentinel object like `object()` cached as a module-level constant. - Memory — each character is a dict entry (~120 bytes minimum) plus the sub-dict overhead. For million-word dictionaries, switch to a more compact representation like a DAWG or a packed array-trie.

What NOT to say: - "Use a class for each node" — works but more code; the nested-dict version is shorter and faster for interview-sized inputs. - "Use a hashset of all words" — that's not a trie; loses the prefix-query capability.

Common follow-up: "How would you implement autocomplete returning all words with a given prefix?" — Walk the prefix to find the sub-trie root. Then DFS the sub-trie, accumulating characters; each time you hit a terminal marker, emit the accumulated string. Returns all matches in $O(\text{matches} \times \text{max_word_length})$.

Stand-out line: "Nested dict is the trie that fits in a function — class-based tries are for when you've outgrown the interview and need typed nodes with extra metadata."

```

class Trie:
    def __init__(self) -> None:
        self.root: dict = {}

    def insert(self, word: str) -> None:
        node = self.root
        for c in word:
            node = node.setdefault(c, {})
        node["#"] = True

    def search(self, word: str) -> bool:
        node = self.root
        for c in word:
            if c not in node:
                return False
            node = node[c]
        return "#" in node

    def starts_with(self, prefix: str) -> bool:
        node = self.root
        for c in prefix:
            if c not in node:
                return False
            node = node[c]
        return True

```

Q30 · Invert a binary tree — the recursion that famously didn't get someone hired.

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Postman, JP Morgan · Topic: Tree manipulation

The question (interviewer's verbatim phrasing):

Write a function that takes the root of a binary tree and inverts it — every node's left and right children are swapped.

The 30-second answer: Recursion in three lines. `def invert(node): if node: node.left, node.right = invert(node.right), invert(node.left); return node`. The tuple-swap on the LHS is the Python-idiomatic way to do it in one expression. $O(n)$ time visiting every node, $O(h)$ space for recursion. The iterative variant uses a BFS or DFS frontier and swaps children at each pop.

Step-by-step thought process: 1. State the recurrence: `invert(node)` returns the inverted subtree rooted at `node`. Swap children, recurse on both. Base case at `None` returns `None`. 2. Walk the tuple-swap.

`node.left, node.right = invert(node.right), invert(node.left)` evaluates both right-hand sides first, then assigns. This is the safe pattern; doing them as two separate statements (`tmp = node.left; node.left = invert(node.right); node.right = invert(tmp)`) is

also correct but more code. 3. The iterative version. BFS the tree with a deque; for each node popped, swap its children, push the original children (now in swapped positions). $O(n)$ time, $O(w)$ space where w is the max width. 4. Mention the famous tweet — Homebrew's author was rejected from Google for not being able to invert a binary tree on a whiteboard. The question became shorthand for "easy interview puzzle you should be able to write under pressure."

Edge cases to mention: - Empty tree — return None; the base case handles it. - Single node — return it unchanged (no children to swap). - Already-symmetric tree (e.g., perfectly balanced with mirror values) — the inversion is still a different tree object; equality and identity diverge.

What NOT to say: - "I can't invert a binary tree" — the joke that ended an engineering career. Just write the three lines. - "Use a custom Node class with an invert method" — overkill; a function on the existing `TreeNode` is what the panel wants.

Common follow-up: "What's the iterative version?" — Same algorithm with an explicit stack or queue. Push root; pop, swap children, push original children. Done when the container is empty.

Stand-out line: "Three lines, one tuple-swap, base case at None — and you'll never get caught off-guard on the interview question that became an industry meme."

```
from collections import deque

def invert_recursive(node: TreeNode | None) -> TreeNode | None:
    if node is None:
        return None
    node.left, node.right = invert_recursive(node.right), invert_recursive(node.left)
    return node

def invert_iterative(root: TreeNode | None) -> TreeNode | None:
    if root is None:
        return None
    q: deque[TreeNode] = deque([root])
    while q:
        node = q.popleft()
        node.left, node.right = node.right, node.left
        if node.left:
            q.append(node.left)
        if node.right:
            q.append(node.right)
    return root
```

Section E — Graphs

Five questions on graph representations, BFS, DFS, cycle detection, and topological sort. The idiomatic-Python angle here is adjacency lists as `dict[node, list[node]]` (or `defaultdict(list)`) — almost never `Edge` and `Vertex` classes, which is what panels see from candidates porting Java

code into Python. Indian product-co interviews lean on graphs in round two because they test the combination of recursion-or-iteration with a non-trivial visited-set discipline.

Q31 · How do you represent a graph in idiomatic Python?

Tags: Difficulty: Easy · Asked at: Razorpay, Flipkart, Swiggy, Microsoft India, JP Morgan · Topic: Graph representation

The question (interviewer's verbatim phrasing):

Show me how you'd model a graph in Python for a BFS or DFS problem.

The 30-second answer: Adjacency list as `dict[node, list[node]]` for directed graphs, or as `defaultdict(list)` for cleaner build code. Each key is a node; the value is the list of its neighbours. For weighted graphs, the values are `list[tuple[neighbour, weight]]`. Adjacency matrices (a 2D list) are simpler for dense graphs but waste memory for sparse ones — for typical interview graphs, adjacency list wins. For huge production graphs, `networkx` is the library; for the interview, write the dict yourself.

Step-by-step thought process: 1. State the choice. Adjacency list is the default. Sparse graphs (most interview graphs) use $\sim O(V + E)$ memory in adjacency list vs $O(V^2)$ in matrix. 2. Walk the construction. `graph = defaultdict(list); for u, v in edges: graph[u].append(v)`. For undirected graphs, add both directions: `graph[u].append(v); graph[v].append(u)`. 3. Weighted variant. `defaultdict(list); for u, v, w in edges: graph[u].append((v, w))`. The tuple keeps the neighbour and weight together; BFS/DFS unpack as needed. 4. Mention `networkx`. For production code or analytical work — community detection, centrality, max-flow — `networkx` is the right answer. For interview DSA, the panel wants the dict, not the library.

Edge cases to mention: - Nodes with no outgoing edges — they don't appear as keys in `defaultdict` until you access them; this is fine because graph traversal usually pre-populates by iterating edges first. - Self-loops — `graph[u].append(u)` is allowed; the traversal logic must guard against infinite loops with a visited set. - Parallel edges — two edges between the same pair of nodes. The list-of-neighbours layout allows them; the set-of-neighbours layout (`defaultdict(set)`) silently dedupes them. Pick consciously.

What NOT to say: - "Use a class for each node and a class for each edge" — that's Java-translated-into-Python; the dict is shorter and faster for the interview. - "Use a 2D list always" — only correct for dense graphs (E close to V^2). For sparse, it's $O(V^2)$ memory wasted.

Common follow-up: "What's the trade-off between adjacency list and adjacency matrix?" — List uses $O(V + E)$ memory and gives $O(1)$ "iterate neighbours" but $O(\text{degree})$ "is there an edge $u \rightarrow v$ " lookup.

Matrix uses $O(V^2)$ memory and gives $O(1)$ "edge?" lookup but $O(V)$ "iterate neighbours." Most graph algorithms iterate neighbours, so list wins for typical workloads.

Stand-out line: " `defaultdict(list)` is the graph in three lines — for interview code, that's the entire data structure."

```
from collections import defaultdict

# directed graph
edges = [("a", "b"), ("a", "c"), ("b", "d")]
graph: defaultdict[str, list[str]] = defaultdict(list)
for u, v in edges:
    graph[u].append(v)

# undirected graph
def build_undirected(edges: list[tuple[str, str]]) -> defaultdict[str, list[str]]:
    g: defaultdict[str, list[str]] = defaultdict(list)
    for u, v in edges:
        g[u].append(v)
        g[v].append(u)
    return g

# weighted graph
weighted: defaultdict[str, list[tuple[str, int]]] = defaultdict(list)
for u, v, w in [("a", "b", 5), ("b", "c", 3)]:
    weighted[u].append((v, w))
```

Q32 · BFS on a graph — finding shortest path by edge count.

Tags: Difficulty: Medium · Asked at: Razorpay, Flipkart, Microsoft India, Goldman, JP Morgan · Topic: BFS

The question (interviewer's verbatim phrasing):

Given an unweighted graph and two nodes, find the shortest path between them by number of edges.

The 30-second answer: BFS with a `deque` as frontier, a `set` for visited, and a `dict` for the parent map (to reconstruct the path). Start from source, BFS outward, mark each newly seen node's parent. When the target is dequeued, walk the parent map backward to reconstruct. $O(V + E)$ time and space. For unweighted graphs, BFS gives the shortest path because it explores by level; for weighted graphs, you'd need Dijkstra.

Step-by-step thought process: 1. Frontier is a deque; visited is a set; parent is a dict. Three containers, each with a clear job. 2. Walk the BFS loop. Popleft from the frontier. For each neighbour, if not visited:

mark visited, set parent, push to frontier. When you pop left the target, you're done. 3. Path reconstruction. From target, follow `parent[target]` backward until you hit source. Reverse to get source-to-target order. 4. Tag the bidirectional optimization. BFS from both ends simultaneously, stopping when frontiers meet, halves the visited set in dense graphs. Useful but more code; mention only if pressed.

Edge cases to mention: - Source equals target — return `[source]` immediately. - No path exists — BFS exhausts the frontier without finding target; return None or empty list. - Disconnected graph — BFS only explores the connected component containing source; unreachable nodes are correctly absent from parent.

What NOT to say: - "Use DFS for shortest path" — DFS finds a path, not the shortest. BFS is the right tool for unweighted shortest path. - "Use a list and `pop(0)`" — $O(n^2)$ traversal; use deque.

Common follow-up: "How would you find the shortest path in a weighted graph?" — Dijkstra's with `heapq`. Pop the lowest-cost node, relax edges (update distances if shorter via this node), push relaxed neighbours. $O((V + E) \log V)$ with a binary heap.

Stand-out line: "BFS for unweighted, Dijkstra for weighted, A for weighted-with-heuristic — knowing which tool fits the cost model is half the algorithmic vocabulary."*

```
from collections import deque

def shortest_path(graph: dict[str, list[str]], source: str, target: str) -> list[str] | None:
    if source == target:
        return [source]
    visited = {source}
    parent: dict[str, str] = {}
    q: deque[str] = deque([source])
    while q:
        node = q.popleft()
        for neighbour in graph.get(node, []):
            if neighbour in visited:
                continue
            visited.add(neighbour)
            parent[neighbour] = node
            if neighbour == target:
                path = [target]
                while path[-1] != source:
                    path.append(parent[path[-1]])
                return list(reversed(path))
            q.append(neighbour)
    return None
```

Q33 · Cycle detection in a directed graph — the three-colour DFS.

Tags: Difficulty: Hard · Asked at: Microsoft India, Goldman, JP Morgan, Razorpay, Flipkart · Topic: Cycle detection

The question (interviewer's verbatim phrasing):

Given a directed graph, return True if it contains a cycle.

The 30-second answer: Three-colour DFS. Each node is WHITE (unvisited), GRAY (on the current DFS stack), or BLACK (fully explored). On a DFS edge into a GRAY node, you've found a back-edge — that's a cycle. Iterate over all nodes; for each WHITE one, DFS from it. If any DFS finds a back-edge, return True. $O(V + E)$ time and space. For undirected graphs, the rule is different — any DFS edge to a visited node that's not the parent is a cycle.

Step-by-step thought process: 1. State the three colours. WHITE means "haven't seen yet." GRAY means "currently on the recursion path from the root." BLACK means "finished — recursed and returned." The GRAY set is what distinguishes a cycle (back-edge into the current path) from a re-discovery (cross-edge to a previously finished node, which is fine in a DAG). 2. Walk the DFS. On entering a node, mark it GRAY. For each neighbour: if WHITE, recurse; if GRAY, you've found a cycle; if BLACK, skip. On exiting, mark BLACK. 3. Iterate over all nodes (not just one source) — disconnected components might contain cycles even if your start node doesn't. 4. Mention Kahn's algorithm. The topological-sort-based cycle detection. Compute in-degrees, repeatedly remove nodes with in-degree 0, decrement neighbours. If the queue empties before all nodes are removed, a cycle remains. Same complexity, different style.

Edge cases to mention: - Self-loop ($u \rightarrow u$) is a cycle of length 1 — the three-colour DFS catches it on the first neighbour iteration. - Multiple components — iterate over all nodes; don't trust starting from any single source. - Undirected graph confusion — for undirected, you must skip the parent when checking neighbours; otherwise every edge looks like a back-edge. The three-colour DFS is for directed.

What NOT to say: - "BFS for cycle detection" — works for undirected but is awkward for directed; DFS is the natural fit because the recursion stack tracks "on the current path." - "Use a single visited set" — without the GRAY/BLACK distinction, you can't tell back-edges from cross-edges. Will produce false positives.

Common follow-up: "How would you find all the cycles, not just detect one?" — Tarjan's strongly-connected-components algorithm — every SCC with more than one node, or any SCC with a self-loop, is a cycle. Linear-time and a classic next-step interview question.

Stand-out line: "Three colours is the discriminator — GRAY tracks the current path, and a GRAY-on-GRAY edge is the back-edge that defines a cycle in a directed graph."


```

WHITE, GRAY, BLACK = 0, 1, 2

def has_cycle(graph: dict[str, list[str]]) -> bool:
    colour: dict[str, int] = {node: WHITE for node in graph}

    def dfs(node: str) -> bool:
        colour[node] = GRAY
        for neighbour in graph.get(node, []):
            if colour.get(neighbour, WHITE) == GRAY:
                return True                # back-edge: cycle
            if colour.get(neighbour, WHITE) == WHITE and dfs(neighbour):
                return True
        colour[node] = BLACK
        return False

    return any(colour[n] == WHITE and dfs(n) for n in graph)

```

Q34 · Topological sort — Kahn's algorithm and the DFS post-order variant.

Tags: Difficulty: Medium · Asked at: Microsoft India, Goldman, JP Morgan, Razorpay, Flipkart · Topic: Topological sort

The question (interviewer's verbatim phrasing):

Given a DAG of task dependencies (each edge means "u must be done before v"), return a valid execution order.

The 30-second answer: Kahn's algorithm. Compute in-degrees of every node. Push all nodes with in-degree zero into a queue. Pop a node, emit it, decrement the in-degrees of its neighbours; if any neighbour's in-degree drops to zero, push it. Continue until the queue is empty. If the result doesn't include every node, there was a cycle. $O(V + E)$ time. The DFS variant pushes nodes onto a stack in post-order and emits them reversed — same complexity, sometimes cleaner code.

Step-by-step thought process: 1. Lead with Kahn's. The intuition is iterative: at any moment, the nodes with no remaining incoming edges are valid next tasks. Process them in any order, decrement neighbours, repeat. 2. Walk the implementation. `in_degree` from a single pass over edges. `q = deque(n for n in graph if in_degree[n] == 0)`. Loop: pop, emit, decrement neighbours' `in_degree`, push when `in_degree` hits zero. 3. Cycle detection comes for free. If the output's length is less than the node count, there's a cycle (the nodes in the cycle never reach in-degree zero). 4. Mention the DFS variant. DFS each unvisited node; on return from each subtree, push the node onto a stack. After all DFS calls, reverse the stack — that's the topological order. Same complexity, no in-degree bookkeeping, but recursive depth is a concern for deep DAGs.

Edge cases to mention: - Empty graph — return empty list. Kahn's loop runs zero times. - Multiple valid orderings — Kahn's emits in whatever order the queue gives; if you need a deterministic order, sort the queue (or use a `heapq` to get the lexicographically smallest valid order). - Multiple roots (multiple in-degree-zero nodes) — Kahn's handles them naturally by starting with all of them in the queue.

What NOT to say: - "Sort the nodes alphabetically" — only valid by accident. - "BFS from any node" — BFS gives a traversal order, not a topological order, unless you also track in-degrees.

Common follow-up: *"How would you return the lexicographically smallest topological order?"* — Replace the `deque` with a `heapq` (min-heap of node names). Each "pop the next zero-in-degree node" becomes a heap-pop. $O((V + E) \log V)$.

Stand-out line: *"Kahn's gives you topological order and cycle detection from one structure — the in-degree counter is the algorithm; everything else is plumbing."*

```
from collections import deque, defaultdict

def topological_sort(graph: dict[str, list[str]]) -> list[str] | None:
    in_degree: defaultdict[str, int] = defaultdict(int)
    nodes = set(graph)
    for u in graph:
        for v in graph[u]:
            in_degree[v] += 1
            nodes.add(v)
    q: deque[str] = deque(n for n in nodes if in_degree[n] == 0)
    order: list[str] = []
    while q:
        node = q.popleft()
        order.append(node)
        for neighbour in graph.get(node, []):
            in_degree[neighbour] -= 1
            if in_degree[neighbour] == 0:
                q.append(neighbour)
    if len(order) != len(nodes):
        return None  # cycle detected
    return order
```

Q35 · When should you reach for `networkx` instead of writing the graph by hand?

Tags: Difficulty: Easy · Asked at: Razorpay, Postman, Microsoft India, Goldman, Flipkart · Topic: Library choice

The question (interviewer's verbatim phrasing):

When is `networkx` the right call in a backend service, and when would you write the graph yourself?

The 30-second answer: `networkx` is the right call for analytical work — community detection, centrality, max-flow, shortest paths across many sources, importing GraphML or GEXF. It's the wrong call inside a hot loop or for tight memory budgets — it's pure Python, ~10× slower than a hand-rolled dict-of-lists, and stores nodes as full Python objects. For interview DSA: write the graph yourself. For analyst dashboards, ETL jobs, recommendation prototypes: `networkx`. For production graph databases: use Neo4j or a native graph engine, not `networkx`.

Step-by-step thought process: 1. Anchor on use case. `networkx` is a library for graph *analysis*. You import a graph from a file or build it from a list, then you run algorithms — `shortest_path`, `pagerank`, `betweenness_centrality` — and pull out numbers. 2. Walk the perf trade-off. A hand-rolled `dict[node, list[node]]` is about 10× faster and 5× smaller than the equivalent `networkx Graph` for typical operations. `networkx` wraps every node in a `NodeView` and every edge in an `EdgeView`; the overhead adds up. 3. Decision rule. (a) Will the graph fit in memory and live longer than one function call? Probably `networkx`. (b) Will you traverse it inside a request handler under SLA? Hand-rolled. (c) Will you do non-trivial analysis (centrality, communities)? Definitely `networkx`. 4. Mention the production graph DBs. For graphs that outgrow RAM, persist in a database with a query language — Neo4j (Cypher), Amazon Neptune, ArangoDB. Don't try to scale `networkx` to billions of nodes; it wasn't built for that.

Edge cases to mention: - For directed graphs use `networkx.DiGraph`; for undirected `networkx.Graph`. Mixing them up silently gives wrong analysis results. - `networkx.shortest_path` has fast algorithms baked in (Dijkstra, A*) — don't write your own when you have `networkx`. - Pickling a `networkx` graph captures everything but is slow and unportable across versions. For long-lived storage, dump to JSON or GraphML.

What NOT to say: - "Always use `networkx`" — wrong in hot loops; the Python overhead is real. - "Never use libraries in production" — false dichotomy; `networkx` is the right call for analytical pipelines.

Common follow-up: "You're computing PageRank on a 10-million-node graph. What's the right stack?" — Not pure `networkx` — too slow at that scale. Either `networkx` + `scipy.sparse` (which `networkx` supports via `nx.pagerank_scipy`) for the in-memory case, or move to a distributed graph engine like Spark GraphX or a dedicated graph DB.

Stand-out line: "*networkx for analytics, dict-of-lists for hot loops, graph DB for production — three tiers, one decision tree.*"

```
import networkx as nx

# analytical work – networkx is the right call
g = nx.DiGraph()
g.add_edges_from([("a", "b"), ("b", "c"), ("a", "c")])
nx.pagerank(g) # one call, full algorithm

# hot-loop work – write the dict yourself
from collections import defaultdict
graph: defaultdict[str, list[str]] = defaultdict(list)
for u, v in edges:
    graph[u].append(v)
# now BFS/DFS at full Python speed
```

Section F — Dynamic Programming and Recursion

Five questions on DP, memoization, tabulation, and the recursion-limit concerns that come with deep DP problems in Python. Indian product-co panels lean on DP in round two or three because it's the test of "can you spot the overlapping subproblems and write the recurrence" — and the idiomatic Python angle is `@functools.cache` instead of hand-rolled memo dicts.

Q36 · `functools.cache` vs `functools.lru_cache` — which to use for memoization.

Tags: Difficulty: Easy · Asked at: Razorpay, Microsoft India, Goldman, JP Morgan, Postman · Topic: Memoization

The question (interviewer's verbatim phrasing):

You're memoizing a recursive function. Should you use `functools.cache` or `functools.lru_cache` ?

The 30-second answer: `functools.cache` for memoization where you want unbounded caching — every distinct argument tuple stays cached forever. `functools.lru_cache(maxsize=N)` for bounded caching where you want least-recently-used eviction past N entries. For interview DP problems with bounded state space (subproblems are clearly indexed by a few ints), `@cache` is cleaner and faster — no LRU bookkeeping overhead. `@cache` was added in 3.9 and is equivalent to `lru_cache(maxsize=None)` .

Step-by-step thought process: 1. State the version. `functools.cache` shipped in 3.9. Before that, you used `functools.lru_cache(maxsize=None)` for the same effect. 2. The lru bookkeeping is real overhead. Each call updates the LRU recency list — a doubly-linked list under the hood. For a tight DP loop over millions of states, `@cache` is roughly 25% faster than `@lru_cache(maxsize=None)`. 3. Tag the unhashable-args gotcha. Both decorators require all args to be hashable. If your subproblem state includes a list, convert to a tuple at the call site, or use `frozenset` for sets. Lists fail at runtime. 4. Mention the cache-clear capability. Both decorators add `.cache_clear()` and `.cache_info()` to the function — useful for between-test cleanup and for diagnosing whether memoization is actually hitting.

Edge cases to mention: - Recursion-limit interaction. Memoization doesn't reduce recursion depth — it just reduces the number of recursive calls. A deep DP can still hit `RecursionError` even with caching; tabulation may be the only fix. - Methods. `@cache` on an instance method caches with `self` as part of the key — different instances get different cache entries; the cache lives on the function, not the instance. Use `@cached_property` for instance-bound caching. - Keyword args. `@cache(maxsize=...)` is wrong — it's not parameterized; just `@cache`. `@lru_cache(maxsize=128)` is the parameterized one.

What NOT to say: - "Use a global dict for memoization" — works but more boilerplate and easier to leak between tests. The decorators are the idiom. - "Use `@cache` with `maxsize=...`" — `@cache` doesn't take parameters.

Common follow-up: "Your DP function uses a list as one of its arguments. What happens when you `@cache` it?" — `TypeError` at the first call ("unhashable type: 'list'"). Convert the list to a tuple in the call site: `solve(tuple(state))`. Or design the recurrence to use indices and lengths instead of the slice itself.

Stand-out line: " `@cache` is the one-line memoizer that turns exponential recursion into polynomial DP — and on Python 3.9+ it's strictly better than `@lru_cache(maxsize=None)` ."

```
from functools import cache

@cache
def fib(n: int) -> int:
    if n < 2:
        return n
    return fib(n - 1) + fib(n - 2)

# bounded LRU when memory matters
from functools import lru_cache

@lru_cache(maxsize=10_000)
def expensive(x: int, y: int) -> int:
    ...
```

Q37 · Memoization vs tabulation — when to switch.

Tags: Difficulty: Medium · Asked at: Microsoft India, Goldman, JP Morgan, Razorpay, Flipkart · Topic: DP styles

The question (interviewer's verbatim phrasing):

What's the difference between memoization and tabulation, and when would you switch from one to the other in Python?

The 30-second answer: Memoization is top-down: write the recursive recurrence, decorate with `@cache`, let it fill the cache lazily as it explores. Tabulation is bottom-up: explicitly fill a table (usually a list or dict) from the smallest subproblems to the largest, iteratively. Memoization wins on clarity and on sparse subproblem spaces (only the needed subproblems are computed). Tabulation wins on Python because it sidesteps the recursion limit and is roughly 2× faster (no function-call overhead). Switch to tabulation when (a) the state space is deep enough to hit the recursion limit, or (b) you need rolling-array space optimization that's awkward to express recursively.

Step-by-step thought process: 1. Walk both with Fibonacci. Memoization: `@cache def fib(n): return n if n < 2 else fib(n-1) + fib(n-2)`. Tabulation: `def fib(n): dp = [0, 1]; for i in range(2, n+1): dp.append(dp[-1] + dp[-2]); return dp[n]`. 2. State the recursion-limit trade-off. Python's default is 1000 frames. A linear DP of depth 10k blows the limit recursively but runs fine tabularly. 3. State the perf trade-off. Function-call overhead in Python is high — roughly 100ns per call. Tabulation avoids that; for a million-state DP, it's tens of milliseconds saved. 4. State the space optimization trade-off. Many DPs only need the last few rows (Fibonacci needs only the last two). Tabulation expresses this naturally — keep `prev` and `curr` scalars and roll them. Memoization keeps everything in the cache, harder to compress.

Edge cases to mention: - Sparse state spaces favour memoization. If only 1% of the theoretical state space is actually reached, memoization computes only those; tabulation might fill the whole table. - The decision is sometimes "neither" — for a problem with no overlapping subproblems, neither helps; you want pure recursion (or iteration). - For interview write-up, memoization is usually accepted as the "DP" answer even if tabulation would be faster. The recursive form is closer to the recurrence and easier to read.

What NOT to say: - "Tabulation is always faster" — only when it can exploit ordering; for sparse subproblems memoization can win. - "Memoization is bottom-up" — backwards; memoization is top-down by definition.

Common follow-up: "You're solving knapsack with capacity 100k and 100 items. Which approach?" — Tabulation. The state space is `100 * 100_000 = 10M` cells; memoization at that depth blows the recursion limit. Tabulation iterates the table in $O(N * W) = 10M$ ops; very tractable.

Stand-out line: "Memoization is what you write first because it matches the recurrence; tabulation is what you switch to when Python's recursion limit or perf budget says you must."

```

from functools import cache

# memoization — top-down
@cache
def coin_change_top(coins: tuple[int, ...], amount: int) -> int:
    if amount == 0:
        return 0
    if amount < 0:
        return -1
    best = -1
    for c in coins:
        sub = coin_change_top(coins, amount - c)
        if sub != -1 and (best == -1 or sub + 1 < best):
            best = sub + 1
    return best

# tabulation — bottom-up
def coin_change_bottom(coins: list[int], amount: int) -> int:
    dp = [0] + [float("inf")] * amount
    for a in range(1, amount + 1):
        for c in coins:
            if c <= a:
                dp[a] = min(dp[a], dp[a - c] + 1)
    return dp[amount] if dp[amount] != float("inf") else -1

```

Q38 · Coin change — minimum number of coins to make a target sum.

Tags: Difficulty: Medium · Asked at: TCS, Razorpay, Microsoft India, Goldman, JP Morgan · Topic: Classic DP

The question (interviewer's verbatim phrasing):

Given a list of coin denominations and a target amount, return the minimum number of coins that sum to the target. Return -1 if impossible.

The 30-second answer: Bottom-up DP. $dp[a]$ = minimum coins to make amount a . Base: $dp[0] = 0$. Recurrence: $dp[a] = \min(dp[a - c] + 1 \text{ for } c \text{ in coins if } c \leq a)$. Initialize unreachable amounts to infinity; if $dp[\text{target}]$ is still infinity at the end, return -1. $O(\text{amount} \times \text{len}(\text{coins}))$ time, $O(\text{amount})$ space. Tabulation is cleaner than memoization here because the state space is dense and the iteration order is obvious.

Step-by-step thought process: 1. Walk the recurrence. To make amount a , you need at least one coin; whichever coin c you pick, you then need to make $a - c$ with the remaining coins. So $dp[a] = 1 + \min(dp[a - c] \text{ for } c \text{ in coins if } a - c \geq 0)$. 2. Initialize the DP table. $dp[0] = 0$ (zero coins make zero). All other $dp[a] = \text{infinity}$, meaning "unreachable until proven

otherwise." 3. Iterate amount from 1 to target. For each amount, scan coins and update. 4. Result is `dp[target]`. If it's infinity, no combination works; return -1.

Edge cases to mention: - Target = 0 — return 0 (no coins needed). Initialize `dp[0] = 0` covers this. - Coin > target — skip it; `c <= a` guard. - Empty coins list — return -1 (or 0 if target is also 0). Guard at the top. - Coin equal to target — `dp[target] = 1 + dp[0] = 1`. Correctly handled.

What NOT to say: - "Greedy: always pick the largest coin that fits" — fails for coin sets like [1, 3, 4] making 6. Greedy says 4 + 1 + 1 = 3 coins; optimal is 3 + 3 = 2 coins. - "Memoize without bounds" — works but tabulation is cleaner for this dense DP.

Common follow-up: "How would you also return the actual coins used?" — Maintain a parent array: `parent[a] = c` if `dp[a]` was updated using coin `c`. Reconstruct by walking `parent[target]`, `parent[target - parent[target]]`, etc. until you hit 0.

Stand-out line: "Coin change is the canonical 'DP, not greedy' question — the panel is checking whether you'd reach for greedy by reflex or stop to consider counterexamples first."

```
def coin_change(coins: list[int], amount: int) -> int:
    if amount < 0:
        return -1
    INF = amount + 1
    dp = [INF] * (amount + 1)
    dp[0] = 0
    for a in range(1, amount + 1):
        for c in coins:
            if c <= a:
                dp[a] = min(dp[a], dp[a - c] + 1)
    return dp[amount] if dp[amount] != INF else -1
```

Q39 · Longest increasing subsequence — the $O(n \log n)$ patience-sort approach.

Tags: Difficulty: Hard · Asked at: Microsoft India, Goldman, JP Morgan, Razorpay, Flipkart · Topic: LIS

The question (interviewer's verbatim phrasing):

Find the length of the longest strictly increasing subsequence of an array. Bonus: do it better than $O(n^2)$.

The 30-second answer: Patience-sort DP with `bisect`. Maintain a `tails` array where `tails[i]` is the smallest possible tail of an increasing subsequence of length `i+1`. For each new element `x`, find the leftmost position in `tails` where `x` could go using `bisect.bisect_left` and either replace that slot (if `x` would shrink a tail) or append (if `x` extends the longest). Final length is

`len(tails)` . $O(n \log n)$ time, $O(n)$ space. The $O(n^2)$ DP — `dp[i] = 1 + max(dp[j] for j < i if nums[j] < nums[i])` — is the brute-force version that the bonus asks you to beat.

Step-by-step thought process: 1. State both approaches and their complexities. Naive DP is $O(n^2)$; patience-sort DP is $O(n \log n)$. The bonus phrasing means they want the better one. 2. Walk the invariant. `tails[i]` is the smallest tail of any length-(i+1) increasing subsequence seen so far. Keeping it smallest gives future elements the maximum chance of extending it. 3. Walk the per-element update. For new value `x`, use `bisect_left(tails, x)` to find where it goes. If the position equals `len(tails)`, `x` extends the longest — append. Otherwise replace `tails[pos]` (shrinking that length's tail to give future elements more room). 4. Note that `tails` itself is NOT the LIS — it's an auxiliary that gives the correct *length*. Reconstructing the actual subsequence needs additional bookkeeping (predecessor indices).

Edge cases to mention: - Empty array — return 0; the loop runs zero times. - Strictly increasing input — `tails` grows by one every iteration; result is `n`. - Strictly decreasing input — every new element replaces `tails[0]`; result is 1. - Equal values — strict increasing uses `bisect_left` (replace at the first equal); non-strict would use `bisect_right` (append after equals).

What NOT to say: - "Sort the array and return its length" — completely wrong; LIS is a subsequence, order matters. - "Use a recursive $O(2^n)$ explore-all" — fails on inputs of length > 30.

Common follow-up: "What if you wanted the longest non-decreasing subsequence instead of strictly increasing?" — Change `bisect_left` to `bisect_right`. The right-vs-left determines whether equal values extend or replace.

Stand-out line: "Patience-sort LIS is the trick — `bisect` turns a quadratic DP into $n \log n$, and once you've internalized that, half the 'subsequence with property' problems collapse to the same shape."

```
import bisect

def lis_length(nums: list[int]) -> int:
    tails: list[int] = []
    for x in nums:
        pos = bisect.bisect_left(tails, x)
        if pos == len(tails):
            tails.append(x)
        else:
            tails[pos] = x
    return len(tails)
```

Q40 · Python's recursion limit — what is it, when does it bite, and how do you fix it?

Tags: Difficulty: Medium · Asked at: Razorpay, Microsoft India, Goldman, JP Morgan, Postman · Topic: Recursion limit

The question (interviewer's verbatim phrasing):

What's Python's recursion limit? When does it cause real bugs and what do you do?

The 30-second answer: Default limit is 1000 frames, queried by `sys.getrecursionlimit()` and changed by `sys.setrecursionlimit(N)`. It exists because Python has no tail-call optimization and each frame consumes C-stack memory; running off the C stack would segfault, so Python pre-empts it with a `RecursionError`. The limit bites on (a) deep DP problems on adversarial inputs, (b) deeply nested JSON parsing, (c) recursive directory walks on pathological filesystems, (d) recursive serializers like pickle on deeply linked objects. The three fixes: iteratively rewrite with an explicit stack (most reliable), `sys.setrecursionlimit(10_000)` (works for moderate depths, risks segfault for very deep), or `threading.Thread(target=..., stack_size=...)` with a larger thread stack.

Step-by-step thought process: 1. State the default and the API. 1000 in CPython 3.x.

`getrecursionlimit`, `setrecursionlimit`. There's no per-function override. 2. Walk *why* the limit exists. Each Python frame is roughly 500-1000 bytes of C-stack. The OS default thread stack is ~8MB on Linux. $8\text{MB} / 1\text{KB} = \sim 8000$ frames before segfault. The 1000 default is conservative to keep a safety margin. 3. Walk the three fixes. (a) Iterative rewrite — bulletproof but requires algorithm work. (b) Raise the limit — works for moderate increases; setting it to 1_000_000 on a deeply recursive function will segfault long before. (c) Run on a thread with a bigger stack — `threading.Thread(target=fn, stack_size=64*1024*1024)` gives 64MB; rare but works for pathological cases. 4. Mention the production angle. JSON deserialization with deeply nested objects has bit production services. `json.loads` recursively walks the structure; a 10k-deep nested array crashes. The fix in production is to validate input depth before parsing or to use a streaming parser like `ijson`.

Edge cases to mention: - Memoized recursion doesn't help with recursion depth — it only reduces *redundant* calls. A linear chain of subproblems still recurses linearly. - Generators with `yield from` chain frames. Deep `yield from` chains hit the limit too. `itertools.chain` is the iterative alternative. - The limit is per-thread, not per-process. A spawned thread can have its own limit and stack size.

What NOT to say: - "Python has tail-call optimization" — wrong; Guido has explicitly rejected it. Don't claim it. - "Just set the limit to a billion" — segfault. The limit is a safety net, not the actual C-stack budget.

Common follow-up: "You're getting `RecursionError` in production on a recursive DP. What do you do?" — Three-step diagnosis. (1) Is the depth a function of input size? If yes and inputs are adversarial, the only safe fix is iterative rewrite. (2) Is the depth bounded by a constant just over 1000? Raise the limit modestly (5000) and add a depth-guard in the function. (3) Is the structure deeply linked (graph cycles)? Add a visited set; the recursion was never bounded.

Stand-out line: "The recursion limit is the Python-specific concern that separates 'works on LeetCode' from 'survives production' — iterative rewrite is the bulletproof answer, raising the limit is the patch."

```
import sys
import threading

# inspect / set the limit
sys.getrecursionlimit()          # 1000 default
# sys.setrecursionlimit(10_000)   # moderate raise – risks segfault if too high

# iterative rewrite – preferred
def factorial_iter(n: int) -> int:
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result

# run on a thread with a bigger stack – last resort
def run_deeply():
    deep_recursive_fn(10_000)

threading.stack_size(64 * 1024 * 1024) # 64MB stack
t = threading.Thread(target=run_deeply)
t.start()
t.join()
```

Closing notes

That's forty questions across the six surfaces every Indian backend round leans on — arrays and slicing, hashmaps and sets, heaps, trees, graphs, and DP. The thread through all of them is the same: idiomatic Python beats Java-translated-into-Python in the interview room, because the panel is checking whether you've shipped Python services and not just done LeetCode in Python syntax. `collections.deque` for BFS frontiers, `collections.Counter` and `defaultdict` for grouping, `heapq.merge` and `heapq.nlargest` for streaming, `@functools.cache` for memoization, `bisect` for log-time inserts in sorted lists, nested `dict` for tries, and `defaultdict(list)` for graph adjacency lists — those are the seven imports that turn five-line loops into two-line answers, and they're the imports the panel is hoping to see you reach for.

The companion chapter — Chapter 2, OOP Design — moves from algorithmic data structures into how you architect Python code that's testable, composable, and survives review. Read it next.

Chapter 2 — OOP Design (20 Qs)

Python Interview Prep — 2026 Edition · Learn & Excel · File 07 Backend Track Library

This is the OOP design chapter for the Backend track. Where File 04 Chapter 1 drilled the *mechanics* of classes (MRO, descriptors, dunder methods), this chapter drills the *design choices* — how to actually structure backend code so it stays testable, extensible, and reviewable after twelve people have touched it. Indian backend panels at Razorpay, Cred, Swiggy, Postman, Zerodha, Flipkart, PhonePe, Walmart Labs, JP Morgan, Goldman, and the GCC tier almost always ship two design questions in round 2 — one principle question ("composition vs inheritance, give me an example") and one live-design question ("design a rate limiter for our payment gateway"). The candidates who freeze are the ones who memorised Java textbook OOP and never adapted it to Python's looser, function-first idioms.

Each Q below uses the same shape: **30-second answer** (what you say first when the panel is impatient), **step-by-step thought process** (how to reason out loud), a **code** block (compiles on Python 3.12+), **edge cases to mention** (where panels watch you trip), **what NOT to say** (the anti-patterns that signal you've never shipped this), and a **stand-out line** — the single sentence that signals you've actually shipped a Python service to production.

Section index

- Section A — Design principles in Python (Q1–Q5)
- Section B — Designing for testability (Q6–Q10)
- Section C — Common design patterns, the Python way (Q11–Q15)
- Section D — OOP design interview patterns (Q16–Q20)

Section A — Design Principles in Python

Q1 · Composition vs inheritance — when to pick which

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Flipkart, JP Morgan · Topic: OOP Design

The question (interviewer's verbatim phrasing):

Explain composition vs inheritance with a real backend example. When would you pick one over the other?

30-second answer: Inheritance models an "is-a" relationship — a `RetryablePayment` is a `Payment`. Composition models a "has-a" relationship — a `PaymentService` has a `RetryPolicy`. In Python, the default should be composition because it's looser, more testable, and doesn't lock you into a fragile parent class. Reach for inheritance only when you need polymorphism through a stable base contract (often via `abc.ABC` or `typing.Protocol`) — and even then, prefer composing in behaviour rather than inheriting it.

Step-by-step thought process: - Open with the canonical phrasing: inheritance is "is-a", composition is "has-a". Panels need to hear those exact words before you go deeper. - Give the failure mode of over-inheritance — the deep parent chain

(`BasePayment` → `CardPayment` → `RetryableCardPayment` → `ThreeDSCardPayment`) where one tweak in `BasePayment` breaks four siblings. Indian backend teams ship this bug regularly in payment, notification, and pricing modules. - Composition wins because behaviour is injected as a constructor argument — swap the `RetryPolicy` in tests with a `NoRetryPolicy`, no monkeypatching required. - The Python tilt: because duck typing is free, you rarely need a base class to enable polymorphism. A `Protocol` (PEP 544) gives you the type contract without forcing a runtime inheritance chain. - Don't say "never use inheritance". The standard library uses it well — `Exception` subclassing, custom dict subclasses, ORM models. The rule is *favour composition*, not *forbid inheritance*.

Code:

```

from typing import Protocol
from dataclasses import dataclass

# === Inheritance – works, but couples PaymentService to a class tree ===
class Payment:
    def charge(self, amount: int) -> bool: ...

class RetryablePayment(Payment):
    def charge(self, amount: int) -> bool:
        for _ in range(3):
            if super().charge(amount):
                return True
        return False

# Problem: testing RetryablePayment forces you to test Payment too.
# Adding a new retry strategy = a new subclass = combinatorial explosion.

# === Composition – RetryPolicy is injected, swappable, testable ===
class RetryPolicy(Protocol):
    def run(self, attempt) -> bool: ...

@dataclass
class ExponentialRetry:
    max_attempts: int = 3
    def run(self, attempt) -> bool:
        for i in range(self.max_attempts):
            if attempt():
                return True
        return False

@dataclass
class NoRetry:
    def run(self, attempt) -> bool:
        return attempt()

class PaymentService:
    def __init__(self, gateway, retry: RetryPolicy):
        self.gateway = gateway
        self.retry = retry

    def charge(self, amount: int) -> bool:
        return self.retry.run(lambda: self.gateway.charge(amount))

# In production
svc = PaymentService(razorpay_gateway, ExponentialRetry(max_attempts=5))
# In tests
test_svc = PaymentService(FakeGateway(), NoRetry())

```

Edge cases to mention: - "Composition over inheritance" is a guideline, not a religion. The standard library uses inheritance perfectly well for `Exception`, custom container types, and Django/SQLAlchemy ORM models — that's exactly the polymorphic-base scenario. - Naive composition still couples you if the injected class is concrete. The win comes when you inject against a `Protocol` or

`ABC` , not against a concrete class. - Don't use inheritance just to share code. That's what helper functions, mixins (used sparingly), or composed services are for. Inheritance is for polymorphism.

What NOT to say: - "Inheritance is bad, always use composition." Too absolute — the standard library, Django, SQLAlchemy, and `Exception` hierarchies all use inheritance correctly. The rule is *favour* composition, not forbid inheritance. - "Use multiple inheritance for behaviour reuse." That drags MRO complexity into a problem mixins or composition solves more cleanly. Multiple inheritance signals you haven't thought about whether the relationship is really "is-a".

Stand-out line: *"In Python, composition is the default because duck typing makes the polymorphism free — you don't need a base class to swap behaviour, you just need a `Protocol` and a constructor parameter."*

Common follow-up: *"What's a mixin and when would you use one over composition?"* — A mixin is a class designed to be inherited from alongside others to contribute a slice of behaviour (e.g., `LoggingMixin` , `SerializerMixin`). Use them sparingly when the behaviour really wants to hook into `self` (e.g., serialise the object's own fields); reach for composition first because mixins still drag MRO complexity into the picture.

Q2 · SOLID adapted for Python — what survives, what fades

Tags: Difficulty: Medium · Asked at: Razorpay, JP Morgan, Goldman, Walmart Labs, Cred · Topic: OOP Design

The question (interviewer's verbatim phrasing):

You've probably heard of SOLID in a Java context. Does all of it still apply in Python? Which letters matter more, which matter less?

30-second answer: All five letters still apply in spirit, but the implementations differ because Python has duck typing and first-class functions. SRP (Single Responsibility) is the biggest win — small classes, one reason to change. OCP (Open-Closed) usually manifests as injecting a strategy rather than subclassing. LSP (Liskov) still bites in inheritance chains. ISP (Interface Segregation) maps to keeping `Protocol` s narrow. DIP (Dependency Inversion) maps to passing dependencies through `__init__` rather than hardcoding imports. The L and the I matter less in everyday Python because we use Protocols instead of explicit interface hierarchies.

Step-by-step thought process: - Walk the panel through each letter with a concrete Python adjustment:
- **S — Single Responsibility:** `UserService` should not also send emails. Split into `UserService` and `EmailService` , compose them. This is the most under-applied principle in Indian backend codebases — services tend to balloon to 800 lines. - **O — Open-Closed:** Open for extension via new strategies / new Protocol implementations, closed for modification of the orchestrator. In Python this

usually means injecting a function or a small class, not subclassing. - **L — Liskov Substitution:** A subclass must be substitutable for its parent. Python's duck typing means you'll often hit this at runtime — `AttributeError` when the subclass doesn't honour the contract. `Protocol` s with `runtime_checkable` help. - **I — Interface Segregation:** Don't make clients depend on methods they don't use. In Python, keep `Protocol` s narrow — one method is fine. - **D — Dependency Inversion:** High-level modules depend on abstractions, not concrete classes. In Python that's a `Protocol` parameter in `__init__` , not a Java-style interface. - Stress-test the answer with the "wait, do you really need this?" sceptic — for a 200-line script you don't. SOLID is for the services that grow. - Mention Python-specific lever: because functions are first-class, "inject a function" is a perfectly Pythonic way to do Open-Closed and Strategy without writing a single class.

Code:


```

from typing import Protocol, Callable
from dataclasses import dataclass

# S – Single Responsibility split
@dataclass
class User:
    id: int
    email: str

class UserRepository(Protocol):
    def save(self, user: User) -> None: ...
    def find(self, user_id: int) -> User | None: ...

class EmailSender(Protocol):
    def send(self, to: str, body: str) -> None: ...

# Each class has ONE reason to change
class UserService:
    def __init__(self, repo: UserRepository, mailer: EmailSender):
        self.repo = repo          # D – depend on abstraction
        self.mailer = mailer

    def register(self, email: str) -> User:
        user = User(id=self._next_id(), email=email)
        self.repo.save(user)
        self.mailer.send(email, "Welcome") # composed, not inherited
        return user

    def _next_id(self) -> int:
        return 1 # placeholder

# O – Open-Closed via a function (no subclassing needed)
PricingRule = Callable[[int], int]

def gst_18(amount: int) -> int:
    return amount + amount * 18 // 100

def gst_5(amount: int) -> int:
    return amount + amount * 5 // 100

def price_with(rule: PricingRule, amount: int) -> int:
    return rule(amount)

# Add a new rule = define a new function. Existing code unchanged.

```

Edge cases to mention: - Over-applying SOLID produces 30-file projects for a 200-line problem. Use it as a smell-detector, not a religion. - LSP violation in Python is silent at definition time and explodes at call time — a **Protocol**-typed function happily accepts an object missing the method, you only learn at the failing call. - ISP in Python rarely means "split this Protocol" — it usually means "don't add a sixth method to this Protocol just because you have a new client".

What NOT to say: - "SOLID is a Java thing, it does not apply to Python." It does apply — the implementations just shift (Protocols instead of interfaces, functions instead of Strategy classes). Saying this signals you haven't worked on a long-lived Python service. - "I follow SOLID strictly on every project." Over-application produces 30-file projects for 200-line problems. Panels want SOLID as a smell-detector, not a religion.

Stand-out line: *"SOLID survives in Python, but the D — Dependency Inversion — is the one that pays for itself every sprint, because it's what makes your classes mockable without monkeypatching."*

Common follow-up: *"Show me what Dependency Inversion looks like in a Django view."* — A Django view that calls `EmailService.send(...)` directly is hard-coded; refactor so the view receives an `EmailService` via the request/container or via a service registry, and the view's tests can substitute a fake.

Q3 · Dependency injection in Python — the `__init__` pattern

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Cred, Swiggy, Zerodha · Topic: OOP Design

The question (interviewer's verbatim phrasing):

In Java you'd use Spring for DI. What does dependency injection look like in idiomatic Python?

30-second answer: In Python, dependency injection is usually just constructor injection — pass dependencies into `__init__` as parameters and store them as attributes. No framework needed for 90% of cases. For the remaining 10% (large services, request-scoped dependencies, framework integration), FastAPI's `Depends`, `dependency_injector`, or a simple service-locator pattern fill the gap. The big shift from Java: there is no annotation magic, no `@Autowired` — the wiring is explicit, which is a feature, not a bug.

Step-by-step thought process: - Open with the punchline: Python's DI is constructor injection, full stop. Pass the dependency in, store it, use it. No XML, no annotations, no container. - Show the contrast with the wrong way — hardcoded `import` of a concrete `requests` call, or instantiating the dependency inside the method. Both are unmockable without ugly patching. - Step up to typed DI: type the parameter as a `Protocol`. Now the class advertises what it needs (a thing that can `send`, a thing that can `save`), and any implementation works. - For request-scoped DI in web frameworks: FastAPI's `Depends` is the canonical Python answer for HTTP routes. Flask uses `flask.g` or a factory. Django uses request middleware or a settings-based registry. - Don't bring a heavy DI framework unless your team is genuinely large or you're plugging into a legacy enterprise system. The reflex to install `dependency_injector` for a 15-class service is the Java instinct showing.

Code:

```

from typing import Protocol
import logging

# Define what the dependency must do (Protocol = duck-typed interface)
class Notifier(Protocol):
    def notify(self, message: str) -> None: ...

class EmailNotifier:
    def notify(self, message: str) -> None:
        # would actually send mail
        print(f"emailing: {message}")

class SlackNotifier:
    def __init__(self, webhook: str):
        self.webhook = webhook
    def notify(self, message: str) -> None:
        print(f"slack {self.webhook}: {message}")

# Constructor injection – no framework needed
class OrderProcessor:
    def __init__(self, notifier: Notifier, logger: logging.Logger):
        self.notifier = notifier
        self.logger = logger

    def process(self, order_id: int) -> None:
        self.logger.info(f"processing {order_id}")
        self.notifier.notify(f"order {order_id} processed")

# Wire it once at the edge of the application
proc = OrderProcessor(
    notifier=SlackNotifier("https://hooks.slack.com/..."),
    logger=logging.getLogger("orders"),
)
proc.process(1234)

# In tests
class CapturingNotifier:
    def __init__(self):
        self.messages = []
    def notify(self, message: str) -> None:
        self.messages.append(message)

test_notifier = CapturingNotifier()
test_proc = OrderProcessor(test_notifier, logging.getLogger("test"))
test_proc.process(1)
assert test_notifier.messages == ["order 1 processed"]

```

Edge cases to mention: - The "import inside the method" pattern (`def send(self): import requests; requests.post(...)`) is unmockable without `unittest.mock.patch` aimed at the call site — and call-site patching is brittle because moving the import breaks the patch. - Passing the entire `settings` object as a dependency couples the class to your config shape — pass only what's

needed. - For class-method dependencies (e.g., a singleton DB pool), pass the pool into `__init__`. Don't reach for a module-level global from inside the class — that's the unmockable pattern wearing a Python disguise.

What NOT to say: - "Python needs a DI framework like Spring or Guice to do this properly." It does not. The whole point of Python DI is that constructor parameters are the framework — adding annotation magic obscures the wiring. - "Just use `unittest.mock.patch` to swap dependencies in tests." That's the brittle workaround for code that wasn't designed for DI. Constructor injection makes the patch unnecessary and the test setup readable.

Stand-out line: *"Python's DI is just constructor parameters — and the absence of a framework is the point, because the wiring stays visible in the code instead of hidden in an annotation."*

Common follow-up: *"How does FastAPI's `Depends` differ from plain constructor injection?"* — `Depends` is request-scoped: FastAPI calls the dependency function per request, caches it within the request, and tears it down at the end. Constructor injection wires once at app start; `Depends` wires per request, which matters for things like DB sessions and authenticated users.

Q4 · Single Responsibility — splitting a fat service class

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Swiggy, Postman, Flipkart · Topic: OOP Design

The question (interviewer's verbatim phrasing):

Here's a 400-line `OrderService` class with 20 methods. Walk me through how you'd refactor it.

30-second answer: A class with 20 methods almost always has more than one reason to change. Identify the responsibility clusters — usually persistence, business rules, notifications, external API calls — and split each into its own class. The `OrderService` then becomes a thin orchestrator that composes the new classes via constructor injection. The goal isn't "small classes for the sake of it"; it's that each class has one reason to change and one set of collaborators to mock.

Step-by-step thought process: - Diagnose first: ask the panel "what changes most often in this class?" If the answer is "the email body and the DB query", you have at least two responsibilities. - Cluster methods by collaborator. Methods that touch the DB → `OrderRepository`. Methods that send notifications → `OrderNotifier`. Methods that compute pricing rules → `PricingEngine`. Methods that call payment APIs → `PaymentGateway`. - Keep the original `OrderService` class name and turn it into the orchestrator. Existing callers don't break; the internals are now composed. - Test win: each new class is tested in isolation. The orchestrator's tests inject fakes for all four collaborators. Before the split, every test had to set up the entire 400-line god class. - Watch for the anti-

pattern: splitting *too* finely. A `OrderIdGenerator` class for a one-liner is over-engineering — keep it as a method until it grows.

Code:

```

from typing import Protocol
from dataclasses import dataclass

@dataclass
class Order:
    id: int
    amount: int
    status: str

# Before: one fat class doing everything (not shown – imagine 400 lines)

# After: responsibility-split
class OrderRepository(Protocol):
    def save(self, order: Order) -> None: ...
    def find(self, order_id: int) -> Order | None: ...

class PaymentGateway(Protocol):
    def charge(self, order: Order) -> bool: ...

class OrderNotifier(Protocol):
    def notify_placed(self, order: Order) -> None: ...
    def notify_paid(self, order: Order) -> None: ...

class PricingEngine(Protocol):
    def final_amount(self, base: int) -> int: ...

# OrderService now orchestrates – one reason to change: the flow
class OrderService:
    def __init__(
        self,
        repo: OrderRepository,
        gateway: PaymentGateway,
        notifier: OrderNotifier,
        pricing: PricingEngine,
    ):
        self.repo = repo
        self.gateway = gateway
        self.notifier = notifier
        self.pricing = pricing

    def place(self, order_id: int, base_amount: int) -> Order:
        order = Order(id=order_id, amount=self.pricing.final_amount(base_amount), status="placed")
        self.repo.save(order)
        self.notifier.notify_placed(order)
        return order

    def pay(self, order_id: int) -> Order:
        order = self.repo.find(order_id)
        if order is None:
            raise ValueError("no such order")
        if self.gateway.charge(order):
            order.status = "paid"
            self.repo.save(order)

```

```
self.notifier.notify_paid(order)
return order
```

Edge cases to mention: - Splitting by responsibility, not by file size. A 100-line class with one responsibility is fine; a 50-line class with three responsibilities is not. - The split is a refactor, not a redesign — keep the existing public API of `OrderService` the same so callers don't change. Internals get reorganised. - Resist the urge to add a base class for the four new classes. They have no shared behaviour; they're collaborators, not siblings.

What NOT to say: - "I'll split the class only when it crosses 500 lines." Line count is not the criterion — number of independent reasons to change is. A 100-line class with three responsibilities still needs splitting. - "We will create a `BaseService` for all four classes to share." They share no behaviour. Adding a base class re-introduces the coupling the split was meant to eliminate.

Stand-out line: *"Single responsibility isn't about line count — it's about how many distinct teams or requirements can independently force a change to this file."*

Common follow-up: *"Once you split into four classes, where does the transaction boundary live?"* — In the orchestrator (`OrderService`), because that's the only object with visibility into the whole flow. The repository methods become primitive operations; the orchestrator wraps them in a `with self.uow:` or `@transactional` boundary.

Q5 · "Favour composition over inheritance" — a concrete refactor

Tags: Difficulty: Medium · Asked at: Cred, Razorpay, JP Morgan, Walmart Labs, Postman · Topic: OOP Design

The question (interviewer's verbatim phrasing):

Here's a class hierarchy: `User → AdminUser → SuperAdminUser`. The product team wants a new type that's an admin but with API rate limits. How would you redesign?

30-second answer: The hierarchy is already at three levels and you're being asked to add a fourth orthogonal dimension — that's the smell that inheritance was wrong from the start. Refactor toward composition: `User` is a single class with a `permissions` field and a `rate_limit` field, each backed by a small composed class. New combinations now require zero new classes — they're new configurations.

Step-by-step thought process: - Diagnose: the hierarchy is encoding two independent dimensions — *what permissions you have* and *what rate limits apply*. Inheritance only encodes one dimension at a time, so any second dimension forces a class-explosion (`RateLimitedAdminUser`, `RateLimitedSuperAdminUser`, ...). - Walk through the refactor target: a flat `User` class

composed with a `Permissions` object and a `RateLimit` object. New dimensions = new composed objects, not new subclasses. - Show the immediate win — the product team's "admin with rate limits" becomes one line: `User(permissions=admin_perms, rate_limit=strict_limit)`. No new class. - Cover the "but I want polymorphism" objection: if some code branches on `isinstance(user, AdminUser)`, refactor it to ask the `permissions` object (`user.permissions.is_admin()`). Polymorphism moves from the class hierarchy to the composed object. - Don't preach "no inheritance ever". `Permissions` itself might be a small hierarchy if there's a stable contract. The point is to keep inheritance shallow and tactical.

Code:


```

from dataclasses import dataclass, field
from typing import Protocol

# === Before – class explosion ===
# class User: ...
# class AdminUser(User): ...
# class SuperAdminUser(AdminUser): ...
# class RateLimitedAdminUser(AdminUser): ...  <- new ask = new class
# Combinatorial explosion as orthogonal dimensions grow.

# === After – composition ===
@dataclass(frozen=True)
class Permissions:
    can_read: bool = True
    can_write: bool = False
    can_admin: bool = False

@dataclass(frozen=True)
class RateLimit:
    requests_per_minute: int = 60
    burst: int = 10

@dataclass
class User:
    id: int
    email: str
    permissions: Permissions = field(default_factory=Permissions)
    rate_limit: RateLimit = field(default_factory=RateLimit)

    def is_admin(self) -> bool:
        return self.permissions.can_admin

# Production wiring
regular_user = User(id=1, email="a@x.com")
admin = User(
    id=2, email="b@x.com",
    permissions=Permissions(can_admin=True, can_write=True),
)
# The new "admin with rate limits" requirement – no new class
rate_limited_admin = User(
    id=3, email="c@x.com",
    permissions=Permissions(can_admin=True, can_write=True),
    rate_limit=RateLimit(requests_per_minute=10, burst=2),
)

```

Edge cases to mention: - `field(default_factory=...)` is mandatory for mutable defaults in dataclasses — using `permissions: Permissions = Permissions()` looks like it works and then bites you with the shared-default-instance bug. - The flat `User` can grow too — if it gets a third orthogonal dimension (`Theme` , `Locale`), apply the same pattern again. - Some existing callers will break (`isinstance(u, AdminUser)` returns False for everyone). Migrate them to `u.is_admin()` or `u.permissions.can_admin` first, then delete the subclasses.

What NOT to say: - "Add a new subclass like `RateLimitedAdminUser` to handle the new axis." That's the class-explosion smell. Two orthogonal axes already means the inheritance model is wrong; adding a third subclass per combination compounds the bug. - "Use multiple inheritance to mix the rate-limit axis with the role axis." MRO complexity is not the fix for a design that should have been composition from the start.

Stand-out line: *"Inheritance trees are how OO languages encode one axis of variation; the moment your domain has two orthogonal axes, composition is the only way to avoid a class-explosion."*

Common follow-up: *"What if you do want subclasses for things like `Customer`, `Vendor`, `Employee`?"* — Those are domain types, not behaviour modifiers, so inheritance can be appropriate. Keep the hierarchy shallow (one level) and put orthogonal traits (permissions, rate limits, locale) into composed objects.

Section B — Designing for Testability

Q6 · Making a class mockable — design from the test backwards

Tags: Difficulty: Medium · Asked at: Postman, Razorpay, Cred, Walmart Labs, Goldman · Topic: Testability

The question (interviewer's verbatim phrasing):

What does it mean for a class to be 'mockable', and how do you design for it?

30-second answer: A class is mockable when every external dependency it talks to is passed in (usually via `__init__`) rather than imported and called directly. In tests you swap each dependency for a stub or a `MagicMock`, run the code, and assert the right interactions happened. The opposite — hardcoded imports of `requests`, `boto3`, or a database client buried inside a method — forces `unittest.mock.patch` at the call site, which is brittle and noisy.

Step-by-step thought process: - Define mockable concretely: every collaborator the class talks to is a parameter, not a hardcoded import. - Walk through the cheap version first — `__init__` takes the dependency, stores it on `self`. In tests you instantiate the class with a fake. - For static methods or functions, pass them as parameters too — `def send(notifier=requests.post)` is a Pythonic way to keep the default convenient but the test substitutable. - Stress-test against the reflex of `from foo import bar` inside the class. That's not wrong by itself; it becomes wrong when `bar` is the dependency you'd like to swap in tests. - Show the `unittest.mock` alternative honestly — `@patch("module.path.requests.post")` works, but it's call-site patching and it breaks when the import path changes.

Code:

```

from typing import Protocol

# === Not mockable — hardcoded dependency ===
class WebhookSenderBad:
    def send(self, url: str, payload: dict) -> int:
        import requests # the dependency
        return requests.post(url, json=payload).status_code
# To test: @patch("module.requests.post") — call-site fragile

# === Mockable — dependency passed in ===
class HTTPClient(Protocol):
    def post(self, url: str, json: dict) -> "Response": ...

class Response(Protocol):
    status_code: int

class WebhookSender:
    def __init__(self, http: HTTPClient):
        self.http = http

    def send(self, url: str, payload: dict) -> int:
        return self.http.post(url, json=payload).status_code

# Production
import requests
sender = WebhookSender(http=requests)

# Test
class FakeResponse:
    status_code = 200
class FakeHTTP:
    def __init__(self):
        self.calls = []
    def post(self, url, json):
        self.calls.append((url, json))
        return FakeResponse()

fake = FakeHTTP()
test_sender = WebhookSender(http=fake)
assert test_sender.send("https://x.com/hook", {"a": 1}) == 200
assert fake.calls == [("https://x.com/hook", {"a": 1})]

```

Edge cases to mention: - `unittest.mock.MagicMock` is happy to fake anything — including misspelled method names. Use `spec=` to constrain it to the real class's interface. - A class with seven hardcoded dependencies isn't fixed by mocking — it's a single-responsibility problem dressed up as a testability problem. - Module-level singletons (`db = SQLAlchemy()` , `cache = redis.Redis()`) are unmockable inside the module. Push them to the edge and inject them.

What NOT to say: - "Just use `mock.patch('module.requests.post')` everywhere." Call-site patching is the brittle workaround for code that should have had a seam. Move the dependency into

`__init__` and patching becomes unnecessary. - "Mock the database in unit tests." If a unit test is talking to anything that needs mocking, the design has too many responsibilities. Pure logic should not need mocks at all.

Stand-out line: "A class is mockable when its dependencies are visible in the constructor signature — if I have to read the source to figure out what it calls, the test setup will be a mess."

Common follow-up: "When would you use `monkeypatch` over constructor injection?" — For test setup of module-level state you don't own (e.g., `os.environ`, `time.time`, third-party globals). For code you're writing, constructor injection wins because it documents the dependency.

Q7 · Pure logic vs I/O — the separation that makes tests fast

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Zerodha, JP Morgan · Topic: Testability

The question (interviewer's verbatim phrasing):

Walk me through how you'd structure a function that decides whether to send a low-balance alert.

30-second answer: Split into two layers — a pure function that takes inputs and returns a decision (no DB, no API calls), and an I/O wrapper that fetches the inputs, calls the pure function, and acts on the result. The pure function tests in microseconds without mocks; the wrapper has thin integration tests. This pattern — sometimes called "functional core, imperative shell" — eliminates 80% of the mocking pain in everyday backend code.

Step-by-step thought process: - Diagnose the typical mess: a single function that opens a DB connection, fetches the account, computes the decision, and fires an email — all in 30 lines. Unit-testing it requires mocking the DB, the user fetch, the email sender, and the time. - The split: extract `should_send_low_balance_alert(balance, threshold, last_alert_at, now)` as a pure function. Inputs in, decision out. No side effects, no I/O. - The wrapper: fetches the inputs from the world, calls the pure function, fires the side effects. The wrapper is tiny and easy to integration-test. - Why this wins: the pure function tests at the speed of arithmetic. The decision logic — usually 70% of the bugs — is now covered exhaustively in milliseconds. The wrapper handles the boring "did it call the email API" check. - Don't say "now we don't need integration tests" — you still do. They just become a much smaller surface.

Code:

```

from dataclasses import dataclass
from datetime import datetime, timedelta
from typing import Protocol

# === Pure logic – testable in microseconds ===
@dataclass(frozen=True)
class AlertDecision:
    send: bool
    reason: str

def should_send_low_balance_alert(
    balance: int,
    threshold: int,
    last_alert_at: datetime | None,
    now: datetime,
    cooldown_hours: int = 24,
) -> AlertDecision:
    if balance >= threshold:
        return AlertDecision(send=False, reason="above threshold")
    if last_alert_at and now - last_alert_at < timedelta(hours=cooldown_hours):
        return AlertDecision(send=False, reason="within cooldown")
    return AlertDecision(send=True, reason="low balance")

# === I/O shell – thin wrapper ===
class AccountRepo(Protocol):
    def get_balance(self, account_id: int) -> int: ...
    def get_last_alert_at(self, account_id: int) -> datetime | None: ...
    def record_alert_at(self, account_id: int, when: datetime) -> None: ...

class Mailer(Protocol):
    def send(self, account_id: int, subject: str) -> None: ...

class LowBalanceAlerter:
    def __init__(self, repo: AccountRepo, mailer: Mailer, threshold: int = 100):
        self.repo = repo
        self.mailer = mailer
        self.threshold = threshold

    def check(self, account_id: int, now: datetime) -> AlertDecision:
        decision = should_send_low_balance_alert(
            balance=self.repo.get_balance(account_id),
            threshold=self.threshold,
            last_alert_at=self.repo.get_last_alert_at(account_id),
            now=now,
        )
        if decision.send:
            self.mailer.send(account_id, "low balance")
            self.repo.record_alert_at(account_id, now)
        return decision

# Tests: 5 lines, no mocks, exhaustive
def test_no_alert_above_threshold():

```

```
d = should_send_low_balance_alert(200, 100, None, datetime(2026, 1, 1))
assert d.send is False
```

Edge cases to mention: - "Pure" in Python means no I/O and no mutating arguments.

`datetime.now()` inside a function is impure (it depends on the clock) — accept `now` as a parameter. - Logging from a "pure" function is technically impure but usually pragmatic. Decide as a team and stay consistent. - Resist the urge to make the pure function "almost pure" by adding a single read from a config. That re-introduces every problem the split was meant to fix.

What NOT to say: - "Pure functions are a functional-programming thing, not relevant in an OOP service." Wrong axis. The pure-vs-I/O split is about test speed and deterministic logic; it composes fine with classes (functional core, imperative shell). - "I'll just mock `time.time()` and the DB in every test." Each mock is a test-only contract you have to keep in sync with production. Pulling the I/O out eliminates the mocks entirely for the logic layer.

Stand-out line: *"The decision logic should be testable without spinning up Postgres — once you split pure logic from I/O, your test suite stays under five seconds even at 10,000 tests."*

Common follow-up: *"How would you test the I/O wrapper?"* — Either with fakes that record calls (`CapturingMailer` , `FakeRepo`) and assert the right calls happened, or with a small integration test against a real DB in CI.

Q8 · `@dataclass` for value objects — when and why

Tags: Difficulty: Easy · Asked at: Razorpay, Swiggy, Postman, Cred, Walmart Labs · Topic: Testability

The question (interviewer's verbatim phrasing):

When would you reach for `@dataclass` over a regular class?

30-second answer: Reach for `@dataclass` whenever the class is mostly data — a value object, a DTO, a configuration record, a domain entity without much behaviour. It auto-generates `__init__` , `__repr__` , `__eq__` , and optionally `__hash__` , which means tests can compare instances with `==` instead of asserting field-by-field. For frozen value objects use `@dataclass(frozen=True)` ; for entities that mutate, plain `@dataclass` is fine. Hand-roll a class only when you need significant behaviour, custom `__init__` logic, or `__slots__` for memory.

Step-by-step thought process: - Anchor on the time savings: a 30-line class with `__init__` , `__repr__` , and `__eq__` collapses to a 3-line `@dataclass` . Every line you don't write is a line that can't have a bug. - The `==` superpower: `Order(id=1, amount=500) == Order(id=1, amount=500)` is `True` because `@dataclass` auto-generates `__eq__` . Tests get readable — `assert result == Order(id=1, amount=500)` . - `frozen=True` for value objects:

immutability prevents the "did I accidentally mutate the cached version" class of bug. Frozen dataclasses are also hashable, so they work as dict keys. - `slots=True` (Python 3.10+) for memory-tight cases — it strips `__dict__` and locks the attribute set. Useful for high-cardinality value objects (e.g., a million Order records in memory). - Don't use `@dataclass` for everything. Classes with heavy `__init__` validation, custom constructor signatures, or significant behaviour are clearer as plain classes.

Code:

```
from dataclasses import dataclass, field
from typing import Self

# Frozen value object – hashable, comparable, immutable
@dataclass(frozen=True, slots=True)
class Money:
    amount_paise: int
    currency: str = "INR"

    def __add__(self, other: "Money") -> Self:
        if self.currency != other.currency:
            raise ValueError("currency mismatch")
        return Money(self.amount_paise + other.amount_paise, self.currency)

# Equality works without writing __eq__
assert Money(50000) == Money(50000)
# Hashable, so it works in sets and dict keys
prices = {Money(50000), Money(100000)}

# Mutable entity
@dataclass
class Order:
    id: int
    items: list[str] = field(default_factory=list) # NOT items=[]
    status: str = "pending"

    def add_item(self, sku: str) -> None:
        self.items.append(sku)

o1 = Order(id=1)
o1.add_item("razorpay-tee")
assert o1 == Order(id=1, items=["razorpay-tee"]) # readable test
```

Edge cases to mention: - `field(default_factory=list)` for mutable defaults. Writing `items: list[str] = []` raises `ValueError` at class-definition time precisely because dataclasses catch the mutable-default bug — but newcomers still try it. - `frozen=True` makes the instance immutable, but if a field is itself mutable (a list), you can still `instance.items.append(...)`. True immutability needs tuples/frozensets all the way down. - `eq=True` is the default; if you customise `__hash__`, set `eq=False` or also pass `frozen=True` — otherwise Python silently sets `__hash__ = None`.

What NOT to say: - "`@dataclass` is just sugar for `__init__`." It's also `__repr__`, `__eq__`, optional `__hash__`, ordering, and the slot/frozen flags. The `__eq__` alone is the test-readability win.

- "Use `@dataclass` for everything, including domain entities with logic." For pure data, yes. For classes with substantial behaviour, custom invariants, or non-trivial `__init__`, a regular class is clearer.

Stand-out line: "`@dataclass` is the cheapest readability upgrade in the language — three decorators replace 30 lines of boilerplate, and the `==` for free makes every test that compares objects two lines shorter."

Common follow-up: "What's the difference between `@dataclass` and `typing.NamedTuple` and `pydantic.BaseModel`?" — `NamedTuple` is tuple-based (positional + named, but limited mutation); `@dataclass` is the standard-library choice for value/entity objects; Pydantic adds validation, coercion, and JSON serialisation — use it at the application boundary, not internally.

Q9 · Protocol-based DI — typing dependencies without a base class

Tags: Difficulty: Medium · Asked at: Postman, Cred, Razorpay, JP Morgan, Walmart Labs · Topic: Testability

The question (interviewer's verbatim phrasing):

You've mentioned `Protocol` a few times. Why use it over an abstract base class?

30-second answer: `Protocol` (PEP 544) gives you structural typing — any object with the right methods satisfies the protocol, without inheriting from anything. That matches Python's duck typing instead of fighting it. Use `ABC` when you also want runtime enforcement (`isinstance` checks, "you must implement these methods or instantiation fails"); use `Protocol` when you just need a type contract for static analysis and tests. For most DI in modern Python, `Protocol` is the right pick.

Step-by-step thought process: - Define structural vs nominal typing in one breath: with `ABC`, a class must explicitly inherit to count; with `Protocol`, it just needs the right method shapes. - Show the testability win: a `Protocol`-typed parameter accepts any object — production class, fake test class, `MagicMock(spec=Notifier)`. None of them inherit from the protocol; they all satisfy it structurally. - Stress-test against the runtime-check question: by default, `isinstance(x, MyProtocol)` raises. Add `@runtime_checkable` to enable it — but it only checks method *names*, not signatures. - When to use `ABC`: when you want to fail fast on instantiation if a subclass forgets a method, or when you have shared default behaviour to inherit. - The hybrid pattern: define the protocol for callers to type against, and an internal abstract base class for shared behaviour. Most code only needs the protocol.

Code:


```

from typing import Protocol, runtime_checkable
from abc import ABC, abstractmethod

# Protocol – structural typing
class CacheBackend(Protocol):
    def get(self, key: str) -> str | None: ...
    def set(self, key: str, value: str, ttl: int) -> None: ...

# Any object with these two methods satisfies CacheBackend
class RedisCache:
    def get(self, key: str) -> str | None: return None
    def set(self, key: str, value: str, ttl: int) -> None: pass

class InMemoryCache:
    def __init__(self): self.store: dict[str, str] = {}
    def get(self, key: str) -> str | None: return self.store.get(key)
    def set(self, key: str, value: str, ttl: int) -> None: self.store[key] = value

def cached_lookup(cache: CacheBackend, key: str) -> str | None:
    return cache.get(key)

# Both work – no inheritance required
cached_lookup(RedisCache(), "user:1")
cached_lookup(InMemoryCache(), "user:1")

# === ABC alternative – explicit inheritance + runtime enforcement ===
class CacheBackendABC(ABC):
    @abstractmethod
    def get(self, key: str) -> str | None: ...
    @abstractmethod
    def set(self, key: str, value: str, ttl: int) -> None: ...

# This would fail at instantiation
# class HalfBaked(CacheBackendABC):
#     def get(self, key): return None
# HalfBaked() -> TypeError: Can't instantiate abstract class with abstract method 'set'

@runtime_checkable
class Closeable(Protocol):
    def close(self) -> None: ...

class File:
    def close(self) -> None: pass

assert isinstance(File(), Closeable) # works because @runtime_checkable

```

Edge cases to mention: - `Protocol` without `@runtime_checkable` doesn't support `isinstance` — you'll get `TypeError`. Decide upfront which protocols need runtime checks. - `@runtime_checkable` only checks method *names*, not signatures or return types. A class with `def get(self):` (wrong arity) still passes `isinstance`. - Don't mix `Protocol` and `ABC` in the same hierarchy. Pick one for each contract; using both is confusing and rarely needed.

What NOT to say: - "Always use `ABC` for interfaces because that's how Java does it." Python's runtime-checked `ABC` works, but `Protocol` matches Python's duck-typing model better and does not force callers to subclass. - " `Protocol` makes my code slower because of runtime checks." It does not — `Protocol` is a static-typing construct by default. Runtime cost appears only if you opt in to `@runtime_checkable` and call `isinstance` .

Stand-out line: " `Protocol` is duck typing with a type checker — you get the structural flexibility Python had all along, plus mypy will tell you when a caller is missing a method."

Common follow-up: "Can a `Protocol` have default method implementations?" — Yes, since Python 3.8 — define the method body in the `Protocol` , and any conforming class can inherit it (if they choose to subclass) or implement their own.

Q10 · The seam pattern — designing where the test cuts in

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Goldman, Cred, JP Morgan · Topic: Testability

The question (interviewer's verbatim phrasing):

When you say a class has a 'seam', what do you mean? Give an example.

30-second answer: A seam is a place in the code where you can substitute one implementation for another without editing the surrounding code — usually a constructor parameter, a passed-in function, or a `Protocol` -typed attribute. Tests use seams to swap real dependencies for fakes; feature flags use seams to swap strategies at runtime. Designing for seams up front is what "designing for testability" actually means.

Step-by-step thought process: - Define the term: a seam is a substitution point. Michael Feathers' "Working Effectively with Legacy Code" coined it — and the metaphor (a seam in fabric, where you can take it apart and put it back together) is exactly right. - The cheapest seam in Python is a constructor parameter: `def __init__(self, clock=time.time)` . Production uses the real clock; tests pass `lambda: 1700000000.0` . - The next seam is a `Protocol` -typed dependency — anyone matching the shape can plug in. - Functions passed in as parameters are seams too. Don't always make a class — a function that takes a `send_email: Callable[[str, str], None]` parameter has a seam built in. - The opposite of seams: hardcoded globals, module-level state, `import` inside methods. Each one closes a seam and makes the code harder to test.

Code:

```

from typing import Callable, Protocol
from datetime import datetime

# Seam 1 – constructor parameter with a sensible default
class SessionToken:
    def __init__(self, secret: str, now: Callable[[], datetime] = datetime.utcnow):
        self.secret = secret
        self.now = now          # the seam

    def issued_at(self) -> datetime:
        return self.now()

# Test
def fake_now():
    return datetime(2026, 1, 1, 12, 0, 0)

t = SessionToken("hunter2", now=fake_now)
assert t.issued_at() == datetime(2026, 1, 1, 12, 0, 0)

# Seam 2 – Protocol-typed dependency
class IDGenerator(Protocol):
    def next_id(self) -> int: ...

class CountingIDs:
    def __init__(self): self.n = 0
    def next_id(self) -> int:
        self.n += 1
        return self.n

class OrderBook:
    def __init__(self, ids: IDGenerator):
        self.ids = ids
        self.orders: list[int] = []

    def place(self) -> int:
        oid = self.ids.next_id()
        self.orders.append(oid)
        return oid

# Test with a predictable ID generator
book = OrderBook(ids=CountingIDs())
assert book.place() == 1
assert book.place() == 2

```

Edge cases to mention: - Default-arg seams (`now=datetime.utcnow`) are bound once at function definition time. That's fine for callables; if you ever put a *value* (`now=datetime.utcnow()`) you'd freeze the timestamp at import. Different bug. - Too many seams = noisy constructor. Group related seams into a small dependency object if the count climbs past four or five. - A seam is only as good as the test using it. Designing seams without writing the tests that use them is a code-review smell.

What NOT to say: - "Seams are an extra concept I don't need — I'll just write the code and figure out testing later." Test-after frequently means redesign-after, because the production code has no place for the test to cut in. - "Every parameter is a seam." Not in any useful sense. A seam is a deliberate injection point for a dependency that has a production version and a test version — not every config knob is one.

Stand-out line: *"A seam is a place where the test cuts in — and if there isn't a seam where your test wants to be, you're about to either rewrite the production code or write a brittle mock."*

Common follow-up: *"What's the difference between a seam and a mock?"* — A seam is a design choice in production code; a mock is a test-only object that plugs into the seam. Without a seam, mocks need `unittest.mock.patch` at the call site, which is the fragile workaround.

Section C — Common Design Patterns, the Python Way

Q11 · Strategy pattern — just use a function

Tags: Difficulty: Easy · Asked at: Razorpay, Postman, Swiggy, Cred, Zerodha · Topic: Design Patterns

The question (interviewer's verbatim phrasing):

Implement the Strategy pattern in Python.

30-second answer: In Python, the Strategy pattern is usually just a function passed as a parameter. The "context class with a strategy field" version from the GoF book works fine, but it's overkill when the strategy is one method. The Pythonic form: type the strategy as a `Callable`, accept it as a constructor argument, default to a sensible function. Class-based Strategy still has its place when the strategy has state or multiple methods.

Step-by-step thought process: - State the Java instinct first — a `Strategy` interface, three concrete classes, a context holding a strategy. Show that the Python form collapses to two lines. - The Pythonic shift: functions are first-class. A "strategy" is a function with a known signature. Pass it directly. - When to bring classes back: if the strategy needs configuration or holds state (a retry strategy with `max_attempts`, a tax calculator with rates), a small class is cleaner than a closure factory. - For a panel demonstration, do both versions — the function version to show the idiomatic answer, the class version to show you can do the formal pattern when needed. - Mention the type win: `Callable[[int], int]` is a one-line type alias for the strategy contract. No `Protocol` ceremony needed.

Code:

```

from typing import Callable, Protocol
from dataclasses import dataclass

# === Function-as-strategy – idiomatic Python ===
TaxRule = Callable[[int], int]

def gst_18(amount: int) -> int: return amount + amount * 18 // 100
def gst_5(amount: int) -> int: return amount + amount * 5 // 100
def no_tax(amount: int) -> int: return amount

class Invoice:
    def __init__(self, amount: int, tax_rule: TaxRule = no_tax):
        self.amount = amount
        self.tax_rule = tax_rule

    def total(self) -> int:
        return self.tax_rule(self.amount)

assert Invoice(1000, gst_18).total() == 1180
assert Invoice(1000, gst_5).total() == 1050

# === Class-as-strategy – when the strategy has state ===
class TaxStrategy(Protocol):
    def apply(self, amount: int) -> int: ...

@dataclass
class TieredTax:
    threshold: int
    low_rate: int
    high_rate: int

    def apply(self, amount: int) -> int:
        rate = self.low_rate if amount < self.threshold else self.high_rate
        return amount + amount * rate // 100

inv = Invoice.__class__(1500) # placeholder; real Invoice would accept TaxStrategy
# In a real codebase you'd parametrise Invoice over TaxRule | TaxStrategy

```

Edge cases to mention: - A function strategy is unhashable as a dict key only when it's a lambda holding a closure; module-level functions are hashable. Tiny detail, occasionally bites in caching layers. - Storing the strategy on `self` means it's serialised when you pickle the object — functions defined at module level pickle fine; lambdas don't. - Don't make a class for every Strategy because "patterns". A 3-line function is fine.

What NOT to say: - "I'll create an abstract `Strategy` class with a single `execute` method." That's the Java translation. In Python, that class collapses to a `Callable` type hint and a function. - "Strategy needs a context class to hold the current strategy." Optional, not required. The function or strategy object can be passed directly to where it's used.

Stand-out line: "Java's Strategy pattern is a workaround for not having first-class functions — in Python the strategy is the function, and the pattern collapses to a parameter."

Common follow-up: "How would you let users register new strategies at runtime?" — A registry:

`STRATEGIES: dict[str, TaxRule] = {}` and a `register("gst_18", gst_18)` helper. Or use entry points if you want plugin-style discovery across packages.

Q12 · Observer pattern — properties, callbacks, and signals

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Swiggy, Flipkart · Topic: Design Patterns

The question (interviewer's verbatim phrasing):

Implement the Observer pattern in Python.

30-second answer: Three common forms in Python: (1) a list of callbacks the subject invokes on change — the simplest and the most common; (2) `@property` with a setter that fires the callbacks; (3) framework signals like Django's `signals` or `blinker` for app-wide event buses. For a single class with a handful of listeners, callbacks-on-a-list is the idiomatic pick. Bring out signal libraries when the publisher and subscribers are decoupled across modules.

Step-by-step thought process: - Open with the simplest form: the subject keeps `self._listeners: list[Callable]`, a `subscribe` method appends to it, and any mutator calls each listener. - Step up to the property form when the trigger is a state change rather than an explicit event — `@price.setter` fires the callbacks if the value actually changed. - Mention the framework forms — Django signals (`pre_save` , `post_save`) are observer-pattern infrastructure for cross-module decoupling; `blinker` is the standalone library. - Watch out for the common bug: listeners that fail break the publisher. Wrap each callback in try/except so one bad listener doesn't take down the chain. - For async code, listeners might need to be coroutines — keep the publisher form consistent (`async def notify` and `await listener(event)`).

Code:

```

from typing import Callable, TypeVar, Generic
from dataclasses import dataclass, field

T = TypeVar("T")

# Form 1 – list of callbacks
class PriceFeed:
    def __init__(self):
        self._listeners: list[Callable[[int], None]] = []
        self._price = 0

    def subscribe(self, listener: Callable[[int], None]) -> None:
        self._listeners.append(listener)

    def update(self, price: int) -> None:
        if price != self._price:
            self._price = price
            for cb in self._listeners:
                try:
                    cb(price)
                except Exception as exc:
                    # one bad listener shouldn't break the others
                    print(f"listener error: {exc}")

feed = PriceFeed()
feed.subscribe(lambda p: print(f"alert: {p}"))
feed.subscribe(lambda p: print(f"log: {p}"))
feed.update(100)
feed.update(100) # no notify – same value

# Form 2 – property with setter that publishes
@dataclass
class Account:
    _balance: int = 0
    _listeners: list[Callable[[int], None]] = field(default_factory=list)

    @property
    def balance(self) -> int:
        return self._balance

    @balance.setter
    def balance(self, value: int) -> None:
        old = self._balance
        self._balance = value
        if old != value:
            for cb in self._listeners:
                cb(value)

    def watch(self, cb: Callable[[int], None]) -> None:
        self._listeners.append(cb)

```

Edge cases to mention: - Listeners holding references to the subject create cycles — the GC handles them eventually, but for tight memory cases use `weakref.WeakMethod` for bound-method listeners. - Iterating `self._listeners` while a listener might subscribe a new one mutates the list under you — iterate over a snapshot (`list(self._listeners)`) or use a queue. - Synchronous notify can block the publisher if a listener is slow. For long-running listeners, fire-and-forget into a queue or use an async runtime.

What NOT to say: - "Define an `Observer` interface and an `Observable` interface." Java does that because it has no first-class callbacks. In Python the listener is the function — no interface needed. - "Observers must run in the same thread as the publisher." Not necessarily — for slow listeners, decouple via a queue or async task. The decision is about latency tolerance, not pattern fidelity.

Stand-out line: *"Observer is just a list of callbacks and a loop — the pattern's complexity in Java comes from the interface ceremony, and Python skips most of it."*

Common follow-up: *"How would you implement event bubbling across nested objects?"* — Each child has a parent reference and forwards events upward, or you wire a shared event bus (`blinker.signal('order_paid')`) that any node can publish to and any node can subscribe to.

Q13 · Singleton — use a module instead

Tags: Difficulty: Easy · Asked at: Razorpay, Postman, Swiggy, Cred, Zerodha · Topic: Design Patterns

The question (interviewer's verbatim phrasing):

Implement a Singleton in Python.

30-second answer: The Pythonic Singleton is a module. Modules are imported once, cached in `sys.modules`, and every importer gets the same object — that's the singleton guarantee for free. The class-based forms (`__new__` override, decorator, metaclass) all work, but they're rarely needed in Python because the module pattern is simpler and more honest. Reach for the class form only when you genuinely need the class API (subclassing, lifecycle methods) and modules feel awkward.

Step-by-step thought process: - Start with the punchline: "in Python, use a module." Modules are singletons by definition — `import config` returns the same object on every call from every module. - The class-based forms exist and panels sometimes want to see them — be ready to write `__new__` override, decorator, and metaclass versions. But frame them as "here's how, and here's why you usually don't". - Threading: modules import safely once even under concurrent imports (the import lock handles it). Class-based singletons need `threading.Lock` around the `__new__` check in multi-threaded contexts. - The strongest singleton case: a connection pool or a configuration object. Both are perfectly served by a module with a `pool = create_pool(...)` at the top. - Mention the testability problem: singletons are global state, and global state is hard to test. Even the module form requires care — most

test-suite pain in Python comes from `from module import the_singleton` patterns that can't be swapped.

Code:

```

# === The Pythonic Singleton – a module ===
# file: db.py
# pool = create_connection_pool(url=os.environ["DB_URL"])
#
# Anywhere else:
# from db import pool
# pool.execute(...)
# Every importer gets the same pool. That's the singleton guarantee.

# === Class-based form (when you really need it) ===
class Config:
    _instance = None

    def __new__(cls, *args, **kwargs):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance

    def __init__(self):
        # __init__ runs every time Config() is called – guard with a flag
        if hasattr(self, "_initialised"):
            return
        self._initialised = True
        self.host = "localhost"
        self.port = 5432

c1 = Config()
c2 = Config()
assert c1 is c2

# === Decorator form ===
def singleton(cls):
    instances: dict = {}
    def get_instance(*args, **kwargs):
        if cls not in instances:
            instances[cls] = cls(*args, **kwargs)
        return instances[cls]
    return get_instance

@singleton
class Logger:
    def __init__(self):
        self.lines: list[str] = []
    def log(self, msg: str) -> None:
        self.lines.append(msg)

a = Logger()
b = Logger()
assert a is b

```

Edge cases to mention: - The `__new__`-based singleton's `__init__` runs on every call — guard it with a flag or move initialisation into `__new__`. - A module-level singleton is also global state. Treat it

like a global — pass it explicitly to anything that uses it, so tests can swap it. - Threading: `__new__` singletons need a lock if instantiation happens under contention; module imports already have one.

What NOT to say: - "I'll override `__new__` so my class always returns the same instance." Works, but a module-level instance does the same job with less ceremony and clearer intent. - "Singletons are fine as a global because there's only one." A singleton is still global state — it makes testing harder and hides dependencies. Treat it like a global and inject it where needed.

Stand-out line: *"In Python, the Singleton pattern is the module system — and the moment you write a `_instance` class attribute, ask yourself whether the design wouldn't be simpler as a module."*

Common follow-up: *"How do you reset a singleton for testing?"* — Either expose a `reset()` classmethod (and call it in test setup), or design around DI so tests don't need the singleton at all. The latter is the cleaner answer.

Q14 · Factory — classmethod constructors over Factory classes

Tags: Difficulty: Easy · Asked at: Razorpay, Cred, Postman, JP Morgan, Walmart Labs · Topic: Design Patterns

The question (interviewer's verbatim phrasing):

Show me the Factory pattern in Python.

30-second answer: The Pythonic Factory is a `classmethod` named after the construction intent — `Order.from_json(payload)`, `Order.from_db_row(row)`, `Order.placeholder()`. Each one is an alternate constructor that returns a configured instance. Separate Factory classes from the GoF book are rarely needed because classmethods cover 95% of the use cases and live right next to the class they construct. Reach for a separate Factory class only when the construction is complex enough to deserve its own state or when you need polymorphism across factories.

Step-by-step thought process: - Open with the canonical Python form: `classmethod` constructors. `datetime.fromisoformat`, `dict.fromkeys`, `pathlib.Path.cwd` are all stdlib examples — point them out. - The intent matters in the name: `from_json`, `from_dict`, `from_row`, `from_env`, `default`. Reading the name tells you what data shape it expects. - The `cls` argument matters — using `cls(...)` instead of `Order(...)` makes the classmethod inheritance-friendly. `Subclass.from_json(payload)` returns a `Subclass` instance. - When to escalate to a Factory class: when construction needs configuration of its own (a registry, an environment, a connection pool), and when multiple classes are constructed from the same source. A `PaymentMethodFactory` that returns `Card`, `UPI`, or `Wallet` based on input is a legitimate Factory. - Don't confuse with Abstract Factory — that's about creating families of related objects. Even rarer in Python; usually a `dict` of factory functions does the job.

Code:

```

from __future__ import annotations
from dataclasses import dataclass
from typing import Any

@dataclass
class Order:
    id: int
    amount: int
    currency: str

    @classmethod
    def from_json(cls, payload: dict[str, Any]) -> "Order":
        return cls(
            id=payload["id"],
            amount=int(payload["amount"]),
            currency=payload.get("currency", "INR"),
        )

    @classmethod
    def from_db_row(cls, row: tuple) -> "Order":
        oid, amount, currency = row
        return cls(id=oid, amount=amount, currency=currency)

    @classmethod
    def placeholder(cls) -> "Order":
        return cls(id=0, amount=0, currency="INR")

o = Order.from_json({"id": 1, "amount": "500"})
p = Order.from_db_row((2, 700, "USD"))

# === Factory class – when polymorphic construction is the point ===
class PaymentMethod:
    def charge(self, amount: int) -> bool: ...

class Card(PaymentMethod):
    def charge(self, amount: int) -> bool: return True

class UPI(PaymentMethod):
    def charge(self, amount: int) -> bool: return True

class PaymentMethodFactory:
    _registry: dict[str, type[PaymentMethod]] = {"card": Card, "upi": UPI}

    @classmethod
    def create(cls, kind: str) -> PaymentMethod:
        if kind not in cls._registry:
            raise ValueError(f"unknown payment method: {kind}")
        return cls._registry[kind]()

    @classmethod
    def register(cls, kind: str, method_cls: type[PaymentMethod]) -> None:
        cls._registry[kind] = method_cls

```

```
method = PaymentMethodFactory.create("upi")
assert method.charge(500) is True
```

Edge cases to mention: - Using `Order(...)` directly inside a classmethod hardcodes the class — subclasses inherit the method but lose polymorphism. Always use `cls(...)`. - A factory classmethod that returns a different type than the class is legal Python but confusing. Either return `cls` instances or rename the method to signal it. - Factory class + registry is great until two modules register the same key. Decide whether last-write-wins or a clash raises.

What NOT to say: - "I'll write a separate `OrderFactory` class with a `create_from_dict` method." That's the Java translation; a `classmethod` on `Order` itself is discoverable through autocomplete and avoids the orphan class. - "Factory classes are needed for testability." They are not — the classmethod is just as easy to call, and the test can pass alternate inputs the same way.

Stand-out line: *"The Factory pattern in Python is a `classmethod` — the alternate constructor lives on the class it builds, so the user finds it through autocomplete instead of needing to know a separate Factory class exists."*

Common follow-up: *"What about `__init_subclass__` for auto-registering subclasses?"* — Excellent for plugin systems: every subclass of `PaymentMethod` auto-registers itself in the factory's registry via `__init_subclass__`, so adding a new method only requires defining the class.

Q15 · Adapter — `Protocol` + a thin wrapper

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Cred, JP Morgan, Walmart Labs · Topic: Design Patterns

The question (interviewer's verbatim phrasing):

Explain Adapter with a Python example. When would you reach for it?

30-second answer: An Adapter wraps an existing class whose interface doesn't match what your code wants, and presents the interface your code wants. In Python this is usually a thin wrapper class — sometimes a single function — that satisfies a `Protocol`. Use it when integrating a third-party SDK whose API doesn't match your internal contract, or when migrating from one library to another and you want to swap implementations without rewriting callers.

Step-by-step thought process: - Open with a concrete scenario — your code wants a `Cache` with `get` / `set` / `delete`, but the third-party Redis client uses `r.get`, `r.set`, `r.del_`. Write an adapter that exposes your interface and delegates internally. - The minimum viable adapter: a class with `__init__(self, wrapped)` and the methods you want, each delegating to the wrapped object. - Why this beats subclassing the third-party class: you don't inherit its public surface area, you only

expose the methods your code uses. Refactors stay local. - The migration case: you're swapping `boto3` for an in-house API. Wrap both behind a `Storage` Protocol, write one adapter for each, and swap by configuration. Callers don't change. - Caveat: adapters add a hop. For hot-path code, document the wrapping so reviewers know what to expect. For most application code the cost is invisible.

Code:

```
from typing import Protocol

# Internal contract your codebase wants
class Cache(Protocol):
    def get(self, key: str) -> str | None: ...
    def set(self, key: str, value: str, ttl: int) -> None: ...
    def delete(self, key: str) -> None: ...

# Third-party client with a slightly different interface
class ThirdPartyRedis:
    def __init__(self): self._d: dict[str, str] = {}
    def fetch(self, k: str) -> str | None: return self._d.get(k)
    def store(self, k: str, v: str, expires_in_seconds: int) -> None: self._d[k] = v
    def remove(self, k: str) -> None: self._d.pop(k, None)

# Adapter: presents Cache, delegates to ThirdPartyRedis
class ThirdPartyRedisAdapter:
    def __init__(self, client: ThirdPartyRedis):
        self._client = client

    def get(self, key: str) -> str | None:
        return self._client.fetch(key)

    def set(self, key: str, value: str, ttl: int) -> None:
        self._client.store(key, value, expires_in_seconds=ttl)

    def delete(self, key: str) -> None:
        self._client.remove(key)

# In production
cache: Cache = ThirdPartyRedisAdapter(ThirdPartyRedis())
cache.set("user:1", "abhishek", ttl=60)
assert cache.get("user:1") == "abhishek"

# In tests - a fake satisfying the same Protocol
class InMemoryCache:
    def __init__(self): self._d: dict[str, str] = {}
    def get(self, key): return self._d.get(key)
    def set(self, key, value, ttl): self._d[key] = value
    def delete(self, key): self._d.pop(key, None)
```

Edge cases to mention: - An adapter that just renames methods is fine. An adapter that adds logic (caching, retries, logging) is no longer an adapter — it's a Decorator. Don't mix the two; if you need both,

layer them: Decorator(Adapter(client)). - Multi-method adapters that only forward one method are smell — refactor to a function or to a properly minimal Protocol. - Don't subclass the third-party class to add the new methods. That inherits its entire API surface, which usually drags in deprecation warnings, breaking changes, and surprise behaviour on upgrades.

What NOT to say: - "Just call the SDK directly from the business logic — it's only one place." Until it's not. The day you swap Razorpay for Stripe (or even update the SDK), every direct call has to be touched. - "Adapter and Decorator are the same pattern." They are not. Adapter changes an interface; Decorator adds behaviour at the same interface. Mixing them in one class makes the role unclear.

Stand-out line: *"Adapter is the pattern that makes third-party libraries replaceable — you wrap once at the boundary, and the rest of the codebase doesn't know or care which vendor is behind it."*

Common follow-up: *"What's the difference between Adapter and Facade?"* — Adapter changes an interface to match what you wanted; Facade simplifies a complex subsystem into a smaller interface. Adapter usually wraps one class; Facade usually wraps several.

Section D — OOP Design Interview Patterns

Q16 · Design a rate limiter class

Tags: Difficulty: Hard · Asked at: Razorpay, Swiggy, Flipkart, Postman, Cred · Topic: Design Interview

The question (interviewer's verbatim phrasing):

Design a rate limiter class that allows N requests per minute per key.

30-second answer: Use a token-bucket algorithm — each key gets a bucket of N tokens, tokens refill at a steady rate, each request consumes one. The class needs `try_acquire(key) -> bool` as the main API, internal storage for per-key buckets, and a clock dependency (injectable for tests). Wrap mutating operations in a lock for thread safety. For multi-process or distributed scenarios, back the storage with Redis using atomic Lua scripts; for in-process, a `dict[str, TokenBucket]` is enough.

Step-by-step thought process: - Pick the algorithm first: token bucket (smooth, allows bursts up to bucket size) or sliding window (precise but heavier). For interviews, token bucket is the canonical pick and easier to explain. - API contract: `try_acquire(key)` returns True if allowed, False if throttled. Optionally `retry_after(key)` returns seconds until next token. - Internal state: `dict[str, TokenBucket]` where each `TokenBucket` carries the current token count and a last-refill timestamp. On every `try_acquire`, compute tokens added since last refill, cap at bucket size, decrement, return result. - Inject the clock: `now: Callable[[], float] = time.monotonic`. Tests pass a fake clock, production uses `monotonic` (immune to wall-clock jumps). - Thread safety: wrap mutations in a `threading.Lock`. For high-concurrency cases, shard locks by key prefix. -

Mention distributed: for cluster-wide limits, use Redis `INCR` with `EXPIRE`, or a Lua script that does refill + decrement atomically. The Python class becomes a thin wrapper around the Redis call.

Code:

```
from dataclasses import dataclass, field
from typing import Callable
from threading import Lock
import time

@dataclass
class TokenBucket:
    capacity: int
    refill_per_sec: float
    tokens: float = field(init=False)
    last_refill: float = field(init=False)

    def __post_init__(self):
        self.tokens = float(self.capacity)
        self.last_refill = 0.0

class RateLimiter:
    def __init__(
        self,
        capacity: int,
        refill_per_sec: float,
        now: Callable[[], float] = time.monotonic,
    ):
        self.capacity = capacity
        self.refill_per_sec = refill_per_sec
        self.now = now
        self._buckets: dict[str, TokenBucket] = {}
        self._lock = Lock()

    def try_acquire(self, key: str, tokens: int = 1) -> bool:
        with self._lock:
            bucket = self._buckets.get(key)
            current = self.now()
            if bucket is None:
                bucket = TokenBucket(self.capacity, self.refill_per_sec)
                bucket.last_refill = current
                self._buckets[key] = bucket

            # refill since last check
            elapsed = current - bucket.last_refill
            bucket.tokens = min(
                self.capacity,
                bucket.tokens + elapsed * self.refill_per_sec,
            )
            bucket.last_refill = current

            if bucket.tokens >= tokens:
                bucket.tokens -= tokens
                return True
            return False

    def retry_after(self, key: str, tokens: int = 1) -> float:
        bucket = self._buckets.get(key)
```

```

        if bucket is None or bucket.tokens >= tokens:
            return 0.0
        needed = tokens - bucket.tokens
        return needed / self.refill_per_sec

# Test with a controllable clock
class FakeClock:
    def __init__(self, t: float = 0.0): self.t = t
    def __call__(self) -> float: return self.t
    def advance(self, dt: float) -> None: self.t += dt

clock = FakeClock()
limiter = RateLimiter(capacity=5, refill_per_sec=1.0, now=clock)
assert all(limiter.try_acquire("user:1") for _ in range(5)) # uses up bucket
assert limiter.try_acquire("user:1") is False              # throttled
clock.advance(2.0)                                          # refill 2 tokens
assert limiter.try_acquire("user:1") is True
assert limiter.try_acquire("user:1") is True
assert limiter.try_acquire("user:1") is False

```

Edge cases to mention: - Use `time.monotonic`, not `time.time`. Wall-clock can jump backwards (NTP sync), which would let the bucket refill faster than reality. - The first call for a key initialises the bucket — be careful not to refill from `last_refill=0.0` in that first call. Set `last_refill = now()` on creation. - For very high throughput, the dict grows without bound. Add expiry: a background sweep removes buckets that haven't been touched in N minutes, or use a bounded LRU cache.

What NOT to say: - "I'll use `time.sleep` to throttle requests." That blocks the caller and ties up a worker — not a rate limiter, just a slow path. Reject (429) or queue with bounded wait instead. - "One in-process limiter is enough — we will not scale horizontally." The moment you add a second app server, every limit doubles unless the state is shared (Redis with atomic Lua, or a token-vending service).

Stand-out line: *"Token bucket is the algorithm; the design points are injecting the clock for testability, locking around state mutation, and being explicit about whether you need in-process or distributed semantics."*

Common follow-up: *"How would you make this work across three application servers?"* — Move the buckets to Redis; use a Lua script for atomic refill-and-decrement so two servers can't both consume the last token. The Python class becomes a thin wrapper around the Redis call.

Q17 · Design a session store

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Flipkart, JP Morgan, Cred · Topic: Design Interview

The question (interviewer's verbatim phrasing):

Design a session store class. Sessions expire after 30 minutes of inactivity.

30-second answer: A `SessionStore` exposes `create(user_id) -> token`, `get(token) -> Session | None`, `refresh(token)`, and `delete(token)`. Internally, sessions are keyed by a cryptographically random token (`secrets.token_urlsafe`), each carrying a `last_seen_at`. `get` returns `None` if the session is expired, and `refresh` updates `last_seen_at`. Back the storage with a `Protocol` so production uses Redis and tests use a dict. Garbage collection is either lazy (purge on read) or a background sweep.

Step-by-step thought process: - Outline the public API up front so the panel knows where you're going — `create`, `get`, `refresh`, `delete`. Optionally `delete_all_for_user` for "log out all devices". - Token generation matters: `secrets.token_urlsafe(32)` for unguessable. Don't roll a random with `random.random()` — that's a CVE waiting to happen. - Storage abstraction: a `SessionBackend Protocol` with `put`, `get`, `pop`. The store uses the backend; production wires Redis, tests wire a dict. Same design lever as the rate limiter. - Expiry: lazy is simpler — on `get`, check `now - last_seen_at > 30min`, return `None` if expired (and delete it). Background sweep is for memory pressure and is optional. - Refresh policy: sliding window (refresh on every read) is the friendlier default. Fixed window (no refresh) is stricter; pick based on requirements. - Mention the security gotchas: invalidate sessions on password change, store `user_id` and *not* sensitive data in the session, sign the token if you can't store server-side.

Code:

```
from dataclasses import dataclass
from datetime import datetime, timedelta
from typing import Callable, Protocol
import secrets

@dataclass
class Session:
    user_id: int
    created_at: datetime
    last_seen_at: datetime

class SessionBackend(Protocol):
    def put(self, token: str, session: Session) -> None: ...
    def get(self, token: str) -> Session | None: ...
    def pop(self, token: str) -> None: ...

class InMemoryBackend:
    def __init__(self): self._data: dict[str, Session] = {}
    def put(self, token, session): self._data[token] = session
    def get(self, token): return self._data.get(token)
    def pop(self, token): self._data.pop(token, None)

class SessionStore:
    def __init__(
        self,
        backend: SessionBackend,
        ttl: timedelta = timedelta(minutes=30),
        now: Callable[[], datetime] = datetime.utcnow,
    ):
        self.backend = backend
        self.ttl = ttl
        self.now = now

    def create(self, user_id: int) -> str:
        token = secrets.token_urlsafe(32)
        current = self.now()
        self.backend.put(token, Session(user_id, current, current))
        return token

    def get(self, token: str) -> Session | None:
        session = self.backend.get(token)
        if session is None:
            return None
        if self.now() - session.last_seen_at > self.ttl:
            self.backend.pop(token)
            return None
        return session

    def refresh(self, token: str) -> bool:
        session = self.get(token)
        if session is None:
            return False
        session.last_seen_at = self.now()
```

```

        self.backend.put(token, session)
        return True

    def delete(self, token: str) -> None:
        self.backend.pop(token)

# In tests
class FakeClock:
    def __init__(self, t: datetime): self.t = t
    def __call__(self) -> datetime: return self.t
    def advance(self, dt: timedelta) -> None: self.t += dt

clock = FakeClock(datetime(2026, 1, 1, 12, 0, 0))
store = SessionStore(InMemoryBackend(), now=clock)
token = store.create(user_id=42)
assert store.get(token) is not None
clock.advance(timedelta(minutes=31))
assert store.get(token) is None # expired and cleaned up

```

Edge cases to mention: - Use `secrets.token_urlsafe`, not `random` or `uuid.uuid4` (`uuid4` is not strictly cryptographically random in all implementations). The token is the credential — treat it as such. - Don't store passwords, payment info, or PII in the session. Store `user_id` and look up the rest from the DB. - If you do sliding refresh, expose a max-lifetime cap too (e.g., 7 days regardless of activity) — otherwise active sessions never expire and you can't force re-login.

What NOT to say: - "Sessions in memory are fine — we will move to Redis later." That commits you to a redesign the day you add a second app server, because in-memory sessions do not survive cross-instance traffic. - "Just use the framework's default session." Fine for prototypes; the panel is asking the design question because they want to hear what you would build, not what Django gave you for free.

Stand-out line: *"The session store is two abstractions — the storage backend (Redis or memory) and the expiry policy (sliding or fixed) — and getting both injectable is what makes the class testable and switchable in production."*

Common follow-up: *"How do you invalidate a user's sessions on password change?"* — Either a `delete_all_for_user(user_id)` that scans by user (cheap in Redis with a per-user index, expensive in a plain dict), or include a session version in the user record and check it on every `get`.

Q18 · Design a config loader (env + file + defaults)

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Cred, JP Morgan, Walmart Labs · Topic: Design Interview

The question (interviewer's verbatim phrasing):

Design a config loader that reads from environment variables first, then a YAML file, then defaults. Show how you'd test it.

30-second answer: A `Config` dataclass declares the schema with defaults. A `ConfigLoader` reads from a chain of sources in priority order — env vars, then a file source, then the defaults baked into the dataclass — and returns a populated `Config`. Each source is a `Protocol` so tests can inject a `DictSource` instead of touching `os.environ` or the filesystem. The loader is pure: same inputs, same `Config`, no side effects.

Step-by-step thought process: - Schema first: a `Config` dataclass with typed fields and defaults. The defaults become the lowest-priority source. - Sources as Protocols: `def get(self, key: str) -> str | None`. Implementations: `EnvSource` (reads `os.environ`), `YamlSource` (reads a YAML file), `DictSource` (in-memory, for tests). - Loader walks the source chain in priority order. First non-None wins. Apply type conversion (`str → int`, `str → bool`) at the loader boundary so the `Config` fields are properly typed. - Validation: fail loud on missing required fields. Don't return half-populated config — fail at startup, not on first 500. - Testability lever: the loader takes the source chain as a constructor argument. Tests build a chain of `DictSource` and inject; no filesystem, no env mutation. - Don't reach for a library (`pydantic-settings`, `dynaconf`) on first draft. The hand-rolled version is 30 lines and teaches the design.

Code:

```

from dataclasses import dataclass, fields
from typing import Protocol, Any

@dataclass
class Config:
    db_url: str = "postgres://localhost/dev"
    port: int = 8000
    debug: bool = False

class ConfigSource(Protocol):
    def get(self, key: str) -> str | None: ...

class DictSource:
    def __init__(self, data: dict[str, str]):
        self.data = data
    def get(self, key: str) -> str | None:
        return self.data.get(key)

class EnvSource:
    def __init__(self, prefix: str = "APP_"):
        import os
        self._env = os.environ
        self.prefix = prefix
    def get(self, key: str) -> str | None:
        return self._env.get(f"{self.prefix}{key.upper()}")

def _coerce(raw: str, target_type: type) -> Any:
    if target_type is bool:
        return raw.lower() in {"1", "true", "yes", "on"}
    if target_type is int:
        return int(raw)
    if target_type is float:
        return float(raw)
    return raw

class ConfigLoader:
    def __init__(self, sources: list[ConfigSource]):
        self.sources = sources # priority order: first wins

    def load(self) -> Config:
        values: dict[str, Any] = {}
        for f in fields(Config):
            for source in self.sources:
                raw = source.get(f.name)
                if raw is not None:
                    values[f.name] = _coerce(raw, f.type if isinstance(f.type, type) else str)
                    break
        return Config(**values)

# Production wiring
# loader = ConfigLoader([EnvSource(prefix="APP_"), YamlSource("/etc/app.yaml")])
# config = loader.load()

```



```
# Test
test_loader = ConfigLoader([
    DictSource({"db_url": "postgres://test/db", "debug": "true"}),
])
config = test_loader.load()
assert config.db_url == "postgres://test/db"
assert config.debug is True
assert config.port == 8000 # falls through to default
```

Edge cases to mention: - The `fields(Config)` introspection lets you avoid repeating the schema in two places. Don't hand-list keys — they'll drift from the dataclass. - Type coercion is where bugs live. `"false"` is truthy as a Python string — the `_coerce` helper has to handle the string-to-bool conversion explicitly. - Validation belongs in the dataclass `__post_init__`, not in the loader. Loader's job is to populate fields; the dataclass enforces invariants on those fields.

What NOT to say: - "Read `os.environ` directly inside the service." That couples the service to the environment and breaks tests that don't have those variables set. Push config reads to the edge and pass the config object in. - "We will store secrets in the YAML config in the repo." Never — secrets belong in a secrets manager, not in version control. The chain is `EnvSource` → `SecretsManagerSource` → `YamlSource` → `defaults`.

Stand-out line: "The config loader's superpower is testability — once the env and file sources are behind a Protocol, the entire loader logic can be tested without touching `os.environ` or the filesystem."

Common follow-up: "What about secrets — do you load them the same way?" — Same loader, different source (`AwsSecretsManagerSource` , `VaultSource`). Keep secrets out of YAML in the repo; the source chain becomes `EnvSource` → `SecretsManagerSource` → `YamlSource` → `defaults`.

Q19 · Design a cache with TTL

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Swiggy, JP Morgan · Topic: Design Interview

The question (interviewer's verbatim phrasing):

Design an in-memory cache with TTL support. Multiple readers, one writer.

30-second answer: A `TTLCache` exposes `get(key)`, `set(key, value, ttl)`, and `delete(key)`. Internally, entries are tuples of `(value, expires_at)`. `get` returns the value if `now < expires_at`, otherwise removes and returns `None` — lazy expiry. For concurrent access, wrap mutations in a `threading.Lock`; for high-read scenarios, use `RLock` and minimise the critical section. Cap memory with an LRU eviction policy when the cache hits a size limit. Inject the clock and the lock for testability.

Step-by-step thought process: - API contract first: `get`, `set` (with optional ttl), `delete`, `clear`. Optionally `size()` and an eviction callback. - Storage: `dict[str, tuple[Any, float]]` — value plus expiry timestamp. Tuples are cheap and immutable, good cache citizens. - Expiry strategy: lazy is simplest — check on `get`, evict if expired. Cheap, no background thread. Add a periodic sweep only if memory pressure demands it. - Concurrency: `threading.Lock` around mutations. `get` reads can be lock-free if you accept stale reads of just-expired entries, but the canonical answer is to lock both reads and writes for correctness. - Eviction: when the cache hits `max_size`, evict the oldest entry (FIFO) or the least-recently-used (LRU). `collections.OrderedDict` gives you LRU for free with `move_to_end`. - Testability: inject the clock, optionally inject the lock for tests that care about contention. The clock injection is what lets you test TTL behaviour without `time.sleep(31)`.

Code:

```

from collections import OrderedDict
from threading import Lock
from typing import Any, Callable
import time

class TTLCache:
    def __init__(
        self,
        max_size: int = 1024,
        default_ttl: float = 60.0,
        now: Callable[[], float] = time.monotonic,
    ):
        self.max_size = max_size
        self.default_ttl = default_ttl
        self.now = now
        self._store: OrderedDict[str, tuple[Any, float]] = OrderedDict()
        self._lock = Lock()

    def get(self, key: str) -> Any | None:
        with self._lock:
            entry = self._store.get(key)
            if entry is None:
                return None
            value, expires_at = entry
            if self.now() >= expires_at:
                del self._store[key]
                return None
            # LRU: mark as most-recently-used
            self._store.move_to_end(key)
            return value

    def set(self, key: str, value: Any, ttl: float | None = None) -> None:
        with self._lock:
            ttl = ttl if ttl is not None else self.default_ttl
            expires_at = self.now() + ttl
            self._store[key] = (value, expires_at)
            self._store.move_to_end(key)
            while len(self._store) > self.max_size:
                self._store.popitem(last=False) # evict oldest

    def delete(self, key: str) -> None:
        with self._lock:
            self._store.pop(key, None)

    def __len__(self) -> int:
        return len(self._store)

# Test with a controllable clock
class FakeClock:
    def __init__(self, t: float = 0.0): self.t = t
    def __call__(self) -> float: return self.t
    def advance(self, dt: float) -> None: self.t += dt

```

```

clock = FakeClock()
cache = TTLCache(max_size=3, default_ttl=10.0, now=clock)
cache.set("a", 1)
assert cache.get("a") == 1
clock.advance(11.0)
assert cache.get("a") is None      # expired and removed
cache.set("a", 1); cache.set("b", 2); cache.set("c", 3); cache.set("d", 4)
assert len(cache) == 3             # 'a' evicted (LRU)

```

Edge cases to mention: - Lazy expiry means an expired entry counts toward `max_size` until it's read. For tight memory, run a periodic sweep. - `OrderedDict.move_to_end` mutates the dict — you can't iterate while moving. Iterate over a snapshot if you need to inspect. - Hashing the key matters — string keys are fine. Complex keys (tuples of mutable objects) can break or cause collisions; document what's hashable.

What NOT to say: - "Use a background thread to evict expired entries on every TTL tick." Overkill for most workloads — lazy expiry on read covers it, and a periodic sweep is cheaper than a per-key timer. - "`functools.lru_cache` is enough for everything." It is not — `lru_cache` has no TTL and no eviction-on-time, so stale entries live until the size limit forces them out.

Stand-out line: "Lazy expiry plus `OrderedDict` gives you a TTL cache with LRU eviction in 40 lines — the design points are injecting the clock for test speed and being deliberate about whether reads need the lock."

Common follow-up: "How would you make this thread-safe under high contention?" — Shard the cache into N internal caches keyed by `hash(key) % N`, each with its own lock. Lock contention drops by Nx. Or move to a lock-free library if Python's contention is the bottleneck (rare in CPython because of the GIL).

Q20 · Design a thread-safe counter

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Postman, JP Morgan, Walmart Labs · Topic: Design Interview

The question (interviewer's verbatim phrasing):

Design a thread-safe counter class. Now make it work across multiple processes.

30-second answer: For in-process, a `Counter` class with a `threading.Lock` around increments and reads. `with self._lock: self._value += 1`. For multi-process, the lock has to live in shared memory — use `multiprocessing.Value` (a shared C int) with its built-in lock, or `multiprocessing.Manager().Value`, or push the counter to Redis with `INCR`. The design point

is naming the concurrency boundary: threads share memory and need a `threading.Lock`; processes don't share memory and need either shared-memory primitives or an external store.

Step-by-step thought process: - Threads first — the GIL serialises Python bytecode, but `x += 1` is NOT atomic at the Python level (it's load, add, store). Concurrent threads can interleave and lose increments. The `threading.Lock` fixes it. - Reads also need the lock for tear-free semantics — without it, a reader can see a half-updated value (rare on CPython int, but the contract should be explicit). - Multi-process: `threading.Lock` is process-local and useless across forks. Use `multiprocessing.Value("i", 0)` which carries its own lock, or `Value("i", 0, lock=multiprocessing.Lock())` for an explicit one. - Stress-test the GIL question: "but the GIL serialises Python anyway, why do I need a lock?" Because the GIL releases between bytecodes; `x += 1` decomposes to LOAD, ADD, STORE and the GIL can release between LOAD and STORE on another thread. - For distributed counters across machines, redis `INCR` is the canonical pick — atomic on the server, simple client. The Python class wraps the Redis call.

Code:

```

from threading import Lock, Thread
from multiprocessing import Value, Process

# === Thread-safe in-process counter ===
class Counter:
    def __init__(self, initial: int = 0):
        self._value = initial
        self._lock = Lock()

    def increment(self, by: int = 1) -> int:
        with self._lock:
            self._value += by
            return self._value

    def decrement(self, by: int = 1) -> int:
        return self.increment(-by)

    @property
    def value(self) -> int:
        with self._lock:
            return self._value

# Demonstration
counter = Counter()
def worker():
    for _ in range(10000):
        counter.increment()

threads = [Thread(target=worker) for _ in range(10)]
for t in threads: t.start()
for t in threads: t.join()
assert counter.value == 100000      # no lost updates

# === Multi-process counter via shared memory ===
class ProcessSafeCounter:
    def __init__(self, initial: int = 0):
        # Value carries its own lock by default
        self._value = Value("i", initial)

    def increment(self, by: int = 1) -> int:
        with self._value.get_lock():
            self._value.value += by
            return self._value.value

    @property
    def value(self) -> int:
        with self._value.get_lock():
            return self._value.value

# === Distributed counter (sketch) ===
class RedisCounter:
    def __init__(self, redis_client, key: str):
        self._redis = redis_client

```

```

    self._key = key
    def increment(self, by: int = 1) -> int:
        return self._redis.incrby(self._key, by)
    @property
    def value(self) -> int:
        return int(self._redis.get(self._key) or 0)

```

Edge cases to mention: - The classic mistake: assuming `x += 1` is atomic because of the GIL. It is not — `dis.dis("x += 1")` shows it's `LOAD_NAME`, `LOAD_CONST`, `BINARY_OP`, `STORE_NAME`. The GIL can release between any two bytecodes. - `threading.Lock` is reentrant-unsafe: if the same thread tries to acquire it twice, it deadlocks. Use `RLock` if your code path might recurse into itself. - For multi-process, *don't* try to share a `threading.Lock` across forks. Each forked process gets its own copy with no synchronisation between them.

What NOT to say: - "The GIL guarantees `x += 1` is atomic, so no lock is needed." It does not — `dis.dis` will show you it's multiple bytecodes, and the GIL can release between them. - "Use `multiprocessing.Lock` for multi-thread code." Wrong tool — `multiprocessing.Lock` is for cross-process synchronisation. Use `threading.Lock` (or `RLock`) inside one process.

Stand-out line: *"The GIL doesn't make Python operations atomic — it serialises bytecode execution, but most Python statements decompose into multiple bytecodes, and the GIL is free to release in the gap."*

Common follow-up: *"What's the performance cost of the lock?"* — For a simple counter under low contention, microseconds per call. Under high contention (many threads incrementing), the lock can dominate. If profiling shows this, shard the counter (N internal counters, sum on read) — same trick as the sharded cache.

Chapter wrap-up

These 20 questions cover the OOP design surface area you'll be drilled on in any Indian backend interview round 2 or 3 — the principles (composition, SOLID, DI, SRP), the testability moves (mockable design, pure-vs-I/O split, dataclass value objects, Protocol-based contracts, seam pattern), the patterns in their Pythonic forms (Strategy as a function, Observer as a callback list, Singleton as a module, Factory as a classmethod, Adapter as a thin wrapper), and the live-design classics (rate limiter, session store, config loader, TTL cache, thread-safe counter).

The thread running through all 20 answers: **Python's looseness is a feature**. Where Java needs interfaces and factories and dependency-injection frameworks to enable polymorphism and testability, Python gets the same outcome with `Protocol`s and constructor parameters and module-level functions. Bringing Java-textbook OOP into a Python interview is the single most common reason confident developers get downlevelled — the panel can tell when someone hasn't internalised the language's idioms.

Drill these 20 in your prep, and you'll have something to say to almost every OOP design question an Indian backend panel will throw at you.

Chapter 3 — Django Essentials (25 Questions)

Python Interview Prep — 2026 Edition · Learn & Excel · File 07 Backend Track Library

This chapter is the Django round-one cleanup that catches confident developers who can wire up a `urls.py` but freeze the moment the interviewer pulls them into the ORM internals or the request lifecycle. Twenty-five questions tuned for the Indian backend market — heavy on the questions Razorpay, Cred, Postman, Zerodha, Swiggy, and the Service MNCs (TCS / Infosys / Wipro / Cognizant / Accenture) actually ask in 2026. Every answer is checked against Django 4.2 LTS and 5.x; where 5.x changed something (constraints, async views, `aaggregate`), the tag calls it out. Where a behaviour goes back to 3.x and still holds, the tag says "Django 3.x+".

Every question follows the same shape — tags, the interviewer's verbatim phrasing, a 30-second answer, the step-by-step "what to say out loud" thought process, edge cases to surface, what NOT to say, and the common follow-up. The shape exists because most Indian Django interviews are conversational rather than whiteboard-style; the bottleneck is finding the right words under pressure, not writing the code.

Table of Sections

- Section A — Django ORM (Q1–Q10)
- Section B — Models and Migrations (Q11–Q15)
- Section C — Middleware and Views (Q16–Q20)
- Section D — Common Pitfalls (Q21–Q25)

A note on schema. The questions assume a small set of Indian-flavoured tables you can hold in your head: `Order`, `Payment`, `User`, `Merchant`, `Subscription`, `RefundRequest`, `Webhook`, `LedgerEntry`. Treat them as the same logical schema across questions so the examples chain together — the `Order` from Q3 is the same `Order` you index in Q12 and lock in Q21.

Section A — Django ORM

The ten ORM questions decide whether your Django interview moves on to the architecture round or stops at "we'll get back to you." Every Indian product company that runs Django at scale — Razorpay's

merchant stack, Cred's reward engine, Postman's billing, Zerodha's broker portal, the Service-MNC banking apps built on Django — has been bitten by a QuerySet that did the wrong thing in production, and the round-one questions are designed to surface whether you know the failure modes. The themes that get tested hardest in 2026: when QuerySets evaluate, the N+1 problem and the two tools to fix it, `Q` objects for composable WHERE clauses, transactions and `atomic`, the `F()` expression for race-free updates, and `bulk_create` versus a loop of `.save()` calls. Answer these ten cleanly and you've cleared the database round.

Q1 · Why are Django QuerySets called "lazy"?

Tags: Difficulty: Easy · Asked at: Razorpay, Cred, Postman, TCS, Infosys · Topic: QuerySet evaluation · Django 3.x+

The question (interviewer's verbatim phrasing):

When I write `qs = Order.objects.filter(status='captured')`, does that line hit the database?

The 30-second answer: No. That line builds a QuerySet object — a description of the query — but does not execute it. Django only sends SQL to the database when the QuerySet is *evaluated*, which happens when you iterate it, slice it with a step, call `len()`, `list()`, `bool()`, or `repr()` on it, or call a terminal method like `.exists()`, `.count()`, `.first()`, `.get()`, or `.aggregate()`. Until one of those things happens, the QuerySet is a chainable plan.

Step-by-step thought process: 1. Lead with the definition. Lazy means "no work is done until the result is actually needed." The QuerySet sits there as an in-memory object that knows what SQL it *would* run. 2. Walk through the chain.

`Order.objects.filter(status='captured').exclude(amount__lt=100).order_by('-created_at')` is three QuerySet method calls, each returning a new QuerySet. No SQL has been sent yet — just three Python objects built on top of each other. 3. Then list the evaluation triggers — this is the part interviewers want you to recite. `for o in qs:` (iteration), `list(qs)` (cast), `len(qs)` (length), `bool(qs)` (truthiness in an `if`), `repr(qs)` (printing in the shell), and the terminal queryset methods. 4. Close with the practical implication. Laziness is what lets the Django ORM optimise — because nothing runs until evaluation, the ORM can fold filters, joins, and orderings into a single SQL statement. It is also what lets you pass QuerySets around in code without paying for them until you ask for results.

Edge cases to mention: - Slicing without a step (`qs[:10]`) does *not* evaluate; it adds `LIMIT 10` to the eventual query. Slicing with a step (`qs[::2]`) *does* evaluate, because Python needs the list to step through it. - Calling `.values()` or `.values_list()` returns another QuerySet — still lazy. Only iteration or terminal calls trigger the query. - Django caches the result the first time a QuerySet evaluates.

The second iteration of the *same* QuerySet object does not hit the database again. A different QuerySet built from the same model does.

What NOT to say: - "QuerySets are lazy because they use generators" — wrong. QuerySets are not generators; they cache their results on first evaluation. Saying "generator" tells the interviewer you have not read the source. - "QuerySets hit the database when you build them" — wrong. That is the opposite of lazy.

Common follow-up: *"Name three QuerySet methods that DO evaluate immediately, and three that don't."* — Evaluate immediately: `.exists()`, `.count()`, `.first()`. Stay lazy: `.filter()`, `.exclude()`, `.annotate()`. Memorise the split — the interviewer is checking you can.

Stand-out line: *"Say this and you'll separate yourself: laziness isn't a quirk — it's the contract that lets you compose filters in Python and let the ORM fold them into a single SQL plan at the moment of evaluation."*

```
# No SQL yet — these three lines build a QuerySet in memory
qs = Order.objects.filter(status='captured')
qs = qs.exclude(amount__lt=100)
qs = qs.order_by('-created_at')

# SQL sent on the next line, because iteration evaluates
for order in qs:
    print(order.id)

# SQL sent again on the next line — a new QuerySet, no cache
print(Order.objects.filter(status='captured').count())
```

Q2 · What is the N+1 problem and how do you spot it in Django?

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Swiggy, Postman, Walmart Labs · Topic: N+1 · Django 3.x+

The question (interviewer's verbatim phrasing):

Your API endpoint that lists 100 orders takes 4 seconds in production but feels fine in dev. What is the most common cause, and how do you debug it?

The 30-second answer: Almost always the N+1 problem. The view fires one query for the list of N orders, then one extra query per order to fetch a related field — usually a ForeignKey like `order.user.name` — for a total of N+1 queries. In dev with 5 rows it's 6 queries, no one notices; in production with 100 rows it's 101 queries and the endpoint is gone. You spot it with `django-debug-`

`toolbar` locally, with `django.db.connection.queries` in tests, or with APM in production (New Relic, Datadog).

Step-by-step thought process: 1. Anchor on the shape. The "1" is the parent query — `SELECT * FROM orders LIMIT 100`. The "N" is the per-row follow-up query Django fires when you access a related field that wasn't pre-fetched, like `order.user.email`. 2. Make the cause concrete. A serializer or template loops over the orders and renders the user email per row; Django sees `order.user` is not cached, fires a `SELECT * FROM users WHERE id = ?` per access, and now the endpoint is doing 101 queries. 3. Move to detection. In development, install `django-debug-toolbar` and look at the SQL panel — it shows the query count and duplicate queries with stack traces. In tests, `from django.db import connection; print(len(connection.queries))` after the view runs and assert on the number. In production, the APM trace shows the cascade of single-row SELECTs. 4. Move to the fix. The cure is `select_related` for ForeignKey and OneToOne (single JOIN), and `prefetch_related` for ManyToMany and reverse ForeignKey (one extra query, joined in Python). Q3 and Q4 unpack the difference.

Edge cases to mention: - N+1 also hits the Django admin if you don't override `list_select_related` and the changelist accesses a related field per row. - N+1 hits DRF serializers silently — `serializers.PrimaryKeyRelatedField` is fine, but `serializers.StringRelatedField` or a nested serializer over `order.user` triggers the per-row query. - `assertNumQueries` in Django's test client is the cleanest way to lock in the fix — once the query count is asserted, regressions break the test.

What NOT to say: - "Just cache the queryset" — wrong layer. Caching hides the problem in dev and creates a stale-data bug in production. Fix the query first. - "N+1 only happens with ManyToMany" — wrong. The most common case is a plain ForeignKey accessed inside a loop.

Common follow-up: "Show me with code how `select_related('user')` would change the SQL count from 101 to 1." — Q3 has the answer. The chain `Order.objects.select_related('user').filter(...)` adds an INNER JOIN on `auth_user`, returning user columns alongside order columns in a single SELECT.

Stand-out line: "Say this and you'll separate yourself: N+1 is the bug every senior backend developer spots in code review on muscle memory — `for x in qs: x.related.field` is the signature, and a lock-it-in `assertNumQueries` is how you stop it coming back."

```
# N+1 – 101 queries for 100 orders
orders = Order.objects.filter(status='captured')[:100]
for o in orders:
    print(o.user.email)    # one SELECT per order – this is the "N"

# Fix – 1 query, JOIN built into the SELECT
orders = Order.objects.select_related('user').filter(status='captured')[:100]
for o in orders:
    print(o.user.email)    # no SQL – user already on the row
```

Q3 · `select_related` vs `prefetch_related` — when do you reach for each one?

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Zerodha, JP Morgan India · Topic: ORM joins · Django 3.x+

The question (interviewer's verbatim phrasing):

Explain the difference between `select_related` and `prefetch_related`. Give me a concrete case for each.

The 30-second answer: `select_related` does a SQL JOIN — one query, columns of the related object pulled into the same row. It works for ForeignKey and OneToOne (one-to-one and many-to-one). `prefetch_related` does a separate query for the related rows and stitches them together in Python. It works for ManyToMany and reverse ForeignKey (one-to-many in the other direction). Use `select_related` when the relationship is "this row has exactly one of those." Use `prefetch_related` when it is "this row has many of those."

Step-by-step thought process: 1. Start with the SQL shape. `select_related('user')` rewrites the query as `SELECT order.*, user.* FROM orders INNER JOIN auth_user ON ...` — one query, wider result rows. 2. Then `prefetch_related('payments')` shape. Django runs the parent query first — `SELECT * FROM orders` — collects the order IDs, fires a second query `SELECT * FROM payments WHERE order_id IN (...)`, and assembles the children onto each parent in Python. Two queries instead of N+1. 3. Explain the why behind the split. JOIN on a many-relation would multiply rows (one order × five payments = five rows), which is why Django refuses to use `select_related` for many-side relations — it would force the ORM to deduplicate in Python and the JOIN cost would explode with selectable. 4. State the choice rule. Forward-FK → `select_related`. Reverse-FK → `prefetch_related`. ManyToMany → `prefetch_related`. OneToOne → `select_related` (forward) or `prefetch_related` (reverse, often).

Edge cases to mention: - `prefetch_related` supports a `Prefetch()` wrapper with a custom queryset — useful for filtering or ordering the related rows before they're stitched.

`prefetch_related(Prefetch('payments', queryset=Payment.objects.filter(status='captured')))` - `select_related` cascades through multiple FK hops — `select_related('order__merchant__owner')` builds a triple JOIN. - Mixing both in the same chain is normal — `Order.objects.select_related('user').prefetch_related('payments')` is one of the most common patterns at Razorpay-scale.

What NOT to say: - " `select_related` is just faster, always use it" — wrong. JOINS widen rows and waste bandwidth when the related table has many columns you don't need. - "Both make the same SQL" — wrong. The whole point of the split is the SQL strategy differs.

Common follow-up: "What happens if you call `select_related('payments')` where `payments` is a reverse ForeignKey?" — Django raises `FieldError: Invalid field name(s) given in select_related: 'payments'`. Reverse-FK is many-side; use `prefetch_related`.

Stand-out line: "Say this and you'll separate yourself: pick by the cardinality of the relation, not by what feels fast — JOIN for one-to-one, IN-list-and-stitch for one-to-many, and the ORM will tell you when you got it wrong with a `FieldError`."

```
# Forward FK — JOIN, one query
qs = Order.objects.select_related('user', 'merchant').filter(created_at__gte=yesterday)

# Reverse FK + M2M — two extra queries, no row multiplication
qs = Merchant.objects.prefetch_related(
    'orders',                                # reverse FK
    'tags',                                  # M2M
    Prefetch('payouts', queryset=Payout.objects.filter(status='paid'))
)
```

Q4 · Q objects — when do you reach for them?

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Swiggy, Goldman Sachs India · Topic: complex queries · Django 3.x+

The question (interviewer's verbatim phrasing):

I want to filter for orders where the status is 'captured' OR the amount is over 10000. How do I express that in the ORM?

The 30-second answer: With `Q` objects. `Q` lets you build arbitrary boolean combinations — OR, AND, NOT — on top of the usual filter syntax. The query becomes

`Order.objects.filter(Q(status='captured') | Q(amount__gt=10000))`. You can also negate with `~Q(...)` and combine across as many fields as you need. The default `filter(**kwargs)` only supports AND between keyword arguments; the moment you need OR or grouped logic, reach for `Q`.

Step-by-step thought process: 1. State the operator set. `&` is AND, `|` is OR, `~` is NOT. Parentheses give you precedence control. The expression `Q(a=1) | (Q(b=2) & ~Q(c=3))` translates to `WHERE a = 1 OR (b = 2 AND c <> 3)`. 2. Show the mental model. `filter(a=1, b=2)` is sugar for `filter(Q(a=1) & Q(b=2))`. Once you internalise that, the OR case is just swapping the operator. 3. Move to composition. `Q` objects can be built up dynamically — store them in a list and reduce with `operator.or_` to build a filter from runtime input. This is how dashboards with optional filter checkboxes work without writing string SQL. 4. Note the gotcha. `Q` objects mix with `__`

lookups normally — `Q(merchant__name__startswith='Razor')`. They can also reference `F()` expressions for column-to-column comparisons — `Q(amount__gt=F('refundable_limit'))`.

Edge cases to mention: - A bare `Q(...)` not passed to `filter` does nothing; it's just an expression object. - `exclude` with multiple kwargs ANDs them inside the NOT — `exclude(a=1, b=2)` is `WHERE NOT (a = 1 AND b = 2)`, which is *not* the same as `exclude(a=1).exclude(b=2)` (NOT a AND NOT b). - Combining `Q` with a `__in` lookup on a subquery lets you do correlated subqueries inside an OR clause.

What NOT to say: - "Use raw SQL for OR" — that signals you don't know the ORM has `Q`. Use raw SQL only when the ORM cannot express the query at all. - "Q objects are slower" — they compile to the same SQL as the kwargs form; there's no runtime cost difference.

Common follow-up: "How would you compose a `Q` filter from a list of (field, value) pairs at runtime?" — Build a list of `Q(**{f'{field}': value})`, then `from functools import reduce; from operator import or_; reduce(or_, qs_list)` to OR them.

Stand-out line: "Say this and you'll separate yourself: `Q` is the moment the ORM stops being a kwargs trick and becomes an algebra — `&`, `|`, `~`, and `F()` give you every boolean shape SQL can express without ever leaving Python."

```
from django.db.models import Q

# Static — status captured OR amount > 10000
qs = Order.objects.filter(Q(status='captured') | Q(amount__gt=10000))

# Dynamic — build the filter from runtime input
from functools import reduce
from operator import or_

filters = [Q(merchant__name__icontains=name) for name in user_input_names]
if filters:
    qs = Order.objects.filter(reduce(or_, filters))
```

Q5 · When do you reach for raw SQL inside Django?

Tags: Difficulty: Medium · Asked at: Cred, Razorpay, Goldman Sachs India, Walmart Labs · Topic: raw SQL escape hatch · Django 3.x+

The question (interviewer's verbatim phrasing):

The Django ORM is great, but when does it fall short, and what's the right way to drop down to raw SQL?

The 30-second answer: Reach for raw SQL when the ORM cannot express the query — window functions on a complex partition, recursive CTEs, vendor-specific operators, or queries that need a hint the planner won't pick on its own. Use `Model.objects.raw('SELECT ...')` when you want results back as model instances, and `connection.cursor()` for fully ad-hoc queries that don't map to a model. Always parameterise — never interpolate user input into a string.

Step-by-step thought process: 1. Frame the choice. The ORM covers 95% of queries cleanly. The remaining 5% are CTEs, window functions on funky partitions, vendor-specific syntax (Postgres `LATERAL`, `DISTINCT ON`), and performance hints. For those, drop down. 2. Show the two APIs. `Model.objects.raw(sql, params)` returns a `RawQuerySet` that you can iterate to get model instances — useful when the result columns align with model fields. `with connection.cursor() as cursor: cursor.execute(sql, params); cursor.fetchall()` returns plain tuples — useful for analytics or reporting. 3. Hammer the parameterisation rule. Always pass parameters as the second argument (`params`) — never `f-string` user input into the SQL. The cursor or `raw()` handles SQL escaping. Concatenating with `%s` in Python is the SQL injection that finds you in security review. 4. Caveat the cost. Raw SQL bypasses migrations, signals on save, model validation, and the ORM's `QuerySet` caching. You lose the moat. Document the raw query so the next developer knows why it exists.

Edge cases to mention: - Postgres-specific operations like `INSERT ... ON CONFLICT DO UPDATE` are not first-class in Django 4.x — use `bulk_create(..., update_conflicts=True, update_fields=[...])` in Django 4.1+ or drop to raw SQL on older Django. - `RawQuerySet` is lazy — it does not run the SQL until iterated, same as a regular `QuerySet`. - `cursor.execute` with `executemany` is the right call for bulk parameterised statements; `bulk_create` is usually cleaner but `executemany` wins on edge cases.

What NOT to say: - "I always use raw SQL — the ORM is too slow" — wrong stance for an interview. The ORM compiles to the same SQL most of the time; reach for raw only when needed. - "String-format the user input into the SQL" — this is the wrong answer; expect to be cut.

Common follow-up: "Show me a window-function query that ranks payments per merchant, using raw SQL inside Django." — Use `connection.cursor()` and `ROW_NUMBER() OVER (PARTITION BY merchant_id ORDER BY captured_at)`. Pass any `merchant_id` filter as a parameter.

Stand-out line: "Say this and you'll separate yourself: raw SQL is an escape hatch, not a stance — name the exact reason (CTE, window, vendor syntax, planner hint) the ORM cannot do this, parameterise every variable, and leave a comment explaining what the next developer is looking at."


```

from django.db import connection

# Raw SQL with model mapping
orders = Order.objects.raw(
    """
    SELECT *
    FROM orders
    WHERE created_at >= %s
    AND id IN (SELECT order_id FROM payments WHERE status = %s)
    """,
    [cutoff, 'captured']
)

# Fully ad-hoc – for analytics or reporting
with connection.cursor() as cursor:
    cursor.execute(
        "SELECT merchant_id, COUNT(*) FROM orders WHERE created_at >= %s GROUP BY 1",
        [cutoff]
    )
    rows = cursor.fetchall()

```

Q6 · `transaction.atomic` — what does it guarantee, and where do you wrap it?

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Zerodha, JP Morgan India, Postman · Topic: transactions · Django 3.x+

The question (interviewer's verbatim phrasing):

Your payment endpoint writes to two tables — `payments` and `ledger_entries`. If the second write fails, the first must be rolled back. How do you guarantee that in Django?

The 30-second answer: Wrap both writes in `transaction.atomic()`. As a context manager — `with transaction.atomic(): ...` — or as a decorator on the view function. Inside the block, every database operation is part of one SQL transaction; any unhandled exception rolls the whole block back. Outside the block, Django's default is autocommit, where each statement commits on its own. For multi-step money flows, `atomic` is non-negotiable.

Step-by-step thought process: 1. State the default. Django runs in autocommit mode — every ORM call (`.save()`, `.create()`, `.update()`, `.delete()`) commits immediately. There is no transaction wrapping unless you create one. 2. Move to `atomic`. As a context manager: `with transaction.atomic(): payment.save(); LedgerEntry.objects.create(...)`. If `LedgerEntry.objects.create` raises, the `payment.save()` is rolled back too. Both succeed or neither does. 3. Nesting. `atomic` blocks can nest — the inner one uses a SAVEPOINT and only rolls back to that savepoint on failure, leaving the outer block's earlier work intact. The outer block is the real

transaction; inner ones are checkpoints. 4. The decorator pattern. `@transaction.atomic` on a view function wraps the whole view in one transaction — convenient but coarse. For long views with external API calls, you don't want the transaction held open across a network round-trip; use the context manager around only the database work.

Edge cases to mention: - `transaction.atomic` only covers the *default* database. With multiple databases, pass `using='alias' : transaction.atomic(using='ledger')`. Cross-database atomicity is *not* guaranteed by Django. - A `try / except` *inside* an atomic block swallows the exception but does *not* prevent the rollback — Django still marks the transaction as dirty and will roll back at the end. Use `transaction.set_rollback(False)` if you genuinely want to keep going (rare). - `on_commit` hooks: `transaction.on_commit(lambda: send_email(...))` defers a callback until the transaction commits successfully. Critical for not sending emails for rolled-back orders.

What NOT to say: - "Django auto-rolls-back on any exception" — wrong without `atomic`. In autocommit mode, the failed statement rolls back but earlier statements already committed. - "Wrap the whole view in atomic to be safe" — partially wrong; for views with slow external calls, the open transaction holds row locks and can cause connection-pool starvation.

Common follow-up: *"What happens if you raise an exception inside a nested `atomic` block — does the outer block roll back?"* — Only if you let the exception propagate. The inner `atomic` rolls back to its savepoint; the outer block sees the exception and rolls back too unless you catch it before it leaves the outer block.

Stand-out line: *"Say this and you'll separate yourself: `atomic` is the database boundary; `on_commit` is the side-effect boundary — together they're the only way to guarantee an email never goes out for an order that got rolled back."*

```
from django.db import transaction

@transaction.atomic
def settle_payment(payment_id):
    payment = Payment.objects.select_for_update().get(pk=payment_id)
    payment.status = 'settled'
    payment.save()

    LedgerEntry.objects.create(
        payment=payment,
        debit=payment.amount,
        kind='settlement',
    )

    # Email AFTER commit only — never for rolled-back transactions
    transaction.on_commit(lambda: notify_merchant.delay(payment.merchant_id))
```

Q7 · `bulk_create` vs a loop of `.save()` — when does it matter?

Tags: Difficulty: Medium · Asked at: Cred, Razorpay, Postman, Swiggy, Walmart Labs · Topic: bulk writes · Django 3.x+

The question (interviewer's verbatim phrasing):

You need to insert 10,000 rows into the `webhook_events` table from a batch job. Why is `bulk_create` faster than looping over `.save()` ?

The 30-second answer: A loop of `.save()` is 10,000 separate INSERT statements, each its own round-trip and its own implicit transaction. `bulk_create` issues a single INSERT statement with 10,000 VALUES rows (batched into chunks, by default unlimited but configurable), so it pays one network round-trip and one transaction boundary for the whole batch. On a real network, the difference is two orders of magnitude — 30 seconds versus 300 milliseconds.

Step-by-step thought process: 1. Name the cost. Round-trip to the database is the dominant cost in a hot loop — typically 0.5-2 ms per call against a local Postgres, much more across availability zones. 10,000 round-trips at 1 ms each is 10 seconds before any CPU work. 2. Walk the `bulk_create` shape. Build a list of model instances *without saving*: `objs = [WebhookEvent(payload=p) for p in batch]`. Then `WebhookEvent.objects.bulk_create(objs)` issues a single multi-row INSERT. 3. Cover the `batch_size` parameter. `bulk_create(objs, batch_size=500)` splits the work into chunks of 500 — important on databases that limit the number of parameters per statement (Postgres allows 65,535; SQLite is much lower). 4. State the trade-offs. `bulk_create` does *not* call `save()`, does *not* fire `pre_save` / `post_save` signals, does *not* populate the primary key on the in-memory objects on every backend (Postgres returns IDs in Django 4.0+, but MySQL with `RETURNING` only since MySQL 8.x), and skips model `clean()` validation. If you depend on any of those, you have to call `.save()` per object — or refactor your dependencies.

Edge cases to mention: - Django 4.1+ added `bulk_create(..., update_conflicts=True, update_fields=[...], unique_fields=[...])` — Postgres `INSERT ... ON CONFLICT DO UPDATE`. This is the modern upsert and avoids the race in `get_or_create` (see Q21). - `bulk_update` is the sibling for UPDATE — pass a list of modified instances and a list of fields to update. - `ignore_conflicts=True` skips duplicates silently — useful for idempotent insert pipelines, but the inserted instances' `pk` is not populated.

What NOT to say: - "`.save()` in a loop is fine, the database is fast" — wrong. Round-trip dominates; a hot loop of saves is the most common Django perf bug. - "`bulk_create` fires signals" — wrong. It explicitly skips signals, which is a feature (perf) and a footgun (audit logs).

Common follow-up: "You need to insert 10,000 webhook events AND update a counter on the parent Merchant for each batch. How do you keep that consistent?" — Wrap both calls in `transaction.atomic`, use `bulk_create` for the events, then a single

`Merchant.objects.filter(pk__in=...).update(event_count=F('event_count') + ...)` using the `F` expression from Q8.

Stand-out line: "Say this and you'll separate yourself: round-trip is the cost — `bulk_create` collapses N round-trips into one `INSERT`, and the trade is signals and `clean()` validation, so name the trade out loud."

```
from webhooks.models import WebhookEvent

# Slow — 10,000 round-trips
for payload in batch:
    WebhookEvent.objects.create(payload=payload)

# Fast — one INSERT statement (or a handful with batch_size)
objs = [WebhookEvent(payload=p, merchant_id=m) for p, m in batch]
WebhookEvent.objects.bulk_create(objs, batch_size=500)
```

Q8 · The `F()` expression — what problem does it solve?

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Zerodha, JP Morgan India, Goldman Sachs India · Topic: atomic updates · Django 3.x+

The question (interviewer's verbatim phrasing):

I want to increment a counter on a row — say `merchant.event_count` — without race conditions. Why is `merchant.event_count += 1; merchant.save()` wrong?

The 30-second answer: That two-line sequence does a `SELECT` into Python, increments in Python, and writes the new value back. Two concurrent requests both read the old value, both increment to `old+1`, both write `old+1` — and one increment is lost. The fix is

`Merchant.objects.filter(pk=m).update(event_count=F('event_count') + 1)`. `F()` references the column on the database side, so the increment happens inside a single SQL `UPDATE ... SET event_count = event_count + 1` — atomic against any number of concurrent updates.

Step-by-step thought process: 1. Diagnose the race. Read-modify-write across two processes loses updates. This isn't a Django bug; it's how any read-then-write does without locking. 2. Introduce `F()`. The `F` expression is a reference to a field by name, evaluated by the database.

`F('event_count') + 1` compiles to the SQL fragment `event_count + 1`, executed atomically inside the `UPDATE`. 3. Show the shape.

`Merchant.objects.filter(pk=m).update(event_count=F('event_count') + 1)`. This is one SQL statement, no Python-side read, no race. 4. Note the breadth. `F()` works in `filter`,

`annotate` , `aggregate` , and `update` . Column-to-column comparisons like `Order.objects.filter(refunded_amount__gt=F('amount'))` are a common use; arithmetic in `annotate` for derived fields like net amount is another.

Edge cases to mention: - After an `F()` -based `update` , the Python instance is stale — it still holds the old value. Refresh from the database with `instance.refresh_from_db()` if you need the new number in code. - `F()` does *not* protect against TOCTOU on a *condition* — `filter(stock__gt=0).update(stock=F('stock') - 1)` can still oversell if you check the count separately. Use `select_for_update` for the conditional case (see Q21). - `F()` works with date arithmetic — `F('created_at') + timedelta(days=30)` for expiry computations.

What NOT to say: - "Use a Python lock to prevent the race" — wrong layer. Python locks don't cross processes or pods. - "`F()` is just for performance" — wrong. The atomicity, not the speed, is the point.

Common follow-up: "How would you decrement `stock` by 1 only when stock is positive, in a way that's safe under load?" — `Order.objects.filter(stock__gt=0).update(stock=F('stock') - 1)` returns the row count; if the count is 0, the row was unavailable. The filter and the update are evaluated atomically by Postgres / MySQL inside the single UPDATE.

Stand-out line: "Say this and you'll separate yourself: `F()` isn't a perf trick — it's how you make the database, not Python, the source of truth for the increment, which is the only way that survives two pods racing to the same row."

```
from django.db.models import F

# Race-prone — DO NOT do this
merchant = Merchant.objects.get(pk=m)
merchant.event_count += 1
merchant.save()

# Race-safe — single UPDATE on the DB side
Merchant.objects.filter(pk=m).update(event_count=F('event_count') + 1)

# Column-to-column comparison in a filter
overdrawn = Order.objects.filter(refunded_amount__gt=F('amount'))
```

Q9 · When does a QuerySet actually evaluate? Name every trigger.

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Infosys, Wipro · Topic: QuerySet evaluation · Django 3.x+

The question (interviewer's verbatim phrasing):

Walk me through every Python operation that forces a Django QuerySet to hit the database.

The 30-second answer: Five categories. (1) Iteration — `for o in qs:`. (2) Length — `len(qs)`. (3) Cast to list — `list(qs)`. (4) Truthiness — `bool(qs)` or `if qs:`. (5) Slicing with a step — `qs[::2]`. Plus the terminal methods that return non-QuerySet values: `.exists()`, `.count()`, `.first()`, `.last()`, `.get()`, `.aggregate()`, `.in_bulk()`, `.earliest()` when used as an iterable. Repr (`print(qs)` in the shell) also evaluates, because it needs to render rows.

Step-by-step thought process: 1. Group the triggers by why they need data. Iteration and cast-to-list need every row. `len` and `bool` need at least a count or one row. Slicing-with-step needs the materialised list to step through. Terminal methods return scalars or single instances, which require the SQL to actually run. 2. Pair each trigger with the SQL it issues. `len(qs)` runs a `SELECT COUNT(*)` if the QuerySet wasn't already evaluated; if it *was* already evaluated, it returns the cached length without SQL. `bool(qs)` runs a `SELECT ... LIMIT 1` to check for the existence of at least one row. `.exists()` is the same but explicit and slightly more efficient. 3. Mention the cache. Once a QuerySet evaluates (by iteration or list cast), Django caches the result in `qs._result_cache`. Subsequent iterations of the *same* object don't re-query. A different QuerySet built from the same model with the same filters is a different object — no cache. 4. Call out the misleading non-triggers. `qs[5:10]` adds OFFSET 5 LIMIT 5 but does *not* evaluate. `qs.filter(...)`, `qs.exclude(...)`, `qs.order_by(...)`, `qs.values()`, `qs.values_list()`, `qs.select_related()`, `qs.prefetch_related()`, `qs.annotate()`, `qs.distinct()` — all return new QuerySets, all stay lazy.

Edge cases to mention: - `len(qs)` versus `qs.count()`: `count()` always runs `SELECT COUNT(*)`. `len(qs)` runs `COUNT(*)` only if the QuerySet hasn't been evaluated yet — if it has, it returns the cached list length. So `len` is sometimes "free" and sometimes a full query — use `count()` when you only need the number and don't want the rows. - `bool(qs)` versus `qs.exists()`: `exists()` is `SELECT 1 ... LIMIT 1`. `bool(qs)` is similar but populates the cache, which is wasteful if you only needed the yes/no. - `qs.iterator()` evaluates row by row using a server-side cursor (Postgres) — does not cache, avoids loading all rows into memory. Use for million-row exports.

What NOT to say: - "filter triggers a query" — wrong; filter is lazy. - "QuerySets evaluate when you assign them to a variable" — wrong; assignment is just storing a reference.

Common follow-up: "You have a QuerySet you'll iterate twice in the same view. How do you avoid hitting the database twice?" — Either cast to a list once (`orders = list(qs)`) and iterate the list, or iterate the QuerySet once normally — its result cache lives until the QuerySet object is garbage-collected.

Stand-out line: "Say this and you'll separate yourself: every senior Django developer can recite the five evaluation triggers in their sleep — that's the test the interviewer is running."

```

qs = Order.objects.filter(status='captured')    # no SQL yet

if qs:                                         # SQL – LIMIT 1 to check existence
    pass

count = len(qs)                              # SQL – full SELECT, populates cache
list_a = list(qs)                            # NO SQL – uses the cache from len()
for o in qs:                                  # NO SQL – same cache
    pass

# But a fresh QuerySet doesn't share the cache:
qs2 = Order.objects.filter(status='captured')
print(qs2.count())                           # SQL – different QuerySet object

```

Q10 · How do you spot the N+1 problem in code review?

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Walmart Labs · Topic: N+1 detection · Django 3.x+

The question (interviewer's verbatim phrasing):

You're reviewing a Django pull request. What patterns in the code tell you there's probably an N+1 problem?

The 30-second answer: Five patterns. (1) A `for` loop over a QuerySet that touches `obj.<related_field>` inside the loop. (2) A DRF serializer with a nested serializer over a ForeignKey or reverse relation. (3) A Django template `{% for %}` that calls `{{ obj.related.field }}`. (4) An admin `list_display` that includes a related field without `list_select_related`. (5) Any code calling `obj.related_set.all()` inside a loop. If you see one of these and the queryset above isn't using `select_related` or `prefetch_related`, raise it on the review.

Step-by-step thought process: 1. Loop + dotted attribute access on a relation. The visual cue is `for o in orders: ... o.user.name ...`. Each dotted access fires a SELECT unless `select_related('user')` was on the queryset. 2. Nested serializers. `class OrderSerializer: user = UserSerializer()` reads `instance.user` per row. The fix is `OrderSerializer.setup_eager_loading` or to override `get_queryset` on the ViewSet to add `select_related('user')`. 3. Template relations. `{{ order.user.email }}` in a `{% for %}` block is N+1 unless the view passed in a `select_related` 'd queryset. 4. Admin pages. `ModelAdmin.list_display = ('id', 'user_email')` hits the user table per row unless `list_select_related = ('user',)` is set. This is the most-shipped N+1 in Indian Django stacks.

5. `obj.related_set.all()` in a loop. Reverse FK per parent — classic N+1. Fix with `prefetch_related('related_set')` upstream.

Edge cases to mention: - The `assertNumQueries` test helper is the cleanest way to lock the fix — once you assert "this endpoint runs in 3 queries," any regression breaks the test. - `django-debug-toolbar` shows the SQL panel with duplicate-query detection and stack traces in dev. Install it day one. - `django-silk` is the equivalent for production-shaped profiling — sampling instead of always-on.

What NOT to say: - "I'd run the code and check the response time" — too slow, won't fail in dev. Look for the patterns in the diff. - "N+1 is fine if N is small" — fine until it isn't. Make the code O(1) queries; future-you will thank you.

Common follow-up: "How do you assert in a test that an endpoint runs in exactly N queries?" — `self.assertNumQueries(N): response = self.client.get(...)`. The context manager captures `connection.queries` and asserts the count.

Stand-out line: "Say this and you'll separate yourself: N+1 is spotted by reading the diff, not by running the code — the patterns are visual, and once you've named them once, you'll catch them in every PR."

```
# Patterns to flag in review:

# 1. Loop + dotted relation
for order in Order.objects.filter(status='captured'):
    print(order.user.email)                                # N+1 unless select_related('user')

# 2. Reverse-FK inside a loop
for m in Merchant.objects.all():
    for o in m.orders.all():                                # N+1 unless prefetch_related('orders')
        ...

# Test that locks the fix
def test_order_list_is_one_query(self):
    Order.objects.create(...) ; Order.objects.create(...) ; Order.objects.create(...)
    with self.assertNumQueries(2):                            # 1 for orders, 1 for prefetch
        self.client.get('/api/orders/')

```

Section B — Models and Migrations

The five questions in this section are the ones interviewers use to separate "I've used Django models" from "I've owned a Django schema in production." Razorpay, Cred, and Zerodha lean on these hardest because their schemas have evolved across years of merger, migration, and feature growth — and they want to hire engineers who can extend a model, ship a migration, and not break the world. The themes that get tested: model fields and their `on_delete` options, custom managers as the primary "encapsulate

query logic" tool, signals and why they cause more outages than they solve, the `Meta` class, and migration trade-offs around zero-downtime, data migrations, and swappable models.

Q11 · `CharField` , `TextField` , `EmailField` , `JSONField` — when do you reach for each?

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Postman, Cognizant · Topic: model fields · Django 3.x+

The question (interviewer's verbatim phrasing):

Walk me through the difference between `CharField` , `TextField` , `EmailField` , and `JSONField` . When do you pick which?

The 30-second answer: `CharField` is bounded text with a required `max_length` , backed by `VARCHAR(n)` — use it for names, statuses, codes, short identifiers. `TextField` is unbounded text, backed by `TEXT` — use it for blog bodies, error logs, long descriptions. `EmailField` is `CharField` with built-in email validation. `JSONField` is native JSON (jsonb on Postgres, JSON on MySQL 5.7+, falls back to a text column with serialization on SQLite) — use it for semi-structured payloads where the schema may evolve, but never as a way to avoid designing a schema.

Step-by-step thought process: 1. Anchor on storage. `CharField(max_length=255)` is `VARCHAR(255)` on every backend. `TextField` is `TEXT` — no length limit, slightly slower to seek inside on some backends. 2. State the validation overlay. `EmailField` runs Django's email regex on `full_clean()` . `URLField` does URL validation. Both are conveniences over `CharField` with a validator — pick them for explicitness. 3. `JSONField` is the modern shape since Django 3.1, when it moved from `django.contrib.postgres` to core. On Postgres it uses `jsonb` and supports key-path lookups (`payload__user__email`). On MySQL 5.7+ it uses JSON. Falls back gracefully on SQLite for tests. 4. The picking rule. Bounded → `CharField` . Unbounded → `TextField` . Validated string subtype → `EmailField` / `URLField` / `SlugField` . Nested / variable schema → `JSONField` . Money — *never* `FloatField` ; use `DecimalField(max_digits=12, decimal_places=2)` . Times — `DateTimeField` ; with timezone support enabled.

Edge cases to mention: - On Postgres, `CharField` and `TextField` perform almost identically — `VARCHAR(n)` is just `TEXT` with a length check. On MySQL, `VARCHAR(255)` is indexed differently from `TEXT` , so the choice matters for indexing. - `JSONField` lookups can be indexed with a `GinIndex` on Postgres (see the SQL pack File 06 on `jsonb_path_ops` vs `jsonb_ops`). Don't index a `JSONField` unless you actually query into it. - `null=True` versus `blank=True` : `null` is database-level (the column is NULLABLE), `blank` is form-level (the field is optional in forms). For text fields, the convention is `blank=True` with default `''` rather than `null=True` , to avoid two "empty" states.

What NOT to say: - "Use TextField for everything to avoid migrations later" — wrong; CharField max_length is part of validation and schema hygiene. - "JSONField is for storing relations" — wrong; you lose foreign-key integrity and JOIN performance.

Common follow-up: "How do you add a unique constraint to an EmailField, case-insensitively, in Postgres?" — Either use CITEXT via django.contrib.postgres.fields.CICharField, or add a UniqueConstraint with a function-based index on Lower('email') in Meta.constraints.

Stand-out line: "Say this and you'll separate yourself: the field type is the first line of validation — EmailField, DecimalField, and JSONField exist so business invariants live in the schema, not in Python."

```
from django.db import models

class Merchant(models.Model):
    name = models.CharField(max_length=200)
    email = models.EmailField(unique=True)
    description = models.TextField(blank=True, default='')
    onboarded_at = models.DateTimeField(auto_now_add=True)
    payout_balance = models.DecimalField(max_digits=14, decimal_places=2, default=0)
    settings = models.JSONField(default=dict, blank=True)
```

Q12 · What is a custom manager, and when do you reach for one?

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Goldman Sachs India · Topic: managers · Django 3.x+

The question (interviewer's verbatim phrasing):

Tell me about Django managers. When would you write a custom one, and how is it different from a custom QuerySet method?

The 30-second answer: A manager is the entry point to the database for a model — Model.objects is the default. A custom manager encapsulates query logic ("active users," "captured payments from last week") so that callers don't reimplement the filter at every call site. The modern pattern is to write a custom QuerySet subclass with your methods, then expose them on the manager with MyQuerySet.as_manager() — that way the methods are chainable.

Step-by-step thought process: 1. State the role. The manager is what turns Order into something you can query. Calling Order.objects.all() is calling the default manager's all method. 2. Show the simple custom-manager pattern. class ActiveOrderManager(models.Manager): def get_queryset(self): return

`super().get_queryset().filter(deleted_at__isnull=True)` . Then `class Order:`
`objects = models.Manager(); active = ActiveOrderManager()` .

`Order.active.all()` filters out soft-deleted rows automatically. 3. Show the modern chainable-method pattern. Define a `QuerySet` subclass with named methods, then `objects =`

`MyQuerySet.as_manager()` . The methods now chain like

`Order.objects.captured().for_merchant(m).recent()` — composable, reusable, testable.

4. Cover the default-manager rule. The first manager defined on the model is the default manager — it's what migrations, ForeignKey reverse-relations, and the admin all use. If you replace `objects` with a manager that filters rows out, you'll lose those rows from migrations and reverse relations. Keep an unfiltered manager defined first.

Edge cases to mention: - Custom managers are the right place for `bulk_create` -style helper methods that group multiple model-level operations into a named method —

`Subscription.objects.renew_batch(rows)` is cleaner than calling a free function. - A custom `QuerySet` method that uses `self.filter(...)` chains; a custom *manager* method that returns a raw value (like a count or a single row) cannot chain. Use the `QuerySet` pattern for everything that returns a `QuerySet`. - Multiple managers on one model: `Order.objects = default Manager()` and `Order.captured = CapturedManager()` . The first one is the default for the ORM machinery.

What NOT to say: - "Just write a `@classmethod` on the model" — works for simple cases but doesn't chain, doesn't compose with other queriesets, doesn't show up in reverse relations. - "Managers are deprecated" — wrong; they're a core part of the ORM.

Common follow-up: "Write a custom `QuerySet` that supports

`Order.objects.captured().for_merchant(m).recent(days=7)` ." — Define a `OrderQuerySet` with three methods returning `self.filter(...)` , then `objects = OrderQuerySet.as_manager()` on the model.

Stand-out line: "Say this and you'll separate yourself: write the methods on a `QuerySet` subclass and expose with `as_manager()` — the chainability is what turns query logic into a domain vocabulary the next developer can read like English."

```
from django.db import models
from django.utils import timezone
from datetime import timedelta

class OrderQuerySet(models.QuerySet):
    def captured(self):
        return self.filter(status='captured')

    def for_merchant(self, merchant):
        return self.filter(merchant=merchant)

    def recent(self, days=7):
        return self.filter(created_at__gte=timezone.now() - timedelta(days=days))

class Order(models.Model):
    status = models.CharField(max_length=20)
    merchant = models.ForeignKey('Merchant', on_delete=models.PROTECT)
    created_at = models.DateTimeField(auto_now_add=True)
    objects = OrderQuerySet.as_manager()

# Chains cleanly
qs = Order.objects.captured().for_merchant(m).recent(days=14)
```

Q13 · Signals — `pre_save`, `post_save`, and why are they dangerous?

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Zerodha, JP Morgan India · Topic: signals · Django 3.x+

The question (interviewer's verbatim phrasing):

What's a Django signal? Give me an example, and tell me why senior engineers warn against them.

The 30-second answer: A signal is a publish-subscribe hook that fires on a lifecycle event — `pre_save`, `post_save`, `pre_delete`, `post_delete`, `m2m_changed`. You register a receiver function and Django calls it when the event fires. They're dangerous because the side effect lives in a file far from the model that triggers it; new developers can't see why a save mysteriously sends an email or writes a ledger entry. Most teams replace signals with explicit calls — a `model.save()` override or a service-layer function — once the codebase grows.

Step-by-step thought process: 1. State the mechanism. `from django.db.models.signals import post_save; @receiver(post_save, sender=Order) def on_order_save(sender, instance, created, **kwargs): ...`. Receivers must accept `**kwargs` because Django passes a moving set of arguments. 2. Show a common shape. After an Order is created, send a

confirmation email or write to a ledger. After a User is saved, create or update a related Profile. 3. Move to the why-dangerous list. (a) Hidden side effects — the signal receiver lives in `signals.py`, the model lives in `models.py`, the import that registers the receiver lives in `apps.py`'s `ready()` method. New devs spend an hour finding the cause of an audit-log write. (b) Test brittleness — every test that touches the model fires the signal; mocking is fiddly. (c) `bulk_create` and `update()` bypass *signals* entirely, so any guarantee you thought you had is silently broken in batch jobs. 4. State the modern alternative. Explicit. Write a `Service` class or override `Model.save()` and call the side effect directly. The call graph is in the file you're reading; the audit-log write is impossible to miss.

Edge cases to mention: - `transaction.on_commit(callback)` is the right way to defer a signal-style side effect until the transaction commits. Sending an email in `post_save` without `on_commit` means rolled-back transactions still send emails. - `pre_save` runs before the SQL INSERT/UPDATE — you can mutate `instance` and the changes will be saved. `post_save` runs after the SQL — your mutations won't persist unless you call `instance.save()` again, which is a recursion trap. - Signals defined on an abstract base model fire for every concrete subclass — sometimes desired, often surprising.

What NOT to say: - "Signals are best practice" — wrong stance for any serious Django codebase. They're a useful tool, not the default. - "Signals are async" — wrong. They run synchronously in the same transaction as the trigger. Use Celery for actual async.

Common follow-up: *"How would you refactor a `post_save` signal that sends an email into something more testable and explicit?"* — Move the email logic into a service function called from the view after `model.save()`, wrap the whole thing in `transaction.atomic`, and defer the email with `transaction.on_commit`. Tests then mock the service function directly.

Stand-out line: *"Say this and you'll separate yourself: signals are an implicit edge in the call graph — explicit service calls make the side effect impossible to miss when reading the file, which is the whole audit trail every senior engineer wants."*

```

from django.db.models.signals import post_save
from django.dispatch import receiver
from django.db import transaction

# Signal – the implicit version
@receiver(post_save, sender=Order)
def on_order_created(sender, instance, created, **kwargs):
    if created:
        transaction.on_commit(
            lambda: send_order_email.delay(instance.id)
        )

# Modern replacement – explicit service function called from the view
def create_order(merchant, amount):
    with transaction.atomic():
        order = Order.objects.create(merchant=merchant, amount=amount)
        LedgerEntry.objects.create(order=order, debit=amount)
        transaction.on_commit(lambda: send_order_email.delay(order.id))
    return order

```

Q14 · The `Meta` class — what lives there and why?

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Postman, Wipro · Topic: Meta class · Django 3.x+

The question (interviewer's verbatim phrasing):

What's the `class Meta` inside a Django model for? Name the options you've actually used.

The 30-second answer: `Meta` is the model's "settings" inner class — options that don't fit as fields. The ones you'll use weekly: `ordering` for default sort, `verbose_name` / `verbose_name_plural` for admin labels, `db_table` to override the table name, `indexes` for custom indexes, `constraints` for `UniqueConstraint` and `CheckConstraint`, `abstract = True` for base models, `unique_together` (legacy — use `UniqueConstraint`), `permissions` for custom auth permissions. Nothing in `Meta` is a column; everything is configuration.

Step-by-step thought process: 1. Frame the role. `Meta` carries options that describe the model — defaults, constraints, indexes, labels — without being a field on it. 2. The greatest-hits list. `ordering = ['-created_at']` sets the default sort order. `indexes = [models.Index(fields=['merchant', 'created_at'])]` adds a composite index. `constraints = [models.UniqueConstraint(fields=['merchant', 'reference_id'], name='unique_merchant_reference')]` is the modern way to declare uniqueness — `unique_together` is legacy. 3. Call out `abstract = True`. Inheriting from an abstract base

copies fields and `Meta` options to children but does *not* create a database table for the parent. Use for `TimestampedModel` and similar shared base classes. 4. `db_table` to override the default `<app_label>_<model_name>`. Useful when migrating a legacy schema or matching an external table name.

Edge cases to mention: - `ordering` adds an `ORDER BY` to *every* query by default — costly if the field isn't indexed. Don't set it unless the cost is justified. - Children of an abstract base inherit `Meta` *unless* they define their own; if they do, they have to manually carry forward parent options they want. - `UniqueConstraint` supports a `condition` argument for partial-unique constraints — `UniqueConstraint(fields=['email'], condition=Q(deleted_at__isnull=True), name='unique_active_email')`. Cleaner than a "deleted-aware" Python check.

What NOT to say: - "`Meta` is for putting fields you don't want to expose" — wrong; fields go on the model, not in `Meta`. - "Always set ordering to make queries predictable" — wrong; default ordering is a perf footgun.

Common follow-up: "How do you declare a check constraint that says `amount > 0` on a model?" — `Meta.constraints = [models.CheckConstraint(check=Q(amount__gt=0), name='order_amount_positive')]`. Django generates the SQL `CHECK (amount > 0)`.

Stand-out line: "Say this and you'll separate yourself: `Meta.constraints` and `Meta.indexes` are how you put invariants into the schema where Postgres enforces them — Python-level checks fail silently under `bulk_create`, but a database constraint never does."

```
from django.db import models
from django.db.models import Q

class TimestampedModel(models.Model):
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        abstract = True

class Order(TimestampedModel):
    merchant = models.ForeignKey('Merchant', on_delete=models.PROTECT)
    reference_id = models.CharField(max_length=64)
    amount = models.DecimalField(max_digits=12, decimal_places=2)
    status = models.CharField(max_length=20)

    class Meta:
        ordering = ['-created_at']
        indexes = [
            models.Index(fields=['merchant', 'created_at']),
            models.Index(fields=['status']),
        ]
        constraints = [
            models.UniqueConstraint(
                fields=['merchant', 'reference_id'],
                name='unique_merchant_reference',
            ),
            models.CheckConstraint(
                check=Q(amount__gt=0),
                name='order_amount_positive',
            ),
        ]
```

Q15 · Migrations — zero-downtime patterns, data migrations, swappable models.

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Zerodha, JP Morgan India, Walmart Labs · Topic: migrations · Django 3.x+

The question (interviewer's verbatim phrasing):

You need to rename a column on a table with 50 million rows that takes traffic 24/7. How do you ship that migration without downtime?

The 30-second answer: You don't rename. You ship it as a multi-step migration: add the new column (nullable, default null), backfill in chunks via a data migration or background job, deploy code that writes to both columns and reads from the new one, deploy code that reads only from the new column, drop

the old column in a later migration. Each step is independently reversible and never holds an ACCESS EXCLUSIVE lock for more than a few seconds. The same pattern applies to most schema changes on hot tables.

Step-by-step thought process: 1. Frame the cost. On Postgres, `ALTER TABLE ... RENAME COLUMN` takes an `ACCESS EXCLUSIVE` lock — fast, but it blocks every concurrent read and write for the duration. `ALTER TABLE ... ADD COLUMN ... DEFAULT <value>` rewrites the table on older Postgres (12 fixed that for constant defaults). Backfilling a column on 50M rows in one UPDATE locks rows en masse. 2. The expand-migrate-contract recipe. (a) Expand — add the new column nullable, keep the old. (b) Migrate — backfill in chunks (`Model.objects.filter(new_col__isnull=True)[:1000]` updated in a loop, with sleeps). Deploy app code that dual-writes. (c) Contract — switch reads to the new column, then drop the old column in a separate migration. 3. Data migrations. `python manage.py makemigrations --empty appname` generates an empty migration; inside, use `RunPython(forwards, backwards)` for code-driven data changes. Always provide a backward function — even if it's a no-op — so rollbacks work. 4. Swappable models. `AUTH_USER_MODEL = 'myapp.User'` lets you replace the built-in User without forking. Reference it in your ForeignKeys as `settings.AUTH_USER_MODEL`, never `auth.User`, so apps remain portable across projects.

Edge cases to mention: - Adding a column with a non-null default still rewrites the table on Postgres < 11 — chunk the backfill. Postgres 11+ optimised constant defaults to a metadata change. - A `RunPython` data migration runs in a transaction by default; for backfills on large tables, set `atomic = False` on the migration class and chunk the updates yourself. - Constraint additions — `ADD CONSTRAINT ... NOT VALID` followed by `VALIDATE CONSTRAINT` is the zero-downtime way on Postgres. Django doesn't emit this out of the box; write a `RunSQL` migration.

What NOT to say: - "Take a maintenance window" — fine for hobby projects, not for Razorpay-scale. The interview is testing whether you know the multi-step alternative. - "Just rename in code with a `db_column` mapping" — partial fix; you still need a migration when you eventually drop the old column.

Common follow-up: "How do you safely add a `NOT NULL` constraint to an existing nullable column with 50M rows?" — Add a check constraint as NOT VALID first, backfill nulls in chunks, then VALIDATE the constraint, then alter the column to NOT NULL. Three migrations, no long lock.

Stand-out line: "Say this and you'll separate yourself: expand-migrate-contract is the zero-downtime contract every senior engineer ships — never rename, always shadow, and never hold an ACCESS EXCLUSIVE lock on a hot table."


```
# 0042_add_new_column.py – expand step
from django.db import migrations, models

class Migration(migrations.Migration):
    dependencies = [('orders', '0041_previous')]
    operations = [
        migrations.AddField(
            model_name='order',
            name='reference_uuid',
            field=models.UUIDField(null=True, blank=True),
        ),
        migrations.AddIndex(
            model_name='order',
            index=models.Index(fields=['reference_uuid'], name='ord_ref_uuid_idx'),
        ),
    ]

# 0043_backfill_reference_uuid.py – migrate step
import uuid
from django.db import migrations

def forwards(apps, schema_editor):
    Order = apps.get_model('orders', 'Order')
    qs = Order.objects.filter(reference_uuid__isnull=True)
    while True:
        batch = list(qs[:1000])
        if not batch:
            break
        for o in batch:
            o.reference_uuid = uuid.uuid4()
        Order.objects.bulk_update(batch, ['reference_uuid'], batch_size=500)

def backwards(apps, schema_editor):
    pass # no-op; keeping the backfilled values is harmless

class Migration(migrations.Migration):
    atomic = False # chunk updates outside a single transaction
    dependencies = [('orders', '0042_add_new_column')]
    operations = [migrations.RunPython(forwards, backwards)]
```

Section C — Middleware and Views

Middleware and views are where the request lifecycle gets tested. The five questions in this section are the ones interviewers reach for when they want to check that the candidate understands what Django actually does between `wsgi.py` and your view function — and what happens on the way back. The shape of the round-one questions is consistent across Razorpay, Cred, Postman, and the Service MNCs: middleware execution order, class-based versus function-based views, `LoginRequiredMixin`, the

`dispatch` method, and `get_context_data`. Answer these and you've cleared the framework round.

Q16 · Middleware — what is it and what is the execution order?

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Goldman Sachs India, Cognizant · Topic: middleware · Django 3.x+

The question (interviewer's verbatim phrasing):

What is Django middleware? If I have `MIDDLEWARE = [A, B, C]` in settings, in what order do they run for a request and a response?

The 30-second answer: Middleware is a chain of callables that wrap every request and response. On the way in, they run top to bottom — `A`, then `B`, then `C` — each one calling the next via `get_response(request)`. The view runs after `C`. On the way out, the response unwinds in reverse — `C` first, then `B`, then `A`. Think of it as a Russian-doll: each middleware wraps the next, so what goes in first comes out last.

Step-by-step thought process: 1. State the shape. Modern middleware (since Django 1.10) is a class with `__init__(self, get_response)` and `__call__(self, request)`. The `__call__` typically reads `request`, calls `response = self.get_response(request)`, and returns the (possibly modified) `response`. 2. Walk the order. Request goes through `A.__call__`, which calls `get_response`, which is `B.__call__`, which calls `get_response`, which is `C.__call__`, which calls `get_response`, which is the view. Each `__call__` runs its pre-view code, calls the next, then runs its post-view code on the way back. 3. Mention the legacy hooks. `process_view(request, view_func, view_args, view_kwargs)`, `process_exception(request, exception)`, and `process_template_response(request, response)` still work — Django calls them at specific points if defined. `process_view` fires after URL resolution but before the view; useful for permission checks. 4. State the practical implication. Put expensive work (database queries, auth lookups) low in the stack so cheap rejects (HTTPS redirect, security headers) happen first. `SecurityMiddleware` and `SessionMiddleware` come early; custom audit logging late.

Edge cases to mention: - An exception inside the view skips the remaining middleware's normal response path; instead, each middleware's `process_exception` is called from the bottom up. If none handles it, Django renders the 500 page. - Returning a response from a middleware *before* calling `get_response` short-circuits — the view never runs. Useful for rate-limit middleware that aborts with `HttpResponse(status=429)`. - Async middleware: Django 3.1+ supports async middleware. Mark with `async_capable = True` and define `__call__` as `async`. Mixing sync and async middleware is allowed; Django adapts via thread-pools.

What NOT to say: - "Middleware runs in alphabetical order" — wrong; the `MIDDLEWARE` list order is authoritative. - "Middleware runs once per server start" — wrong; once per request.

Common follow-up: "Where in the middleware list does `AuthenticationMiddleware` go and why?" — After `SessionMiddleware` (it depends on `request.session`) and before any view-permission middleware (so the view sees `request.user`). The default `MIDDLEWARE` order in `django-admin startproject` is the canonical reference.

Stand-out line: "Say this and you'll separate yourself: middleware is a Russian-doll, not a pipeline — what wraps the request first unwraps the response last, and getting the auth-after-session order wrong is the silent 500 that ships on Friday."

```
# settings.py
MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',          # HTTPS redirect, headers (cheap)
    'django.contrib.sessions.middleware.SessionMiddleware',   # populates request.session
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware', # populates request.user — depends
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
    'orders.middleware.RequestIdMiddleware',                  # custom — sets X-Request-ID
]

# A simple custom middleware
class RequestIdMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        request.request_id = generate_uuid()
        response = self.get_response(request)
        response['X-Request-ID'] = request.request_id
        return response
```

Q17 · Class-based versus function-based views — when do you reach for each?

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, TCS, Infosys · Topic: views · Django 3.x+

The question (interviewer's verbatim phrasing):

Class-based or function-based views — which one do you reach for, and why?

The 30-second answer: Function-based views are simpler to read at a glance — a function takes a `request`, returns a `response`. Use them for one-off endpoints, simple API views, or anything where

the logic is mostly procedural. Class-based views shine when you have *patterns* — CRUD across many models, mixins for auth or permissions, structured form handling. The honest answer for the Indian backend market in 2026: FBVs for the explicit, CBVs when you reach for the generic ones (`ListView` , `DetailView` , `CreateView`) to avoid rewriting CRUD boilerplate. For DRF, you use CBVs whether you wanted to or not.

Step-by-step thought process: 1. Define each. An FBV is a plain function — `def my_view(request): return HttpResponse(...)` . A CBV is a class that inherits from `View` (or one of its subclasses); URL routing uses `MyView.as_view()` to get a callable. 2. The strength of FBVs. Readable top to bottom. No inheritance to chase. Easy to refactor. Good for narrow, one-off endpoints. 3. The strength of CBVs. Inheritance lets you compose behaviour via mixins — `LoginRequiredMixin` , `UserPassesTestMixin` , `PermissionRequiredMixin` . The generic CBVs — `ListView` , `DetailView` , `CreateView` , `UpdateView` , `DeleteView` — encapsulate the most common CRUD patterns with sensible hooks (`get_queryset` , `get_context_data`). 4. The decision rule. Reach for CBVs when (a) you're doing CRUD on a model, (b) you'd duplicate auth/permission code across multiple views, (c) you want the structured form-handling of `FormView` . Reach for FBVs otherwise. Avoid the impulse to convert all FBVs to CBVs for the sake of consistency — readability beats uniformity.

Edge cases to mention: - DRF's `APIView` and `ViewSet` are CBVs by design — most Indian API codebases on Django are CBV-heavy via DRF. - Async views are easier with FBVs in Django 4.x — `async def my_view(request): ...` . CBVs added async support via `View.as_view()` recognising async `dispatch` in Django 4.2+. - CBV mixins can be a maintainability trap when too many mixins compose. The MRO becomes a debugging exercise; flatten when it stops being readable.

What NOT to say: - "CBVs are best practice" — incomplete. They're a tool for shared patterns, not a default. - "FBVs are deprecated" — wrong; both are first-class in modern Django.

Common follow-up: "Convert this FBV into a CBV using `ListView` ." — Be ready to walk through `model = X` , `queryset = X.objects.filter(...)` , `paginate_by = N` , `template_name = '...'` , and `get_context_data` .

Stand-out line: "Say this and you'll separate yourself: pick the view shape that makes the next reader's job easier — FBVs for one-offs, generic CBVs for CRUD patterns, and never convert one to the other for the sake of uniformity."

```
# FBV – explicit and direct
def order_list(request):
    qs = Order.objects.select_related('merchant').filter(status='captured')[:50]
    return render(request, 'orders/list.html', {'orders': qs})

# CBV – same thing using ListView
from django.views.generic import ListView

class OrderListView(ListView):
    model = Order
    template_name = 'orders/list.html'
    context_object_name = 'orders'
    paginate_by = 50

    def get_queryset(self):
        return Order.objects.select_related('merchant').filter(status='captured')

# URL wiring
path('orders/', OrderListView.as_view(), name='order-list'),
```

Q18 · `LoginRequiredMixin` — how does it work, and what's the gotcha?

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Postman, Accenture · Topic: auth mixins · Django 3.x+

The question (interviewer's verbatim phrasing):

How do you require login on a class-based view? Show me the mixin and the gotcha about its position.

The 30-second answer: Inherit from `LoginRequiredMixin` *before* the view class — `class OrderListView(LoginRequiredMixin, ListView): ...`. The mixin's `dispatch` checks `request.user.is_authenticated`; if not, it redirects to `LOGIN_URL`. The gotcha is the order in the MRO: `LoginRequiredMixin` must come *first* (leftmost) so its `dispatch` runs before the view's `dispatch`. Putting it after `ListView` does nothing because Python's MRO walks left to right.

Step-by-step thought process: 1. State the FBV alternative briefly. `@login_required` decorator on a function view does the same thing. 2. CBV mechanics. `LoginRequiredMixin` is a thin class that defines `dispatch(self, request, *args, **kwargs)`. Its `dispatch` checks `request.user.is_authenticated`; if false, returns a redirect; if true, calls `super().dispatch(...)`. 3. MRO matters. Python resolves methods by walking the class hierarchy left to right. `(LoginRequiredMixin, ListView)` means `dispatch` is resolved on `LoginRequiredMixin` first. `(ListView, LoginRequiredMixin)` resolves on `ListView` first.

— and `ListView`'s `dispatch` never calls the mixin's, so the auth check is skipped. 4.
 Configuration knobs. `LOGIN_URL` in settings (or `login_url` on the class) controls the redirect target. `redirect_field_name` controls the `?next=` parameter; `raise_exception = True` makes the mixin raise `PermissionDenied` instead of redirecting — useful for APIs.

Edge cases to mention: - For DRF API views, use `permission_classes = [IsAuthenticated]` instead — `LoginRequiredMixin` redirects to a login URL, which is wrong for a JSON API. - Combine with `UserPassesTestMixin` for arbitrary predicates: override `test_func(self): return self.request.user.merchant == self.get_object().merchant`. Comes after `LoginRequiredMixin` in the MRO. - `PermissionRequiredMixin` checks Django's permission system: `permission_required = 'orders.view_order'`. Also after `LoginRequiredMixin`.

What NOT to say: - "The order in the MRO doesn't matter" — wrong; it's the whole point of the question. - "`LoginRequiredMixin` blocks anonymous users from the whole site" — wrong; only the views that inherit from it.

Common follow-up: "What's the equivalent for an FBV?" — The `@login_required` decorator from `django.contrib.auth.decorators`. Use `@method_decorator(login_required)` on `dispatch` if you really need to mix paradigms on a CBV.

Stand-out line: "Say this and you'll separate yourself: MRO position is the bug — `LoginRequiredMixin` after the view class compiles fine and silently lets anonymous traffic through, which is the security CVE shape every Django auditor reads first."

```
from django.contrib.auth.mixins import LoginRequiredMixin
from django.views.generic import ListView

# Correct — mixin first
class OrderListView(LoginRequiredMixin, ListView):
    model = Order
    login_url = '/accounts/login/'
    redirect_field_name = 'next'

# WRONG — silently broken (mixin after view; its dispatch never runs)
class BrokenOrderListView(ListView, LoginRequiredMixin):
    model = Order
```

Q19 · The `dispatch` method — what's it for and when do you override it?

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, JP Morgan India · Topic: CBV internals · Django 3.x+

The question (interviewer's verbatim phrasing):

What does `dispatch` do on a Django CBV, and when would you override it?

The 30-second answer: `dispatch(self, request, *args, **kwargs)` is the entry point on every class-based view. It looks at `request.method`, finds the matching handler (`get`, `post`, `put`, etc.) on the class, and calls it. You override `dispatch` when you want a check or transformation that runs before any HTTP-method-specific code — auth, rate-limiting, redirecting on a model-state check, or any "guard" that applies to every request to this view.

Step-by-step thought process: 1. Frame the role. The base `View.dispatch` does method lookup: `handler = getattr(self, request.method.lower(), self.http_method_not_allowed); return handler(request, *args, **kwargs)`. That's it. 2. Show the typical override shape. `def dispatch(self, request, *args, **kwargs): if some_check: return HttpResponseForbidden(...); return super().dispatch(request, *args, **kwargs)`. The pre-check runs once for any HTTP method. 3. Common reasons to override. Auth checks for non-standard sessions (e.g., a webhook signature verifier instead of `LoginRequiredMixin`). Logging the request before the handler runs. Mutating `request` (attaching a tenant from a subdomain). Rate-limiting that needs the resolved view context. 4. Note the alternative. If your guard fits the mixin pattern (auth, permission, predicate), prefer a mixin — it's more reusable. Override `dispatch` only when the check is view-specific or doesn't fit the mixin shape.

Edge cases to mention: - Always call `super().dispatch(...)` unless you are deliberately short-circuiting — otherwise the view doesn't run. - Overriding `dispatch` and using `LoginRequiredMixin` both override `dispatch`; the MRO determines which runs first. `LoginRequiredMixin` calls `super().dispatch` after its check, so your `dispatch` runs next. - Async `dispatch` is supported in Django 4.2+; mark with `async def dispatch(...)` and the framework adapts.

What NOT to say: - "`dispatch` is only called for AJAX requests" — wrong; it's called for every request. - "Don't override `dispatch`; override `get` and `post` separately" — sometimes right, but not when the guard is shared across HTTP methods.

Common follow-up: "How would you log every request to a CBV with the elapsed time?" — Override `dispatch` to record `start = time.perf_counter()`, call `super().dispatch`, then log `time.perf_counter() - start` with the request method and path.

Stand-out line: "Say this and you'll separate yourself: `dispatch` is the only place that runs once for every HTTP method — guards that apply to GET and POST equally belong here, never duplicated across handler methods."

```

from django.views.generic import View
from django.http import JsonResponse
import time, logging

logger = logging.getLogger(__name__)

class WebhookView(View):
    def dispatch(self, request, *args, **kwargs):
        # Verify webhook signature before any handler runs
        if not self.verify_signature(request):
            return JsonResponse({'error': 'invalid signature'}, status=403)

        start = time.perf_counter()
        try:
            return super().dispatch(request, *args, **kwargs)
        finally:
            elapsed_ms = (time.perf_counter() - start) * 1000
            logger.info('webhook %s %s in %.1fms', request.method, request.path, elapsed_ms)

    def post(self, request, *args, **kwargs):
        # method-specific handling runs only if dispatch passed signature check
        ...

    def verify_signature(self, request):
        ...

```

Q20 · `get_context_data` — when do you override it?

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Cognizant, Postman · Topic: CBV templates · Django 3.x+

The question (interviewer's verbatim phrasing):

What is `get_context_data`, and when would you override it in a class-based view?

The 30-second answer: `get_context_data(self, **kwargs)` returns the dictionary passed to the template as context. On generic views (`ListView`, `DetailView`, `TemplateView`) it already gives you the model's queryset or instance under a sensible key. You override it to add extra values that the template needs but the base CBV doesn't compute — counts, related objects, form instances, anything precomputed for the template that isn't the main queryset.

Step-by-step thought process: 1. State the base behaviour. `ListView.get_context_data` returns `{'object_list': queryset, 'page_obj': ..., 'paginator': ...}`. `DetailView` returns `{'object': instance, '<model>': instance}`. 2. Walk through the override pattern. Call `super().get_context_data(**kwargs)` to get the base dict, mutate it with

the extra keys, return it. Always merge — never replace — so you don't lose the base context. 3. Show a common shape. A merchant detail page where the template needs the merchant *and* the recent payouts *and* the dispute count: `context['payouts'] = self.object.payouts.recent(); context['dispute_count'] = self.object.disputes.unresolved().count()`. The view encapsulates the data prep. 4. Mention the alternative for one-off keys. You can pass them via `extra_context = {...}` on the class — fine for static values, useless for dynamic ones because it's evaluated at import time.

Edge cases to mention: - `get_context_data` is called once per request, after `get_queryset` (for `ListView`) or `get_object` (for `DetailView`). `self.object` is set by then in `DetailView`. - Database queries in `get_context_data` count against the view's query budget — don't fetch what the template doesn't render. - For DRF, the analogue is `get_serializer_context` for context that the serializer needs (current user, request).

What NOT to say: - "`get_context_data` is just for forms" — wrong; it's for any extra template context. - "Pass extra context via `request.session`" — wrong place; sessions are for cross-request state.

Common follow-up: "How would you add a `pending_dispute_count` to every merchant detail page without duplicating code across views?" — Either subclass `DetailView` once with the shared `get_context_data` and inherit from it, or add a `merchant_context` template tag that the template renders directly without view-side prep.

Stand-out line: "Say this and you'll separate yourself: always `super().get_context_data` then merge — never replace, because the base CBV is the one populating pagination, `object_list`, and form state the template silently depends on."

```
from django.views.generic import DetailView

class MerchantDetailView(DetailView):
    model = Merchant
    template_name = 'merchants/detail.html'

    def get_queryset(self):
        return Merchant.objects.select_related('owner').prefetch_related('tags')

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context['recent_payouts'] = self.object.payouts.recent(days=30)
        context['pending_dispute_count'] = self.object.disputes.filter(status='pending').count()
        context['settings_form'] = MerchantSettingsForm(instance=self.object)
        return context
```

Section D — Common Pitfalls

The five questions here are the production-incident questions — the bugs that ship to staging, look fine, and bite in production when traffic doubles or an environment variable is missing. Every Indian backend interviewer has lived through at least one of these; they ask about them because they want to hire engineers who won't repeat them. The themes: the `get_or_create` race condition, `settings.py` organization across environments, the secret key in version control, `ALLOWED_HOSTS`, and `DEBUG = True` in production. Each of these has shipped a CVE somewhere; each of these is a one-liner to fix once you know it exists.

Q21 · `get_or_create` — when is it race-prone and how do you fix it?

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Zerodha, JP Morgan India, Walmart Labs · Topic: race conditions · Django 3.x+

The question (interviewer's verbatim phrasing):

```
Subscription.objects.get_or_create(user=u, defaults={'plan':  
'starter'})
```

 — what's the race condition, and how do you make it safe?

The 30-second answer: Between the SELECT and the INSERT, another request can insert the same row. Both requests do `SELECT WHERE user_id = u` → no row found → both INSERT → one wins, the other gets `IntegrityError`. The fix has three good shapes: (1) wrap the column in a `UniqueConstraint` and catch `IntegrityError` — Django's `get_or_create` does this internally, but you must declare the uniqueness. (2) Use `update_or_create` which is safer because it tries the INSERT first then falls back to UPDATE. (3) Take a `select_for_update` row lock on a parent row before the get-or-create, so only one transaction at a time runs the check. For the simple case, (1) is enough; for transactional flows where you need stronger guarantees, (3) is the gold standard.

Step-by-step thought process: 1. Diagnose the race. `get_or_create` does a SELECT, then if nothing found, an INSERT. Between those two steps, anything can happen on another connection — including the row you're about to insert appearing. 2. State Django's actual behaviour. Django's `get_or_create` catches `IntegrityError` and retries the SELECT, so on a properly-declared unique constraint it returns the other transaction's row. *Without* the unique constraint, the `IntegrityError` propagates and you crash. 3. The unique-constraint requirement is the load-bearing piece. Without `UniqueConstraint(fields=['user'], name='unique_subscription_user')` on `Subscription`, two concurrent `get_or_create` calls can both insert two different rows. 4. State `update_or_create` as the cleaner pattern when you want the values updated either way — common for "upsert" flows where the input values are authoritative. 5. The strongest guarantee. `with transaction.atomic():`

`Subscription.objects.select_for_update().filter(user=u).first()` then proceed. The lock blocks concurrent transactions on the matching rows.

Edge cases to mention: - `get_or_create` returns `(object, created)` — the boolean tells you if the row was inserted or already existed. - `select_for_update` requires being inside `transaction.atomic` — Django enforces this. - On Postgres, you can also use `bulk_create(..., update_conflicts=True)` (Django 4.1+) for a single-statement upsert, which is race-safe by design.

What NOT to say: - "Use a Python lock" — wrong layer; doesn't cross processes. - "Wrap it in `transaction.atomic` and it's safe" — wrong; atomic alone doesn't prevent two transactions from both reading "no row" simultaneously.

Common follow-up: "Rewrite this to use Postgres `INSERT ... ON CONFLICT DO UPDATE` via Django 4.1+ `bulk_create` ." —

`Subscription.objects.bulk_create([Subscription(user=u, plan='starter')], update_conflicts=True, update_fields=['plan'], unique_fields=['user'])` . One SQL statement, atomic.

Stand-out line: "Say this and you'll separate yourself: `get_or_create` is only race-safe when paired with a `UniqueConstraint` — without it, two concurrent calls produce two rows and the bug ships silently until billing notices."

```
from django.db import IntegrityError, transaction
from django.db.models import UniqueConstraint

# Step 1 – declare uniqueness so get_or_create can recover
class Subscription(models.Model):
    user = models.ForeignKey('User', on_delete=models.CASCADE)
    plan = models.CharField(max_length=20)

    class Meta:
        constraints = [
            UniqueConstraint(fields=['user'], name='unique_subscription_user'),
        ]

# Step 2 – get_or_create with the constraint in place, race-safe
sub, created = Subscription.objects.get_or_create(
    user=u,
    defaults={'plan': 'starter'},
)

# Strongest guarantee – select_for_update
with transaction.atomic():
    existing = Subscription.objects.select_for_update().filter(user=u).first()
    if existing is None:
        sub = Subscription.objects.create(user=u, plan='starter')
    else:
        sub = existing
```

Q22 · `settings.py` organisation — how do you split per environment?

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, TCS, Infosys, Wipro · Topic: settings · Django 3.x+

The question (interviewer's verbatim phrasing):

A single `settings.py` works for a hobby project but breaks in production. How do you organise settings across dev, staging, and prod?

The 30-second answer: Either a folder-style split (`settings/__init__.py` , `settings/base.py` , `settings/dev.py` , `settings/prod.py` , each importing from `base` and overriding what changes) or a single `settings.py` plus a typed config layer like `django-environ` , `pydantic-settings` , or `dynaconf` that reads from environment variables. The folder pattern is the Django convention; the env-var pattern is the 12-factor app convention. Combine both — folder-split settings that load secrets and toggles from env vars — for the production-grade setup.

Step-by-step thought process: 1. State the failure mode. One `settings.py` with all envs in `if DEBUG: ...` blocks ends up with secrets in the file, hardcoded hosts, and dev-only middleware that makes it to production. 2. The folder split. `settings/base.py` has everything common. `settings/dev.py` does `from .base import *` and overrides `DEBUG = True` , `DATABASES = {sqlite}` , etc. `settings/prod.py` does the same with the production overrides. `DJANGO_SETTINGS_MODULE` env var selects which one — set it to `myproject.settings.dev` locally, `myproject.settings.prod` in production. 3. The env-var layer. Use `django-environ` or `os.environ` to read `DATABASE_URL` , `SECRET_KEY` , `ALLOWED_HOSTS` , `EMAIL_BACKEND_DSN` , etc., from the environment. Never put secrets in any settings file — even `prod.py` should reference env vars for sensitive values. 4. Folder pattern + env vars is the production-grade composition. `base.py` reads `os.environ['SECRET_KEY']` — fails loudly if missing in production. `prod.py` adds `SECURE_HSTS_SECONDS` , `SECURE_SSL_REDIRECT` , etc.

Edge cases to mention: - The `python manage.py runserver --settings=myproject.settings.dev` flag is the per-command override. - For per-developer differences, a `settings/local.py` (gitignored) that imports from `dev` and overrides further is the conventional escape hatch. - For container envs, `python manage.py check --deploy` runs Django's deployment checklist and flags things like `DEBUG=True` and missing `SECURE_HSTS_SECONDS` .

What NOT to say: - "Use `if os.environ.get('PROD'): ...` inside one `settings.py`" — works for two settings, breaks the moment you have three environments or thirty toggles. - "Settings shouldn't depend on env vars" — wrong stance; 12-factor and prod-ready Django are env-var first.

Common follow-up: "What goes in `base.py` versus `prod.py` ?" — `base.py` has `INSTALLED_APPS` , `MIDDLEWARE` , `TEMPLATES` , the parts that don't change. `prod.py` overrides

`DEBUG`, `ALLOWED_HOSTS`, `DATABASES`, `SECURE_*` settings, `EMAIL_BACKEND` to point at the real provider.

Stand-out line: *"Say this and you'll separate yourself: folder-split for structure, env vars for secrets — the file that ships to the repo never knows the production database password, and `python manage.py check --deploy` is the CI gate that proves it."*

```
# settings/base.py
import os
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent.parent

# Secrets — always from env, fail loudly if missing
SECRET_KEY = os.environ['DJANGO_SECRET_KEY']

INSTALLED_APPS = ['django.contrib.admin', 'orders', 'merchants', ...]

MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    ...
]

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': os.environ['DB_NAME'],
        'USER': os.environ['DB_USER'],
        'PASSWORD': os.environ['DB_PASSWORD'],
        'HOST': os.environ.get('DB_HOST', 'localhost'),
        'PORT': os.environ.get('DB_PORT', '5432'),
    }
}

# settings/dev.py
from .base import *
DEBUG = True
ALLOWED_HOSTS = ['localhost', '127.0.0.1', '*.ngrok.io']
INSTALLED_APPS += ['debug_toolbar']
MIDDLEWARE = ['debug_toolbar.middleware.DebugToolbarMiddleware', *MIDDLEWARE]
INTERNAL_IPS = ['127.0.0.1']

# settings/prod.py
from .base import *
DEBUG = False
ALLOWED_HOSTS = os.environ['DJANGO_ALLOWED_HOSTS'].split(',')
SECURE_SSL_REDIRECT = True
SECURE_HSTS_SECONDS = 31536000
SECURE_HSTS_INCLUDE_SUBDOMAINS = True
SECURE_HSTS_PRELOAD = True
SESSION_COOKIE_SECURE = True
CSRF_COOKIE_SECURE = True
```

Q23 · The `SECRET_KEY` — what is it, and why must it never be in version control?

Tags: Difficulty: Easy · Asked at: Razorpay, Cred, Postman, TCS, Wipro · Topic: security · Django 3.x+

The question (interviewer's verbatim phrasing):

What does Django's `SECRET_KEY` do, and what's the threat model if it leaks?

The 30-second answer: `SECRET_KEY` is the cryptographic master key Django uses to sign session cookies, password-reset tokens, CSRF tokens, the messages framework's cookie store, and any `signing.Signer`-stamped data in your app. If it leaks, an attacker can forge a session cookie for any user — including admins — and bypass authentication entirely. It must live in environment variables or a secrets manager, never in `settings.py` checked into git. If it does leak, rotate it immediately; every signed token issued before the rotation is now invalid (which is what you want).

Step-by-step thought process: 1. State what it signs. Session cookies — Django serialises and signs the session ID. Password-reset URLs — token signed with `SECRET_KEY`. CSRF tokens — derived from `SECRET_KEY`. The messages framework's cookie storage. `signing.Signer` everywhere a developer touches it. 2. The threat model. Anyone with `SECRET_KEY` can forge a session cookie that claims to be `user_id=1` (often the admin). They can also forge password-reset links that grant access without ever sending the user an email. CSRF protection is also undermined. 3. State the storage rule. Environment variable, AWS Secrets Manager, GCP Secret Manager, HashiCorp Vault, your secrets layer of choice. Never `.py` checked into git — even private repos leak via forks, employee laptops, and accidental pushes to the wrong remote. 4. State the rotation procedure. Replace `SECRET_KEY`, re-deploy. Every signed token in flight invalidates — every user is logged out and has to log back in. This is the correct behaviour; partial revocation is not supported.

Edge cases to mention: - `SECRET_KEY_FALLBACKS` (Django 4.1+) — a list of previous keys Django will accept for verification but won't sign new tokens with. Use it during rotation so existing sessions don't all die at once. - Some test suites hardcode `SECRET_KEY = 'dummy'` in `test_settings.py`. That's fine because tests don't issue real tokens — just don't ship that file's `SECRET_KEY` to prod. - The minimum length is 50 characters. Django generates one at `startproject` time; you can re-generate with `python -c "from django.core.management.utils import get_random_secret_key; print(get_random_secret_key())"`.

What NOT to say: - "It's just a random string, no big deal" — wrong; it's the master signing key. - "If it leaks, just delete the file from git history" — wrong; git history doesn't matter, anyone who pulled the repo has it. Rotate the key.

Common follow-up: "You discover `SECRET_KEY` was committed three months ago. Walk me through the response." — Rotate immediately (new key in env, deploy). Use `SECRET_KEY_FALLBACKS` with the old key for a brief window if you want to avoid mass-logout, otherwise mass-logout is fine. Audit logs for

suspicious sessions in the leak window. Rotate any user-side tokens (API keys, password-reset tokens that may still be unredeemed).

Stand-out line: "Say this and you'll separate yourself: `SECRET_KEY` signs every cookie, token, and password-reset link in your app — treat it as the master credential, and when it leaks, rotate first and audit second."

```
# settings/base.py – correct
import os
SECRET_KEY = os.environ['DJANGO_SECRET_KEY']

# Rotation: keep the old key for token verification, sign new tokens with the new one
SECRET_KEY_FALLBACKS = [os.environ.get('DJANGO_SECRET_KEY_OLD', '')]
SECRET_KEY_FALLBACKS = [k for k in SECRET_KEY_FALLBACKS if k] # drop empty

# WRONG – committed to git, leaked the moment anyone pushes
# SECRET_KEY = 'django-insecure==#!@-r2j8m...'
```

Q24 · `ALLOWED_HOSTS` — what is it, and what breaks if you get it wrong?

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Postman, Cognizant · Topic: deployment · Django 3.x+

The question (interviewer's verbatim phrasing):

What does `ALLOWED_HOSTS` do, and what happens if I leave it as `[]` in production?

The 30-second answer: `ALLOWED_HOSTS` is the list of `Host` header values Django will accept. With `DEBUG = False` and `ALLOWED_HOSTS = []`, every request fails with `DisallowedHost` and a 400 error — Django refuses to serve anything. The setting exists to defend against HTTP Host header attacks, where an attacker sends a `Host` header pointing at a domain you don't own, and Django generates URLs (password-reset emails, redirects) using that attacker-controlled host. Set it to your actual production domains, comma-split from env: `ALLOWED_HOSTS = os.environ['DJANGO_ALLOWED_HOSTS'].split(',')`.

Step-by-step thought process: 1. State the role. Django inspects `request.META['HTTP_HOST']` and checks it against `ALLOWED_HOSTS`. No match → `SuspiciousOperation: DisallowedHost` → 400 response. 2. The threat model. Without `ALLOWED_HOSTS`, an attacker can craft requests with a fake `Host` header. Django uses `request.get_host()` to build links — for password resets, OAuth redirects, social auth callbacks. If the link points at the attacker's domain, the user clicks and lands on the attacker's site with credentials in the URL. 3. State the pattern. `['razorpay.com', 'www.razorpay.com']`,

`'api.razorpay.com']` . Wildcard subdomain: `'.razorpay.com'` matches every subdomain. `'*'` matches everything — *do not* use in production. 4. The `Debug=True` exception. When `DEBUG = True` , Django auto-allows `localhost` , `127.0.0.1` , `:::1` for developer convenience. In production with `DEBUG = False` , you must declare every host.

Edge cases to mention: - Behind a load balancer, the `Host` header is whatever the client sent. Make sure your LB doesn't rewrite it to its internal name, or `ALLOWED_HOSTS` won't match. AWS ALB, GCP HTTPS LB, and Cloudflare preserve the original `Host` header by default. - Health-check endpoints from the LB often send the LB's IP as `Host` — either add the IP to `ALLOWED_HOSTS` or carve out a public health-check URL that doesn't go through Django. - `'*'` in `ALLOWED_HOSTS` plus `DEBUG = False` disables the check entirely — accept everything. Convenient, insecure, never ship.

What NOT to say: - "ALLOWED_HOSTS is optional" — wrong; with `DEBUG = False` , it's required. - "Use `['*']` for simplicity" — works, but reopens the Host-header attack surface.

Common follow-up: *"Your prod app returns 400 errors on every request right after deploy. Where do you look?"* — Check `ALLOWED_HOSTS` . The most common cause is the env var being unset or pointing at the wrong domain.

Stand-out line: *"Say this and you'll separate yourself: `ALLOWED_HOSTS` exists because `request.get_host()` is what Django uses to build password-reset URLs — get it wrong, and the next reset email points users at the attacker's domain."*

```
# settings/prod.py
import os

# Read from env, comma-separated
ALLOWED_HOSTS = [h.strip() for h in os.environ.get('DJANGO_ALLOWED_HOSTS', '').split(',') if h.strip()]

# Example value at deploy:
# DJANGO_ALLOWED_HOSTS=razorpay.com,www.razorpay.com,api.razorpay.com

# For multi-tenant subdomains:
# ALLOWED_HOSTS = ['.razorpay.com', 'razorpay.com']
# The leading dot is the wildcard subdomain marker
```

Q25 · `DEBUG = True` in production — what's the worst thing that happens?

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Cred, Wipro · Topic: deployment · Django 3.x+

The question (interviewer's verbatim phrasing):

A junior developer pushed `DEBUG = True` to production. Why is that an incident, and what do you check?

The 30-second answer: With `DEBUG = True`, any unhandled exception renders Django's full debug page — stack trace, request body, environment variables, settings dump including database credentials, and SQL queries with parameters. Every error reveals the application's internals to anyone who can hit the URL. On top of that, `DEBUG = True` disables `ALLOWED_HOSTS` checking (loosely), retains every SQL query in `connection.queries` so memory grows, and serves static files from the dev server. It's an incident, not a misconfiguration.

Step-by-step thought process: 1. Walk the debug page exposure. On a 500, Django renders `technical_500_response` — a full HTML page with the stack, local variables at each frame, the request `META` dict (which leaks env vars and auth headers), the full settings module (which can include `SECRET_KEY` if not env-sourced), and the SQL log. 2. The DB credential leak. If `DATABASES` is in `settings.py` as a hardcoded dict, the debug page renders it. Even if it's env-sourced, the URL string is visible. Anyone hitting an error URL has your prod DB credentials. 3. Memory grows. Django keeps every SQL query and its parameters in `connection.queries` for the lifetime of the process when `DEBUG = True`. On a long-running gunicorn worker, this is a memory leak. 4. Static files. Django auto-serves `/static/` from the dev server when `DEBUG = True`. In production, your reverse proxy (nginx) should serve those. With `DEBUG = True`, the request goes through Django and adds latency.

Edge cases to mention: - `DEBUG_PROPAGATE_EXCEPTIONS = True` (Django 5.x) controls whether middleware also sees the exception. Doesn't change the leak. - `python manage.py check --deploy` flags `DEBUG = True` as a security issue. Wire it into your CI pipeline. - If you must view the debug page in staging-like envs, use `DEBUG = False` with `INTERNAL_IPS` and Django Debug Toolbar — toolbar still works in production-like setup for IPs you whitelist.

What NOT to say: - "It's fine for a few hours" — wrong; any 500 in that window leaks credentials. - "DEBUG just turns on the toolbar" — wrong; it changes error pages, `ALLOWED_HOSTS` strictness, query log retention, and static-file serving.

Common follow-up: "How do you prevent this from happening again?" — Two layers: (a) `DEBUG = os.environ.get('DJANGO_DEBUG', 'false').lower() == 'true'` with the env var unset in prod, fails closed. (b) CI step that runs `python manage.py check --deploy --fail-level WARNING` against the prod settings module — fails the build if `DEBUG` is `True`.

Stand-out line: "Say this and you'll separate yourself: `DEBUG = True` in production isn't a misconfiguration — it's a credential leak waiting for the first 500 error, and `python manage.py check --deploy` is the one-line CI gate that stops it."

```
# settings/base.py – fails closed
import os

# DEBUG defaults to False unless the env var explicitly says otherwise
DEBUG = os.environ.get('DJANGO_DEBUG', 'false').lower() == 'true'

# settings/dev.py – explicit override for local
from .base import *
DEBUG = True

# settings/prod.py – explicit and unambiguous
from .base import *
DEBUG = False

# CI step:
# DJANGO_SETTINGS_MODULE=myproject.settings.prod python manage.py check --deploy --fail-level WA
```

Closing notes for this chapter

Twenty-five questions, four sections, all tuned for the Indian Django backend interview in 2026 — Razorpay, Cred, Postman, Zerodha, Swiggy, the Service MNCs that have years of Django in production, and the GCCs that hire Django talent to run their internal portals. The questions you'll see in your loop will rarely be these exact ones, but they will live inside this distribution. Round one tests whether you can explain QuerySet laziness, the N+1 problem, and the difference between `select_related` and `prefetch_related` without hedging. Round two pushes deeper — transactions, F-expressions, migration patterns, the dispatch method, the `get_or_create` race. Round three is the failure-mode round — `DEBUG = True`, `ALLOWED_HOSTS`, the `SECRET_KEY` rotation question — where the interviewer is checking whether you've been in production long enough to have lived through one of these.

The shape of the answer is what wins. Lead with the headline (the 30-second answer). Walk the mechanism (the step-by-step). Surface the edge cases. Avoid the wrong answers. Say the stand-out line at the close. If you do that consistently across these twenty-five, you have cleared the Django round.

Next chapter: Flask + FastAPI essentials.

Chapter 4 — Flask + FastAPI (25 Qs)

Python Interview Prep — 2026 Edition · Learn & Excel · File 07 Backend Track Library

If you are interviewing for a backend Python role in India in 2026, this chapter is the one the panel uses to separate the candidate who has *shipped* a service from the one who has *built a tutorial*. Razorpay,

Postman, Swiggy, Zerodha, Cred, PhonePe and Meesho run almost everything on FastAPI now. Older Flask codebases still exist — Flipkart's internal tools, large parts of Freshworks, plenty of Service-MNC client engagements at TCS, Infosys, Wipro — but the new build at almost every Indian product company starts on FastAPI. The interview reflects that split. Round one tests your fluency with both, round two tests whether you can reason about request lifecycles, async, dependency injection and deployment topologies.

The 25 questions below are sequenced from framework basics outward into production. Section A is Flask 3.x — the app factory, blueprints, the request and application contexts, before/after hooks, Jinja2, and the WSGI background that explains every Flask design decision. Section B is FastAPI 0.110+ — dependency injection with `Depends`, Pydantic v2 models for validation, async versus sync endpoints, automatic OpenAPI generation, path versus query versus body parameters, response models, status codes, and `BackgroundTasks`. Section C is async in depth — when FastAPI actually beats Flask, the trap of mixing sync and async, `asyncio.create_task` inside endpoints, WebSockets, and the ASGI versus WSGI distinction. Section D is production — uvicorn and gunicorn with workers, Pydantic v2 patterns, exception handlers, middleware ordering, and CORS configuration.

Every Q is in the standard Step-by-Step Answer format: tags, the question phrased as an interviewer would say it, the 30-second answer, the step-by-step thought process you say out loud, edge cases the panel is listening for, what NOT to say, and the common follow-up. Code targets Flask 3.0+, FastAPI 0.110+, Pydantic 2.x, and uvicorn 0.27+ unless noted.

Table of Sections

- Section A — Flask basics (Q1–Q5)
- Section B — FastAPI fundamentals (Q6–Q15)
- Section C — Async support in depth (Q16–Q20)
- Section D — Production patterns (Q21–Q25)

Section A — Flask Basics

Flask 3.0 shipped in late 2023; 3.1 in 2025. The Flask you are tested on in 2026 is the modern one — type hints on view functions, `Flask.cli` integration, blueprint URL prefixes, and the cleaner context propagation that landed across the 2.x line. Interviewers at Service MNCs lean on Flask because their codebases still run it; interviewers at Indian product cos use Flask questions as a comparator — "now do the same thing in FastAPI." Five questions here, sequenced from project structure to template layer.

Q1 · Flask App Factory Pattern — Why Not Just `app = Flask(__name__)`

Tags: Difficulty: Medium · Asked at: TCS, Infosys, Wipro, Razorpay, Freshworks · Topic: Flask Structure

The question (interviewer's verbatim phrasing):

Most Flask tutorials start with `app = Flask(__name__)` at module top level. Walk me through why a real codebase uses an app factory instead.

The 30-second answer: The app factory pattern wraps the construction of the `Flask` instance inside a function — typically `create_app(config_name)` — so the same code can build a development app, a production app, a test app, and import-time tooling without the side effects of having a live `Flask` object at module load. It is the only sane pattern when you have multiple configurations, when extensions need late binding, when you are running pytest with multiple app fixtures, or when you want to avoid the circular-import minefield that comes with extensions attached to a top-level `app`.

Step-by-step thought process: 1. The top-level pattern instantiates Flask at import time. That means importing the module starts loading config, connecting to extensions like SQLAlchemy, and running module-level decorators — even when all you wanted was to read a constant from the file. 2. The factory pattern moves all of that into `create_app(config=None)`. Extensions are instantiated at module level *without* an app — `db = SQLAlchemy()` — and then bound inside the factory via `db.init_app(app)`. That binding is what lets you have one app instance for production and a different one for tests, sharing zero state. 3. Inside the factory, you read configuration (`app.config.from_object(config or "config.ProductionConfig")`), register blueprints, register error handlers, and return the app. Nothing leaks at import time. 4. The WSGI server — gunicorn, uWSGI — calls the factory through an entry-point string like `app:create_app()`. Flask's CLI auto-detects factories named `create_app` or `make_app` in modules named `app.py` or `wsgi.py`, so `flask run` works without configuration.

Edge cases to mention: - Tests use the factory directly: `app = create_app("config.TestConfig")` inside a pytest fixture, with `app.test_client()` as the request runner. - Multiple blueprints with overlapping URLs are registered inside the factory in a deterministic order — the first match wins. - The `current_app` proxy resolves to the active app inside a request context, so view functions written for the factory pattern do not import the app object directly — they use `current_app`.

What NOT to say: - "App factories are for big apps; small apps can skip them." — Wrong heuristic. The factory pattern costs you ten lines of boilerplate; the cost of *not* using it shows up the day you add pytest or a second config. - "It's just a code-style preference." — It is not. It changes the import graph, the testing story, and the extension-binding model.

Common follow-up: "Show me how you would write the factory for an app with a SQLAlchemy DB, a config object, and two blueprints — `api` and `admin`."

Stand-out line: *"Say this and you'll separate yourself: the app factory isn't ceremony — it's the only pattern that lets pytest spin up a fresh app per fixture without leaking extension state from the last test."*

```
# app/__init__.py
from flask import Flask
from flask_sqlalchemy import SQLAlchemy

db = SQLAlchemy()

def create_app(config_name: str = "config.ProductionConfig") -> Flask:
    app = Flask(__name__)
    app.config.from_object(config_name)

    db.init_app(app)

    from app.api import api_bp
    from app.admin import admin_bp
    app.register_blueprint(api_bp, url_prefix="/api/v1")
    app.register_blueprint(admin_bp, url_prefix="/admin")

    return app

# wsgi.py – gunicorn entry point
from app import create_app
app = create_app()
```

Q2 · Flask Blueprints — What They Buy You

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Wipro, Cognizant, Freshworks, Postman · Topic: Flask Structure

The question (interviewer's verbatim phrasing):

What is a Flask blueprint and what problem does it solve?

The 30-second answer: A blueprint is a lazy, deferred way to register routes, error handlers, and template paths against an app. You write your routes against the blueprint, then register the blueprint against the app at startup with a URL prefix. The problem it solves: keeping a multi-module Flask codebase from collapsing into one giant `app.py` with hundreds of `@app.route` decorators, while also enabling URL-prefix mounting so the same blueprint can live under `/api/v1` in production and `/api/v2` in a staged migration.

Step-by-step thought process: 1. Without blueprints, every route lives on the app: `@app.route("/orders")`. In a 30-endpoint codebase that becomes unreadable, and you cannot move a feature behind a versioned prefix without changing every decorator. 2. A blueprint groups related routes:

`orders_bp = Blueprint("orders", __name__)` followed by `@orders_bp.route("/")` for GET `/orders` and `@orders_bp.route("/<int:order_id>")` for GET `/orders/123`. The blueprint is registered with `app.register_blueprint(orders_bp, url_prefix="/orders")` — the prefix is applied at registration, not at decoration. 3. Blueprints have their own `before_request`, `after_request`, `errorhandler`, and `template_folder`, all scoped. A blueprint can ship its own templates alongside its Python code, which makes it the right unit for a "feature module." 4. The name in `Blueprint("orders", __name__)` is the *namespace* for endpoints — `url_for("orders.detail")` resolves the `detail` view inside the `orders` blueprint. That namespace is how you avoid endpoint collisions when two blueprints both have a `detail` view.

Edge cases to mention: - Blueprints can be nested as of Flask 2.0 —

`parent_bp.register_blueprint(child_bp, url_prefix="/sub")` — useful for very large APIs but rare in practice. - The `url_prefix` on `register_blueprint` overrides the `url_prefix` set on the blueprint constructor; the registration-time prefix wins. - A blueprint registered twice with two prefixes works — same routes mounted at both URLs. Rare, but it is the migration pattern when you are dual-running `/api/v1` and `/api/v2`.

What NOT to say: - "Blueprints are just file organisation." — They are also lazy registration and per-blueprint hooks; that is the design point. - "I don't need blueprints because I only have one module." — Today. The cost of *adding* blueprints later is real because every `url_for("view_name")` needs to become `url_for("blueprint_name.view_name")`.

Common follow-up: "What is the difference between `current_app` and the app object you imported at top level?"

Stand-out line: "Say this and you'll separate yourself: blueprints are deferred registration, not file organisation — the prefix lives at `register_blueprint` time, which is what makes versioned API mounting a one-line change."

```
# app/api/__init__.py
from flask import Blueprint, jsonify

api_bp = Blueprint("api", __name__)

@api_bp.route("/health")
def health():
    return jsonify(status="ok")

@api_bp.route("/orders/<int:order_id>")
def get_order(order_id: int):
    # ... fetch and return
    return jsonify(id=order_id)

@api_bp.errorhandler(404)
def not_found(err):
    return jsonify(error="not found"), 404
```

Q3 · Request Context vs Application Context — The Two Stacks

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Postman, Goldman Sachs India, JPMC · Topic: Flask Internals

The question (interviewer's verbatim phrasing):

Flask has two contexts — application context and request context. Walk me through the difference, when each is pushed, and what proxies live on each.

The 30-second answer: The application context is pushed when Flask is doing app-level work — handling a request, running a CLI command, or executing background code inside an explicit `with app.app_context():` block. It exposes `current_app` and the `g` namespace. The request context is pushed when a request arrives and is popped when the response is sent; it exposes `request` and `session`. The two are nested — every request context pushes an application context underneath it, but you can have an application context *without* a request context (CLI, scheduled jobs, tests). Both are stored in `contextvars` under the hood since Flask 2.x — so they are async-aware and isolated per task.

Step-by-step thought process: 1. Name the four context-local proxies up front: `current_app`, `g`, `request`, `session`. The first two live on the application context; the second two on the request context. 2. Push order: when a request arrives, Flask first pushes an application context (so `current_app` works), then pushes a request context (so `request` works). When the response is sent, the request context is popped first, then the application context. That nesting is what lets `current_app.config["FOO"]` work inside a view. 3. The CLI case is the cleanest illustration of the split: `flask shell` pushes only an application context — `current_app` works, `g` works, but `request` raises `RuntimeError: working outside of request context`. That is why custom CLI commands that need to do DB work use `with app.app_context():` and never expect a `request`. 4. `g` is request-scoped despite living on the application context — it gets reset on every new request because the application context is re-pushed each time. Use it for "stash this for the duration of the request" data: the authenticated user, a fetched config, a cached lookup. Do *not* use it for cross-request state — that needs a real cache.

Edge cases to mention: - Pre-Flask 2.0 the contexts were stored in `werkzeug.local.LocalStack`, which was thread-local. From 2.0 they moved to `contextvars`, which means they propagate correctly into async functions and `asyncio.Task`s spawned inside a view. - Manually pushing contexts in tests or scripts: `with app.test_request_context("/"):` pushes both contexts so you can call view functions directly; `with app.app_context():` pushes only the app context. - `g` is not the same as `flask.session` — `g` is in-memory per request; `session` is a signed cookie persisted across requests.

What NOT to say: - "The contexts are thread-local." — Outdated. They are `contextvars`-based since 2.0 and async-safe. - "`g` lives forever for the lifetime of the app." — Wrong. `g` is request-scoped. - "You can use `request` anywhere." — Outside a request context (e.g. in a CLI command), `request` raises a clear `RuntimeError`.

Common follow-up: "How would you write a CLI command that runs a DB migration but needs the app config?"

Stand-out line: "Say this and you'll separate yourself: the two contexts exist because CLI commands need config but not a request — getting the boundary right is what lets the same `current_app` work in a `flask run` worker and a `flask seed-data` Click command."

```
from flask import Flask, current_app, request, g

@app.before_request
def load_user():
    # Runs inside both contexts — g and request both work
    g.user_id = request.headers.get("X-User-Id")

@app.cli.command("seed-data")
def seed_data():
    # Runs inside app context only — current_app works, request doesn't
    with current_app.app_context():
        # current_app.config["DB_URL"] is available
        print(f"Seeding into {current_app.config['DB_URL']}")
```

Q4 · `before_request` and `after_request` — The Hook Lifecycle

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Postman, Freshworks · Topic: Flask Lifecycle

The question (interviewer's verbatim phrasing):

Walk me through Flask's request hooks — `before_request`, `after_request`, `teardown_request` — and tell me what runs even when an exception is raised.

The 30-second answer: `before_request` runs after the request context is pushed but before the view function. `after_request` runs after the view function returns, but *only* if the view did not raise an unhandled exception — and it must accept and return the response. `teardown_request` runs always, exception or not, after the response has been sent, and is the right place for releasing resources. The most common bug is putting cleanup in `after_request` and discovering that 500 errors leak connections.

Step-by-step thought process: 1. Order on the success path: context push, `before_request` (multiple, in registration order), view function, `after_request` (multiple, in *reverse* registration order — like middleware unwinding), `teardown_request`, context pop. 2. Order on the failure path: context push, `before_request`, view function raises, error handler runs, `after_request` is *skipped*, `teardown_request` still runs with the exception passed in as an argument. 3. The reverse-order on `after_request` is the LIFO middleware pattern — if `before_request` A wraps the response with header X and `after_request` B wraps it with header Y, B runs first on the way out so X is the outermost header. This matters when you are layering authentication, logging and CORS. 4. `teardown_request` is where DB connection cleanup, telemetry flushing, and Sentry span closing belong. Returning from a `teardown_request` does not modify the response — the response has already been sent. 5. Blueprint-scoped versions exist: `@bp.before_request` only fires for routes inside that blueprint. The app-level `@app.before_request` fires for every route.

Edge cases to mention: - A `before_request` handler that returns a non-None value short-circuits — the view function does not run. That is how authentication-rejection patterns work: return a 401 response from `before_request`, the view never sees the request. - `after_request` must return a response — even if you only want to add a header, you must `return response`. - `teardown_appcontext` is the sibling for the application context — runs even on CLI commands. Use it for cleanup that should happen even outside requests.

What NOT to say: - "`after_request` always runs." — Wrong. It is skipped on unhandled exceptions. Use `teardown_request` for "always." - "Hooks run in registration order on the way out." — Wrong. `after_request` runs in reverse registration order. - "I don't need `teardown_request` because Flask cleans up automatically." — Flask cleans up *its* state. Your DB connection, your background tracer, your file handle are not Flask's problem.

Common follow-up: *"How would you instrument every request with a timing log that captures both success and failure?"*

Stand-out line: *"Say this and you'll separate yourself: `after_request` is for success-path response mutation; `teardown_request` is for always-runs cleanup — confuse them and your 500s leak DB connections silently in production."*

```
import time
from flask import g, request, current_app

@app.before_request
def start_timer():
    g.start_ts = time.perf_counter()

@app.after_request
def add_timing_header(response):
    duration_ms = (time.perf_counter() - g.start_ts) * 1000
    response.headers["X-Response-Time-Ms"] = f"{duration_ms:.2f}"
    return response # must return the response

@app.teardown_request
def log_request(exc):
    duration_ms = (time.perf_counter() - g.start_ts) * 1000
    current_app.logger.info(
        "method=%s path=%s status=%s duration_ms=%.2f exc=%s",
        request.method, request.path,
        "exception" if exc else "ok",
        duration_ms, exc,
    )
```

Q5 · Jinja2 Autoescape, WSGI, and Why FastAPI Isn't a Direct Replacement

Tags: Difficulty: Medium · Asked at: TCS, Infosys, Freshworks, Razorpay, Postman · Topic: Flask Internals

The question (interviewer's verbatim phrasing):

Flask runs on WSGI and uses Jinja2 for templates. Both of those choices came from the early 2010s. Why do they still matter in 2026, and why isn't FastAPI a one-for-one replacement?

The 30-second answer: WSGI is the synchronous Python web protocol — one request, one OS thread (or one greenlet), the framework returns a response object. Flask is a WSGI framework, and that means it sits behind gunicorn or uWSGI with a worker pool, scaling out by adding processes. Jinja2 is its server-side templating engine, with autoescape on by default for `.html` and `.htm`. FastAPI runs on ASGI — the async sibling protocol — and does not ship a server-side templating story. If your app renders HTML on the server (admin panels, marketing pages, internal tools), Flask is still the better fit; if your app only returns JSON (most modern APIs), FastAPI's ASGI model and Pydantic integration win.

Step-by-step thought process: 1. WSGI in one line: a Python callable that takes `environ` (a dict) and `start_response` (a callback) and returns an iterable of bytes. Flask's `Flask.__call__` is exactly that callable. It is synchronous by design — every request occupies a worker for its full lifetime. 2. Jinja2

is Flask's default template engine. Autoescape is enabled for `.html`, `.htm`, `.xml`, `.xhtml` — meaning `{{ user_name }}` in a template is HTML-escaped even if the data is hostile. You opt out per-block with `{% autoescape false %}` or per-variable with `{{ raw_html|safe }}` — the latter being the right footgun question for an interview. 3. WSGI does *not* support WebSockets or long-poll streaming natively. You need ASGI for those. If your app needs real-time features (chat, live dashboards, notifications) Flask is the wrong choice — FastAPI or Starlette ships with WebSocket support out of the box. 4. The migration path is real: Flask plus the `asgiref` adapter can run inside an ASGI server, and FastAPI plus `Jinja2Templates` can render HTML. Neither is a zero-friction port — Flask's mature template ecosystem (template inheritance, macros, `flask_login`, `flask_wtf`) doesn't have one-for-one FastAPI equivalents.

Edge cases to mention: - Autoescape applies to *strings* — `Markup("trusted html")` (or `|safe`) marks a string as already-escaped. Wrap everything from the DB as if it is hostile; only `Markup` what you have audited. - Flask 3.x added `async def` view support via `asgiref` — but the request still runs in a thread pool under the hood, so async Flask views do not give you the ASGI concurrency profile. - Jinja2 is a separate library — used by Ansible, Salt, Sphinx, even FastAPI's `Jinja2Templates`. Knowing Jinja2 is portable knowledge; it does not lock you to Flask.

What NOT to say: - "WSGI is obsolete." — It is the runtime for a giant fraction of running Python services. Not obsolete; not the right pick for new async-first APIs. - "Jinja2 is auto-secure." — Autoescape is on by default *for HTML files*. It is off for `.txt` templates by default; for emails this matters. - "FastAPI does everything Flask does." — Not server-side templating, not the Flask extension ecosystem, not the same auth-helper maturity.

Common follow-up: *"If I asked you to render an HTML report inside a FastAPI service, how would you do it?"*

Stand-out line: *"Say this and you'll separate yourself: WSGI versus ASGI isn't a perf debate — it's a model debate, and the right call is Flask for server-rendered HTML, FastAPI for JSON APIs with WebSockets or streaming."*

```
# Flask + Jinja2 – autoescape is on for .html
from flask import render_template
@app.route("/orders/<int:order_id>")
def order_page(order_id: int):
    order = fetch_order(order_id) # may contain user-supplied strings
    return render_template("order.html", order=order)
    # {{ order.notes }} is HTML-escaped; {{ order.notes|safe }} is not

# FastAPI + Jinja2Templates – same engine, ASGI runtime
from fastapi import FastAPI, Request
from fastapi.templating import Jinja2Templates
templates = Jinja2Templates(directory="templates")

@app.get("/orders/{order_id}")
def order_page(order_id: int, request: Request):
    order = fetch_order(order_id)
    return templates.TemplateResponse("order.html", {"request": request, "order": order})
```

Section B — FastAPI Fundamentals

FastAPI is the framework most Indian product cos default to for new services in 2026. Razorpay's internal services, Postman's APIs, Zerodha's risk-engine endpoints, Swiggy's delivery-mesh layer, Cred's reward-rules service — all FastAPI. The interview accordingly drills you on dependency injection (the **Depends** system), Pydantic v2 validation, async-vs-sync endpoint semantics, OpenAPI generation, and the request-parameter parsing rules. The 10 questions below cover the canonical surface — if you can answer all of them, you can pass the FastAPI-fluency bar at every product-co panel.

Q6 · Dependency Injection with **Depends** — What It Actually Does

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Swiggy, Cred, Zerodha, Microsoft India · Topic: FastAPI DI

The question (interviewer's verbatim phrasing):

Walk me through what **Depends** does in FastAPI. Why is it called dependency injection and not just function composition?

The 30-second answer: **Depends(callable)** tells FastAPI: before you call this endpoint, call this other callable, pass the result as the parameter the endpoint declared. FastAPI inspects the callable's signature, resolves its own parameters from the request (path, query, body, headers, other **Depends** calls), and calls it. The result is cached for the duration of a single request, so the same dependency

referenced from three places computes once. That caching plus the recursive resolution — dependencies of dependencies — plus the integration with the request lifecycle (cleanup via `yield`-based generators) is what makes it dependency injection rather than just call composition.

Step-by-step thought process: 1. Mechanics: `def endpoint(user: User = Depends(get_current_user))`. FastAPI sees `Depends(get_current_user)`, inspects `get_current_user`'s signature, finds it needs say `token: str = Header(...)`, parses the header from the incoming request, calls `get_current_user(token=...)`, and passes the returned `User` as the `user` parameter. 2. Recursion: `get_current_user` itself can declare `db: Session = Depends(get_db)`. FastAPI walks the graph — DB session is resolved first, then `get_current_user`, then your endpoint runs. No manual wiring. 3. Per-request caching: if `get_db` is depended on by `get_current_user`, `get_current_org`, and your endpoint directly, it runs *once* per request. The same `Session` object is passed to all three. Toggle with `Depends(get_db, use_cache=False)` if you need a fresh result. 4. Cleanup with `yield`: `def get_db(): db = SessionLocal(); try: yield db; finally: db.close()`. FastAPI treats the generator as a context manager — code before `yield` runs at injection time, code after `yield` runs after the response is sent. That is how DB sessions, file handles, and traced spans are reliably closed. 5. Application-wide dependencies: `app.dependency_overrides[get_db] = test_get_db` is the test-injection seam. The override applies for every endpoint that depends on `get_db`, which is how you swap a real DB for SQLite-in-memory inside pytest.

Edge cases to mention: - `Depends()` with no argument means "use the same name as the parameter type's class" — rarely used, but valid. - Dependencies declared at the router or app level run for every request through that router/app, even if the endpoint doesn't reference them — the right place for "every endpoint requires a valid API key." - Class-based dependencies: `class Pagination: def __init__(self, skip: int = 0, limit: int = 100): ...` then `Depends(Pagination)`. The class is instantiated like a function call; the parameters are parsed from the request.

What NOT to say: - "`Depends` is just a function call." — It is also signature introspection, per-request caching, yield-based cleanup, and override hooks. None of those are plain function call semantics. - "Use `Depends` only for DB sessions." — Auth, rate limiting, request-scoped logging, feature flags, pagination — every cross-cutting concern goes through `Depends`. That is the design. - "It's slow because of reflection." — Resolution happens once per route at startup; per-request cost is a dict lookup. Reflection is *not* the bottleneck.

Common follow-up: "How would you write a `Depends` that requires a valid JWT and returns the decoded user object?"

Stand-out line: "Say this and you'll separate yourself: `Depends` is request-scoped caching plus yield-based cleanup plus override hooks — that combination is why every cross-cutting concern (auth, rate limiting, DB sessions, feature flags) routes through it, not why DB sessions alone do."

```

from fastapi import Depends, FastAPI, Header, HTTPException
from sqlalchemy.orm import Session

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

def get_current_user(
    authorization: str = Header(...),
    db: Session = Depends(get_db),
) -> User:
    token = authorization.removeprefix("Bearer ").strip()
    try:
        payload = jwt.decode(token, SECRET, algorithms=["HS256"])
    except jwt.PyJWTError:
        raise HTTPException(status_code=401, detail="invalid token")
    user = db.query(User).get(payload["sub"])
    if not user:
        raise HTTPException(status_code=401, detail="user not found")
    return user

@app.get("/me")
def me(user: User = Depends(get_current_user)) -> dict:
    return {"id": user.id, "email": user.email}

```

Q7 · Pydantic Models for Request and Response Validation

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Swiggy, Cred, Zerodha, Walmart Labs · Topic: Pydantic

The question (interviewer's verbatim phrasing):

Why does FastAPI use Pydantic for request and response models? What does it give you that `dict` and manual parsing don't?

The 30-second answer: A Pydantic model declares the shape, types, and constraints of a request body or response payload. FastAPI parses the JSON body, coerces the values into the model, runs validation, and either calls your endpoint with a typed object or returns a 422 with a structured error message — all before your code runs. On the way out, declaring a `response_model` filters and serialises the return value, so internal fields never leak to clients. The win versus manual parsing is threefold: validation runs in compiled C (fast), the schema is the source of truth for OpenAPI/Swagger generation, and the same model can be reused inside your service layer with full editor autocomplete.

Step-by-step thought process: 1. Inbound: declare `class OrderIn(BaseModel): user_id: int; items: list[Item]; notes: str | None = None`. Use it as `def create_order(body: OrderIn)`. FastAPI reads the JSON body, validates against `OrderIn`, calls your function with `body.user_id` already an `int`. 2. Validation runs through Pydantic v2's Rust-based core. Constraints — `Field(gt=0, le=100)`, regex on strings, `field_validator` for custom rules — fail with structured errors. The response is a 422 with `loc / msg / type` for each failure, which the frontend can show field-by-field. 3. Outbound: `@app.get("/orders/{id}", response_model=OrderOut)` filters the return value. Even if your function returns a SQLAlchemy `Order` with twenty columns, only the fields declared on `OrderOut` are serialised. Sensitive fields — `password_hash`, `internal_notes` — stay server-side by simply not being on the response model. 4. OpenAPI: the model schema is auto-extracted into the OpenAPI document at `/openapi.json`. Swagger UI at `/docs` and ReDoc at `/redoc` render forms with example payloads from the model. The schema is the documentation; no separate writing. 5. Reuse: the same `OrderIn` model used in the endpoint can be used inside your business logic — `def place_order(req: OrderIn) -> OrderOut` reads cleanly, mypy types it correctly, and the test suite builds payloads with `OrderIn(user_id=42, items=[...])` directly.

Edge cases to mention: - `model_config = ConfigDict(from_attributes=True)` enables building a response model from an ORM object's attributes — required when you `return orm_user` and expect Pydantic to read `orm_user.id`, `orm_user.email`. - `response_model_exclude_unset=True` strips fields that weren't explicitly set — useful for PATCH-style partial responses. - Pydantic v2 changed the API: `.dict()` is now `.model_dump()`, `.parse_obj()` is now `.model_validate()`, `Config` class is now `model_config = ConfigDict(...)`. Code on the internet from 2022 will mix the two — be explicit which version you are on.

What NOT to say: - "Pydantic is slow because it's Python." — Pydantic v2 is mostly Rust under the hood; it is faster than manual `isinstance` checks for non-trivial schemas. - "I prefer to validate in my view function." — Then you re-implement OpenAPI generation, structured errors, and editor autocomplete by hand. Don't. - "`response_model` is optional." — Functionally yes; for sensitive endpoints, omitting it leaks every attribute on the object you return.

Common follow-up: "How would you write a request body with a nested list of items, each with a quantity that must be at least 1?"

Stand-out line: "Say this and you'll separate yourself: the `response_model` is the leak gate — without it, every internal field on the ORM object ships to the client, which is the data-leak class of bug security review catches at the second pass."


```

from pydantic import BaseModel, Field, ConfigDict
from fastapi import FastAPI

class Item(BaseModel):
    sku: str = Field(min_length=3, max_length=32)
    quantity: int = Field(gt=0, le=999)

class OrderIn(BaseModel):
    user_id: int
    items: list[Item] = Field(min_length=1)
    notes: str | None = None

class OrderOut(BaseModel):
    model_config = ConfigDict(from_attributes=True)
    id: int
    user_id: int
    total_paise: int # only the fields we want to expose

@app.post("/orders", response_model=OrderOut, status_code=201)
def create_order(body: OrderIn) -> OrderOut:
    order = orders_service.create(body) # ORM object with many fields
    return order # filtered through OrderOut on the way out

```

Q8 · `async def` vs `def` — When FastAPI Cares

Tags: Difficulty: Hard · Asked at: Razorpay, Swiggy, Postman, Cred, Zerodha, Microsoft India · Topic: FastAPI Async

The question (interviewer's verbatim phrasing):

FastAPI lets you write an endpoint as `def` or `async def`. Walk me through what actually happens differently for each, and tell me when picking the wrong one is a production-blocking mistake.

The 30-second answer: An `async def` endpoint runs directly on the event loop — FastAPI awaits it, and the loop is free to service other requests while your code is awaiting I/O. A plain `def` endpoint is offloaded to a thread pool (`anyio` 's default thread pool, 40 threads by default) so it doesn't block the event loop. The production-blocking mistake is writing `async def` and then calling synchronous blocking I/O inside it — a sync `requests.get`, a sync `psycpg2.execute`, a `time.sleep` — which freezes the event loop and stops every other request from progressing. Pick `async def` only if every I/O call inside is genuinely awaitable; otherwise pick plain `def` and let the thread pool handle it.

Step-by-step thought process: 1. `async def` route: FastAPI does `await endpoint(...)`. Anything synchronous inside that endpoint runs on the event loop thread and *blocks every other request*.

A `time.sleep(1)` inside an async endpoint pauses every concurrent request for 1 second. 2. Plain `def` route: FastAPI offloads to `anyio.to_thread.run_sync` — your endpoint runs in a worker thread, the event loop keeps spinning. Concurrency is limited by the thread pool size (default 40). Blocking code inside is *fine*. 3. The rule: if your endpoint awaits async libraries end-to-end (`httpx.AsyncClient`, `asyncpg`, `aioredis`, `motor`) write `async def`. If it uses sync libraries (`requests`, `psycopg2`, `pymongo`, `redis-py` in sync mode), write plain `def` and let the thread pool absorb the blocking. Mixing the two inside an `async def` is the production fire. 4. Dependencies follow the same rule. A `Depends(get_db)` written as `def get_db()` is offloaded if the endpoint is sync, awaited if it is async (and your sync `get_db` will need to do its work without blocking the loop, or you must use `async def get_db`). 5. The diagnostic for "is this async-safe?": grep the function body for any function call you do not recognise as async. If `db.query(...)`, `redis.get(...)`, `requests.get(...)` appears inside an `async def`, you have a bug.

Edge cases to mention: - `anyio` thread-pool size is tunable: `from anyio import to_thread; to_thread.current_default_thread_limiter().total_tokens = 100`. Bump it if you have many slow sync endpoints; do it before the first request. - `async def` endpoint that does only CPU-bound Python work (a big regex, a JSON parse, a hash) blocks the event loop just as badly as `time.sleep`. Wrap CPU-heavy work in `await asyncio.to_thread(...)` or `await loop.run_in_executor(None, ...)` to offload. - A common mistake is writing `async def` with `requests.get(...)` "because it works in dev." It does — until a slow upstream pauses your loop and concurrent requests time out under load.

What NOT to say: - "`async def` is always faster." — Faster *for non-blocking I/O*. For blocking I/O, plain `def` is faster because the thread pool gives real concurrency. - "Async is more modern, just always use it." — Bad heuristic. The right pick depends on which libraries you call inside. - "FastAPI is async, so I have to write `async def`." — No. FastAPI happily runs plain `def` endpoints; the thread-pool offload is by design.

Common follow-up: "You have a FastAPI endpoint that calls a downstream HTTP API. Show me the right way to write it both with sync `requests` and with async `httpx`."

Stand-out line: "Say this and you'll separate yourself: `async def` with `requests.get` inside is the FastAPI incident — it works in dev, freezes the loop in prod, and the bug stays invisible until traffic doubles."

```
import asyncio, httpx, requests
from fastapi import FastAPI

app = FastAPI()

# Plain def – sync requests, runs in the thread pool. Loop stays unblocked.
@app.get("/sync")
def sync_endpoint():
    resp = requests.get("https://api.example.com/data", timeout=5)
    return resp.json()

# async def – async httpx, runs on the loop. True concurrency.
@app.get("/async")
async def async_endpoint():
    async with httpx.AsyncClient(timeout=5) as client:
        resp = await client.get("https://api.example.com/data")
    return resp.json()

# BUG – async def + sync requests. Blocks the loop. Don't ship this.
@app.get("/broken")
async def broken_endpoint():
    resp = requests.get("https://api.example.com/data", timeout=5) # blocks loop
    return resp.json()
```

Q9 · Automatic OpenAPI Docs — How and Why

Tags: Difficulty: Easy · Asked at: Razorpay, Postman, Swiggy, Cred, Microsoft India · Topic: FastAPI OpenAPI

The question (interviewer's verbatim phrasing):

FastAPI ships with `/docs` and `/redoc` out of the box. How does that work, and what determines what shows up there?

The 30-second answer: FastAPI introspects every registered route — its path, method, parameter names and types, dependency signatures, request body model, response model, status codes, and response examples — and builds an OpenAPI 3.1 JSON document served at `/openapi.json`. `/docs` renders that document with Swagger UI; `/redoc` renders it with ReDoc. Pydantic models contribute their JSON schemas directly. The result is interactive API documentation generated from the same code that serves the requests — no separate spec file, no documentation drift.

Step-by-step thought process: 1. Schema sources: path params from the route string `/orders/{order_id}`, query params from default arguments with primitives, body params from Pydantic models, headers from `Header(...)`, response shape from the `response_model` argument on the

decorator. 2. Docstring + summary: FastAPI uses the route function's docstring as the endpoint description, and the `summary=` keyword on the decorator as the short title. Adding a docstring is free documentation: `def get_order(order_id: int): """Fetch a single order by ID."""`. 3. Tags + grouping: `@app.get("/orders", tags=["orders"])` groups endpoints under the "orders" section in Swagger UI. Use tags consistently across blueprints/routers and the docs page becomes navigable. 4. Customisation: `app = FastAPI(title="Razorpay Orders API", version="2.3.1", description="...", contact={...})` controls the top-of-page metadata. Override `app.openapi_url`, `app.docs_url`, `app.redoc_url` to disable or relocate the docs (commonly disabled in production for public APIs, kept enabled behind admin auth for internal ones). 5. Schema export: `python -c "from main import app; import json; print(json.dumps(app.openapi()))"` dumps the spec to stdout for piping into client-generator tooling (`openapi-generator-cli`, `oapi-codegen`). That is how typed clients in JS, Go, Java are kept in sync with backend changes.

Edge cases to mention: - OpenAPI generation happens at startup the first time `/openapi.json` is requested, then cached. Tweaks to docstrings require a server restart to reflect. - `include_in_schema=False` on a route hides it from `/docs` and `/openapi.json` — use for internal health checks, debug endpoints, and admin tooling. - Custom OpenAPI schema: override `app.openapi` to return a modified dict — useful for adding security schemes, server URLs, or vendor extensions like `x-rate-limit`.

What NOT to say: - "FastAPI generates docs by parsing comments." — It parses types and signatures; comments only contribute via the docstring. - "Swagger UI is the only renderer." — ReDoc ships too; you can swap in any OpenAPI renderer. - "OpenAPI docs are a development-only feature." — Many product cos publish them as their public API documentation directly.

Common follow-up: *"How would you disable `/docs` in production while keeping `/openapi.json` for internal tooling?"*

Stand-out line: *"Say this and you'll separate yourself: the OpenAPI doc isn't a side-effect of FastAPI — it's the source of truth that drives typed clients in JS, Go, and Java, which is why a docstring on every endpoint is the cheapest documentation contract you'll ever ship."*

```
import os
from fastapi import FastAPI

app = FastAPI(
    title="Razorpay Orders API",
    version="2.3.1",
    description="Order management for the merchant dashboard",
    docs_url="/docs" if os.getenv("ENV") != "production" else None,
    redoc_url=None,          # disable ReDoc everywhere
    openapi_url="/openapi.json", # keep the JSON for internal tooling
)

@app.get("/internal/healthz", include_in_schema=False)
def healthz():
    return {"status": "ok"}
```

Q10 · Path Params vs Query Params vs Body — How FastAPI Decides

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Swiggy, Cred, Zerodha, Freshworks · Topic: FastAPI Parameters

The question (interviewer's verbatim phrasing):

Walk me through the rules FastAPI uses to decide whether a parameter is a path param, a query param, or part of the body.

The 30-second answer: The rule is mechanical. If a parameter name appears inside the route's `{name}` placeholders, it is a path parameter. Otherwise, if its type is a Pydantic `BaseModel`, it is a body parameter (single body model, expected as JSON). Otherwise, if its type is a primitive (`int`, `str`, `bool`, `list[str]`, etc.) it is a query parameter. You can override any of these by explicitly wrapping the default with `Path(...)`, `Query(...)`, `Body(...)`, `Header(...)`, `Cookie(...)`, or `Form(...)` — the explicit form wins and also lets you attach validation and OpenAPI metadata.

Step-by-step thought process: 1. Path: `@app.get("/orders/{order_id}")` plus `def get_order(order_id: int)` — the placeholder name matches the parameter name. FastAPI parses `order_id` out of the URL and casts to `int`; non-int raises 422 before your code runs. 2. Query: `def list_orders(limit: int = 50, status: str | None = None)` — neither name is in the path, both are primitives, so they are parsed from the query string `?limit=20&status=paid`. Defaults make them optional; missing required query params return 422. 3. Body: `def create_order(body: OrderIn)` — `OrderIn` is a `BaseModel`, so FastAPI parses the request body as JSON, validates it against `OrderIn`, and passes the typed object. 4. Multiple body models: if you declare two `BaseModel` params, FastAPI expects a JSON body that contains both as top-level

keys: `{"order": {...}, "discount": {...}}` . Embed a single model inside a key with `Body(..., embed=True)` for consistency. 5. Explicit override: `def list(q: str = Query(min_length=3, max_length=64))` or `def get(id: int = Path(gt=0))` . Useful when the default-based detection isn't what you want, or when you need validation constraints, alias, or `deprecated=True` on a parameter.

Edge cases to mention: - `list[str]` as a query param parses repeated keys: `?tag=fast&tag=cheap` becomes `["fast", "cheap"]` . Single value `?tag=fast` becomes `["fast"]` . - A non- `BaseModel` parameter that you want from the body needs `Body(...)` : `def f(name: str = Body(...))` — without `Body(...)` , FastAPI assumes it is a query param. - `Header(..., alias="X-Request-Id")` lets you map a header with hyphens to a Python identifier with underscores.

What NOT to say: - "FastAPI guesses based on the HTTP method." — It does not; the rule is based on signature, not method. - "Body params have to be a `BaseModel` ." — Not strictly; you can use `Body(...)` to pull primitives or `dict` from the body. - "Path params can be optional." — Not in FastAPI's routing — path params are required by definition. Optional means it is a query param.

Common follow-up: "Show me an endpoint that takes a path id, two query filters, and a JSON body."

Stand-out line: "Say this and you'll separate yourself: parameter placement isn't FastAPI magic — it's a mechanical rule based on signature, and `Path` / `Query` / `Body` / `Header` are the explicit knobs when you need validation or to override the default."

```
from fastapi import FastAPI, Path, Query
from pydantic import BaseModel

class PaymentUpdate(BaseModel):
    status: str
    reason: str | None = None

@app.patch("/merchants/{merchant_id}/payments/{payment_id}")
def update_payment(
    merchant_id: int = Path(gt=0),
    payment_id: int = Path(gt=0),
    notify: bool = Query(default=False, description="Send webhook on update"),
    actor: str = Query(min_length=3, max_length=64),
    body: PaymentUpdate = ...,          # JSON body
) -> dict:
    # All five parameters arrive typed and validated
    return {"merchant_id": merchant_id, "payment_id": payment_id, "status": body.status}
```

Q11 · Response Models — Filtering, Validation, and the Leak Problem

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Cred, Goldman Sachs India · Topic: FastAPI Response

The question (interviewer's verbatim phrasing):

You return a SQLAlchemy model from a FastAPI endpoint and the response includes a `password_hash` field. What went wrong, and how do you fix it?

The 30-second answer: The endpoint did not declare a `response_model`, so FastAPI serialised every attribute of the returned object — including the password hash. The fix is to declare a `response_model` on the route decorator that lists only the fields the client should see. FastAPI will run your return value through the response model on the way out, dropping unlisted fields entirely. This is a security boundary, not just a presentation choice.

Step-by-step thought process: 1. Declare a `UserOut(BaseModel)` with `model_config = ConfigDict(from_attributes=True)` (the v2 spelling of v1's `orm_mode = True`) and only the fields safe to expose: `id`, `email`, `display_name`. Omit `password_hash`, `internal_notes`, `risk_score`. 2. Wire it: `@app.get("/users/{id}", response_model=UserOut)`. Now the endpoint may return a full `User` ORM object; FastAPI calls `UserOut.model_validate(user)` and serialises only the model's declared fields. 3. Beyond filtering, the response model validates types — if `display_name` is `str | None` and your ORM returns an `int` by accident, the validation fails fast in your test suite, not in production. 4. Two related knobs: `response_model_exclude_unset=True` strips fields that weren't explicitly set on the model (useful for partial-update responses where you don't want to send back null defaults); `response_model_by_alias=True` serialises under the field aliases (useful when the API contract uses `userId` but the model uses `user_id`). 5. The strongest version of this pattern is "one model per direction" — `UserIn` for incoming, `UserOut` for outgoing, `UserDB` for the ORM. Never use the same model in all three positions; it inevitably leaks something eventually.

Edge cases to mention: - `response_model` runs on whatever you return — including `dict`. If you return `{"password_hash": "..."} literally, the model will drop it. Good safety net. - A response_model of list[UserOut] filters each item. Useful for list endpoints; common to forget the list[] wrapper. - For internal admin endpoints where leaking is fine, omit the response model and accept the leakage cost — but make it a conscious decision, not an oversight.`

What NOT to say: - "Just remove the field from the ORM model." — Wrong layer. The ORM model needs the field; the *response* should not include it. - "I'll set it to `None` before returning." — Brittle and forgettable. The response model is the single declarative source of truth. - "Pydantic does this by default." — No. Without an explicit `response_model`, every attribute on the returned object is fair game for serialisation.

Common follow-up: *"How would you write a response model that supports both a list endpoint and a detail endpoint where detail includes extra fields?"*

Stand-out line: *"Say this and you'll separate yourself: one model per direction — `UserIn`, `UserOut`, `UserDB` — because every shared model eventually leaks the field you forgot was sensitive."*

```

from pydantic import BaseModel, ConfigDict
from fastapi import FastAPI

class UserListItem(BaseModel):
    model_config = ConfigDict(from_attributes=True)
    id: int
    display_name: str

class UserDetail(UserListItem):
    email: str
    created_at: datetime
    # password_hash deliberately absent

@app.get("/users", response_model=list[UserListItem])
def list_users() -> list[User]:
    return db.query(User).all() # each filtered through UserListItem

@app.get("/users/{id}", response_model=UserDetail)
def get_user(id: int) -> User:
    return db.query(User).get_or_404(id) # filtered through UserDetail

```

Q12 · Status Codes — The Defaults FastAPI Picks and How to Override

Tags: Difficulty: Easy · Asked at: Razorpay, Postman, Freshworks, Swiggy · Topic: FastAPI Response

The question (interviewer's verbatim phrasing):

When you write a FastAPI endpoint, what HTTP status code is returned by default, and how do you set a different one for a successful POST that creates a resource?

The 30-second answer: By default, every endpoint returns 200 OK on success. For a POST that creates a resource, the convention is 201 Created; you set it with `status_code=201` on the decorator:

`@app.post("/orders", status_code=201)`. For a 204 No Content on a DELETE that returns nothing, declare `status_code=204` and return `None`. For dynamic status codes, inject the `Response` object and set `response.status_code = 202` inside the function.

Step-by-step thought process: 1. Static status: `@app.post("/orders", status_code=201)` — every successful response from this endpoint gets a 201. Validation failures still return 422; `HTTPException` still wins with whatever code it raises. 2. Use `status.HTTP_201_CREATED` from `fastapi import status` rather than the bare number — readable, mypy-checkable, less likely to typo a 504 as 405. 3. 204 No Content is the standard for "I did the thing, nothing to say." Declare `status_code=204` and return `None`. Browsers and clients treat any body on a 204 as malformed; FastAPI omits the body automatically. 4. Dynamic per-request: inject `response: Response` into your endpoint, set `response.status_code = 202` for one branch and `response.status_code =`

200 for another. The `response_model` still runs on whatever you return. 5. Raising errors: `raise HTTPException(status_code=404, detail="not found")` returns a JSON body `{"detail": "not found"}` with the requested status. Use it for client-fault conditions; let server-fault conditions surface as 500 via the exception handler (Q23).

Edge cases to mention: - A POST that creates *and* returns the created object should still use 201; the body is the new resource, and the `Location` header optionally points to it. - 422 is FastAPI's default for request validation failures; you do not get to override that by setting `status_code=400` on the endpoint — it is owned by Pydantic's validator. - Streaming responses (`StreamingResponse`) bypass the response model; status codes are set on the response object directly.

What NOT to say: - "Use 200 for everything." — Cliche-rich anti-pattern; clients depend on the standard codes. - "201 is just a convention; 200 works fine." — Some clients (and almost every API design review at a product co) treat the wrong status code as a bug. - "I'll set the status code by returning a tuple like Flask does." — That is Flask. In FastAPI it does not work the same way.

Common follow-up: *"How would you write an idempotent PUT endpoint that returns 201 on create and 200 on update?"*

Stand-out line: *"Say this and you'll separate yourself: status codes are part of the API contract — 201 on POST-create, 204 on DELETE, 202 for accepted-not-yet-done — and `from fastapi import status` is how you keep mypy honest about them."*

```
from fastapi import FastAPI, Response, status

@app.post("/orders", status_code=status.HTTP_201_CREATED, response_model=OrderOut)
def create_order(body: OrderIn) -> Order:
    return orders_service.create(body)

@app.delete("/orders/{id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_order(id: int) -> None:
    orders_service.delete(id)
    return None # body is omitted on 204

@app.put("/orders/{id}", response_model=OrderOut)
def upsert_order(id: int, body: OrderIn, response: Response) -> Order:
    order, created = orders_service.upsert(id, body)
    response.status_code = status.HTTP_201_CREATED if created else status.HTTP_200_OK
    return order
```

Q13 · BackgroundTasks — Fire-and-Forget Without a Queue

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Swiggy, Cred, Freshworks · Topic: FastAPI Background

The question (interviewer's verbatim phrasing):

You need to send a welcome email after a user signs up, but you do not want to make the user wait for SMTP. FastAPI has a `BackgroundTasks` feature. Walk me through what it is, when to use it, and when it is the wrong tool.

The 30-second answer: `BackgroundTasks` is an injected dependency that lets you queue a function to run *after* the response has been sent, on the same worker process. You inject it into the endpoint signature, call `background_tasks.add_task(send_email, to=user.email)`, and FastAPI returns the response immediately, then runs the task. It is the right tool for short, fire-and-forget side effects on the same process — sending an email, hitting a webhook, writing an analytics event. It is the *wrong* tool for anything that needs durability, retries, or cross-process scaling — those go to Celery, RQ, Dramatiq, or an SQS/SNS pipeline.

Step-by-step thought process: 1. Mechanics: declare `background_tasks: BackgroundTasks` as a parameter. FastAPI sees the type and injects an empty task list. `background_tasks.add_task(callable, *args, **kwargs)` appends; multiple tasks run in registration order, serially. 2. Lifecycle: FastAPI returns the response from your endpoint to the client *first*, then runs the queued tasks. The user sees the success page before the email is sent. 3. Failure handling: an exception inside a background task is logged but does not surface to the client (the response has already gone out). You are responsible for retries, dead-lettering, and observability inside the task itself. 4. Sync vs async: a `def` task runs in the thread pool (same as a sync endpoint); an `async def` task is awaited on the event loop. The same blocking-call hazard applies — don't put `requests.get` inside an `async def` task. 5. The decision tree: use `BackgroundTasks` if (a) the task is fast (a few seconds max), (b) you can tolerate losing it on worker restart, and (c) the work is bounded — a single email, a single webhook. Use Celery/RQ/SQS if (a) you need durability across worker restarts, (b) you need retries with backoff, (c) you need to scale workers separately from the API, or (d) the task can take minutes.

Edge cases to mention: - A worker restart (SIGTERM, OOM, deploy) between response-send and task-run *drops* the task. There is no on-disk durability. - Tasks queued from inside a dependency (via `Depends`) also run after the response — useful for "log the request after the response is sent without slowing the response." - Class-based background tasks: pass a class with `__call__` for stateful tasks. Rare but valid.

What NOT to say: - "BackgroundTasks is FastAPI's task queue." — It is not a queue in the durable sense. It is in-process fire-and-forget. - "Use BackgroundTasks for everything async." — Wrong heuristic. Use it for short, lossy, in-process work; use a real queue for everything else. - "Tasks run in parallel." — They run sequentially in the order added.

Common follow-up: *"What changes in the design if the email send must be guaranteed delivered even if the worker crashes?"*

Stand-out line: "Say this and you'll separate yourself: `BackgroundTasks` is in-process fire-and-forget — losing the task on worker restart is the default, so for anything that needs durability, retries, or scaling, the answer is Celery or SQS, not `add_task`."

```
from fastapi import BackgroundTasks, FastAPI
import logging

logger = logging.getLogger(__name__)

def send_welcome_email(email: str, name: str) -> None:
    try:
        smtp_client.send(to=email, subject=f"Welcome {name}!", body=WELCOME_BODY)
    except Exception:
        logger.exception("welcome email failed for %s", email) # log; can't raise

@app.post("/signup", status_code=201, response_model=UserOut)
def signup(body: SignupIn, background_tasks: BackgroundTasks) -> User:
    user = users_service.create(body)
    background_tasks.add_task(send_welcome_email, email=user.email, name=user.name)
    return user # response sent, then email fires
```

Q14 · Path Operation Decorators — POST, PUT, PATCH, DELETE Semantics

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Postman, Swiggy · Topic: FastAPI REST

The question (interviewer's verbatim phrasing):

Walk me through when you use POST vs PUT vs PATCH vs DELETE on a FastAPI endpoint, and what the REST conventions are around request bodies and idempotency.

The 30-second answer: POST creates a new resource, returns 201 with the created object — not idempotent (calling it twice creates two resources). PUT replaces a whole resource at a known URL — idempotent (calling it twice leaves the resource in the same state). PATCH partially updates a resource — also idempotent if done right (you send only the fields that changed). DELETE removes the resource — idempotent (already-gone is not an error). FastAPI's `@app.post`, `@app.put`, `@app.patch`, `@app.delete` decorators map to these one-to-one.

Step-by-step thought process: 1. POST: `@app.post("/orders", status_code=201, response_model=OrderOut)`. Takes a body, creates a resource, returns it. Two POSTs with the same body = two orders. The collection URL `/orders` is the target, not a specific resource. 2. PUT: `@app.put("/orders/{id}", response_model=OrderOut)`. Takes a body that represents the *entire* desired state of the resource at `/orders/{id}`. If the resource exists, replace it; if not, create it (some APIs return 201; most return 200). Idempotent because the body fully specifies the end state. 3.

PATCH: `@app.patch("/orders/{id}", response_model=OrderOut)`. Takes a body with only the fields to change. Pydantic model should use `Optional` / `| None` for every field with a sentinel-aware default, and you apply only the explicitly-provided fields. Idempotent only if the patch is the same each time. 4. DELETE: `@app.delete("/orders/{id}", status_code=204)`. No body, no response body. Idempotent: if the order is already deleted, return 204 anyway (some APIs prefer 404; pick one and document it). 5. The reason these matter: clients, proxies and frameworks (browsers, retry middleware, CDNs) make assumptions based on the method. PUTs and DELETes get auto-retried by browsers; POSTs do not. Picking the wrong method leaks correctness bugs through that machinery.

Edge cases to mention: - PATCH semantics are not fully standard — JSON Merge Patch (RFC 7396) is the common convention; JSON Patch (RFC 6902) is more explicit and rare. Most APIs default to merge. - POST is allowed at any URL — `/orders/{id}/cancel` is a POST to a custom action. The "no path on POST" rule is a convention, not REST law. - DELETE with a body is technically legal in HTTP but most proxies and clients drop it. Avoid.

What NOT to say: - "PUT is for updates and POST is for creates." — Roughly right but incomplete; the deeper rule is idempotence and target. - "PATCH and PUT are the same." — They are not; PUT requires the whole body, PATCH takes a delta. - "DELETE should return 200 with a body confirming deletion." — Sometimes useful; more idiomatic to return 204 No Content.

Common follow-up: *"Write a PATCH endpoint that updates only the fields the client sends, with the rest preserved."*

Stand-out line: *"Say this and you'll separate yourself: the deep rule is idempotence and target, not 'POST creates and PUT updates' — browsers and retry middleware auto-retry PUT and DELETE on the assumption you're telling the truth about it."*

```
from pydantic import BaseModel
from fastapi import FastAPI

class OrderPatch(BaseModel):
    notes: str | None = None
    status: str | None = None

@app.patch("/orders/{id}", response_model=OrderOut)
def patch_order(id: int, body: OrderPatch) -> Order:
    order = orders_service.get_or_404(id)
    patch = body.model_dump(exclude_unset=True) # only the keys the client sent
    for field, value in patch.items():
        setattr(order, field, value)
    db.commit()
    return order
```

Q15 · APIRouter — Modularising a FastAPI Codebase

Tags: Difficulty: Easy · Asked at: Razorpay, Postman, Swiggy, Cred · Topic: FastAPI Structure

The question (interviewer's verbatim phrasing):

A FastAPI app at scale has dozens of endpoints. What's the equivalent of Flask's blueprints, and how do you structure a multi-module FastAPI codebase?

The 30-second answer: `APIRouter` is FastAPI's blueprint-equivalent — you create routers per feature, register routes against them, and include them on the main app with a prefix and tags. Routers can have their own dependencies that apply to every route in the router (handy for "every endpoint here requires authentication"), their own exception handlers, and their own prefix. A typical structure is `app/routers/orders.py` defining `router = APIRouter(prefix="/orders", tags=["orders"])`, then `app.include_router(orders.router)` inside the factory.

Step-by-step thought process: 1. Create: `from fastapi import APIRouter; router = APIRouter(prefix="/orders", tags=["orders"])`. Then `@router.get("/")` for `GET /orders` and `@router.get("/{id}")` for `GET /orders/{id}`. The prefix is applied at inclusion, the tags propagate to every route in the router. 2. Include: in `main.py` or your app factory, `from app.routers import orders, users, payments` then `app.include_router(orders.router); app.include_router(users.router); app.include_router(payments.router)`. The order of inclusion is the order Swagger UI displays them. 3. Router-level dependencies: `router = APIRouter(prefix="/admin", dependencies=[Depends(require_admin)])`. Every route in the router runs `require_admin` before the route. Cleaner than decorating each endpoint. 4. Nested inclusion: a router can include another router, useful for `/api/v1/orders/...` style nested versioning. `v1_router.include_router(orders.router); app.include_router(v1_router, prefix="/api/v1")`. 5. Override at inclusion: `app.include_router(orders.router, prefix="/api/v2/orders", tags=["orders-v2"])` lets you mount the same router at multiple URLs — useful during versioned migrations.

Edge cases to mention: - Tags on the router and tags on the route are concatenated; you can add per-route tags without losing the router-level grouping. - `include_router` is the only sanctioned way to attach routes from a router; importing the router and reassigning routes manually breaks OpenAPI generation. - Router-level `responses={401: {"description": "Unauthorized"}}` adds response documentation to every route in the router — useful for advertising common error envelopes.

What NOT to say: - "FastAPI doesn't have blueprints." — `APIRouter` is the equivalent. - "Just put everything in main.py." — Works for a tutorial; not for a 50-endpoint service. - "Routers are just for organisation." — They are also dependency-scope boundaries and tag-scope boundaries. Both matter at scale.

Common follow-up: "How would you structure a FastAPI codebase with versioned APIs — v1 and v2 of orders?"

Stand-out line: "Say this and you'll separate yourself: `APIRouter` is a dependency-scope boundary and a tag-scope boundary, not just a file-split — router-level

`dependencies=[Depends(require_auth)]` is how you guarantee every endpoint in the router gets the check."

```
# app/routers/orders.py
from fastapi import APIRouter, Depends
from app.deps import require_auth

router = APIRouter(prefix="/orders", tags=["orders"], dependencies=[Depends(require_auth)])

@router.get("/", response_model=list[OrderOut])
def list_orders():
    return orders_service.list()

@router.get("/{id}", response_model=OrderOut)
def get_order(id: int):
    return orders_service.get_or_404(id)

# app/main.py
from fastapi import FastAPI
from app.routers import orders, users, payments

def create_app() -> FastAPI:
    app = FastAPI(title="Merchant API", version="2.3.1")
    app.include_router(orders.router, prefix="/api/v1")
    app.include_router(users.router, prefix="/api/v1")
    app.include_router(payments.router, prefix="/api/v1")
    return app

app = create_app()
```

Section C — Async Support in Depth

This section is where mid-level Indian-product-co panels separate the FastAPI-tutorial graduate from the engineer who has actually shipped an async service. The questions here are the round-two material — when does async actually pay off, what breaks when you mix sync and async, how do you fire-and-forget within a request, how do you use WebSockets, and what is the ASGI/WSGI line. Five questions; every one is a real interview signal at Razorpay, Postman, Swiggy, Cred and Zerodha.

Q16 · When FastAPI Is Actually Faster Than Flask

Tags: Difficulty: Hard · Asked at: Razorpay, Postman, Swiggy, Cred, Zerodha · Topic: Async Performance

The question (interviewer's verbatim phrasing):

FastAPI is "fast." Be precise — when is it actually faster than Flask, and when is the difference negligible?

The 30-second answer: FastAPI is meaningfully faster than Flask in two regimes: high-concurrency I/O-bound workloads (many endpoints awaiting a DB, a Redis call, an upstream HTTP) and in raw request-parsing throughput when Pydantic v2 is doing the validation in Rust. For purely CPU-bound endpoints, or for workloads where the bottleneck is downstream latency, the difference between FastAPI and Flask collapses to noise — the framework is not on the critical path. The honest framing is "FastAPI's async model lets a single worker handle thousands of concurrent in-flight requests; Flask's WSGI model handles one per worker thread." Throughput at high concurrency is where you see it; raw single-request latency is where you don't.

Step-by-step thought process: 1. The async win: an `async def` endpoint that awaits `httpx`, `asyncpg`, `aioredis` releases the event loop on every I/O. One worker can have 5,000 concurrent in-flight requests. Flask, on a worker pool of 40 threads, tops out at 40 concurrent in-flight requests. For I/O-bound workloads with long awaits (an upstream payment gateway taking 500ms), FastAPI handles 100x the concurrency on the same hardware. 2. The Pydantic win: Pydantic v2's core is Rust. Body parsing, validation, response serialisation — all faster than Flask's `request.get_json()` + manual validation. On JSON-heavy endpoints with complex schemas, this is a measurable throughput edge even at low concurrency. 3. The CPU-bound case: if your endpoint does heavy CPU work (regex over a big string, a tight Python loop, JSON computation), the event loop blocks anyway. FastAPI is no faster than Flask here; both saturate one core per request. 4. The downstream-dominated case: if your endpoint's wall-clock is 95% downstream latency (a slow DB query, a slow upstream API), the framework choice is invisible — both Flask and FastAPI add < 5ms. Pick whichever your team is fluent in. 5. The benchmarking honesty: published benchmarks (TechEmpower, framework-perf shootouts) measure trivial JSON-echo endpoints at extreme concurrency. Real applications rarely look like that. Use your own profiler.

Edge cases to mention: - Flask's async support (`async def` views) is implemented via `asgiref` and runs the view in a thread pool — so async Flask views do NOT give you the FastAPI concurrency profile, they give you Flask + an extra hop. - gunicorn + sync workers caps your concurrency at `workers * 1`; gunicorn + gthread caps at `workers * threads`; uvicorn workers under FastAPI cap at `workers * (event loop's in-flight capacity)`. - Pydantic v2 vs v1: if you are stuck on v1 for compatibility, you lose the Rust win — about a 5–50x serialisation speedup depending on schema complexity.

What NOT to say: - "FastAPI is always faster than Flask." — Only for the right workload. - "FastAPI is twice as fast." — Vague; benchmarks vary by an order of magnitude depending on the workload. - "Flask is slow." — Flask is fine; it has a different concurrency model.

Common follow-up: "Walk me through a benchmarking setup you would use to decide whether to port a Flask service to FastAPI."

Stand-out line: "Say this and you'll separate yourself: FastAPI's win is concurrency on I/O-bound workloads, not raw throughput on every endpoint — port the service when concurrent in-flight requests are the bottleneck, not because the benchmark shootout said so."

Q17 · The Sync-Inside-Async Trap — How It Blocks the Loop

Tags: Difficulty: Hard · Asked at: Razorpay, Swiggy, Postman, Cred, Walmart Labs · Topic: Async Pitfalls

The question (interviewer's verbatim phrasing):

An engineer on your team wrote an `async def` endpoint that calls `requests.get(...)`. Production starts dropping connections under load. Explain what's happening and what the fix is.

The 30-second answer: The endpoint is `async def` so FastAPI awaits it on the event loop. `requests.get(...)` is synchronous and blocks the thread for the duration of the HTTP call. While that thread is blocked, the event loop cannot service any other request — every other in-flight async request stalls. Under load, those stalls compound, downstream clients time out, connections back up, and the worker starts dropping. The fix is one of two: change the endpoint to `def` (FastAPI offloads to the thread pool, loop stays free), or change `requests.get` to `httpx.AsyncClient().get(...)` with an `await` (the call yields to the loop).

Step-by-step thought process: 1. Diagnose: an `async def` endpoint runs on the loop thread. If the loop thread is blocked, every concurrent async task waits. The symptom is wall-clock latency that scales with concurrency even though "CPU is fine." 2. Confirm: `py-spy dump --pid PID` on the worker process shows the event loop thread stuck inside `requests` socket recv. That is the smoking gun. 3. Two valid fixes. Option A: change `async def endpoint(...)` to `def endpoint(...)`. FastAPI offloads to the anyio thread pool (default 40 workers); the loop stays free. Each request takes a thread, but threads are cheap and the loop is unblocked. 4. Option B: change `requests.get(...)` to `await httpx.AsyncClient().get(...)`. The call yields to the loop; the loop services other tasks during the wait. Higher concurrency ceiling than option A. 5. The deeper rule: an `async def` endpoint must `await` for every I/O it performs. If you can't easily port the dependency to async, write a plain `def` endpoint and let the thread pool handle it.

Edge cases to mention: - `asyncio.run_in_executor(None, blocking_func)` lets you offload a blocking call from within an `async def` to the default thread pool, but you must remember to use it — the bug pattern is forgetting and just calling the sync function directly. - CPU-bound code inside `async def` is the same trap with a different cause — a tight Python loop or a big regex blocks the loop. `await asyncio.to_thread(...)` is the offload escape hatch. - This is the single most common production bug in early FastAPI codebases. Lint for it: `grep async def` files for calls to `requests`, `urllib`, `psycopg2`, `redis-py` (sync mode), `pymongo`.

What NOT to say: - "Just add a try/except around `requests.get` ." — Doesn't fix anything; the call still blocks. - "Use threading inside the async function." — `requests.get` from inside `asyncio.to_thread(...)` works but is uglier than just making the endpoint `def` . Pick the right tool. - "Increase the worker count." — Treats the symptom, not the cause. The async worker only has one loop; more workers don't help a single worker's loop.

Common follow-up: "How would you write a linter check that catches this in code review?"

Stand-out line: "Say this and you'll separate yourself: every senior FastAPI engineer greps `async def` files for `requests` , `psycopg2` , `pymongo` and sync-mode `redis-py` — the sync-inside-async pattern is the single most common production fire in early FastAPI codebases."

```
# BUG – async def + blocking call. One slow upstream stalls all concurrent requests.
@app.get("/orders/{id}/details")
async def details_broken(id: int) -> dict:
    upstream = requests.get(f"https://payments/{id}", timeout=5) # blocks loop
    return upstream.json()

# Fix A – make the endpoint plain def, thread pool absorbs the blocking.
@app.get("/orders/{id}/details")
def details_sync(id: int) -> dict:
    upstream = requests.get(f"https://payments/{id}", timeout=5)
    return upstream.json()

# Fix B – keep async, switch to httpx.AsyncClient and await.
@app.get("/orders/{id}/details")
async def details_async(id: int) -> dict:
    async with httpx.AsyncClient(timeout=5) as client:
        upstream = await client.get(f"https://payments/{id}")
    return upstream.json()
```

Q18 · `asyncio.create_task` Inside an Endpoint — When and Why

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Postman, Swiggy, Goldman Sachs India · Topic: Async Patterns

The question (interviewer's verbatim phrasing):

Inside an `async def` endpoint, when would you reach for `asyncio.create_task` instead of just `await` ? And what are the gotchas?

The 30-second answer: You use `asyncio.create_task` when you want to start an awaitable running concurrently with other work in the same endpoint — typically when you have two independent I/O calls and want both in flight at the same time. The gotchas are three: the task can be garbage-

collected if nothing holds a reference to it, exceptions raised inside the task are silently swallowed unless you `await` it or attach a callback, and a task that outlives the request is *not* a substitute for `BackgroundTasks` — it gets cancelled when the event loop shuts down or the worker restarts. For "fan out two requests and combine the results," use `asyncio.gather`. For "fire and forget after the response," use `BackgroundTasks` or a real queue.

Step-by-step thought process: 1. `await` is sequential — `await call_a(); await call_b()` runs them back to back. If each takes 200ms, total is 400ms. 2.

`asyncio.create_task(call_a()); asyncio.create_task(call_b())` schedules both immediately; you then `await` them to collect results. Total is `max(200ms, 200ms) = 200ms`. The shortcut for "create both and await both" is `await asyncio.gather(call_a(), call_b())`. 3. Gotcha 1 — garbage collection: `asyncio.create_task` only weakly references the task. If you do `asyncio.create_task(call())` and don't hold the result, the task can be GC'd mid-flight. Keep a strong reference: assign to a variable, or `app.state.bg_tasks` set. 4. Gotcha 2 — exception swallowing: if a task raises and you never `await` it, the exception is logged at the next event-loop tick and then forgotten. Add a `task.add_done_callback(handle_error)` or always `await` the task. 5. Gotcha 3 — lifetime: a task that runs longer than the request is NOT durable. If the worker is recycled (timeout, deploy, OOM), the task is cancelled mid-flight. Use `BackgroundTasks` for fire-and-forget within the request lifetime; use a real queue (Celery, RQ, SQS) for "must survive worker restart."

Edge cases to mention: - `asyncio.TaskGroup` (Python 3.11+) is the structured-concurrency replacement for `gather` — exceptions propagate cleanly and child tasks are auto-cancelled on error. Prefer it in new code. - `asyncio.shield(task)` prevents cancellation propagation — useful when an upstream cancel should not abort an in-flight critical write. - The "create_task and don't await" anti-pattern is how you build "fire and forget" within a request — but it has every gotcha above. `BackgroundTasks` is almost always the right call for that intent.

What NOT to say: - "`create_task` is the way to run things in parallel." — `gather` and `TaskGroup` are the cleaner APIs for parallel-then-collect. - "Just `create_task` and move on." — Drops exceptions, GC's the task, no observability. - "Use `create_task` for background work." — That's `BackgroundTasks`'s job; `create_task` has worse lifetime guarantees.

Common follow-up: "Write an endpoint that fetches three independent upstream APIs in parallel and combines the result."

Stand-out line: "Say this and you'll separate yourself: reach for `gather` (or `TaskGroup` on 3.11+) for fan-out-and-combine — `create_task` without holding a reference is the GC-eats-the-task bug that ships silently."

```
import asyncio, httpx
from fastapi import FastAPI

@app.get("/dashboard/{user_id}")
async def dashboard(user_id: int) -> dict:
    async with httpx.AsyncClient(timeout=5) as client:
        # asyncio.gather – runs all three concurrently, returns when all done
        orders, payments, support = await asyncio.gather(
            client.get(f"https://orders/users/{user_id}"),
            client.get(f"https://payments/users/{user_id}"),
            client.get(f"https://support/users/{user_id}"),
        )
    return {
        "orders": orders.json(),
        "payments": payments.json(),
        "support": support.json(),
    }

# Python 3.11+ – TaskGroup is the structured-concurrency form
async def dashboard_v2(user_id: int) -> dict:
    async with httpx.AsyncClient(timeout=5) as client:
        async with asyncio.TaskGroup() as tg:
            t1 = tg.create_task(client.get(f"https://orders/users/{user_id}"))
            t2 = tg.create_task(client.get(f"https://payments/users/{user_id}"))
            t3 = tg.create_task(client.get(f"https://support/users/{user_id}"))
        return {"orders": t1.result().json(), "payments": t2.result().json(), "support": t3.result().json()}
```

Q19 · WebSockets in FastAPI — Endpoint, Lifecycle, Broadcast

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Swiggy, Zerodha · Topic: FastAPI WebSockets

The question (interviewer's verbatim phrasing):

FastAPI supports WebSockets natively. Walk me through how you declare one, the lifecycle of a connection, and how you would broadcast a message to all connected clients.

The 30-second answer: A WebSocket endpoint is declared with `@app.websocket("/ws/path")` and takes a `WebSocket` parameter. Inside, you call `await websocket.accept()` to complete the handshake, then loop on `await websocket.receive_text()` (or `receive_json`, `receive_bytes`) to read messages and `await websocket.send_text(...)` to write. Disconnects raise `WebSocketDisconnect`, which you catch to clean up. For broadcast, you maintain a connection-manager singleton — a `set[WebSocket]` of accepted connections, plus `connect`, `disconnect`, `broadcast` methods. FastAPI does the handshake; you own the routing logic.

Step-by-step thought process: 1. Declaration: `@app.websocket("/ws/orders")` registers a WS handler at that path. The function takes `websocket: WebSocket` and is `async def`. 2. Handshake: `await websocket.accept()` upgrades the HTTP connection to WebSocket. Skip the call and the client sees a refusal. 3. The loop: `while True: data = await websocket.receive_text()` blocks on the next message. Pair with `await websocket.send_text(...)` to reply. Use `receive_json` / `send_json` for structured messages. 4. Disconnect: `except WebSocketDisconnect: ...` catches a client hangup. Always clean up — remove the connection from any broadcast set, close any DB cursors, log the event. 5. Broadcast pattern: a `ConnectionManager` class holds `active: set[WebSocket]`. `connect(ws)` accepts and adds; `disconnect(ws)` removes; `broadcast(message)` iterates and `await` sends to every active connection, catching per-connection failures so one dead client doesn't block the broadcast.

Edge cases to mention: - WebSocket endpoints do not get the `Depends` dependency tree the same way HTTP endpoints do — only a subset of dependencies work, and behaviour around DB sessions changes. Authenticate via the initial `Sec-WebSocket-Protocol` or a query-string token, not via a `Depends` that reads headers. - A WebSocket dropped mid-send raises `RuntimeError`. Wrap each send in a try/except so one dead connection doesn't break the broadcast loop. - For real-scale broadcast you outgrow the single-process pattern and need a pub/sub backplane — Redis pub/sub, NATS, or a dedicated WebSocket service. FastAPI ships the receiving side; the fan-out is your design.

What NOT to say: - "FastAPI's WebSocket support is experimental." — It is production-ready and used at scale (Postman, Razorpay, Cred). - "WebSockets are just long-lived HTTP." — They are a different protocol after the upgrade; the multiplexing rules differ. - "Just `create_task` to broadcast." — Hides errors and leaks tasks; the explicit manager pattern is cleaner.

Common follow-up: *"How would you authenticate a WebSocket connection?"*

Stand-out line: *"Say this and you'll separate yourself: a connection-manager singleton plus a per-send try/except is the broadcast pattern — one dead client should never break the fan-out, and `Depends` doesn't fully work on WS endpoints so auth lives in the query-string token."*

```

from fastapi import FastAPI, WebSocket, WebSocketDisconnect

class ConnectionManager:
    def __init__(self) -> None:
        self.active: set[WebSocket] = set()

    async def connect(self, ws: WebSocket) -> None:
        await ws.accept()
        self.active.add(ws)

    def disconnect(self, ws: WebSocket) -> None:
        self.active.discard(ws)

    async def broadcast(self, message: str) -> None:
        for ws in list(self.active):
            try:
                await ws.send_text(message)
            except Exception:
                self.disconnect(ws)

manager = ConnectionManager()

@app.websocket("/ws/orders")
async def orders_stream(websocket: WebSocket):
    await manager.connect(websocket)
    try:
        while True:
            msg = await websocket.receive_text()
            await manager.broadcast(f"client said: {msg}")
    except WebSocketDisconnect:
        manager.disconnect(websocket)

```

Q20 · ASGI vs WSGI — The One-Line Difference That Changes Everything

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Swiggy, Cred, Microsoft India · Topic: Protocols

The question (interviewer's verbatim phrasing):

What is the difference between WSGI and ASGI, and why does it matter for picking Flask versus FastAPI?

The 30-second answer: WSGI is a synchronous protocol — the framework exposes a callable that takes `environ` and `start_response`, returns an iterable of bytes, and the server (gunicorn, uWSGI) calls it once per request. ASGI is the async sibling — the framework exposes an async callable that takes `scope`, `receive`, `send`, and the server (uvicorn, hypercorn) drives it as an event-loop coroutine. ASGI supports WebSockets, long-poll streaming, server-sent events, and lifespan events

natively; WSGI does not. Flask is WSGI; FastAPI (and Starlette underneath) is ASGI. Picking ASGI gets you async I/O, WebSockets and lifespan hooks; picking WSGI gets you a mature ecosystem and simpler debugging.

Step-by-step thought process: 1. WSGI shape: one Python function `app(environ, start_response) -> Iterable[bytes]`. Synchronous, request-scoped, one shot. Defined in PEP 3333 (2010). 2. ASGI shape: one async function `app(scope, receive, send)`. `scope` describes the connection, `receive` is an async callable that returns events (request bodies, WebSocket messages), `send` is an async callable that emits events (responses, WebSocket messages). Defined in 2018, formalised over the next few years. 3. The big differences ASGI adds: native async/await, native WebSockets, native server-sent events, lifespan events (startup/shutdown hooks), long-poll streaming. WSGI lacks all of these — projects like `gevent` and `eventlet` faked async on WSGI but with serious trade-offs. 4. Server matrix: WSGI → gunicorn (with sync, gthread, gevent workers), uWSGI, mod_wsgi. ASGI → uvicorn, hypercorn, daphne. Modern deployments are `gunicorn` with `uvicorn` workers — gunicorn for process supervision, uvicorn for ASGI. 5. Migration: WSGI apps can run inside ASGI via `WSGIMiddleware` (and vice versa via `asgiref.WsgiToAsgi`); useful for incremental ports. Native is faster and exposes more features.

Edge cases to mention: - Flask 2.0+ added `async def` view support via `asgiref` but Flask itself is still a WSGI framework — the async views run inside a thread pool, so you don't get the ASGI concurrency profile. - Some Starlette features (lifespan, WebSockets) leak through to FastAPI; FastAPI's `@app.on_event("startup")` is a thin wrapper over the ASGI lifespan protocol — superseded by `lifespan=...` arg in newer FastAPI. - The WSGI vs ASGI divide is invisible in trivial endpoints; it dominates the design of anything that needs streaming or long-lived connections.

What NOT to say: - "ASGI is WSGI 2.0." — Different design. ASGI is async-first. - "Flask is WSGI so it can't do anything async." — Flask supports `async def` views since 2.0 (with caveats). - "Always pick ASGI." — Match the framework to the workload. WSGI is fine for synchronous JSON APIs.

Common follow-up: *"How would you deploy a FastAPI app on a server that only supports WSGI?"*

Stand-out line: *"Say this and you'll separate yourself: ASGI versus WSGI is the protocol decision, not a perf decision — pick ASGI when you need WebSockets, streaming, or lifespan hooks, and WSGI when the synchronous request-response shape is the whole story."*

```
# WSGI – Flask. Synchronous callable.
# server: gunicorn -w 4 wsgi:app
from flask import Flask
app = Flask(__name__)

# ASGI – FastAPI. Async callable, lifespan + WebSockets supported.
# server: gunicorn -w 4 -k uvicorn.workers.UvicornWorker main:app
from contextlib import asynccontextmanager
from fastapi import FastAPI

@asynccontextmanager
async def lifespan(app: FastAPI):
    # startup
    app.state.db = await create_engine_async("postgresql://...")
    yield
    # shutdown
    await app.state.db.dispose()

app = FastAPI(lifespan=lifespan)
```

Section D — Production Patterns

This section is the round-three material — questions that come up when the panel believes you can write FastAPI but wants to know whether you can run it. Deployment topology, request validation under Pydantic v2, exception handling, middleware ordering, and CORS. Five questions; missing any of these signals you have never owned the deploy.

Q21 · uvicorn + gunicorn + Workers — The Production Deployment Topology

Tags: Difficulty: Hard · Asked at: Razorpay, Postman, Swiggy, Cred, Zerodha, Walmart Labs · Topic: Deployment

The question (interviewer's verbatim phrasing):

Walk me through how you deploy a FastAPI service in production. What are uvicorn and gunicorn, why use both, and how do you size workers?

The 30-second answer: The standard topology is gunicorn as the process supervisor, running uvicorn workers as the ASGI runtime. gunicorn handles worker lifecycle — restarts on crash, graceful reloads on signal, max-requests recycling — while uvicorn handles the ASGI protocol and the event loop. The invocation is `gunicorn -w N -k uvicorn.workers.UvicornWorker main:app`. Worker count

starts at `(2 * CPU cores) + 1` for I/O-bound apps and falls to `CPU cores` for CPU-bound. Each worker is one process with one event loop; concurrency *within* a worker scales with async tasks, not threads. Behind that you put nginx or a cloud load balancer for TLS termination and connection pooling.

Step-by-step thought process: 1. Why both: uvicorn alone (`uvicorn main:app --workers 4`) runs ASGI workers but with weaker process supervision — no graceful restart on signal, fewer hooks, less observable. gunicorn adds the production process management while delegating ASGI to uvicorn. 2. Worker count: for I/O-bound services start at `2 * cores + 1` (the classic gunicorn heuristic) and tune. Each worker has its own event loop; async concurrency *inside* a worker is high (thousands of in-flight async tasks), so you rarely need more workers than cores. 3. Worker recycling: `--max-requests 1000 --max-requests-jitter 100` recycles each worker after roughly 1,000 requests with random jitter. Defensive against slow memory leaks and stale connection pools; the jitter prevents thundering-herd restarts. 4. Timeouts: `--timeout 60` kills a worker that hasn't responded in 60s — protects against runaway requests. For async workloads, the timeout should be generous (60–120s) because the worker is supposed to handle many concurrent in-flight requests; killing it kills *all* of them. 5. Behind it: nginx or an AWS ALB / GCP LB for TLS termination, HTTP/2, connection pooling and X-Forwarded-For propagation. Configure FastAPI's `proxy_headers=True` (the uvicorn flag) and trust the proxy's `X-Forwarded-For`, otherwise `request.client.host` is always the proxy IP.

Edge cases to mention: - For CPU-bound workloads, each worker is one core's worth of throughput. Adding more workers than cores fights the scheduler. For I/O-bound workloads, async concurrency inside the worker dominates — more workers buys little. - The `--worker-class` for plain Flask is `gthread` or `sync`; for FastAPI it must be `uvicorn.workers.UvicornWorker` (or the newer `uvicorn-worker` package). Wrong worker class is the #1 deployment bug — the app runs but loses async concurrency. - For zero-downtime deploys, gunicorn's `--reload` is dev-only; production uses `kill -HUP` to gracefully reload workers, or you run two replicas behind a load balancer and drain one at a time.

What NOT to say: - "Just use uvicorn alone in production." — Works but loses process supervision; debugging worker crashes becomes painful. - "More workers always helps." — Diminishing returns past `2 * cores`; threads/coroutines inside the worker scale further. - "Use the default timeout." — gunicorn's default is 30s, which is too tight for many APIs and will kill long-running async tasks.

Common follow-up: *"How would you structure a Dockerfile for this deployment?"*

Stand-out line: *"Say this and you'll separate yourself: gunicorn for process supervision, uvicorn workers for ASGI — and `--max-requests` with jitter is the cheap memory-leak insurance every long-running async worker needs."*


```
# Dockerfile — FastAPI + gunicorn + uvicorn
FROM python:3.12-slim AS base
WORKDIR /app
COPY pyproject.toml uv.lock ./
RUN pip install --no-cache-dir uv && uv sync --frozen --no-dev
COPY . .

EXPOSE 8000
CMD ["uv", "run", "gunicorn", \
    "-w", "4", \
    "-k", "uvicorn.workers.UvicornWorker", \
    "--max-requests", "1000", \
    "--max-requests-jitter", "100", \
    "--timeout", "60", \
    "--access-logfile", "-", \
    "--bind", "0.0.0.0:8000", \
    "app.main:app"]
```

Q22 · Pydantic v2 Validation Patterns That Show Up in Interviews

Tags: Difficulty: Hard · Asked at: Razorpay, Postman, Cred, Swiggy, Zerodha · Topic: Pydantic

The question (interviewer's verbatim phrasing):

Pydantic v2 changed a lot from v1. Walk me through the validator patterns you actually use in production — field validators, model validators, computed fields, settings.

The 30-second answer: The v2 validator API is `field_validator` for single-field checks (replacing v1's `validator`), `model_validator` for cross-field invariants (replacing v1's `root_validator`), and `computed_field` for derived properties that should appear in serialisation but are not stored. `BaseSettings` moved out of pydantic core into `pydantic-settings` as a separate package — it reads from env vars and `.env` files with explicit type coercion. The defining v2 patterns are: validators are decorators on classmethods with `mode="before"` or `mode="after"`, `model_config = ConfigDict(...)` replaces the v1 `class Config`, and the serialisation API is `.model_dump()` instead of `.dict()`.

Step-by-step thought process: 1. `field_validator` for single fields: validates and optionally transforms one field's value. `mode="before"` runs on the raw input (great for normalising input formats); `mode="after"` runs after the type coercion (great for invariants). The function is a classmethod that returns the value. 2. `model_validator` for cross-field rules: "if `payment_method` is `card` then `card_number` is required" lives here. `mode="after"` runs after all field validation, gives you the fully-constructed `self`, and you `return self` (possibly modified). 3. `computed_field` for derived properties: a `@computed_field @property` on the

model is serialised as part of `.model_dump()`. Use for `total_paise = quantity * unit_price_paise` that the client should see but isn't stored. 4. `model_config = ConfigDict(...)` is where you configure `from_attributes=True` (the old `orm_mode`), `populate_by_name=True` (for alias-based input), `str_strip_whitespace=True`, etc. The old `class Config:` syntax still works but `ConfigDict` is the canonical v2 form. 5. `BaseSettings` is now `from pydantic_settings import BaseSettings`. It reads env vars by default, plus `.env` via `model_config = SettingsConfigDict(env_file=".env")`. Type-coerces everything: `PORT: int = 8000` reads `PORT=8000` from env and gives you an int.

Edge cases to mention: - `field_validator(..., mode="before")` runs *before* type coercion, so the input might be a `str` even if the field is declared as `int`. Useful for normalising "yes"/"no" → bool, "₹1,000" → int. - `model_validator(mode="after")` runs after individual field validators; raising `ValueError` from inside it surfaces as a 422 with structured `loc`. - Pydantic v2 supports `RootModel` for models whose root is not a dict (a list, a single value). Use for endpoints that expect a top-level array body: `class UserList(RootModel[list[UserIn]])`.

What NOT to say: - "v2 is the same as v1." — The API surface broke in many places; v1 code does not run unchanged. - "Use `validator` and `root_validator`." — Those are v1; v2 wants `field_validator` and `model_validator`. - "Pydantic settings is in core." — Moved to `pydantic-settings` in v2.

Common follow-up: "Show me a model that validates an email, normalises it, and enforces that the password is at least 12 characters with at least one digit."

Stand-out line: "Say this and you'll separate yourself: v2's `field_validator`, `model_validator`, and `computed_field` are the three axes — single-field, cross-field, derived — and once you internalise the split, every validation question is a one-decorator answer."

```

from pydantic import BaseModel, ConfigDict, EmailStr, field_validator, model_validator, computed_field
from pydantic_settings import BaseSettings, SettingsConfigDict

class SignupIn(BaseModel):
    model_config = ConfigDict(str_strip_whitespace=True)

    email: EmailStr
    password: str
    confirm_password: str

    @field_validator("email", mode="after")
    @classmethod
    def lower_email(cls, v: str) -> str:
        return v.lower()

    @field_validator("password")
    @classmethod
    def password_strength(cls, v: str) -> str:
        if len(v) < 12:
            raise ValueError("password must be at least 12 characters")
        if not any(c.isdigit() for c in v):
            raise ValueError("password must contain a digit")
        return v

    @model_validator(mode="after")
    def passwords_match(self) -> "SignupIn":
        if self.password != self.confirm_password:
            raise ValueError("passwords do not match")
        return self

class OrderOut(BaseModel):
    items: list[Item]
    @computed_field
    @property
    def total_paise(self) -> int:
        return sum(i.quantity * i.unit_price_paise for i in self.items)

class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", env_prefix="APP_")
    db_url: str
    port: int = 8000
    debug: bool = False

```

Q23 · Exception Handlers — Catching, Translating, and Not Leaking

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Cred, Swiggy, JPMC · Topic: Error Handling

The question (interviewer's verbatim phrasing):

A downstream service raises a custom exception. How do you turn that into a clean 502 response in FastAPI without writing try/except in every endpoint?

The 30-second answer: Register an exception handler with

`@app.exception_handler(YourException)`. The function is called whenever any endpoint raises that exception type (or a subclass), and it returns a `JSONResponse` (or any `Response`) with the status code and body you choose. FastAPI ships handlers for `HTTPException` and `RequestValidationError` out of the box; you add your own for domain exceptions. The win is that your endpoint code stays clean — `raise UpstreamDownError("payments")` — while the handler centralises the JSON shape, the logging, and the status code.

Step-by-step thought process: 1. Declare the exception: `class`

`UpstreamDownError(Exception):` `def __init__(self, service: str):` `self.service = service`. No HTTP semantics yet; this is a pure domain exception. 2. Register the handler:

`@app.exception_handler(UpstreamDownError)` decorating an `async def` that takes `(request: Request, exc: UpstreamDownError)` and returns a

`JSONResponse(status_code=502, content={"error": "upstream_down", "service": exc.service})`. The function is called on every match. 3. Inheritance: a handler for a base class catches all subclasses. Useful for "any `IntegrityError` from SQLAlchemy → 409 Conflict" patterns.

4. Built-in handlers: `HTTPException` produces a 4xx/5xx with `{"detail": ...}`;

`RequestValidationError` produces a 422 with the structured Pydantic error list. You can override these — register your own `@app.exception_handler(RequestValidationError)` to reshape the 422 envelope to match your API's error contract. 5. Log inside the handler: every domain exception that surfaces to a handler is a real production event. Log with the request path, the exception type, and the user context so observability tooling can alert on spikes.

Edge cases to mention: - Exception handlers do not run inside `BackgroundTasks` — those exceptions are logged by the task runner but not routed through your handler. Wrap each background task with try/except internally. - A handler that itself raises is not retried; the response becomes a 500 from the default ASGI error path. Keep handlers boring. -

`Starlette.exceptions.HTTPException` and `fastapi.HTTPException` are not the same class. FastAPI's adds a `headers` argument; if you register a handler for one, register both to avoid accidentally missing the other.

What NOT to say: - "Use try/except in every endpoint." — That's exactly what exception handlers are for; they DRY up the code. - "FastAPI handles all exceptions automatically." — Only `HTTPException` and `RequestValidationError`. Everything else surfaces as 500 unless you handle it. - "Just return 500 for any unexpected exception." — Loses the chance to translate domain errors to the right HTTP code.

Common follow-up: "How would you reshape FastAPI's default 422 response to match a `{"error": {"code": "...", "details": [...]}}` envelope?"

Stand-out line: *"Say this and you'll separate yourself: register handlers for domain exceptions so endpoint code stays `raise UpstreamDownError('payments')` — the handler centralises the JSON envelope, the status code, and the logging, and the endpoint stays readable."*

```
from fastapi import FastAPI, Request, HTTPException
from fastapi.exceptions import RequestValidationError
from fastapi.responses import JSONResponse

class UpstreamDownError(Exception):
    def __init__(self, service: str, cause: Exception | None = None):
        self.service = service
        self.cause = cause

@app.exception_handler(UpstreamDownError)
async def upstream_down(request: Request, exc: UpstreamDownError) -> JSONResponse:
    logger.warning("upstream_down service=%s path=%s cause=%r",
                  exc.service, request.url.path, exc.cause)
    return JSONResponse(
        status_code=502,
        content={"error": "upstream_down", "service": exc.service},
    )

@app.exception_handler(RequestValidationError)
async def validation_error(request: Request, exc: RequestValidationError) -> JSONResponse:
    # Reshape Pydantic's default to match our API's error envelope
    return JSONResponse(
        status_code=422,
        content={
            "error": {"code": "validation_failed", "details": exc.errors()},
        },
    )

@app.get("/orders/{id}/payment-status")
async def payment_status(id: int) -> dict:
    try:
        async with httpx.AsyncClient() as client:
            r = await client.get(f"https://payments/{id}", timeout=5)
            r.raise_for_status()
    except (httpx.HTTPError, httpx.TimeoutException) as e:
        raise UpstreamDownError(service="payments", cause=e)
    return r.json()
```

Q24 · Middleware Ordering — Why It Matters and Common Bugs

Tags: Difficulty: Hard · Asked at: Razorpay, Postman, Swiggy, Cred, Goldman Sachs India · Topic: Middleware

The question (interviewer's verbatim phrasing):

FastAPI middleware runs in a specific order. Walk me through how registration order maps to execution order, and the bugs that come from getting it wrong.

The 30-second answer: Middleware in FastAPI is registered in the order you call

`app.add_middleware`, but it executes in *reverse* — the last added is the outermost on the request and the innermost on the response. So registration order matters: TrustedHost first, then CORS, then a request-ID middleware means TrustedHost is the innermost (closest to the route) and request-ID is the outermost (first to see the request, last to touch the response). The common bug is registering CORS *after* an authentication middleware that rejects the request before CORS can attach its headers — the OPTIONS preflight gets a 401 with no Access-Control headers and the browser refuses to send the real request.

Step-by-step thought process: 1. Mechanics: every middleware wraps the next one.

`app.add_middleware(A)` then `app.add_middleware(B)` results in `B(A(route))` — B runs first on the way in, A runs first on the way out. 2. The CORS-before-auth bug: if auth rejects with a 401 before CORS runs, the response has no `Access-Control-Allow-Origin` header. The browser sees the missing header and reports a CORS failure to JavaScript — masking the real cause (the 401). Register CORS *last* so it is outermost and always attaches its headers. 3. The request-ID pattern: a middleware that reads or generates a request ID, attaches it to `request.state`, attaches it to the response header, and includes it in every log line — must be outermost so every other middleware and the route can read `request.state.request_id`. Register it last. 4. The session middleware: if you use `SessionMiddleware`, it must run before any code that reads `request.session`. Register it before middlewares that depend on the session. 5. The GZip middleware: needs to be inside CORS (so CORS headers are not gzipped at the wrong stage) and inside body-reading middleware. Register GZip relatively early so most other middleware sits outside it.

Edge cases to mention: - Middleware that calls `await call_next(request)` and then modifies the response is doing a wrap-then-modify pattern; if it doesn't await, the next middleware never runs. Forgetting the await is a common test-flake source. - An exception raised inside a middleware skips the rest of the chain and goes directly to the exception handlers. Logging middleware should `try/except` around `call_next` to log the failure. - `BaseHTTPMiddleware` is the Starlette-provided class for writing custom middleware; for performance-critical paths there is a lower-level ASGI middleware protocol that avoids some overhead but is harder to write.

What NOT to say: - "Middleware runs in registration order." — Wrong on the response path; it runs in reverse. - "Order doesn't matter; just register everything." — CORS-after-auth is the classic counter-example. - "Use one big middleware for everything." — Defeats the composition; harder to test and reuse.

Common follow-up: "Write a request-ID middleware that propagates an `X-Request-Id` header through the request and the response."

Stand-out line: *"Say this and you'll separate yourself: middleware executes in reverse-registration order — last added is outermost — and the CORS-before-auth ordering bug is the one every senior FastAPI engineer has shipped a fix for at least once."*

```
import uuid
from fastapi import FastAPI, Request
from fastapi.middleware.cors import CORSMiddleware
from fastapi.middleware.gzip import GZipMiddleware
from starlette.middleware.base import BaseHTTPMiddleware

class RequestIDMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request: Request, call_next):
        rid = request.headers.get("X-Request-Id") or str(uuid.uuid4())
        request.state.request_id = rid
        response = await call_next(request)
        response.headers["X-Request-Id"] = rid
        return response

app = FastAPI()
# Order matters. Reversed at execution, so register inner-to-outer.
app.add_middleware(GZipMiddleware, minimum_size=1024)           # innermost
app.add_middleware(RequestIDMiddleware)                        # middle
app.add_middleware(                                           # outermost
    CORSMiddleware,
    allow_origins=["https://merchant.razorpay.com"],
    allow_methods=["*"],
    allow_headers=["*"],
)
```

Q25 · CORS Configuration — What Actually Has to Be Right

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, Swiggy, Cred, Freshworks, Zerodha · Topic: CORS

The question (interviewer's verbatim phrasing):

Walk me through configuring CORS in FastAPI for a service that needs to accept requests from a specific frontend domain with credentials.

The 30-second answer: Use `CORSMiddleware` from `fastapi.middleware.cors`. The required arguments are `allow_origins` (a list of exact URLs — `["https://merchant.razorpay.com"]`, never `["*"]` if you also want credentials), `allow_methods` (the HTTP methods to allow, often `["*"]`), `allow_headers` (the headers the client may send, often `["*"]`), and `allow_credentials=True` if cookies / auth headers must cross origins. The browser's preflight `OPTIONS` request gets the right headers automatically; the

middleware also rejects requests from origins not in the list. The four most common mistakes are:

`allow_origins=["*"]` with `allow_credentials=True` (spec forbids it; browser rejects), forgetting `expose_headers` for custom headers the frontend reads (`X-Request-Id` , pagination metadata), registering CORS too early in the middleware chain (auth rejects before CORS runs), and using a regex without thinking through the staging-vs-production domain pattern.

Step-by-step thought process: 1. The simplest correct configuration:

`app.add_middleware(CORSMiddleware, allow_origins=["https://merchant.razorpay.com"], allow_credentials=True, allow_methods=["*"], allow_headers=["*"])` . Tight origin list, broad methods/headers, credentials on. 2. The wildcard credential trap: `allow_origins=["*"]` plus `allow_credentials=True` is rejected by every modern browser. The spec says credentials require a specific origin to be reflected, not the wildcard. Either list real origins or drop credentials. 3. Per-environment origins:

`allow_origin_regex=r"https://(merchant|admin)\.(razorpay|razorpay-staging)\.com"` matches a controlled pattern. Avoid `.*` regexes — you will eventually accept an attacker's subdomain. 4. `expose_headers` for custom headers: if your API returns `X-Request-Id` or `X-Total-Count` and the frontend needs to read them via JavaScript, list them in `expose_headers` . The browser hides un-exposed headers from JS even if the server sent them. 5. Middleware ordering: CORS should be outermost (registered last) so its headers attach to every response — including 401s from auth middleware, 422s from validation, 500s from exceptions. Otherwise the browser sees a "CORS error" when the real cause is a different status code.

Edge cases to mention: - Preflight caching: `max_age=600` tells the browser to cache the preflight response for 10 minutes, reducing OPTIONS request volume. Default is browser-dependent but usually a few seconds. - A regex pattern is compiled once and matched per request; a list comparison is faster for ≤ 10 origins. For ≤ 10 explicit origins, prefer the list. - `allow_methods=["*"]` does NOT include `OPTIONS` — that is always allowed because it is the preflight method. List the methods your endpoints actually expose.

What NOT to say: - "Just allow `*` for everything." — Insecure and incompatible with credentials. - "CORS is enforced by the server." — Wrong. CORS is enforced by the *browser*; the server only sends the headers and the browser decides whether to forward the response to JavaScript. Non-browser clients ignore CORS entirely. - "CORS issues are usually a server bug." — Usually a configuration bug; less often an ordering bug; almost never a "server is broken" bug.

Common follow-up: *"You receive a CORS error in production but only on POST, not on GET. Where do you start debugging?"*

Stand-out line: *"Say this and you'll separate yourself: CORS is enforced by the browser, not the server — every CORS error you see in production is the browser refusing to forward a perfectly valid response to JavaScript, and the fix is almost always in the header config, not the application code."*


```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()

ALLOWED_ORIGINS = [
    "https://merchant.razorpay.com",
    "https://admin.razorpay.com",
    "https://merchant-staging.razorpay.com",
]

app.add_middleware(
    CORSMiddleware,
    allow_origins=ALLOWED_ORIGINS,
    allow_credentials=True,                # cookies, Authorization header
    allow_methods=["GET", "POST", "PUT", "PATCH", "DELETE"],
    allow_headers=["Authorization", "Content-Type", "X-Request-Id"],
    expose_headers=["X-Request-Id", "X-Total-Count"], # readable from JS
    max_age=600,                             # preflight cache for 10 min
)

# In an emergency, narrow only the origin list, not the credentials flag.
# Wildcard + credentials is the bug, not the feature.
```

Chapter Recap

Twenty-five questions, four sections. If you can talk through all twenty-five from memory — not verbatim, but the substance — you have cleared the Flask + FastAPI bar at every Indian product company. The interview signal split is consistent: Flask questions test whether you've worked in an older codebase or with Service-MNC clients (Q1–Q5); FastAPI fundamentals test whether you've shipped a service in the last two years (Q6–Q15); async questions are the round-two material that separates the FastAPI-tutorial graduate from the practitioner (Q16–Q20); production patterns are the round-three material that decides whether you can own the deploy (Q21–Q25).

A few cross-cutting habits to walk into the interview with. First, lead every async-versus-sync answer with "what's inside the function body?" — the right pick depends on whether the calls inside are awaitable. Second, never describe the GIL as the cause of a FastAPI concurrency problem; the cause is almost always sync calls in an async function, and the GIL is only the underlying reason that one thread can't bail you out. Third, treat **Depends**, response models, and exception handlers as the three primitives that keep FastAPI codebases from collapsing into spaghetti — every cross-cutting concern goes through one of them.

Combine this chapter with Chapter 5 (REST API Design) and Chapter 6 (DB Interaction) and you have the full backend round-one-to-round-three coverage for any Python-shop interview in India.

Chapter 5 — REST API Design

Backend Track Library · 20 Questions · Python Interview Prep 2026 Edition · Learn & Excel

REST API design is the round-two staple at every Indian backend interview that goes past leetcode. Razorpay will ask you to design an idempotent payment endpoint. Swiggy will ask why your menu endpoint returns 500 instead of 404. Postman will hand you a spec and ask "what's wrong with this URL." Cred and PhonePe will press on rate limiting, JWT vs session, and how you'd version a public API without breaking the existing mobile app. The reason interviewers love this topic is that it sits exactly at the seam between code and product — the candidate who can quote `@app.get("/users/{id}")` but cannot explain why DELETE should be idempotent is the candidate who ships a webhook bug that double-charges a customer.

This chapter has twenty questions across four sections. Section A is resource modelling — verbs, idempotency, nesting, filters, sub-resources. Section B is auth and security — token vs session vs OAuth2, CSRF, CORS, rate limiting, input validation, parameterised queries. Section C is versioning and pagination — URL vs header versioning, breaking vs non-breaking changes, cursor vs offset. Section D is error contracts and production patterns — the HTTP status matrix, structured error bodies, idempotency keys for POST, request IDs for tracing, dedup for "exactly once" delivery.

Answers stay framework-agnostic where the principle is universal, and lean on Django REST Framework (DRF) or FastAPI patterns where the concrete API matters. Most code examples use FastAPI because it is the framework Indian product cos default to in 2026 for new services; the Django REST equivalents are noted inline where they diverge.

Table of Sections

- Section A — Resource modelling (Q1 to Q5)
 - Section B — Auth and security (Q6 to Q10)
 - Section C — Versioning and pagination (Q11 to Q15)
 - Section D — Error contracts and production patterns (Q16 to Q20)
-

Section A — Resource modelling

The shape of your URLs and the choice of verb is the first thing an interviewer looks at. It also happens to be the cheapest thing for you to get right — read the spec, pick the right verb, name resources as plural nouns, stop trying to be clever. The five questions in this section cover the conventions, the idempotency rules that flow out of those conventions, when to nest and when to flatten, the standard

query-parameter vocabulary for filter / sort / paginate, and how to decide between a sub-resource URL and a separate endpoint.

Q1 · The REST verb matrix — GET, POST, PUT, PATCH, DELETE

Tags: Difficulty: Easy · Asked at: TCS, Infosys, Razorpay, Swiggy, Freshworks, Postman · Topic: REST · Verbs

The question (interviewer's verbatim phrasing):

Walk me through GET, POST, PUT, PATCH, DELETE. What does each one mean semantically, and which ones are safe versus idempotent?

The 30-second answer: GET reads. POST creates or runs an action. PUT replaces a resource at a known URL. PATCH partially updates. DELETE removes. GET is safe (no server-state change) and idempotent (same call N times = same effect as 1). PUT and DELETE are idempotent but not safe. POST is neither. PATCH is partial-update semantics and is conventionally idempotent only if you design it that way — RFC 5789 does not promise it.

Step-by-step thought process: 1. Define safe vs idempotent first — they are the two properties that organise the matrix. Safe means the server's state is unchanged. Idempotent means repeating the call leaves the server in the same final state as one call. 2. Walk the verbs in order. GET is safe and idempotent; it is the only verb that is safe. POST is neither — it creates a new resource each call, so two POSTs to `/orders` create two orders. 3. PUT is full replacement at a known URL — `PUT /users/42` with a body that contains the entire user record. Idempotent because replacing twice with the same body gives the same record. 4. PATCH is partial — `PATCH /users/42` with a body that contains only the changed fields. Whether it is idempotent depends on the diff format. A JSON Merge Patch that sets `{"status": "active"}` is idempotent. A JSON Patch that says `{"op": "add", "path": "/tags/-", "value": "vip"}` is not — applying it twice appends "vip" twice. 5. DELETE removes. Idempotent because the second DELETE of the same resource produces the same final state (the resource is gone). The status code may change — first DELETE returns 204, second may return 404 — but the state is the same.

Edge cases to mention: - Browsers and proxies aggressively cache GETs. Anything that mutates state via GET is a security smell — search engine crawlers will trigger it. - Some APIs use POST for read-only batch operations (`POST /search` with a big body). That is not RESTful but is common when the query is too large for a URL. - PUT can also create — `PUT /users/abhi-mittal` may create the user if it does not exist (the spec allows it). PUT-as-upsert is one of the cleaner upsert idioms.

What NOT to say: - "POST creates, PUT updates" — partially correct, hides PUT's replace-or-create semantics and PATCH's existence. - "PATCH is just a faster PUT" — they have different semantics. PUT requires the full body; PATCH carries only the diff.

Common follow-up: "If I send the same POST to `/orders` twice, the customer is double-charged. How do you fix that at the HTTP layer?" — Idempotency keys. Covered in Q18.

Stand-out line: "Say this and you'll separate yourself: the matrix is safe versus idempotent versus partial — GET is the only safe one, POST is the only non-idempotent one, and that distinction is what tells you which retries are free and which need a dedup key."

```
# FastAPI — the five verbs on one resource
from fastapi import FastAPI, status
app = FastAPI()

@app.get("/orders/{order_id}")          # safe + idempotent
def get_order(order_id: int): ...

@app.post("/orders", status_code=201)    # neither safe nor idempotent
def create_order(payload: OrderIn): ...

@app.put("/orders/{order_id}")           # idempotent, full replace
def replace_order(order_id: int, payload: OrderIn): ...

@app.patch("/orders/{order_id}")         # partial update
def patch_order(order_id: int, payload: OrderPatch): ...

@app.delete("/orders/{order_id}", status_code=204) # idempotent
def delete_order(order_id: int): ...
```

Q2 · Why are PUT and DELETE idempotent but POST is not

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, PhonePe, Walmart Labs · Topic: REST · Idempotency

The question (interviewer's verbatim phrasing):

Network glitches retry requests. Why is it safe to retry a PUT or DELETE but not a POST, and what does idempotency actually buy you?

The 30-second answer: Idempotency means the final server state after N identical calls equals the state after one. PUT replaces the resource — replacing twice with the same body gives the same row. DELETE removes — removing a removed thing is a no-op. POST creates, so two POSTs create two resources, which is why a retried POST without an idempotency key can produce duplicate orders, double charges, or two webhooks to your downstream.

Step-by-step thought process: 1. Start with the property. Idempotency is about *final state*, not about the response code. The first DELETE returns 204, the second returns 404 — the response differs but the

state does not. 2. Apply it to PUT. `PUT /users/42 {"name": "Abhi"}` writes the row to name=Abhi. The second call writes the same value. State is identical after one or after a hundred. 3. Apply it to POST. `POST /orders {...}` creates a new order each time. State is not identical — a retry made the database grow by two orders, not one. 4. Apply it to PATCH. Depends on the diff. Set-this-field PATCH is idempotent; append-to-list PATCH is not. Design your PATCH semantics with this in mind. 5. Tie it back to why anyone cares: clients (mobile networks, retry middleware, queue-driven workers) will retry. An idempotent endpoint can be retried safely without an idempotency-key dance. A non-idempotent endpoint needs a dedup mechanism — see Q18.

Edge cases to mention: - A PUT that overwrites a `last_modified_at = now()` field is technically non-idempotent in the strictest reading — the state differs in the timestamp. Most reviewers ignore this; the row content is what matters. - A DELETE that triggers a side effect (publishes a Kafka event, sends an email) makes the side effect non-idempotent even if the row deletion is. Wrap side effects in a dedup table. - A POST can be made idempotent by the server treating an Idempotency-Key header as a dedup token. This is exactly how Razorpay, Stripe, and Cred's `Idempotency-Key` work — see Q18.

What NOT to say: - "Idempotency means the response is the same." It does not. State is the same, response may differ. - "GET is idempotent because it does not change state." That is two properties. GET is safe (no state change) and idempotent. Conflating the two loses points.

Common follow-up: *"If POST is not idempotent, how does Razorpay's create-payment endpoint guarantee at-most-once semantics on a flaky network?"* — Idempotency-Key header. Server stores a dedup row keyed on the header value; a retried POST with the same key returns the cached response. Covered in Q18.

Stand-out line: *"Say this and you'll separate yourself: idempotency is about final state, not response code — a second DELETE returning 404 instead of 204 is still idempotent because the world ends up in the same place."*

Q3 · Nesting resources — when to nest and when to flatten

Tags: Difficulty: Medium · Asked at: Swiggy, Meesho, Zerodha, Freshworks · Topic: REST · URL Design

The question (interviewer's verbatim phrasing):

A user has many orders. Each order has many line items. Should the URL be `/users/42/orders/77/items/3` or just `/items/3` ? How deep do you nest?

The 30-second answer: Nest one level deep at most, and only when the parent gives meaningful context for filtering or scoping. `/users/42/orders` is fine. `/orders/77/items` is fine. `/users/42/orders/77/items/3` is two parents deep — drop the user, the order ID alone is unique. Beyond

one level the URL is just decorative; the deep IDs do not constrain access and the duplication invites bugs where the URL says `/users/42/orders/77` but order 77 actually belongs to user 99.

Step-by-step thought process: 1. Ask what the nesting buys. If `/users/42/orders` lets you list orders for one user, it earns its place. If `/users/42/orders/77` returns the same thing as `/orders/77`, the user prefix is dead weight. 2. Watch for the inconsistency trap. If your route accepts `/users/42/orders/77` but order 77 belongs to user 99, do you return 404 (the resource at this URL does not exist), 403 (you cannot access user 99's order), or 200 (ignore the user)? All three are defensible and all three confuse clients. Flatten to `/orders/77` and the question vanishes. 3. Apply the one-level rule. List endpoints can nest — `GET /users/42/orders` returns one user's orders. Item endpoints flatten — `GET /orders/77` instead of `/users/42/orders/77`. 4. Where the nested form is the only sensible one — `/repos/owner/name/issues/3` in GitHub-style APIs, where "owner/name" is the actual identifier — keep it. The rule is about decoration, not about meaningful composite keys. 5. Mention HATEOAS-style links if asked: include the parent in the response body (`{"id": 77, "user_id": 42, "links": {"user": "/users/42"}}`) rather than in the URL path.

Edge cases to mention: - Some teams nest for authorisation reasons — `/users/42/orders/77` means "this URL is callable only by user 42 or admin." Cleaner to do that authorisation in middleware, not URL structure. - Sub-resources with their own lifecycle (comments on a post, addresses on a user) can legitimately nest one level because they only exist in the parent's context. - Do not nest collections inside collections — `/users/42/orders/77/items` is fine; `/users/42/orders/77/items/3/refunds` is route soup.

What NOT to say: - "More nesting is more RESTful." It is not. REST has no rule about nesting depth; the verbs and statelessness are what matter. - "Always nest everything." See above — it leads to the consistency trap.

Common follow-up: "What if the same order needs to be accessible to both the buyer and the seller — would you nest under buyers or sellers or neither?" — Neither. `/orders/77` is one resource with two access-control rules.

Stand-out line: "Say this and you'll separate yourself: nest at most one level — `/users/42/orders` is fine because it scopes the list, but `/users/42/orders/77` is decoration that invites the URL-says-42-but-row-says-99 bug."

```
# DRF – flat resource URL, parent context in body
# urls.py
urlpatterns = [
    path("users/<int:user_id>/orders/", UserOrdersList.as_view()), # list scoped to user
    path("orders/<int:pk>/", OrderDetail.as_view()),               # item is flat
]
```

Q4 · Filter, sort, paginate — the standard query-parameter vocabulary

Tags: Difficulty: Medium · Asked at: Razorpay, Zerodha, Swiggy, Postman, Walmart Labs · Topic: REST · Query Params

The question (interviewer's verbatim phrasing):

Design the GET endpoint that lists orders. The client needs to filter by status, sort by created_at descending, and paginate. What does the URL look like?

The 30-second answer: `GET /orders?status=captured&sort=-created_at&limit=50&cursor=eyJpZCI6MTAwfQ` . Filter as `field=value` . Sort with a signed list — leading minus for descending — and allow comma-separated multi-sort. Paginate with `limit` plus a cursor for big or write-heavy datasets, or `page` plus `per_page` for small static lists. The exact spelling is less important than picking one convention and using it across every list endpoint.

Step-by-step thought process: 1. Filters use named params, one per field.

`status=captured&merchant_id=razorpay_acq_001` . Combine with implicit AND. For OR, take a comma-separated value: `status=captured,refunded` . For ranges, use suffixes:

`amount_gte=1000&amount_lt=5000` . Avoid building a homegrown query-language DSL — it always grows into a bad SQL. 2. Sort uses a single `sort` param with a signed field list. `sort=-created_at`

is descending; `sort=status,-created_at` is "status ascending then created_at descending." 3. Paginate. Two choices — offset/page or cursor. Offset is simple (`?page=3&per_page=50`)

and lets you jump to any page, but it is slow on big tables and inconsistent under writes. Cursor (`?limit=50&cursor=...`) is fast and write-stable but supports only forward / backward navigation. Pick cursor for write-heavy or large datasets. Covered fully in Q14. 4. Mention field selection if it comes up. `fields=id,status,amount` lets the client thin the response. GitHub and Stripe support this; smaller APIs usually skip it. 5. Document the vocabulary and stick to it. Inconsistency across endpoints (`status` on one, `state` on another, `order_status` on a third) is the most common source of integration bugs.

Edge cases to mention: - Cap `limit` . A client that asks for `limit=1000000` will OOM the API server. FastAPI lets you do this with `Query(50, le=200)` . DRF's

`PageNumberPagination.max_page_size = 200` . - Allow `sort` only on indexed columns.

Sorting by an un-indexed column on a million-row table will take down the database. - The `q` param is the convention for free-text search. `GET /products?q=power+bank&category=electronics` .

What NOT to say: - "Filter parameters should go in the request body." Not for GET — bodies on GET are not universally supported and break caches and CDNs. - "Pagination is just LIMIT OFFSET." See Q14 — offset is one strategy, cursor is the other, and the choice matters.

Common follow-up: "How would you let the client get the total count without paying for it on every request?" — Optional `include=count` param. The client opts in when it needs the count for a "showing 1-50 of 12,490" UI, and the API skips the `COUNT(*)` for the common case.

Stand-out line: "Say this and you'll separate yourself: cap `limit`, allow `sort` only on indexed columns, and never build a homegrown query DSL — every one of those grows into a bad SQL the day a customer asks for a feature you did not plan."

```
# FastAPI — typed filter, sort, paginate
from fastapi import Query
from typing import Annotated

@app.get("/orders")
def list_orders(
    status: Annotated[list[str] | None, Query()] = None,
    sort: str = "-created_at",
    limit: Annotated[int, Query(ge=1, le=200)] = 50,
    cursor: str | None = None,
):
    return repo.list_orders(status=status, sort=sort, limit=limit, cursor=cursor)
```

Q5 · Sub-resources versus separate endpoints

Tags: Difficulty: Medium · Asked at: PhonePe, Cred, Meesho · Topic: REST · URL Design

The question (interviewer's verbatim phrasing):

A user has tags. Should "add a tag" be `POST /users/42/tags` or `POST /tags` with `user_id` in the body, or `PATCH /users/42` with the new tag list?

The 30-second answer: Pick based on whether tags have their own lifecycle. If a tag exists independently of any user, model it as `POST /tags` (create the tag) plus `POST /users/42/tags/{tag_id}` (associate). If tags only exist in the context of a user, model it as `POST /users/42/tags` with body `{"name": "vip"}` — sub-resource. If you are just editing the user's tag list as a property, `PATCH /users/42 {"tags": ["vip", "early"]}` is fine for short lists. The choice is about lifecycle and cardinality, not about being RESTful.

Step-by-step thought process: 1. Map the entity. Does "tag" have a stable ID that survives across users (an `Order` is a top-level concept) or does it only mean something inside the user record (an `Address` is a sub-thing of `User`)? 2. Three candidate shapes follow: - **Separate endpoint + association:** `POST /tags` creates a tag, `POST /users/42/tags` associates. Use when tags are first-class and reusable. - **Sub-resource:** `POST /users/42/tags` creates a tag scoped to user 42. Use when the tag would be meaningless without the user. - **PATCH on parent:** `PATCH /users/42 {"tags": [...]}`. Use when the collection is small, bounded, and the client always sends the full new list. 3. Watch the PATCH-on-parent trap — it requires the client to read the current list, modify, and send the full list. Two clients racing each other will lose updates. Use the sub-resource shape if

concurrent edits matter. 4. Watch the separate-endpoint trap — for one-to-many things that have no independent existence (a comment on a post is rarely shared across posts), the separate endpoint adds a join and a registration step the client did not need. 5. Document the lifecycle in the API spec. A reader scanning your OpenAPI doc should see at a glance whether tags live inside users or outside them.

Edge cases to mention: - Bulk-edit collections via `PUT /users/42/tags [...]` (full replace) is fine for tag-set-on-a-user shapes. Idempotent, easy to reason about. Just be aware it deletes anything not in the new list. - For many-to-many, the separate-endpoint shape often gets a dedicated association resource — `POST /memberships {"user_id": 42, "group_id": 7}` — and that is cleaner than nesting twice. - DRF's `@action` decorator and FastAPI's nested routers both make sub-resource shapes ergonomic; do not let framework friction drive the design.

What NOT to say: - "Always use sub-resources, they are more RESTful." There is no rule. The choice is about the underlying data model. - "PATCH on the parent is always wrong." Not for small, owned, set-shaped collections.

Common follow-up: *"How would you handle the race where two admins simultaneously add a different tag to the same user via PATCH?"* — ETag + If-Match. The first request sends If-Match with the current ETag; the second request's ETag is stale, server returns 412 Precondition Failed, client retries with the new state.

Stand-out line: *"Say this and you'll separate yourself: lifecycle decides the URL shape — first-class entities get their own endpoint plus an association resource, owned sub-things nest one level, and small bounded collections live as a PATCH on the parent."*

Section B — Auth and security

Five questions on the parts of API design that show up in the security review and again in the postmortem when something gets exfiltrated. Token auth versus session, CSRF for cookie-based flows, CORS basics, rate limiting via Redis, input validation through Pydantic, and SQL injection mitigated via parameterised queries. None of these are framework-specific in principle, but FastAPI and DRF make some patterns more idiomatic than others — that is called out where it matters.

Q6 · Token auth — JWT, opaque session, OAuth2 — when does each one win

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, PhonePe, Postman, JP Morgan India · Topic: REST · Auth

The question (interviewer's verbatim phrasing):

Walk me through JWT, opaque session tokens, and OAuth2 access tokens. When would you reach for each, and what is the standard mistake on each?

The 30-second answer: JWT is a stateless self-contained token — server signs claims, client carries them, server verifies on each request. Opaque session tokens are server-side state — client carries an ID, server looks up a session row. OAuth2 is the delegation protocol (Authorize Code, Client Credentials, etc.) and it usually issues an opaque access token plus a refresh token. Use JWT for stateless internal service-to-service or short-lived API tokens. Use opaque sessions for user-facing apps where you want fast revocation. Use OAuth2 when a third party needs scoped access to your API on a user's behalf — payment gateways, public dev platforms.

Step-by-step thought process: 1. Distinguish the two axes: who stores the state (server or client) and who issues the token (you or a delegated auth server). 2. JWT puts state in the client. Pros — no DB lookup per request, scales horizontally without sticky sessions, easy cross-service propagation. Cons — revocation is hard (you need a revocation list or short expiries), token size grows with claims, signing key rotation is a chore. 3. Opaque session tokens put state on the server. Pros — instant revocation (delete the row), tiny token, cheap to rotate. Cons — DB or cache lookup on every request, sticky sessions if the store is per-node. 4. OAuth2 is a protocol family. The Authorization Code flow (with PKCE for mobile) is the standard for user-grants-third-party-access. Client Credentials is for service-to-service. The access token issued is usually opaque to the relying party but may be a JWT internally — your API does not care. 5. The standard mistakes: JWT in a cookie without CSRF protection (covered in Q7); using `alg: none`; not rotating signing keys; long-lived JWTs without a revocation story. Opaque tokens: sending them in URLs (they show up in logs); not hashing them at rest. OAuth2: skipping PKCE on mobile clients; storing the refresh token in localStorage; mixing up access and ID tokens (the ID token in OpenID Connect is for identity, not authorisation).

Edge cases to mention: - Refresh tokens — always opaque, always one-time-use, always rotated. The OAuth2 spec calls this refresh token rotation; do it. - Audience and issuer claims (`aud` , `iss`) — a JWT signed for service A should not validate at service B. Check `aud` . - The Authorization header convention is `Authorization: Bearer <token>` for everything bearer-token-shaped — JWT, opaque, OAuth2 access token. Pick this over custom headers.

What NOT to say: - "JWT is more secure than sessions." Neither is inherently more secure. They have different trade-offs and different failure modes. - "OAuth2 is just for login." OAuth2 is for authorisation. OpenID Connect (built on top) is for identity / login.

Common follow-up: "How would you revoke a JWT immediately after a user logs out?" — Short-lived access tokens (5-15 minutes) plus a server-side blocklist keyed on `jti` (JWT ID). Check the blocklist on each request. Some teams skip the blocklist and rely on the short expiry alone, accepting up to 15 minutes of accepted-after-logout traffic for revoked tokens.

Stand-out line: "Say this and you'll separate yourself: JWT trades revocation cost for DB lookups; opaque sessions trade scale for fast revocation — name the trade out loud and pick the side that matches your traffic shape."

```
# FastAPI — bearer-token dependency, JWT verified per request
from fastapi import Depends, HTTPException, status
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
import jwt

bearer = HTTPBearer()

def current_user(creds: HTTPAuthorizationCredentials = Depends(bearer)):
    try:
        claims = jwt.decode(creds.credentials, SIGNING_KEY,
                             algorithms=["HS256"], audience="orders-api")
    except jwt.PyJWTError:
        raise HTTPException(status.HTTP_401_UNAUTHORIZED, "invalid token")
    return claims["sub"]

@app.get("/me")
def me(user_id: str = Depends(current_user)):
    return {"user_id": user_id}
```

Q7 · CSRF for cookie-based auth — what it is, when to defend

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Walmart Labs, Freshworks · Topic: REST · Security

The question (interviewer's verbatim phrasing):

When does an API need CSRF protection, and what does the actual defence look like? Does a bearer-token API need it?

The 30-second answer: CSRF matters when the browser will automatically attach an auth credential to a cross-site request. That is the case for cookies — the browser sends them on any request to the origin. It is not the case for **Authorization: Bearer** headers, which the browser will not auto-attach. So a pure bearer-token API does not need CSRF protection; a cookie-based session API does. Defences: SameSite=Lax/Strict on the cookie, plus a double-submit CSRF token, plus an **Origin** check on state-changing requests.

Step-by-step thought process: 1. Frame the threat. CSRF is "evil-site.com submits a form that hits your-bank.com, browser auto-sends the bank's session cookie, the bank treats it as a legitimate request." The exploit only works because the cookie was auto-attached. 2. Bearer tokens are not auto-attached. The attacker would need to steal the token first (XSS, leaked logs), at which point CSRF is the smaller problem. So a stateless JWT-in-Authorization-header API is CSRF-immune by construction. 3. Cookie-based APIs need three layers. SameSite=Lax on the session cookie (blocks the cross-site form-POST trick for GETs but allows top-level navigations); SameSite=Strict if you can tolerate the UX hit (no

cross-site nav-in carries the cookie). Add a CSRF token — server emits a per-session token in a separate cookie and the JS client echoes it in a header (`X-CSRF-Token`); attacker cannot read the cookie due to same-origin policy, so cannot echo it. Add an `Origin` or `Referer` check as a backstop. 4. Django CSRF middleware does the token-in-header pattern out of the box. DRF's `SessionAuthentication` enforces it. FastAPI ships nothing built-in — wire it through `starlette-csrf` or roll the double-submit pattern manually. 5. Mention `SameSite=None` — required if you genuinely need cross-site cookies (a payment-iframe scenario), but must come with `Secure` and ideally with a CSRF token.

Edge cases to mention: - `SameSite=Lax` is the browser default in 2026, so even forgetting the attribute gets you partial protection. Do not rely on it — set it explicitly. - A GET request is "safe" by REST contract, so the CSRF token is needed only on POST / PUT / PATCH / DELETE. Make sure your state-changing endpoints are not exposed as GETs. - Subdomain trust — `*.example.com` cookies are visible to every subdomain. A compromised marketing subdomain can read the cookie. Scope your auth cookie to the exact host.

What NOT to say: - "Bearer tokens need CSRF protection too." They do not — the browser does not auto-attach them. - "SameSite=Strict alone is enough." It is not — older browsers do not honour it, and your defence-in-depth should include the token pattern.

Common follow-up: *"Your mobile app uses session cookies via WebView. Does CSRF apply?"* — Yes if a browser context can host arbitrary script. Use `SameSite=Strict` and the double-submit token, or migrate the mobile flow to bearer tokens.

Stand-out line: *"Say this and you'll separate yourself: CSRF protection is about whether the browser auto-attaches credentials — cookies yes, bearer tokens no, which is why a pure JWT-in-Authorization-header API is CSRF-immune by construction."*

Q8 · CORS basics — what it is, where it goes wrong

Tags: Difficulty: Medium · Asked at: Postman, Razorpay, Freshworks, Zoho · Topic: REST · Security

The question (interviewer's verbatim phrasing):

Your frontend at `app.example.com` cannot call `api.example.com`. The browser says CORS. Walk me through what is happening and how you fix it.

The 30-second answer: CORS is the browser's same-origin-policy guard rail. By default, JavaScript on `app.example.com` cannot read responses from `api.example.com` because they are different origins. The fix is server-side — `api.example.com` returns `Access-Control-Allow-Origin: https://app.example.com` on responses, plus for non-simple requests responds to a preflight

OPTIONS call with the allowed methods, headers, and credentials. Wide-open `*` is a code smell for any API that accepts credentials.

Step-by-step thought process: 1. State the model. CORS is enforced by the browser, not the server. Curl, mobile apps, and server-to-server traffic ignore it. So a CORS error is always a browser-issued message; the API is working fine. 2. Distinguish simple vs preflighted requests. A simple request (GET / HEAD / POST with a limited content-type and no custom headers) does not preflight. Anything else triggers an OPTIONS preflight with `Access-Control-Request-Method` and `Access-Control-Request-Headers` headers, which the server must answer with allow lists. 3. Walk the response headers. `Access-Control-Allow-Origin` : the origin you trust, or `*` for fully public. `Access-Control-Allow-Methods` : which verbs. `Access-Control-Allow-Headers` : which custom headers (e.g., `Authorization` , `X-Idempotency-Key`). `Access-Control-Allow-Credentials: true` if the client sends cookies — and in this case `Origin` cannot be `*` , must be the specific origin. 4. The standard misconfiguration: `Allow-Origin: *` plus `Allow-Credentials: true` . Browsers refuse this combination. The fix is to echo the request's `Origin` header back if it is in your allowlist. 5. FastAPI: `app.add_middleware(CORSMiddleware, allow_origins=[...])` . DRF: `django-cors-headers` package. Both ship cache-control for the preflight via `max_age` — set it to 600 to avoid re-preflighting every request.

Edge cases to mention: - Preflight is per (origin, method, path) effectively. A misconfigured cache key on your CDN can poison the preflight for the next user. - WebSocket connections do not use CORS; they use the `Origin` header for the handshake. Servers should validate it explicitly. - `Access-Control-Expose-Headers` is required if you want the JS client to read custom response headers like `X-Request-Id` . Otherwise they are invisible to fetch / axios.

What NOT to say: - "Set Allow-Origin to star and be done." Fine for public read-only APIs; a security incident for anything authenticated. - "CORS is server-side security." It is browser-side enforcement of a same-origin contract. Servers cannot rely on it as a security boundary against non-browser clients.

Common follow-up: "Why does `Allow-Origin: *` with credentials fail in the browser?" — Because the credentials are bound to a specific origin (cookie scope, basic auth realm) and the spec disallows wildcarding origin when credentials are in play. The browser refuses the response.

Stand-out line: "Say this and you'll separate yourself: CORS is browser-enforced, not server-enforced — every CORS error is the browser refusing to hand the response to JavaScript, and the fix lives entirely in the response headers."

```
# FastAPI — explicit allowlist for credentialed API
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["https://app.example.com", "https://admin.example.com"],
    allow_credentials=True,
    allow_methods=["GET", "POST", "PUT", "PATCH", "DELETE"],
    allow_headers=["Authorization", "Content-Type", "X-Idempotency-Key"],
    expose_headers=["X-Request-Id"],
    max_age=600,
)
```

Q9 · Rate limiting — token bucket via Redis

Tags: Difficulty: Medium · Asked at: Razorpay, PhonePe, Postman, Cred, Walmart Labs · Topic: REST · Security

The question (interviewer's verbatim phrasing):

Design a rate limiter for a public API. 100 requests per minute per API key. How do you implement it across 10 API nodes?

The 30-second answer: Token bucket in Redis. One Redis key per API key holding the current token count and the last-refill timestamp. On each request, refill tokens based on elapsed time, decrement by one, refuse if negative. Use a Lua script so the read-modify-write is atomic across the 10 nodes. Return `429 Too Many Requests` with a `Retry-After` header when refused, and standard `X-RateLimit-*` response headers on every success.

Step-by-step thought process: 1. Choose the algorithm. Token bucket allows controlled bursts (spend N tokens at once, then wait); leaky bucket smooths to a constant rate; fixed-window counters are the simplest but suffer the "double burst at the window edge" problem. Token bucket is the standard for public APIs. 2. The data: per-API-key bucket holding `{tokens: float, last_refill_ts: float}`. Refill is `tokens += (now - last_refill_ts) * rate`, capped at `bucket_size`. Then deduct one for the current request. If `tokens < 0`, refuse. 3. Atomicity is the whole game. Without it, two concurrent requests both read `tokens=1`, both decrement to `0`, both succeed. Lua in Redis solves this — the script runs single-threaded, so the read-modify-write is a unit. 4. Response shape on success: `X-RateLimit-Limit: 100`, `X-RateLimit-Remaining: 47`, `X-RateLimit-Reset: 1730543400`. On refusal: `429 Too Many Requests` with `Retry-After: 28` and a JSON body explaining the limit. 5. Keying — by API key (paid plans), by user ID (per-account quotas), by IP (anti-abuse on public endpoints), or composite (per-API-key-per-endpoint for finer control). Document the keying so clients can predict the limit.

Edge cases to mention: - Distributed clocks. Use Redis's `TIME` command inside the Lua script for the timestamp, not the application server's clock — otherwise drift between nodes corrupts the bucket. - Storage expiry. Set a TTL on the Redis key (e.g., 1 hour) so abandoned API keys do not fill memory. Refresh the TTL on every access. - The "burst on retry" failure mode. If 1,000 clients hit 429 and all set 30-second Retry-After timers, they re-burst together at $t+30$. Add jitter to `Retry-After` or use a smoothed rate at the client.

What NOT to say: - "Just use Redis INCR with EXPIRE." That is the fixed-window counter and it doubles the limit at the window boundary. Acceptable for rough quotas, wrong for SLA-grade rate limits. - "Rate limit at the load balancer." Possible at the edge, but per-API-key limits need application-level knowledge of the key. Combine — edge limits for raw QPS, app limits for business-tier quotas.

Common follow-up: *"Razorpay's payment-create endpoint rate-limits at 50 RPS per merchant. The merchant integration sometimes retries on timeout and bursts to 200 RPS. How do you handle that without rejecting legitimate retries?"* — Bucket size larger than the rate (e.g., size=200, refill=50/s) absorbs short bursts. Combined with the merchant honouring `Retry-After` plus jitter.

Stand-out line: *"Say this and you'll separate yourself: token bucket plus a Redis Lua script for atomicity is the production-grade rate-limit shape — `INCR` plus `EXPIRE` is the fixed-window version that doubles the limit at the boundary and ships as a CVE on the first sustained burst."*

```
# Token bucket via Redis Lua script — atomic across N nodes
LUA = """
local key = KEYS[1]
local rate, capacity, now, cost = tonumber(ARGV[1]), tonumber(ARGV[2]),
                                tonumber(ARGV[3]), tonumber(ARGV[4])

local data = redis.call("HMGET", key, "tokens", "ts")
local tokens = tonumber(data[1]) or capacity
local ts = tonumber(data[2]) or now
tokens = math.min(capacity, tokens + (now - ts) * rate)
local allowed = tokens >= cost
if allowed then tokens = tokens - cost end
redis.call("HMSET", key, "tokens", tokens, "ts", now)
redis.call("EXPIRE", key, 3600)
return {allowed and 1 or 0, tokens}
"""

def check(api_key: str, rate=50, capacity=200) -> tuple[bool, float]:
    now = time.time()
    allowed, remaining = redis.eval(LUA, 1, f"rl:{api_key}", rate, capacity, now, 1)
    return bool(allowed), remaining
```

Q10 · Input validation and SQL injection — Pydantic and parameterised queries

Tags: Difficulty: Medium · Asked at: TCS, Infosys, Razorpay, Swiggy, JP Morgan India · Topic: REST · Security

The question (interviewer's verbatim phrasing):

Your endpoint takes an `order_id` from the URL and a body with a status string. Walk me through how you validate the input and how you prevent SQL injection.

The 30-second answer: Validate at the edge with Pydantic — typed fields, length limits, allowed-value enums, custom validators for shapes the type system cannot express. Return `422 Unprocessable Entity` with a structured error body on validation failure. For SQL, use parameterised queries — placeholders that the driver substitutes safely, never string concatenation. SQLAlchemy's bound parameters and Django ORM's queryset filters both do this automatically; psycopg's `cursor.execute(sql, (params,))` is the raw-SQL equivalent.

Step-by-step thought process: 1. The principle: never trust the client. Even an "internal" API has a hostile threat model the day someone leaks a token or runs an SSRF chain. 2. Pydantic at the edge. Define an input schema with the exact types — `OrderStatus(str, Enum)` for status, `condecimal(ge=0)` for amount, `EmailStr` for emails. FastAPI integrates Pydantic so you get free type checking on every request; DRF achieves the same via serializers. Reject early — let the framework return 422 with field-level errors. 3. SQL injection defence is parameterised queries. The classic exploit is `f"SELECT * FROM users WHERE name = '{name}'" where name = ""`; `DROP TABLE users; --`. Parameterised queries pass the value through the driver's escape layer, treating it as data not code: `cur.execute("SELECT * FROM users WHERE name = %s", (name,))`. 4. ORM-shaped code is parameterised by default. `User.objects.filter(name=name)` in Django, `session.query(User).filter_by(name=name)` in SQLAlchemy — both bind. The places that go wrong are `cursor.execute()` with f-strings, `raw_sql=name + "..."`, or `Model.objects.extra(where=[...])`. 5. Defence in depth: parameterise plus minimum-privilege DB user (the app role should not have DROP TABLE), plus WAF rules on the obvious patterns, plus query logging that flags abnormal patterns.

Edge cases to mention: - Identifier injection. Parameterised queries bind values, not identifiers — `ORDER BY %s` with `column = "id; --"` does not work. For dynamic column names, validate against a whitelist before interpolating. - NoSQL injection. MongoDB queries can be tricked when JSON bodies are passed straight into `find({...body})` — clients send `{"name": {"$ne": null}}` and dump everything. Validate the shape, do not accept arbitrary operator dicts. - Pydantic strictness — `strict=True` rejects type coercion (the string `"42"` will not silently become an int). The cost is friendliness; the benefit is fewer surprises.

What NOT to say: - "ORMs are SQL-injection-proof." Only for the parts where they bind. `.raw()`, `.extra()`, and string-concat builders are still injectable. - "Sanitise the input by removing single quotes." Blocklist sanitisation always loses. Bind values; do not try to scrub them.

Common follow-up: "Show me a Pydantic schema that rejects an order status that is not in `{'created', 'captured', 'refunded'}`." — Use `Literal` or an `Enum`:

Stand-out line: *"Say this and you'll separate yourself: Pydantic at the edge plus parameterised queries everywhere — never trust the client, never concatenate into SQL, and never assume the ORM covers your `.raw()` escape hatch."*

```
from enum import Enum
from pydantic import BaseModel, Field
from typing import Annotated

class Status(str, Enum):
    created = "created"
    captured = "captured"
    refunded = "refunded"

class OrderPatch(BaseModel):
    status: Status
    notes: Annotated[str, Field(max_length=500)] | None = None

@app.patch("/orders/{order_id}")
def patch_order(order_id: int, payload: OrderPatch):
    # parameterised, no string interpolation
    db.execute(
        "UPDATE orders SET status = %s, notes = %s WHERE id = %s",
        (payload.status.value, payload.notes, order_id),
    )
```

Section C — Versioning and pagination

Versioning is the contract you keep with every client that already ships. Pagination is the contract you keep with every client that loads a list. Both decisions cost very little to make right at version one and become disproportionately expensive to fix once a mobile app, a partner integration, and three internal services depend on the existing shape. Five questions on the principled answers.

Q11 · URL versioning versus header versioning

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, PhonePe, Walmart Labs, JP Morgan India ·

Topic: REST · Versioning

The question (interviewer's verbatim phrasing):

Your API is going from v1 to v2 with breaking changes. Do you put the version in the URL (`/v2/orders`) or in a header (`Accept: application/vnd.example.v2+json`)? Defend your choice.

The 30-second answer: URL versioning (`/v1/orders` , `/v2/orders`) is the pragmatic default for public APIs that real teams use. It is greppable in logs and client code, cacheable by CDNs at the URL level, easy to route at the load balancer, and obvious in API docs. Header versioning is technically purer (the URL identifies the resource, the representation is negotiated) but it surprises engineers, breaks naive CDN caching, and makes copy-paste curl examples non-obvious. Razorpay, Stripe, GitHub, and almost every Indian-team-facing API use the URL form. Pick the URL.

Step-by-step thought process: 1. The two camps. The URL camp says "make it loud, make it obvious, optimise for the engineer reading a log line." The header camp says "the URL points to a resource, the version is a representation detail, content negotiation already exists for this exact purpose." 2. The URL form wins in practice because operations is downstream of theory. A CDN keyed on URL can cache `/v1/orders/42` and `/v2/orders/42` independently. A header-keyed cache needs `Vary: Accept` and rarely gets configured right. 3. The header form is the choice when the API has a long tail of point releases (Stripe uses a date-based header: `Stripe-Version: 2024-04-10`) and you want the URL to remain stable across many minor versions. This is a legitimate choice for a high-churn financial API; it is overkill for the average product. 4. Hybrid: URL major version (`/v1` , `/v2`) plus a date-based header for minor (`X-API-Version: 2026-05-30`). GitHub and Stripe both run a hybrid in practice. The URL bumps for breaking changes; the header takes the optional point updates. 5. Whatever you choose, document the policy. Deprecation timeline, sunset date, what counts as breaking — all in the versioning page.

Edge cases to mention: - Sub-domain versioning (`v1.api.example.com`) is a third option. Useful when v1 and v2 are deployed on different infrastructure. Mostly an artefact of operational separation. - Query-string versioning (`/orders?v=2`) is the worst of both — neither cacheable nor discoverable. Skip. - URI-template route ordering matters. FastAPI/DRF should route `/v2/orders` before `/v1/orders` ; otherwise client typos can fall through to the wrong version.

What NOT to say: - "Versioning is unnecessary if you only ever make backwards-compatible changes." Theoretically true; practically the day you need a breaking change you find out you have no version mechanism. - "Header versioning is more RESTful." It is, by a strict reading; the strict reading is not what real teams optimise for.

Common follow-up: "Razorpay's payment-create v1 expected `amount` as paise (integer), v2 expects `amount` as a Money object with `value` and `currency` . How do you ship v2 without breaking v1 clients?" — Mount v2 on `/v2/payments` with the new schema. Keep `/v1/payments` running with the old schema. Internally route both into the same domain service via two thin adapters that translate the wire format.

Stand-out line: *"Say this and you'll separate yourself: URL versioning wins because operations is downstream of theory — CDN caching, log grep, and curl examples all work better when the version is loud in the path."*

Q12 · Breaking versus non-breaking changes

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, PhonePe · Topic: REST · Versioning

The question (interviewer's verbatim phrasing):

Your API takes a `status` field. Today it accepts `created` and `captured`. You want to add `partial_refund`. Is that a breaking change?

The 30-second answer: Adding a new enum value to a response field is breaking — existing clients that switch-case on the value will hit the default branch and may crash or misbehave. Adding it as an accepted input value is non-breaking. The principle: adding optional inputs and additive output fields is safe; tightening anything, removing anything, or expanding the range of any output the client must handle is breaking. When in doubt, ship behind a version.

Step-by-step thought process: 1. The mental model — clients break when the server starts producing something the client did not expect. Adding a new optional request field does not surprise the server (the server already ignored unknown fields if it was good). Adding a new response field that the client does not have to read is also safe. Expanding what the server can return on an existing field is not safe — switch-case statements without defaults will break. 2. Non-breaking additions: new endpoints, new optional request fields, new response fields, new optional response headers, new HTTP 2xx codes if clients only check the family. 3. Breaking changes: removing a field, renaming a field, changing the type of a field (`int` to `string` is breaking even if "2" still encodes 2), changing a default, narrowing the accepted value set, expanding the produced value set on a fixed-enum field. 4. The 2026-standard advice: ship breaking changes behind a new version. For non-breaking additions, document the version they appeared in (a `since` annotation in your OpenAPI spec) so clients can feature-detect. 5. Compatibility tip: have a contract test suite recorded from your existing client SDKs against the candidate server. Any test that broke is a candidate breaking change you might have missed.

Edge cases to mention: - Status code changes are sneaky. Going from 200 to 201 for a POST is technically a change in the response shape; most clients tolerate it; some do not. Pick once, lock in. - Error-body shape changes are always breaking even if the status code stays the same. Clients parse the structured error. - Pagination semantics changes (offset to cursor, page-size default) almost always break someone.

What NOT to say: - "Adding fields is always safe." Not for fields with restricted enums on output. - "We just bump the minor version for any change." Without a version mechanism, minor version is just a number.

Common follow-up: "You need to add a `partial_refund` status to the orders API. How do you ship it without breaking the existing mobile app?" — Two options. (a) Ship it in `/v2/orders` only; v1 clients see the old enum, v2 clients see the expanded one. (b) Add the new status only to events posted to a new webhook endpoint, leaving the existing endpoint's enum unchanged for a deprecation period.

Stand-out line: "Say this and you'll separate yourself: a new enum value on an output field is breaking because clients switch-case on it — every senior API designer treats range-expansion as a version bump, not a non-breaking addition."

Q13 · Deprecation patterns — how to retire an old version cleanly

Tags: Difficulty: Medium · Asked at: Razorpay, Postman, PhonePe, Cred, Walmart Labs · Topic: REST · Versioning

The question (interviewer's verbatim phrasing):

You shipped v2 six months ago. You want to retire v1. Walk me through how you actually do that without breaking the partner who integrated on v1 two years ago.

The 30-second answer: Phase the retirement. Announce a sunset date (six months out is the minimum for any paid public API). Add a `Deprecation` and `Sunset` HTTP response header to every v1 response right now so clients can detect it programmatically. Email and dashboard-notify identified callers. At T-30 days, return a warning body alongside the regular response. At T-7 days, randomly return 410 Gone for 5% of v1 traffic (a "brownout") so anyone still on v1 finds out before T-0. At T-0, 410 with a `Link` header pointing to the new endpoint.

Step-by-step thought process: 1. Frame the constraint — public APIs are a commitment to your partners. Indian payment APIs especially: Razorpay deprecating an endpoint with two weeks notice would break checkout flows across every merchant in India. 2. Step 1 — add the headers.

`Deprecation: true` (RFC 8594) plus `Sunset: Sat, 30 Nov 2026 00:00:00 GMT`. Some teams add a `Link: <https://docs.example.com/v2-migration>; rel="successor-version"` so the new doc is discoverable. 3. Step 2 — communicate. Identify callers via API-key analytics. Email the contact on file. Show a deprecation banner in the developer dashboard. Update the doc with a sticky deprecation banner. 4. Step 3 — escalation. Logs only at T-90, header warnings at T-60, response-body warnings at T-30, brownouts at T-14 and T-7 (return 410 for 1% then 5% then 25% of requests randomly), full sunset at T-0. Each step gives clients another chance to notice before the final break. 5. Step 4 — the 410 itself. Return `410 Gone` with a body explaining the deprecation, a link to the new endpoint, and a contact for anyone stuck. Do not just turn the route off — silent 404 is worse than a loud 410.

Edge cases to mention: - Some clients ignore the `Deprecation` header and never look at logs until production breaks. Brownouts are the only thing that actually surfaces them. - Long-lived clients

(firmware on POS terminals, embedded SDKs in mobile apps with low update rates) need longer sunsets — 12-18 months not 6. - Status code 410 vs 404: 410 means "this used to exist and is permanently gone," 404 means "no such resource." 410 with the Link header is the right deprecation answer.

What NOT to say: - "Just take v1 down on the sunset date." Without brownouts, the 410 comes as a surprise to the half of clients who never read the email. - "Deprecation header is enough." It is not — manual outreach plus brownouts catch the long tail.

Common follow-up: "What if a paying partner refuses to migrate by the sunset date?" — Decision tree: extend for that one customer via a private endpoint (`/v1-legacy/`) gated to their API key, or hold the line and accept the churn. The right answer depends on contract value and engineering cost of carrying two versions.

Stand-out line: "Say this and you'll separate yourself: brownouts are the only thing that actually surfaces the long tail — engineers ignore deprecation headers and emails, but a 1% random 410 at T-14 gets immediate attention."

Q14 · Cursor-based versus offset-based pagination

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Cred, Walmart Labs · Topic: REST · Pagination

The question (interviewer's verbatim phrasing):

A list endpoint over 50 million orders. Page 1 is fast, page 10000 takes 8 seconds. Diagnose and propose a fix.

The 30-second answer: Offset pagination — `LIMIT 50 OFFSET 499950` for page 10001 — forces the database to scan and discard 499,950 rows before returning the page. That scan grows linearly with the page number. Switch to cursor pagination — the URL carries an opaque cursor encoding the last seen row's sort key, and the next page is

`WHERE (created_at, id) < (cursor_ts, cursor_id) ORDER BY created_at DESC, id DESC LIMIT 50` . The database uses the index range scan, every page is constant time, no scan-and-discard.

Step-by-step thought process: 1. Diagnose offset's cost. `LIMIT 50 OFFSET N` is implemented as "produce N+50 rows, throw the first N away, return the rest." Even with the perfect index, that is N row materialisations. 2. The cursor trick. Sort on a (timestamp, id) tuple that has a covering index. Encode the last returned (created_at, id) into the cursor. Next request reads `WHERE (created_at, id) < (cursor_ts, cursor_id) ORDER BY ... LIMIT 50` — index seek, return 50, done. Cost is constant in N. 3. The trade-off. Cursor pagination supports next / previous but not "jump to page 1000." Offset supports random access. For most product UIs (infinite scroll, "next page" button), cursor is fine. For admin tools needing "page 47," offset is fine because the tables are small. 4. Implementation note.

Encode the cursor as base64 of a small JSON blob to make it opaque, e.g.,

`eyJ0cyI6IjIwMjYtMDUzMzAiLCJpZCI6MTIzfQ` . Resist exposing raw IDs — clients will tamper. 5. Stability under writes. Offset is unstable — if a row is inserted between requests, page 2 will re-show what was on page 1 (or skip it). Cursor is stable because the cursor encodes the seen position. For inserting-into-a-stream UIs (notifications, feeds, payment events), this matters more than the speed argument.

Edge cases to mention: - Ties on the sort key. If two orders have the same `created_at` , sort by `(created_at, id)` so the order is deterministic and the cursor unambiguous. - Bidirectional cursors. Return both `next_cursor` and `prev_cursor` in the response so the client can page backwards. - Total count. Cursor APIs typically do not return total count because computing it kills the speed advantage. If the UI needs a count, ship an opt-in `?include=count` flag.

What NOT to say: - "Just add an index." Index helps the scan but does not eliminate the OFFSET. The scan-and-discard is the issue. - "Cursor pagination is just for infinite scroll." Useful for any consistent-snapshot view of a write-heavy collection.

Common follow-up: *"Razorpay's payments API uses cursor pagination. A merchant builds a reconciliation tool that walks all payments from a date range. How do they handle a payment created during the walk?"* — Cursor pagination shows a stable snapshot from the cursor's perspective. A payment created after the walk started will appear when (and only when) the walk reaches that cursor position. Reconciliation tools usually re-walk overlapping windows for safety.

Stand-out line: *"Say this and you'll separate yourself: `LIMIT 50 OFFSET N` scans N rows and throws them away — every page hurts more than the last, which is why every public-API team eventually rebuilds list endpoints around cursors."*

```
# FastAPI — cursor-based list endpoint
import base64, json

def encode_cursor(ts: str, id: int) -> str:
    return base64.urlsafe_b64encode(json.dumps({"ts": ts, "id": id}).encode()).decode()

def decode_cursor(c: str) -> tuple[str, int]:
    d = json.loads(base64.urlsafe_b64decode(c.encode()))
    return d["ts"], d["id"]

@app.get("/orders")
def list_orders(limit: int = 50, cursor: str | None = None):
    if cursor:
        ts, oid = decode_cursor(cursor)
        rows = db.execute(
            "SELECT id, created_at, status FROM orders "
            "WHERE (created_at, id) < (%s, %s) "
            "ORDER BY created_at DESC, id DESC LIMIT %s",
            (ts, oid, limit + 1),
        ).fetchall()
    else:
        rows = db.execute(
            "SELECT id, created_at, status FROM orders "
            "ORDER BY created_at DESC, id DESC LIMIT %s", (limit + 1),
        ).fetchall()
    has_more = len(rows) > limit
    page = rows[:limit]
    next_cursor = encode_cursor(page[-1]["created_at"], page[-1]["id"]) if has_more else None
    return {"data": page, "next_cursor": next_cursor}
```

Q15 · When offset pagination is still the right call

Tags: Difficulty: Easy · Asked at: Swiggy, Meesho, Freshworks · Topic: REST · Pagination

The question (interviewer's verbatim phrasing):

If cursor is faster and write-stable, why does anyone still use offset pagination?

The 30-second answer: Offset is the right call when the table is small (a few thousand rows), random-access matters (admin UI with a "page 47" link), or the API consumer expects a total count alongside the page. Cursor wins on speed and write-stability; offset wins on UX features and on simplicity for small data. Pick offset for an internal admin dashboard over 5,000 products. Pick cursor for the public payments feed over 50 million rows.

Step-by-step thought process: 1. Small data — offset over 10,000 rows is fast regardless. The query plan reads through the index and discards, but 10,000 is nothing for the database. 2. Random access —

when the user needs to jump to "page 47," cursor cannot help. An admin grid listing products often has this. 3. Total count expectation — offset goes well with `?count=true` returning the total. Cursor APIs that need a count force the server to either compute it expensively or return an estimate (Postgres's `EXPLAIN` row estimates). 4. Ergonomics — offset is `?page=N&per_page=M`, four characters in a URL bar. Cursor requires the client to parse, store, and pass an opaque token. For an internal tool that humans build URLs for, offset wins. 5. The hybrid — DRF's `LimitOffsetPagination` and FastAPI's `fastapi-pagination` support both; you can ship offset for v1 and cursor for v2 of the same endpoint without breaking the URL.

Edge cases to mention: - Offset with a large page number is the classic case where MySQL falls over: `LIMIT 50 OFFSET 1000000` on InnoDB scans 1,000,050 rows. The advice flips fast as the table grows. - Keyset pagination is the formal name for cursor pagination in DB-textbook land. Same idea, different vocabulary. - Skipping ahead a known amount — `?after_id=12345` is a thin cursor, half-way between offset and full opaque cursor.

What NOT to say: - "Cursor is always better." Not for the page-47-of-100 internal admin tool. - "Offset is deprecated." It is not. It is the right tool for small data and random access.

Common follow-up: *"Your internal admin dashboard hits the same API endpoint as your mobile app. The mobile app uses cursor; the admin needs page jumps. What do you do?"* — Two endpoints with shared business logic — `/admin/orders?page=N` for the admin (offset), `/v2/orders?cursor=...` for the mobile (cursor). Same service layer, two transport adapters. Or document that the public API supports cursor only and have the admin tool walk pages internally.

Stand-out line: *"Say this and you'll separate yourself: cursor for write-heavy public feeds, offset for small admin grids with page jumps — the choice is about random access and total-count expectations, not about one being modern and the other deprecated."*

Section D — Error contracts and production patterns

The last five questions cover what separates a passable API from one that holds up at midnight when something goes wrong. The HTTP status matrix, the error-body shape that lets clients programmatically retry, the idempotency-key pattern for POST, the request-ID convention for tracing, and the dedup approach that gives webhooks "exactly once" delivery.

Q16 · The HTTP status code matrix — 200, 201, 204, 400, 401, 403, 404, 409, 422, 429, 500, 503

Tags: Difficulty: Easy · Asked at: TCS, Razorpay, Postman, Cred, JP Morgan India · Topic: REST · Status Codes

The question (interviewer's verbatim phrasing):

Walk me through 200, 201, 204, 400, 401, 403, 404, 409, 422, 429, 500, 503. When do you return each one?

The 30-second answer: 2xx for success. 200 for GET / PUT / PATCH that returns a body; 201 for POST that created a resource (Location header points to it); 204 for DELETE or empty success. 4xx for the client's fault. 400 for malformed input the parser rejected; 401 for missing or invalid credentials; 403 for valid credentials with no permission; 404 for the resource does not exist; 409 for a state conflict (e.g., concurrent update); 422 for valid syntax but semantically invalid (Pydantic validation failure); 429 for rate-limited. 5xx for the server's fault. 500 for an unexpected exception; 503 for the service is up but a dependency is down or it is overloaded.

Step-by-step thought process: 1. The 2xx family. 200 is the workhorse. 201 specifically means "I created a resource" — return the Location header with the new URL. 204 means "success, no body" — DELETE typically, sometimes PUT. 2. 400 vs 422. 400 is "your JSON did not parse" or "you missed a required field at the protocol level." 422 is "your JSON parsed, every field is the right type, but the combination is illegal — start date is after end date, amount exceeds plan limit." FastAPI/Pydantic returns 422 for validation; DRF returns 400 by default but you can shift to 422 in the serializer. 3. 401 vs 403. 401 is "I do not know who you are" — missing or bad token. 403 is "I know who you are and you are not allowed." Use 401 when re-authentication might fix it; 403 when no amount of re-authentication helps. 4. 409 and 429. 409 is the conflict family — concurrent modification (ETag mismatch), duplicate-by-business-key, state-machine illegal transitions. 429 is rate-limit; pair with `Retry-After` header. 5. 5xx is on you. 500 means "something blew up I did not expect" — log the stack, return a request ID, do not leak the stack to the client. 503 is "I am up but cannot serve right now" — dependency down, maintenance, intentional load shedding. Add `Retry-After` so clients back off.

Edge cases to mention: - 405 Method Not Allowed — returned when the URL exists but the verb does not. Pair with `Allow: GET, POST` header listing valid verbs. - 412 Precondition Failed — for ETag / If-Match races. Useful in concurrent-update flows. - 503 with `Retry-After: 30` is the polite way to shed load. Better than a 500 because it signals "retry me," whereas 500 signals "I am broken, do not retry without humans looking."

What NOT to say: - "200 for everything, errors in the body." That is the JSON-RPC style and it loses you HTTP caching, monitoring tools (which key on status code), and client retry heuristics. Use the status codes. - "404 for unauthorised so you do not leak existence." Sometimes correct (resource enumeration defence), but only after you have considered the audit trade-off — 403 vs 404 differs in what the access log captures.

Common follow-up: "You GET /orders/77 but order 77 belongs to another user. Do you return 403 or 404?" — Depends on whether existence is sensitive. For public IDs (Razorpay's payment ID is a hash) the existence of the resource is not secret, so 403 is honest. For sequential IDs where enumeration is a threat, 404 may be safer. Document the choice.

Stand-out line: "Say this and you'll separate yourself: every senior backend engineer can rattle off the 12-code matrix without hedging — 401 means re-auth, 403 means no path, 409 means conflict, 422

means semantically wrong — and getting that vocabulary right is how clients write retry logic that actually works."

Q17 · Structured error responses — the body shape clients can actually code against

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, PhonePe · Topic: REST · Error Handling

The question (interviewer's verbatim phrasing):

Design the error response body for a 422 validation error. The client needs to display field-level errors to the user.

The 30-second answer: Return a top-level `error` object with a stable `code`, a human-readable `message`, a `request_id` for support, and a `details` array for field-level information. Codes are programmatic ("INVALID_AMOUNT"), messages are human ("Amount must be positive"). Field-level entries have the JSON pointer, the error code for that field, and a message. Clients code against the `code` field, never against the `message`. RFC 7807's `application/problem+json` is the standardised flavour of this if you want to follow a spec.

Step-by-step thought process: 1. Two consumers — humans (developers reading the response, end users reading translated UI) and clients (code that branches on the error). Both need to be served. Codes are for code; messages are for humans. 2. Required fields. `code` (machine-readable, stable, SCREAMING_SNAKE_CASE), `message` (human, English, OK for the dev console), `request_id` (so logs can be looked up), `details` (per-field issues for 422; nested operation issues for batch endpoints). 3. Optional fields. `documentation_url` to point to the relevant doc page. `retry_after` (mirroring the header) for 429/503. `errors[]` for batch operations where multiple sub-errors apply. 4. The big rule: code is the contract. If you ever change a code, that is breaking. Bumping a message is fine, bumping a code is a version-2 change. 5. RFC 7807 is the standardised cousin — `{"type": "...", "title": "...", "status": 422, "detail": "...", "instance": "..."} plus the application/problem+json content type. Some teams adopt it; many roll their own shape. Either is defensible, just be consistent.`

Edge cases to mention: - For 401/403, do not leak which credential failed. `INVALID_CREDENTIALS` is enough — do not say "user not found" vs "wrong password" (that is the user-enumeration trap). - For 500, return a minimal body — just `code`, `message="An internal error occurred"`, and `request_id`. Never expose the stack trace. - Internationalisation. Messages are typically English in the body; clients translate using the code. If you must i18n on the server, take the locale from the `Accept-Language` header.

What NOT to say: - "Just return the exception's `str()`." That leaks internals and is not machine-parseable. - "Errors do not need a stable shape." They do — clients code against this.

Common follow-up: *"Razorpay's payment-create returns BAD_REQUEST_ERROR with a code on every failure. How does this help the SDK?"* — The SDK can map specific codes to typed exceptions —

`PaymentDeclinedError` , `InvalidVpaError` — and the merchant's app handles them with type-safe branches instead of string-matching the message.

Stand-out line: *"Say this and you'll separate yourself: codes are the contract, messages are for humans — once you ship a code, you can't rename it without bumping the version, which is why SCREAMING_SNAKE_CASE and a stable taxonomy are the senior-engineer choice."*

```
# FastAPI — global validation error handler with structured body
from fastapi import FastAPI, Request, status
from fastapi.exceptions import RequestValidationError
from fastapi.responses import JSONResponse

@app.exception_handler(RequestValidationError)
async def validation_handler(request: Request, exc: RequestValidationError):
    return JSONResponse(
        status_code=status.HTTP_422_UNPROCESSABLE_ENTITY,
        content={
            "error": {
                "code": "VALIDATION_FAILED",
                "message": "One or more fields are invalid.",
                "request_id": request.headers.get("X-Request-Id"),
                "details": [
                    {
                        "field": ".".join(str(p) for p in e["loc"][1:]),
                        "code": e["type"].upper(),
                        "message": e["msg"],
                    }
                    for e in exc.errors()
                ]
            },
        },
    )
```

Q18 · Idempotency keys for POST — the Razorpay pattern

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, PhonePe, Stripe-style interviewers · Topic: REST · Idempotency

The question (interviewer's verbatim phrasing):

Your POST /payments endpoint charges a card. A flaky network retries it. How do you guarantee that the customer is charged at most once?

The 30-second answer: The client generates an idempotency key (a UUID) and sends it as `Idempotency-Key` header on the POST. The server stores `(key, request_hash, response_status, response_body)` in a dedup table on first acceptance. A retry with the same key and same body returns the stored response without re-processing. A retry with the same key and a different body returns 400 because the client is sending mismatched semantics under one key. Keep dedup entries for 24 hours — long enough for retries, short enough to bound storage.

Step-by-step thought process: 1. The client's job. Generate one UUID per logical action (one checkout = one key). Send it in the `Idempotency-Key` header on the POST. On retry, send the *same* key. 2. The server's job at first request. Acquire a lock on the key (Redis SETNX is the standard). Process the payment. Store the result keyed by `(api_key, idempotency_key)` with a 24-hour TTL. Release the lock. Return the response. 3. The server's job on retry. Look up the key. If found: return the stored response verbatim (same status, same body). Skip processing. 4. The body mismatch defence. Hash the request body and store the hash alongside the response. On retry, compute the hash again; if it differs, the client is misusing the key (e.g., they reused the same key for two different orders). Return 400 with a specific error code so the SDK knows to regenerate the key. 5. The race during first processing. If two retries arrive simultaneously, the second one finds the lock held. It can wait briefly (with a timeout) and then return the stored response, or it can return 409 with `Retry-After` so the client backs off.

Edge cases to mention: - TTL trade-offs. 24 hours is the Stripe / Razorpay default. Longer means more storage but tolerates clients with slow retry queues; shorter risks the dedup expiring before a client retry. Make it configurable per endpoint if you have wildly different SLAs. - Scope. The key is unique per API key, not globally. Two different merchants can both use UUID `abc-123` without colliding. - Make the header mandatory on POSTs that have real cost. A POST without the header is allowed for read-shaped batch POSTs (search), but for /payments-create it should be a 400 if missing.

What NOT to say: - "Just check if a payment with the same amount and card already exists." That is heuristic dedup and it fails — the customer might legitimately want two ₹500 payments to the same merchant a minute apart. - "The database's unique constraint will dedup." Only if the constraint is on a column the client controls, which they usually do not.

Common follow-up: *"What if the first request times out from the client's perspective at exactly the moment the server commits? The server has the row, the client thinks it failed and retries with the same key. What does the second request return?"* — The dedup table has the response; the second request returns it with HTTP 200 (the original status). The client now knows the first attempt succeeded. This is the entire point of the pattern.

Stand-out line: *"Say this and you'll separate yourself: idempotency-key dedup plus body-hash mismatch detection is the Razorpay/Stripe pattern — the key alone is not enough because clients reuse keys by mistake, and the body hash is how you catch them."*

```
# FastAPI — idempotency middleware
import hashlib, json
from fastapi import Request, Response, HTTPException

DEDUP_TTL = 24 * 3600

@app.middleware("http")
async def idempotency(request: Request, call_next):
    if request.method != "POST":
        return await call_next(request)
    key = request.headers.get("Idempotency-Key")
    if not key:
        return await call_next(request)
    api_key = request.state.api_key
    body = await request.body()
    body_hash = hashlib.sha256(body).hexdigest()
    cache_key = f"idem:{api_key}:{key}"
    cached = redis.get(cache_key)
    if cached:
        cached = json.loads(cached)
        if cached["body_hash"] != body_hash:
            raise HTTPException(400, {"code": "IDEMPOTENCY_KEY_REUSED"})
        return Response(cached["body"], cached["status"], headers=cached["headers"])
    # acquire lock, process, store
    if not redis.set(f"{cache_key}:lock", "1", nx=True, ex=60):
        raise HTTPException(409, {"code": "IDEMPOTENT_REQUEST_IN_PROGRESS"})
    response = await call_next(request)
    body_out = b"".join([c async for c in response.body_iterator])
    redis.setex(cache_key, DEDUP_TTL, json.dumps({
        "status": response.status_code, "body": body_out.decode(),
        "headers": dict(response.headers), "body_hash": body_hash,
    }))
    return Response(body_out, response.status_code, headers=response.headers)
```

Q19 · Request IDs for tracing — the X-Request-Id convention

Tags: Difficulty: Easy · Asked at: Razorpay, PhonePe, Postman, Walmart Labs · Topic: REST · Observability

The question (interviewer's verbatim phrasing):

A customer complains "my payment failed yesterday at 3pm." How do you find the exact request in your logs? How does the API help?

The 30-second answer: Generate a request ID at the edge — middleware reads an incoming **X-Request-Id** header if the client sent one, otherwise mints a UUID. Attach it to every log line for the

request via a contextvar. Echo it back in the response header `X-Request-Id` and inside the error body. The customer (or their support engineer) reads the ID from the failed response, hands it to your support, and one grep finds every log line for that request across every service it touched.

Step-by-step thought process: 1. Generate at the edge. The first service to handle the request (API gateway or the outermost middleware) accepts a client-supplied `X-Request-Id` if present, otherwise generates a UUID. Length 16-32 chars, URL-safe. 2. Propagate. The ID rides downstream — every internal RPC call, every queue message, every DB query carries it. In Python this is typically a `ContextVar` plus a log filter that pulls the current value into every log record's `extra` dict. 3. Echo back. `X-Request-Id` on the response header always. In the error body, include `request_id` as a top-level field (see Q17). 4. Index for search. Ingest into your log store (Loki, Cloudwatch, Datadog) with `request_id` as a structured field. A single search by ID returns every line across services, including downstream API call traces. 5. The W3C Trace Context standard (`traceparent` header) is the heavier cousin — it carries a trace ID plus a span ID and supports propagation across HTTP and queue boundaries. For full distributed tracing (OpenTelemetry), use it; for "just give support a string to grep," `X-Request-Id` is enough.

Edge cases to mention: - Validate the client-supplied ID. Cap length, restrict to URL-safe chars, reject anything ridiculous. Otherwise clients can inject log-line spam. - Do not put PII in the ID. UUIDs are fine; customer email is not. - For high-traffic APIs, consider compact IDs (16 chars of base32) to reduce log volume — the UUID's hyphens add up at billion-request scale.

What NOT to say: - "Use the IP and timestamp." Two requests in the same millisecond from behind NAT share the IP and timestamp. - "The trace ID is enough." For the full distributed trace, yes. For surfacing a string to the customer, X-Request-Id is the right convention.

Common follow-up: "How do you tie this to the trace from your front-end?" — Front-end generates a request ID on each XHR call and sends it via `X-Request-Id`. The browser tab's tracing tool (Sentry, OpenTelemetry-web) records the ID. Now the front-end span and the back-end logs share the same key.

Stand-out line: "Say this and you'll separate yourself: every error response carries a `request_id` that the customer can hand to support — one grep across the log store closes the support ticket, and that's the whole observability ROI for a one-line middleware."

```
# FastAPI — request-ID middleware with contextvar logging
import contextvars, logging, uuid
from fastapi import Request

request_id_var = contextvars.ContextVar("request_id", default="-")

class RequestIdFilter(logging.Filter):
    def filter(self, record):
        record.request_id = request_id_var.get()
        return True

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(request_id)s] %(levelname)s %(message)s",
)
logging.getLogger().handlers[0].addFilter(RequestIdFilter())

@app.middleware("http")
async def add_request_id(request: Request, call_next):
    rid = request.headers.get("X-Request-Id") or uuid.uuid4().hex
    request_id_var.set(rid)
    response = await call_next(request)
    response.headers["X-Request-Id"] = rid
    return response
```

Q20 · "Exactly once" via dedup — webhooks and the at-least-once reality

Tags: Difficulty: Hard · Asked at: Razorpay, PhonePe, Cred, Walmart Labs, JP Morgan India · Topic: REST · Delivery Semantics

The question (interviewer's verbatim phrasing):

Razorpay sends you a webhook when a payment succeeds. Sometimes you receive the same event twice. How do you make sure your downstream side effect (sending an order confirmation email) fires exactly once?

The 30-second answer: HTTP delivery is at-least-once. Webhooks retry on any non-2xx, so a receiver that times out mid-processing may get the same event again. The standard pattern is dedup on the receiver — every event carries an event ID, you store processed IDs in a table with a unique constraint on `event_id`, and you wrap "store the dedup row" plus "do the side effect" in one transaction. If the row already exists, you return 200 OK without re-processing. The combination of unique constraint plus transaction is what gives you effective exactly-once.

Step-by-step thought process: 1. Frame the reality. Networks fail, processes crash, retries happen. There is no exactly-once delivery in distributed systems without a coordinator. What we can have is

effectively-once processing — at least one successful processing, never two. 2. The sender's job. Sign the webhook (HMAC over the body with a shared secret), include an `event_id` that is unique per logical event, retry with backoff on any non-2xx, include the original `event_id` on every retry. 3. The receiver's job. Verify the signature first (reject 401 if it fails). Read the `event_id`. Open a transaction. Insert into a `processed_events` table with `event_id` as a primary key — if the insert succeeds, run the side effect, commit. If the insert fails on the unique constraint, the event was already processed, commit and return 200. 4. The side-effect catch. The side effect (sending the email) must be inside the same transaction as the dedup insert, or be itself idempotent. Otherwise: the dedup insert commits, the email send fails, the retry sees the dedup row and returns 200 — but the email never went out. Use a transactional outbox: write the email-to-send row in the same transaction, dispatch it via a polling worker. 5. Retention. Dedup rows live as long as the sender retries — usually 24-72 hours. After that, the sender gives up; the row can be archived.

Edge cases to mention: - Signature verification is non-negotiable. Without it, an attacker forges a "payment captured" event and gets free goods. - Replay window. Include a timestamp in the signed body; reject events older than 5 minutes. Defends against replay attacks where an attacker captures a real webhook and re-sends it. - The retry storm. If the receiver returns 500 for an hour, the sender's queue can grow. When the receiver comes back, the queue floods. Receivers should be idempotent and senders should rate-limit retries.

What NOT to say: - "Use a queue with exactly-once semantics." There is no such queue across an HTTP boundary. The "exactly once" guarantees you read about (Kafka EOS) are for in-cluster producers / consumers, not for HTTP webhooks. - "Just check if we already processed it by querying our domain DB." Possible, but the explicit dedup table is faster and decouples the dedup from the domain schema.

Common follow-up: *"What if the dedup table query itself fails — say Postgres is briefly down? Do you accept the webhook or reject it?"* — Return 500 (or 503). The sender will retry. Returning 200 without persisting means the event is silently dropped. Returning 500 keeps the sender on the hook.

Stand-out line: *"Say this and you'll separate yourself: 'exactly once' doesn't exist over HTTP — what does exist is at-least-once delivery plus a `processed_events` unique constraint inside the same transaction as the side effect, and that combination is what every senior backend engineer means when they say 'effectively exactly-once'."*

```
# FastAPI — webhook receiver with dedup and signed verification
import hmac, hashlib
from fastapi import APIRouter, Request, HTTPException, Depends
from sqlalchemy.exc import IntegrityError

router = APIRouter()

def verify_signature(secret: str, body: bytes, signature: str) -> bool:
    expected = hmac.new(secret.encode(), body, hashlib.sha256).hexdigest()
    return hmac.compare_digest(expected, signature)

@router.post("/webhooks/razorpay")
async def handle(request: Request, db = Depends(get_db)):
    body = await request.body()
    sig = request.headers.get("X-Razorpay-Signature", "")
    if not verify_signature(WEBHOOK_SECRET, body, sig):
        raise HTTPException(401, "bad signature")
    event = json.loads(body)
    event_id = event["id"]
    try:
        with db.begin():
            db.execute("INSERT INTO processed_events (event_id) VALUES (%s)",
                       (event_id,))
            # write to outbox in the SAME transaction
            db.execute(
                "INSERT INTO email_outbox (kind, payload) VALUES (%s, %s)",
                ("order_confirmation", json.dumps(event["payload"])),
            )
    except IntegrityError:
        pass # already processed, idempotent return
    return {"ok": True}
```

That closes Chapter 5. The next chapter — DB interaction — picks up where Q18's idempotency-table and Q20's outbox-table examples left off: SQLAlchemy core vs ORM, connection pooling, transactions, and the N+1 query trap that kills more Django views in India than any other single bug.

Chapter 6 — DB Interaction (15 Questions)

Python Interview Prep — 2026 Edition · Learn & Excel · File 07 Backend Track Library

This chapter is for backend engineers who interview at Indian product companies — Razorpay, Cred, Swiggy, Zerodha, Meesho, PhonePe, Postman, Flipkart — plus GCCs (Walmart Labs, Goldman Sachs India, JP Morgan India, Microsoft India) and the late-stage startup tier. The DB-interaction round is where

round-two or round-three pivots from "do you know Python" to "do you actually run Python services against Postgres in production." Fifteen questions covering the five surfaces that get probed: SQLAlchemy Core vs ORM choice, connection pooling (sync and async drivers, the pool-exhaustion failure mode, N+1 detection), transactions and isolation levels with retry patterns, and the real-world plumbing — Alembic migrations, read-replica routing, Redis as a sidecar cache or session store, and idempotency keys for exactly-once semantics.

The framing is deliberately Django-agnostic. Django's ORM is covered in Chapter 3 of this same file. The questions here assume SQLAlchemy 2.x (which is the version every Indian backend team that isn't on Django is running in 2026), Postgres 15 or 16 as the OLTP store, asyncpg for async paths, and Redis 7 for the cache layer. If your shop runs MySQL 8 the patterns transfer with no change beyond driver swaps; isolation-level defaults are the only meaningful drift.

Each question follows the standard format — tags, the interviewer's verbatim phrasing, a 30-second answer, the step-by-step "what to say out loud" thought process, edge cases, what NOT to say, the common follow-up, and a runnable code snippet where it sharpens the answer. The code is SQLAlchemy 2.x style throughout — `sqlalchemy.orm.DeclarativeBase`, `select()`, `Session`, `async_sessionmaker` — because the 1.x-style `query()` API is what flags a candidate as "hasn't touched the codebase since 2021."

Table of Sections

- Section A — SQLAlchemy Core vs ORM (Q1–Q5)
- Section B — Connection Pooling and N+1 (Q6–Q8)
- Section C — Transactions and Isolation (Q9–Q12)
- Section D — Real-World DB Patterns (Q13–Q15)

Section A — SQLAlchemy Core vs ORM

The first five questions exist because every Indian backend team I've seen mixes Core and ORM in the same codebase, and most engineers cannot articulate where the line is. The interviewer is checking two things: that you know SQLAlchemy 2.x is a unified API (no more `engine.execute` vs `session.query` split), and that you can name the situations where dropping out of the ORM into Core or raw SQL is the right move rather than the lazy move. The Session object, the unit-of-work pattern, the `declarative_base` deprecation in favour of `DeclarativeBase` — these are the surfaces probed in round two at any product company.

Q1 · SQLAlchemy Core vs ORM — When to pick each one

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Swiggy, Postman, Walmart Labs · Topic: SQLAlchemy basics

The question (interviewer's verbatim phrasing):

Your team is building a new FastAPI service that hits Postgres. Walk me through how you'd decide whether each query should use the SQLAlchemy ORM, SQLAlchemy Core, or raw SQL.

The 30-second answer: The ORM is for stateful, object-graph work — load a row, mutate it, flush, commit. SQLAlchemy Core is for SQL-shaped work where you want the typed expression language but don't care about identity-mapped objects — bulk inserts, analytics aggregations, ad-hoc joins that don't fit your mapped classes. Raw SQL via `text()` is the escape hatch for the queries Core can't express cleanly — CTEs with window functions, dialect-specific syntax, or a query you've already tuned in `psql` and don't want to re-translate. The three are not religious choices; a single endpoint commonly uses all three.

Step-by-step thought process: 1. Lead with the mental model: ORM tracks objects, Core composes SQL, raw SQL is the escape hatch. The question isn't "which one is best," it's "which one fits the current statement." 2. Give the ORM use cases. You have a `User` mapped class, the user is changing their email, you want change tracking and validation events to fire — that's ORM. You're returning a list of orders with their items eagerly loaded — also ORM, because the object graph is the deliverable. 3. Give the Core use cases. A nightly batch job that inserts 200k rows into `payments_audit` — Core's `insert()` with `executemany` is 10x faster than `session.add_all()` because it bypasses the identity map and the unit-of-work. An analytics endpoint that groups payments by merchant and date — Core's `select(payments.c.merchant_id, func.sum(payments.c.amount))` is cleaner than mapping a fake class just to hold the aggregate. 4. Give the raw SQL trigger. You wrote a recursive CTE for a category-tree query, it took two hours to tune in `psql`, the planner is happy. Don't re-translate — drop it in as `session.execute(text("WITH RECURSIVE ..."))` with named parameters. The escape hatch is a feature, not a failure. 5. End with the integration point that catches juniors out: SQLAlchemy 2.x unified the API. All three modes go through the same `Session.execute()` or `Connection.execute()` call. You don't switch engines or contexts to mix them — you just pass different constructs to the same `execute()`.

Edge cases to mention: - ORM `session.flush()` round-trips per-object for inserts unless you use `bulk_insert_mappings` or Core's `insert()`. On a 50k-row insert this is the difference between 30 seconds and 300 milliseconds. - Core constructs returned by `select()` are typed expressions, not strings. The dialect compiler turns them into Postgres or MySQL SQL at execute time, so you get cross-dialect portability for free. - Raw `text()` calls must use named parameters (`:user_id`), not f-strings or `%s` formatting — otherwise you reintroduce SQL injection and lose the per-dialect quoting.

What NOT to say: - "Always use the ORM." Wrong for bulk loads, analytics, and many migration paths. This answer says you've never run a production batch job. - "Raw SQL is unsafe." Raw SQL with bound

parameters is exactly as safe as ORM SQL — both go through the DB-API parameter machinery. Unsafe SQL is string-concatenated SQL, regardless of which library you used.

Common follow-up: *"Show me the same 'sum of payments per merchant for last 7 days' query written three ways — ORM, Core, raw SQL."*

Stand-out line: *"Say this and you'll separate yourself: ORM, Core, and raw SQL are not religious choices — they're three constructs that flow through the same `execute()`, and a single endpoint commonly uses all three for object work, bulk loads, and the CTE you already tuned in psql."*

```

from sqlalchemy import create_engine, select, func, text, Column, Integer, Numeric, DateTime
from sqlalchemy.orm import DeclarativeBase, Session, Mapped, mapped_column
from datetime import datetime, timedelta

class Base(DeclarativeBase):
    pass

class Payment(Base):
    __tablename__ = "payments"
    id: Mapped[int] = mapped_column(primary_key=True)
    merchant_id: Mapped[int]
    amount: Mapped[float] = mapped_column(Numeric(12, 2))
    created_at: Mapped[datetime]

engine = create_engine("postgresql+psycopg://user:pass@localhost/billing")
cutoff = datetime.utcnow() - timedelta(days=7)

# ORM-style – typed objects, identity-mapped
with Session(engine) as session:
    stmt = (
        select(Payment.merchant_id, func.sum(Payment.amount).label("total"))
        .where(Payment.created_at >= cutoff)
        .group_by(Payment.merchant_id)
    )
    rows = session.execute(stmt).all()

# Core-style – same SQL, table object, no mapping needed
from sqlalchemy import Table, MetaData
md = MetaData()
payments_t = Table("payments", md, autoload_with=engine)
with engine.connect() as conn:
    stmt = (
        select(payments_t.c.merchant_id, func.sum(payments_t.c.amount).label("total"))
        .where(payments_t.c.created_at >= cutoff)
        .group_by(payments_t.c.merchant_id)
    )
    rows = conn.execute(stmt).all()

# Raw SQL – escape hatch, named parameters
with engine.connect() as conn:
    rows = conn.execute(
        text("SELECT merchant_id, SUM(amount) AS total FROM payments "
            "WHERE created_at >= :cutoff GROUP BY merchant_id"),
        {"cutoff": cutoff},
    ).all()

```

Q2 · The Session and the Unit-of-Work pattern

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Goldman Sachs India, Microsoft India · Topic: SQLAlchemy Session

The question (interviewer's verbatim phrasing):

What is the SQLAlchemy `Session` ? What does it mean when people say it implements the unit-of-work pattern?

The 30-second answer: The `Session` is a workspace that tracks every mapped object you've loaded or created during one logical operation. The unit-of-work pattern means: you mutate objects in Python — change an attribute, append to a relationship, add a new instance — and the Session accumulates those changes. When you call `commit()` (or `flush()`), the Session computes the minimum set of `INSERT`, `UPDATE`, and `DELETE` statements needed to push your in-memory state to the database, orders them to respect foreign keys, and runs them inside a single transaction. You never write SQL for the mutations; the Session does.

Step-by-step thought process: 1. Define the workspace. A Session opens (lazily) one DB connection from the pool, holds an identity map (a `{(class, primary_key) -> object}` dict so the same row loaded twice returns the same Python object), and tracks the state of every mapped instance — transient, pending, persistent, deleted, detached. 2. Explain what unit-of-work means in plain English: it batches your intent. Instead of "every attribute write fires an UPDATE," the Session records that the object is dirty and emits one UPDATE per dirty object at flush time. Multiple writes to the same object collapse into one statement. 3. Walk through a flush. Flush ordering is topological: parents before children for inserts, children before parents for deletes. The Session computes the dependency graph from your mapped foreign keys; you do not have to think about insertion order. 4. Show the lifecycle handshake. `Session.add(obj)` makes the object pending. `Session.flush()` emits the SQL but doesn't commit — useful when you need the auto-generated primary key for a downstream foreign-key insert. `Session.commit()` flushes and commits the underlying transaction; the connection returns to the pool. 5. End with the standard FastAPI / Flask shape: one Session per request. Open at the start, close in a `finally` block, and never share Sessions across threads or across async tasks. This is the rule that prevents 90% of weird "object is not bound to a Session" errors in production.

Edge cases to mention: - The identity map means `session.get(User, 42)` returns the same Python object whether called once or ten times in the same Session — pointer-equal, not value-equal. - `Session.expire_all()` invalidates every loaded object so the next attribute access refetches; useful after a long-running task where the DB may have moved underneath you. - Autoflush (the default) means a `session.execute(select(...))` will flush pending changes first, so your query sees your own writes. Sometimes that's exactly wrong — disable with `with session.no_autoflush:`.

What NOT to say: - "Session is like a database cursor." Wrong abstraction — a cursor is a single-statement primitive; a Session is a transaction-scoped object-tracking workspace. - "I create a global Session at app startup." This is the single most common production bug — global Sessions accumulate state across requests, leak memory, and break concurrency.

Common follow-up: "Show me the FastAPI dependency that gives each request its own Session and closes it cleanly."

Stand-out line: "Say this and you'll separate yourself: Session is a transaction-scoped workspace, not a connection — one per request, never global, and `expire_on_commit=False` is the FastAPI default that stops the 'object is not bound to a Session' error from surfacing after commit."

```
from sqlalchemy.orm import sessionmaker
from fastapi import Depends, FastAPI

SessionLocal = sessionmaker(bind=engine, expire_on_commit=False)
app = FastAPI()

def get_session():
    session = SessionLocal()
    try:
        yield session
    finally:
        session.close()

@app.post("/users/{user_id}/email")
def update_email(user_id: int, new_email: str, session=Depends(get_session)):
    user = session.get(User, user_id)
    if user is None:
        return {"error": "not found"}
    user.email = new_email          # Session marks this object dirty
    session.commit()               # one UPDATE statement, one transaction
    return {"id": user.id, "email": user.email}
```

Q3 · `declarative_base()` vs `DeclarativeBase` — what changed in SQLAlchemy 2.x

Tags: Difficulty: Easy · Asked at: Cred, Swiggy, Postman, JP Morgan India · Topic: SQLAlchemy 2.x mapping

The question (interviewer's verbatim phrasing):

The old SQLAlchemy tutorials all start with `Base = declarative_base()`. The 2.0 docs use a class that inherits from `DeclarativeBase`. What changed and why?

The 30-second answer: `declarative_base()` is a factory function that returns a dynamically-built base class — which works, but type checkers like mypy and Pyright can't see through it, so you lose autocompletion and type safety on every mapped attribute. `DeclarativeBase` is a real class in SQLAlchemy 2.0 that you subclass directly, paired with `Mapped[T]` annotations and `mapped_column()` for column definitions. Same runtime behaviour, full static typing. New projects in 2026 use `DeclarativeBase`; the function form is retained for backward compatibility but produces deprecation warnings in 2.x.

Step-by-step thought process: 1. Frame the old vs new. `Base = declarative_base()` returns an opaque class object built at import time; you couldn't annotate columns with types, you used `Column(Integer, primary_key=True)` and the IDE saw it as a `Column` instance, not an `int`. 2. Introduce the new shape. `class Base(DeclarativeBase): pass` is a normal subclass; mapped columns use `Mapped[int] = mapped_column(primary_key=True)`. Now mypy sees `User.id` as `int`, not `Column(Integer)`. 3. Show the migration mechanics: you can convert table-by-table. SQLAlchemy 2.x supports both styles in the same project. The deprecation warning is emitted once per Base creation, so noisy migrations stay quiet. 4. Call out the imperative mapping form too — `registry.map_imperatively(User, users_table)` — for the rarer case where the class is generated dynamically or comes from a third-party library you don't want to subclass. Declarative is the default; imperative is for advanced cases. 5. End with the practical impact for interviews: code samples that use `declarative_base()` flag your codebase as pre-2021. Even if everything works, a senior reviewer will ask why you haven't migrated.

Edge cases to mention: - `MappedAsDataclass` is a mixin you can add to `DeclarativeBase` to get dataclass-style `__init__` generation — useful for FastAPI request models and tests, available since 2.0. - The `Mapped[Optional[str]]` annotation drives both the Python type and the SQL `NULL`-ability — you no longer specify `nullable=True` separately for most cases. - If you have a generic base mixin (e.g. `TimestampMixin`), it must use `Mapped` annotations too; mixing old-style `Column()` and new-style `mapped_column()` in the same class hierarchy produces confusing errors.

What NOT to say: - "`declarative_base()` was removed." It wasn't — it's still in 2.x as a compatibility shim. Don't lie about removals; the interviewer will catch you. - "`DeclarativeBase` is faster." It isn't — same runtime path; the win is at type-check time, not runtime.

Common follow-up: "Write the same `User` table both ways so I can see the diff."

Stand-out line: "Say this and you'll separate yourself: `DeclarativeBase` plus `Mapped[T]` is the 2.x convention because the type checker can finally see through the base — same runtime, full static typing, and every code sample using `declarative_base()` flags the codebase as pre-2021."

```
# Pre-2.0 style – still works, no static typing
from sqlalchemy import Column, Integer, String
from sqlalchemy.orm import declarative_base

LegacyBase = declarative_base()

class UserOld(LegacyBase):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True)
    email = Column(String(255), nullable=False, unique=True)

# 2.0 style – typed, mypy-friendly, recommended
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column
from typing import Optional

class Base(DeclarativeBase):
    pass

class User(Base):
    __tablename__ = "users"
    id: Mapped[int] = mapped_column(primary_key=True)
    email: Mapped[str] = mapped_column(String(255), unique=True)
    deleted_at: Mapped[Optional[datetime]] = mapped_column(default=None)
```

Q4 · The raw-SQL escape hatch — `text()` and bound parameters

Tags: Difficulty: Easy · Asked at: Razorpay, PhonePe, Walmart Labs, TCS · Topic: raw SQL

The question (interviewer's verbatim phrasing):

When you need to drop into raw SQL inside SQLAlchemy, what does the API look like? Show me a parameterised query.

The 30-second answer: Wrap the SQL string in `sqlalchemy.text()`, mark parameters with a colon prefix (`:user_id`), and pass them as a dict to `Session.execute()` or `Connection.execute()`. The result is a `Result` object you iterate with `.all()`, `.first()`, `.scalar()`, or `.mappings().all()` for dict-shaped rows. The bound-parameter machinery is the same one the ORM uses, so injection is impossible if you never string-interpolate.

Step-by-step thought process: 1. Show the canonical form.

`session.execute(text("SELECT id, email FROM users WHERE id = :uid"), {"uid": 42})` is the entire pattern — no curlies, no `%s`, no f-strings. 2. Explain why this is safe. The driver substitutes parameters at protocol level, after the SQL is parsed. The user's value is never concatenated

into the SQL string. Even if `uid` is `"42; DROP TABLE users"`, Postgres treats it as a single string-typed parameter. 3. Walk through result shaping. The default `.all()` returns a list of `Row` namedtuples — index by position or attribute. Use `.mappings().all()` for `dict`-like access — handy when forwarding to JSON. Use `.scalar()` when you expect one column from one row. 4. Note the typed-execute pattern: `text("SELECT id FROM users WHERE created_at >= :cutoff").bindparams(bindparam("cutoff", type_=DateTime))` lets you pin parameter types when the dialect can't infer them — rare in Postgres, common in MSSQL. 5. End with when not to use raw SQL. If Core can express it cleanly, prefer Core — you get dialect portability and the IDE can autocomplete column names. Raw SQL is for the queries Core makes ugly, not the queries Core makes verbose.

Edge cases to mention: - `IN` clauses with a list parameter need `expanding=True` : `text("WHERE id IN :ids").bindparams(bindparam("ids", expanding=True))` then pass `{"ids": [1, 2, 3]}`. Without `expanding`, the driver sees the list as a single parameter. - `text()` does NOT inspect or rewrite SQL — `text("SELECT * FROM " + user_input)` is a SQL-injection bug. Bound parameters protect values, not identifiers. - `RETURNING` clauses work in raw `text()` exactly as in Core — `INSERT INTO ... RETURNING id` and read with `.scalar()`.

What NOT to say: - "You can't pass parameters to `text()`." You can; it's the entire point. - "Raw SQL bypasses the ORM entirely so it's unsafe." Raw SQL with bound parameters is exactly as safe as ORM SQL. The hazard is string interpolation, not raw SQL.

Common follow-up: "How do you bind a list to an `IN` clause in `text()`?"

Stand-out line: "Say this and you'll separate yourself: raw SQL with bound parameters is exactly as safe as ORM SQL — both flow through the DB-API parameter machinery, and the only injection vector is string interpolation, regardless of which library you used."

```
from sqlalchemy import text, bindparam

# Single scalar parameter
row = session.execute(
    text("SELECT id, email FROM users WHERE id = :uid"),
    {"uid": 42},
).first()

# List parameter – expanding bindparam
stmt = text("SELECT id FROM users WHERE id IN :ids").bindparams(
    bindparam("ids", expanding=True)
)
ids = session.execute(stmt, {"ids": [1, 2, 3, 4]}).scalars().all()

# Mapping-style result – JSON-ready
rows = session.execute(
    text("SELECT id, email FROM users WHERE created_at >= :cutoff"),
    {"cutoff": cutoff},
).mappings().all()
# rows is List[dict]: [{"id": 1, "email": "a@b.com"}, ...]
```

Q5 · Bulk inserts — `bulk_insert_mappings`, Core `insert()`, and when each wins

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Swiggy, Walmart Labs · Topic: bulk operations

The question (interviewer's verbatim phrasing):

You need to insert 100,000 webhook events into Postgres in a single batch job. The naive `for event in events: session.add(event); session.commit()` takes 4 minutes. Get it under 5 seconds.

The 30-second answer: Three escalating options. First, batch commits:

`session.add_all(events); session.commit()` — collapses 100k transactions into one, gets you to roughly 30 seconds. Second, `session.bulk_insert_mappings(Event, [e.__dict__ for e in events])` — skips the unit-of-work, sends multi-row INSERTs in chunks, gets you to roughly 8 seconds. Third, Core `insert()` with `executemany` or Postgres `COPY` — bypasses the ORM entirely, gets you to 2 seconds. For 100k rows the Core path is the right answer; for 10M rows you skip SQLAlchemy and stream `COPY ... FROM STDIN` through `psycopg` directly.

Step-by-step thought process: 1. Diagnose why the naive code is slow. Per-row `add` plus `commit` means one round-trip per row plus one transaction per row. At 1 ms RTT to Postgres on the same VPC that's a hard floor of 100 seconds; 4 minutes is consistent with WAL fsync amplification. 2. Option one — batch commits. One transaction, one flush, one commit. The unit-of-work still emits one `INSERT` per object because it needs the auto-generated PKs, so the round-trip count is still high. Wins maybe 8x over naive. 3. Option two — `bulk_insert_mappings`. Takes a list of dicts (not mapped objects), skips the identity map, skips the unit-of-work change detection, batches into multi-row `INSERT` statements. Returned PKs are NOT populated on your input dicts. Wins maybe 30x over naive. 4. Option three — Core `insert()`.

`conn.execute(events_table.insert(), [{...}, {...}, ...])` is the cleanest 2.x form; the driver pipelines multi-row inserts, no Python-side change tracking, no `RETURNING` unless you ask. Wins maybe 100x over naive. 5. Beyond Core — Postgres `COPY`. For multi-million-row loads, `psycopg.copy()` streams CSV-style data to the server with no parsing per row; it's the only path that hits raw disk bandwidth. SQLAlchemy doesn't wrap `COPY`; reach for `psycopg` directly.

Edge cases to mention: - `bulk_insert_mappings` does NOT fire ORM events (`before_insert`, `after_insert`). If your application logic depends on those — audit logging, search-index sync — the bulk path silently breaks it. - Core `insert()` with `RETURNING id` lets you reclaim auto-generated PKs in bulk; without it the PKs go to the void and you re-query if needed. - Postgres unique-constraint violations roll back the entire batch unless you wrap with `INSERT ... ON CONFLICT DO NOTHING` or split into smaller batches with savepoints.

What NOT to say: - "Just use `session.add_all` ; it's batched." It commits as one transaction but still round-trips per row — partial win, not the answer. - "ORM is always slow for bulk." Half true. The ORM is slow because of unit-of-work bookkeeping, not because of the SQL it emits. The right level of escape hatch beats both extremes.

Common follow-up: "Show me Core `insert()` with `RETURNING` so I can reclaim the IDs in the same round-trip."

Stand-out line: "Say this and you'll separate yourself: the ORM is slow on bulk loads because of unit-of-work bookkeeping, not because of the SQL it emits — Core `insert()` with `executemany` is the right escape hatch at 100k rows, and `COPY` is the only path that hits raw disk bandwidth at 10M."

```
from sqlalchemy import insert

# Option 2 – bulk_insert_mappings (skip unit-of-work, list of dicts)
session.bulk_insert_mappings(
    Event,
    [{"merchant_id": e.merchant_id, "payload": e.payload} for e in events],
)
session.commit()

# Option 3 – Core insert(), batched, with RETURNING
with engine.begin() as conn:
    rows = conn.execute(
        insert(Event.__table__).returning(Event.__table__.c.id),
        [{"merchant_id": e.merchant_id, "payload": e.payload} for e in events],
    ).scalars().all()
    # `rows` is a list of the auto-generated event IDs in input order

# Option 4 – psycopg COPY for 10M+ row loads
import psycopg, io
csv_buf = io.StringIO()
for e in events:
    csv_buf.write(f"{e.merchant_id}\t{e.payload}\n")
csv_buf.seek(0)
with psycopg.connect(dsn) as conn, conn.cursor() as cur:
    with cur.copy("COPY events (merchant_id, payload) FROM STDIN") as cp:
        cp.write(csv_buf.read())
```

Section B — Connection Pooling and N+1

The next three questions are where most candidates lose points. Connection pooling is invisible until production hits it, and then it's the only thing that matters: the FastAPI app freezes, the load balancer 502s, and someone says "the DB is down" when actually the pool is exhausted and 200 requests are queued. The N+1 problem is the corresponding ORM-side failure — the symptom is "the endpoint got

slower as the dataset grew" and the cause is one query per parent row, hidden behind lazy-loaded relationships. Both have known fixes; the interview is checking you can name them on the spot.

Q6 · Why pool connections — sync defaults, async drivers, pool exhaustion

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Swiggy, PhonePe, Goldman Sachs India · Topic: connection pooling

The question (interviewer's verbatim phrasing):

Why does SQLAlchemy pool connections by default? Walk me through what `pool_size` and `max_overflow` mean, and what happens when traffic spikes.

The 30-second answer: Opening a Postgres connection takes 20–50 ms — TCP handshake, TLS, auth, `SET` of session parameters. That's a tax you don't want to pay per request. The pool keeps `N` connections open after first use and hands them out on demand. `pool_size` is the steady-state count (default 5), `max_overflow` is the burst headroom above that (default 10), so the hard ceiling is 15 concurrent checkouts. When traffic spikes past 15, the 16th request blocks for `pool_timeout` seconds (default 30) waiting for a checkin; if none arrives, it raises `TimeoutError`. That's pool exhaustion, and the symptom is "every request slows to 30 seconds, then errors."

Step-by-step thought process: 1. Cost-justify the pool. A Postgres connection is a server-side fork (or, in PG 14+, a thread in some configurations) with shared-buffer attach, role lookup, and search_path setup. Not cheap. Mid-30 ms is normal on a same-VPC TLS connection; over the internet it's 200+ ms. 2. Walk through the dials. `pool_size=5, max_overflow=10` means up to 15 connections; below 5, extras get closed after `pool_recycle` (default 1 hour). Above 5 and below 15, "overflow" connections are opened on demand and closed on return. Above 15, the next checkout blocks. 3. Explain the failure mode. Under sustained load above the ceiling, queue length grows, response times spike, and `pool_timeout` kicks in. Look for the SQLAlchemy `TimeoutError: QueuePool limit of size 5 overflow 10 reached, connection timed out, timeout 30.00`. Set monitoring on this — it's the unambiguous "raise pool_size or kill long-running queries" signal. 4. Size the pool to the workload. The rule of thumb: `pool_size * num_workers <= max_connections_postgres - reserved`. If Postgres caps at 100 and you have 4 gunicorn workers, each worker can safely hold 20 connections; pick `pool_size=15, max_overflow=5` per worker. Over-pooling causes Postgres to OOM; under-pooling causes the symptom above. 5. Note the async path. Async engines (asyncpg, aiomysql) use `AsyncAdaptedQueuePool` by default with the same `pool_size` semantics; you tune the same dials. SQLite's defaults skip pooling because file-DB connections are basically free.

Edge cases to mention: - `pool_pre_ping=True` checks each connection with a lightweight `SELECT 1` before handing it out — catches "the DB restarted, your pooled connections are dead" without the first user seeing the error. - `pool_recycle=1800` forces connections older than 30 minutes to be discarded — important behind AWS RDS and similar managed Postgres that idle-timeouts

at 1800 seconds. - Long-held connections (a request that does heavy CPU work between DB calls) effectively shrink the pool. Either `session.commit()` early or use `engine.connect()` blocks rather than keeping Sessions open across work.

What NOT to say: - "Pooling is just a cache." It is, but the answer that misses the failure modes (exhaustion, stale connections, `pool_pre_ping`) is the answer that says you've never debugged a production outage. - "Just set `pool_size=1000` ." This usually kills Postgres before it helps your app; `max_connections` is the limiting resource on the DB side.

Common follow-up: "What does `pool_pre_ping` actually do and what does it cost?"

Stand-out line: "Say this and you'll separate yourself: pool exhaustion is the symptom 'every request slows to 30 seconds then errors' — `pool_size * workers <= max_connections - reserved` is the sizing rule, and `pool_pre_ping` is the cheap insurance against an RDS idle-timeout silently killing your connections."

```
from sqlalchemy import create_engine

engine = create_engine(
    "postgresql+psycopg://user:pass@db.internal/billing",
    pool_size=15,                # steady-state count per worker
    max_overflow=5,              # burst headroom; hard ceiling = 20
    pool_timeout=10,            # block 10s before raising on exhaustion
    pool_pre_ping=True,         # SELECT 1 before checkout, catches dead conns
    pool_recycle=1800,          # recycle conns older than 30 min (RDS idle timeout)
)

# Async equivalent
from sqlalchemy.ext.asyncio import create_async_engine

async_engine = create_async_engine(
    "postgresql+asyncpg://user:pass@db.internal/billing",
    pool_size=15,
    max_overflow=5,
    pool_pre_ping=True,
)
```

Q7 · Async DB drivers — asyncpg, aiomysql, and the `AsyncSession` shape

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Microsoft India · Topic: async DB

The question (interviewer's verbatim phrasing):

Your FastAPI app needs to be async. What does the async SQLAlchemy stack look like? Which driver do you pick for Postgres and MySQL?

The 30-second answer: Use `create_async_engine` with an async-capable driver — `asyncpg` for Postgres (`postgresql+asyncpg://`), `aiomysql` or `asyncmy` for MySQL (`mysql+aiomysql://`). The session API mirrors the sync one but with `AsyncSession` and `await` on every IO call: `await session.execute(...)` , `await session.commit()` . The unit-of-work, identity map, and pool semantics are the same; only the IO is non-blocking. Don't mix sync and async engines pointing at the same Session — pick one stack per service.

Step-by-step thought process: 1. Name the drivers. Postgres: `asyncpg` is the dominant choice — pure Python protocol implementation, no libpq dependency, fastest async Postgres driver in the ecosystem. Postgres alternative: `psycopg3` has an `async` mode too, slightly slower but works if you already use `psycopg3` sync. MySQL: `aiomysql` (older, stable) or `asyncmy` (newer, faster); SQLAlchemy supports both via `mysql+aiomysql` and `mysql+asyncmy` URLs. 2. Show the shape. Replace `create_engine` with `create_async_engine` . Replace `Session` with `AsyncSession` , and `sessionmaker` with `async_sessionmaker` . Every `execute` , `commit` , `rollback` , `close` becomes an awaitable. 3. Walk through the FastAPI integration. The dependency function is now `async def get_session()` with `async with` instead of `try/finally` . FastAPI handles async dependencies natively; no extra wiring. 4. Call out the gotcha: lazy-loaded relationships do not work in async land by default. Accessing `user.orders` on an `AsyncSession` blows up because the lazy fetch would need to do IO synchronously inside attribute access. Fix: eager-load via `selectinload(User.orders)` at query time, or use `await session.refresh(user, attribute_names=["orders"])` explicitly. 5. End with `expire_on_commit=False` . The default sync behaviour is to expire all objects after commit so the next access refetches — but async refetch from an expired object requires an await chain that can't happen mid-attribute-access. Set `expire_on_commit=False` on `async_sessionmaker` to avoid the trap.

Edge cases to mention: - `asyncpg` uses Postgres' binary protocol and does not honour `?` placeholders — only `$1`, `$2` or SQLAlchemy's bound-parameter machinery. Raw `psycopg3`-style queries with `%s` won't work if you switch drivers. - `asyncpg` 's type codecs are stricter than `psycopg3`'s — `json` columns come back as Python dicts directly (not JSON strings), and `numeric` columns are `Decimal` not `float` . Test your serialisation layer when switching. - The async pool is `AsyncAdaptedQueuePool` and supports the same `pool_size` , `max_overflow` , `pool_pre_ping` knobs. Sizing rules from Q6 transfer.

What NOT to say: - "Async is always faster." It isn't — for CPU-bound work or low-concurrency endpoints the overhead of the event loop exceeds the IO savings. Async wins when you have many concurrent IO-bound requests. - "I use the sync engine inside an async function with `run_in_executor` ." That's the workaround when no async driver exists for your DB; if `asyncpg` is available, use it.

Common follow-up: "Show me a FastAPI route that uses `AsyncSession` end to end."

Stand-out line: "Say this and you'll separate yourself: async SQLAlchemy works because every IO is awaited and lazy-loads are forbidden — `selectinload` at query time plus `expire_on_commit=False` on the sessionmaker stops the two bugs that ship 80% of FastAPI services into production rework."

```

from sqlalchemy.ext.asyncio import create_async_engine, async_sessionmaker, AsyncSession
from sqlalchemy import select
from sqlalchemy.orm import selectinload
from fastapi import Depends, FastAPI

async_engine = create_async_engine(
    "postgresql+asyncpg://user:pass@db.internal/billing",
    pool_size=15, max_overflow=5,
)
AsyncSessionLocal = async_sessionmaker(
    async_engine, expire_on_commit=False, class_=AsyncSession
)

app = FastAPI()

async def get_session():
    async with AsyncSessionLocal() as session:
        yield session

@app.get("/users/{user_id}")
async def get_user(user_id: int, session: AsyncSession = Depends(get_session)):
    stmt = (
        select(User)
        .options(selectinload(User.orders))  # eager-load avoids async lazy bug
        .where(User.id == user_id)
    )
    user = (await session.execute(stmt)).scalar_one_or_none()
    if user is None:
        return {"error": "not found"}
    return {"id": user.id, "orders": [{"id": o.id} for o in user.orders]}

```

Q8 · The N+1 problem — what causes it, how to detect it, three fixes

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Swiggy, Flipkart, Walmart Labs · Topic: N+1 problem

The question (interviewer's verbatim phrasing):

Your `/orders` endpoint returns a list of 50 orders with their line items. It used to take 80 ms; now it takes 2.5 seconds. The DB hasn't changed. Explain what's likely happening and how you'd fix it.

The 30-second answer: Classic N+1: one query loads the 50 orders, then iterating `order.items` triggers one more query per order to lazy-load its items — 51 queries total instead of 1 or 2. Detection is two clicks in any APM tool, or `echo=True` on the engine for local dev. Three fixes.

`joinedload(Order.items)` issues one query with a `LEFT JOIN` — best for to-one and small to-

many. `selectinload(Order.items)` issues two queries — one for orders, one `WHERE items.order_id IN (...)` — best for larger to-many because it doesn't multiply rows. `raiseload()` on the relationship in production prevents lazy loads from ever firing silently, raising an exception that surfaces the bug at code-review time.

Step-by-step thought process: 1. Diagnose by symptom. Latency grew with dataset size, DB is fine, single-row endpoints are fast. That's N+1's signature. Confirm by toggling `echo=True` on the engine or reading the APM span list — 51 separate SQL statements with `WHERE order_id = ?` is the smoking gun. 2. Explain the mechanism. The ORM defaults `relationship()` to `lazy='select'`, which means the relationship is unloaded until first attribute access. The view code does `for order in orders: order.items` and each access fires a fresh query. The endpoint became slow when somebody added the items rendering to the response without changing the query. 3. Fix one — `joinedload`. `select(Order).options(joinedload(Order.items))` emits one `LEFT JOIN`. Works well for to-one (`Order.customer`) and small fan-out to-many. Watch for row multiplication: 50 orders × 8 items each = 400 result rows; the ORM deduplicates but the wire transfer doesn't. 4. Fix two — `selectinload`. Same query as before, plus one extra `SELECT items WHERE order_id IN (id1, id2, ..., id50)`. Two round-trips instead of 51, no row multiplication. The default choice for to-many relationships above ~5 items per parent. 5. Fix three — `raiseload`. In production, set `lazy='raise'` on the relationship or wrap with `options(raiseload('*'))` to make lazy loads raise `InvalidRequestError`. Now N+1 surfaces as a test failure rather than a silent latency bug. The right policy for any service with strict SLOs.

Edge cases to mention: - Async sessions hit N+1 harder — the lazy load IS the bug because it can't even fire (`MissingGreenlet` error). Eager-loading is mandatory in async land. - `joinedload` with pagination (`.limit(20)`) silently breaks: the LIMIT applies to the joined rowset, not the parent. Use `selectinload` whenever you also paginate. - `subqueryload` is the older sibling of `selectinload` — uses a correlated subquery instead of an `IN`. Rarely the right pick in 2.x; `selectinload` is the modern default.

What NOT to say: - "I'll just add a cache." Caching N+1 hides the bug; you've still made 51 queries on cache miss, and the cache will go cold the moment data changes. - "N+1 only matters for slow networks." Wrong — even on localhost, 51 round-trips eat 5–10 ms per query in driver + parser overhead. The cliff is real on the same VPC too.

Common follow-up: "Show me the `selectinload` and the `joinedload` versions of the orders-with-items query and explain when you'd pick each."

Stand-out line: "Say this and you'll separate yourself: `joinedload` for to-one, `selectinload` for to-many that paginates, `raiseload` in production as the silent-regression guard — once you've named the trio, every N+1 is a five-minute fix instead of a Friday-night incident."


```

from sqlalchemy import select
from sqlalchemy.orm import selectinload, joinedload, raiseload

# Broken – N+1 lurking
stmt = select(Order).limit(50)
orders = session.execute(stmt).scalars().all()
for o in orders:
    print(o.items)                # one fresh query per order, total = 51

# Fix A – joinedload (one query with LEFT JOIN)
stmt = (
    select(Order)
    .options(joinedload(Order.items))
    .limit(50)                    # CAUTION: limit applies to joined rowset
)
# Better used with .unique() and small per-parent fan-out

# Fix B – selectinload (two queries, no row multiplication, paginates correctly)
stmt = (
    select(Order)
    .options(selectinload(Order.items))
    .limit(50)
)
orders = session.execute(stmt).scalars().all()

# Fix C – raiseload as a production guard
stmt = (
    select(Order)
    .options(selectinload(Order.items), raiseload('*')) # everything else raises
    .limit(50)
)

```

Section C — Transactions and Isolation

These four questions are where mid-level engineers separate from senior ones. Everyone can say "I use `with session.begin()`," but few can articulate what READ COMMITTED actually permits, when to escalate to REPEATABLE READ or SERIALIZABLE, what a savepoint is for, and the retry-on-serialization-failure pattern that production needs. The questions assume Postgres semantics; MySQL InnoDB defaults to REPEATABLE READ rather than READ COMMITTED, which is the one drift to mention if asked.

Q9 · `with session.begin()` — what the context manager does

Tags: Difficulty: Easy · Asked at: Razorpay, Cred, Postman, TCS · Topic: transactions

The question (interviewer's verbatim phrasing):

Explain what `with session.begin():` does. How is it different from calling `session.commit()` manually?

The 30-second answer: `with session.begin():` is a context manager that begins a transaction on entry and commits on clean exit, or rolls back on any exception. It's the safe shape: you can't forget to commit, you can't forget to rollback on error. Manual `session.commit()` works but requires `try/except/session.rollback()` plumbing. Use `with session.begin():` for almost everything; reach for explicit commit/rollback only when you need to commit-mid-block and continue working.

Step-by-step thought process: 1. State what the manager wraps. On enter, it calls `session.begin()` — start an implicit transaction. On exit without exception, commit. On exit with exception, rollback and re-raise. 2. Contrast with manual. The manual shape is the four-line `try: ... session.commit() except: session.rollback(); raise`. Every engineer writes the broken version of this at least once (forgets the rollback, swallows the exception). The context manager makes the failure mode impossible. 3. Note the SQLAlchemy 2.x specifics. In 2.x, a Session is in "autobegin" mode — it implicitly starts a transaction on first IO and you can manually commit later. `with session.begin():` makes the boundary explicit and the rollback automatic; that's the recommended shape. 4. Walk through the nested form. `session.begin_nested()` opens a SAVEPOINT inside the outer transaction — covered in Q11. The outer `with session.begin():` can still wrap one or more nested blocks. 5. End with the FastAPI integration. The `get_session` dependency yields a session inside `with`, then the route uses `with session.begin():` for its mutations. The dependency's exception handling rolls back the outer transaction if the route raises.

Edge cases to mention: - If autocommit is on (it isn't by default in SQLAlchemy 2.x), `with session.begin()` still works but emits an explicit `BEGIN`. Mixing autocommit and explicit transactions is a footgun. - `session.in_transaction()` returns `True` while inside the `with` block, even before the first SQL — useful for asserting test setup. - Re-raising inside `__exit__` from custom decorators around a session can swallow the rollback; if you wrap a session in another context manager, make sure your `__exit__` returns falsy.

What NOT to say: - "It's just sugar for commit." It's sugar for begin + commit + rollback-on-exception — the rollback half is the important part. - "I don't need transactions for read-only queries." Even reads run in a transaction; the manager just makes it visible and bounded.

Common follow-up: "Show me the same logic with manual `commit/rollback` and tell me what can go wrong."

Stand-out line: "Say this and you'll separate yourself: `with session.begin():` is begin-commit-rollback in one shape — the rollback half is the load-bearing piece, and every senior engineer prefers the context manager precisely because manual `commit/rollback` is where the forgotten-rollback bug ships."

```

from sqlalchemy.orm import Session

# Recommended – context manager handles commit/rollback
def transfer(session: Session, src_id: int, dst_id: int, amount: float):
    with session.begin():
        src = session.get(Account, src_id, with_for_update=True)
        dst = session.get(Account, dst_id, with_for_update=True)
        if src.balance < amount:
            raise InsufficientFunds()
        src.balance -= amount
        dst.balance += amount
    # On exit: commit if no exception; rollback if any exception

# Manual equivalent – easy to get wrong
def transfer_manual(session: Session, src_id: int, dst_id: int, amount: float):
    try:
        src = session.get(Account, src_id, with_for_update=True)
        dst = session.get(Account, dst_id, with_for_update=True)
        if src.balance < amount:
            raise InsufficientFunds()
        src.balance -= amount
        dst.balance += amount
        session.commit()
    except Exception:
        session.rollback()          # forget this line and the next request sees stale state
        raise

```

Q10 · Isolation levels — READ COMMITTED, REPEATABLE READ, SERIALIZABLE

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Goldman Sachs India, JP Morgan India · Topic: isolation

The question (interviewer's verbatim phrasing):

Postgres ships with READ COMMITTED as the default. What changes if I bump my transaction to REPEATABLE READ or SERIALIZABLE? Give me a real example of when you'd reach for each.

The 30-second answer: READ COMMITTED guarantees you never read uncommitted data, but two reads of the same row inside one transaction can return different values (a non-repeatable read), and a re-run of the same `WHERE` can return a different row set (a phantom). REPEATABLE READ takes a snapshot at the first statement and re-reads the same rows from that snapshot — non-repeatable reads and phantoms are gone for already-read rows, but you can still hit write-skew. SERIALIZABLE serialises the entire transaction against a global order — no anomalies of any kind. Cost goes up the ladder: SERIALIZABLE in Postgres uses predicate locks and may abort transactions with `serialization_failure`, which you must retry. Use READ COMMITTED for everything by default;

bump to REPEATABLE READ for read-heavy reports that span multiple queries; reach for SERIALIZABLE when you have explicit invariants like "no two reservations for the same seat" that can't be enforced by a unique constraint.

Step-by-step thought process: 1. Name the three Postgres levels (Postgres skips READ UNCOMMITTED — internally identical to READ COMMITTED). Default is READ COMMITTED. MySQL InnoDB defaults to REPEATABLE READ — call this out if interviewing for a MySQL shop. 2. Walk the anomaly ladder. Dirty read (forbidden by all three). Non-repeatable read (allowed by READ COMMITTED, blocked by REPEATABLE READ and SERIALIZABLE). Phantom read (allowed by READ COMMITTED, blocked by REPEATABLE READ in Postgres because of snapshot isolation, blocked by SERIALIZABLE everywhere). Write skew (allowed by READ COMMITTED and REPEATABLE READ, blocked only by SERIALIZABLE). 3. Give a real READ COMMITTED case. The 99% case — a CRUD endpoint where two reads of the same row inside the request returning different values is fine because the request is reading once. 4. Give a real REPEATABLE READ case. A reporting endpoint that issues five `SELECT` s to build a consistent snapshot ("total payments today + count of refunds today + ..."). Without REPEATABLE READ, the totals can drift between the first and last SELECT because writers commit in between. 5. Give a real SERIALIZABLE case. A booking flow where you check "no row exists for seat X" and then insert a reservation for seat X. At READ COMMITTED two concurrent requests can both pass the check and both insert; at SERIALIZABLE one will abort with `40001 serialization_failure` and your code retries. (The simpler fix when applicable: add a unique constraint on `(event_id, seat_id)`.)

Edge cases to mention: - `set_isolation_level` in SQLAlchemy 2.x is a connection-time setting or per-engine option: `engine.execution_options(isolation_level="REPEATABLE READ")`. You can also set it per-transaction using `connection.execution_options(...)` before `begin()`. - Postgres's REPEATABLE READ is actually SNAPSHOT ISOLATION, which is stricter than the SQL standard's RR — it blocks phantoms too. The standard's RR does not. Don't conflate the two in mixed-DB shops. - SERIALIZABLE's `serialization_failure` errors are not bugs — they are correct behaviour signalling the system protected an invariant. Your code must retry.

What NOT to say: - "SERIALIZABLE is always safer so I'll just set it everywhere." Wrong — abort rates can be high enough to tank throughput on hot rows. Use the lowest level that holds your invariants. - "REPEATABLE READ in MySQL is the same as REPEATABLE READ in Postgres." Subtly wrong — Postgres uses snapshot isolation which blocks phantoms; MySQL InnoDB's RR has gap locks for inserts but per-statement snapshot for SELECTs. Different mechanics, different edge cases.

Common follow-up: "How do you set the isolation level per-transaction in SQLAlchemy 2.x?"

Stand-out line: "Say this and you'll separate yourself: use the lowest isolation that holds your invariants — READ COMMITTED for 99% of CRUD, REPEATABLE READ for consistent reports, SERIALIZABLE only when no unique constraint covers the invariant and you're ready to retry `40001`."

```

from sqlalchemy import create_engine
from sqlalchemy.orm import Session

engine = create_engine("postgresql+psycopg://user:pass@db/billing")

# Per-engine: every Session uses this isolation level
strict_engine = engine.execution_options(isolation_level="SERIALIZABLE")

# Per-transaction: set on the connection before begin()
def daily_report(session: Session):
    with session.connection().execution_options(
        isolation_level="REPEATABLE READ"
    ) as conn:
        # All reads inside this with-block see the same snapshot
        total = conn.execute(text("SELECT SUM(amount) FROM payments WHERE date = :d"),
                              {"d": "2026-05-30"}).scalar()
        count = conn.execute(text("SELECT COUNT(*) FROM payments WHERE date = :d"),
                              {"d": "2026-05-30"}).scalar()
        refunds = conn.execute(text("SELECT SUM(amount) FROM refunds WHERE date = :d"),
                               {"d": "2026-05-30"}).scalar()
        return {"total": total, "count": count, "refunds": refunds}

```

Q11 · Savepoints — `begin_nested()` and partial rollback

Tags: Difficulty: Medium · Asked at: Cred, Razorpay, JP Morgan India · Topic: savepoints

The question (interviewer's verbatim phrasing):

What is a savepoint and when would you actually use one in production code?

The 30-second answer: A savepoint is a labelled marker inside an open transaction that you can roll back to without aborting the whole transaction. SQLAlchemy exposes savepoints via `session.begin_nested()`. The classic use is "try a risky operation inside a larger transaction, and if it fails, undo just that step and continue." In production the canonical use is bulk import — process a batch of rows, savepoint each row, on per-row failure roll back the savepoint and log it, keeping the rest of the transaction alive. Without savepoints a single bad row aborts the whole batch.

Step-by-step thought process: 1. Name the SQL primitive: `SAVEPOINT name`, `ROLLBACK TO SAVEPOINT name`, `RELEASE SAVEPOINT name`. SQLAlchemy wraps these in `begin_nested()` which behaves like a sub-transaction context manager. 2. Show the shape. `with session.begin():` opens the outer transaction; inside, `with session.begin_nested():` opens a savepoint. Exception inside the nested block rolls back to that savepoint only; the outer transaction continues. Clean exit releases the savepoint. 3. Walk through the bulk-import case. You're loading 10,000 CSV rows. Row 4,217 violates a check constraint. Without

savepoints, the whole transaction aborts and 4,216 already-validated rows are lost. With a savepoint per row (or per chunk), one failure costs you one row. 4. Walk through the "try with fallback" case. Insert a row using the natural key; on conflict savepoint-rollback and try with the legacy key format. Cleaner than nested try/except plumbing. 5. Note the cost. Savepoints aren't free — each one writes a marker to the WAL and uses transaction memory. Don't open thousands of savepoints inside a tight loop without measuring; sometimes the right shape is per-1000-row chunks rather than per-row.

Edge cases to mention: - Savepoints survive autoflush, but Session state is rolled back on the inner exit only for objects modified inside the nested block — objects loaded before the savepoint stay attached but may need `session.refresh()` . - A nested begin without an outer transaction implicitly opens one — but the inner block's rollback then aborts that implicit outer too. Always be explicit about the outer `begin()` . - The async equivalent is `async with session.begin_nested():` and works the same way over `AsyncSession` .

What NOT to say: - "Savepoints are for nested transactions." Almost — Postgres and most relational DBs don't actually support fully nested transactions; savepoints are the standard simulation. - "Use savepoints liberally — they're cheap." They are not free, especially under load. Use them where they earn their cost.

Common follow-up: *"Walk me through bulk CSV import with per-row error tolerance using savepoints."*

Stand-out line: *"Say this and you'll separate yourself: savepoints are not nested transactions — they're labelled markers inside one transaction, and the canonical use is per-row CSV import where one bad row should not invalidate the 9,999 good ones."*

```
from sqlalchemy.exc import IntegrityError, DataError

def import_csv_rows(session, rows):
    failures = []
    with session.begin():
        for row in rows:
            try:
                with session.begin_nested():    # SAVEPOINT
                    session.add(Payment(**row))
                # nested block exited cleanly => SAVEPOINT released
            except (IntegrityError, DataError) as exc:
                # nested rolled back to SAVEPOINT; outer txn still alive
                failures.append({"row": row, "error": str(exc.orig)})
    # outer commit: everything that succeeded is now durable
    return failures
```

Q12 · Deadlock handling — detection and the retry pattern

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Swiggy, Goldman Sachs India · Topic: deadlocks

The question (interviewer's verbatim phrasing):

Two endpoints update the same two rows in opposite order and occasionally deadlock. Walk me through what the database does, what error your Python code sees, and the retry pattern you'd write.

The 30-second answer: Postgres has a deadlock detector that fires every `deadlock_timeout` (default 1 second). When it detects a cycle in the lock-wait graph, it picks a victim, kills its transaction, and raises `40P01 deadlock_detected` — SQLAlchemy surfaces it as `OperationalError` with `pgcode == '40P01'`. The fix has two layers. First, make deadlocks rarer by ordering writes consistently — always lock rows in primary-key order. Second, when they still happen, retry: catch the exception, back off with jitter, and re-run the transaction. The wrapper is typically a decorator that retries 3 times before giving up.

Step-by-step thought process: 1. Explain detection. Postgres maintains a wait-for graph among row-level locks. Every `deadlock_timeout` (1s default) it scans for cycles. On detection it picks the transaction holding the youngest lock as the victim, aborts it, and raises the error. The other transactions in the cycle proceed. 2. Identify the error in Python. SQLAlchemy wraps DB-API errors in `sqlalchemy.exc.OperationalError`. The original exception is on `exc.orig` — for psycopg, `exc.orig.pgcode` is `'40P01'` for deadlock and `'40001'` for serialization failure. Both are "retry me" signals; SQLAlchemy 2.x exposes them uniformly. 3. Prevent first, retry second. The prevention rule is "always acquire locks in the same order." If endpoint A locks rows in PK ascending order and endpoint B does too, no deadlock occurs — one always waits for the other. The classic deadlock is account-transfer: A locks src then dst, B locks dst then src. 4. Build the retry decorator. Three attempts with exponential backoff and jitter — $100 \text{ ms} \times 2^{\text{attempt}} + \text{uniform } 0\text{--}50 \text{ ms}$ — covers the 99% of recoverable deadlocks. Beyond three retries you've got a hot row, not a transient deadlock; let it bubble up to the caller. 5. Walk through the deadlock case end-to-end. The retry catches the exception, rolls back (the session is already aborted by the failure), sleeps, opens a fresh transaction, re-runs the work. The second attempt usually succeeds because the other transaction has already released its locks.

Edge cases to mention: - Retrying is only safe for idempotent operations. A POST that creates a payment twice on retry will double-charge — combine the retry pattern with idempotency keys (Q15). - Postgres `40001 serialization_failure` (from `SERIALIZABLE` isolation) uses the same retry path. Catch both pgcodes in the same handler. - The Session is in an unusable state after the error fires — you MUST call `session.rollback()` (or let `with session.begin():` do it) before reusing it. Calling `session.execute()` on an aborted Session raises `PendingRollbackError`.

What NOT to say: - "I'll catch and ignore the deadlock." Ignoring it silently loses the writes the user expected. Retry, don't swallow. - "Set `deadlock_timeout` to 100 ms so it detects faster." Faster detection costs CPU; the default is well-tuned for most workloads. Tune only after measuring.

Common follow-up: "Write the retry decorator. What's the right backoff and what's the cap?"

Stand-out line: *"Say this and you'll separate yourself: deadlocks aren't bugs — they're correct database behaviour signalling you didn't acquire locks in PK order, and the production-grade fix is consistent ordering plus a retry decorator that catches `40P01` and `40001` together."*

```
import random
import time
from functools import wraps
from sqlalchemy.exc import OperationalError

RETRYABLE_PGCODES = {"40001", "40P01"} # serialization_failure, deadlock_detected

def retry_on_serialization(max_attempts: int = 3, base_delay: float = 0.1):
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            last_exc = None
            for attempt in range(max_attempts):
                try:
                    return func(*args, **kwargs)
                except OperationalError as exc:
                    pgcode = getattr(exc.orig, "pgcode", None)
                    if pgcode not in RETRYABLE_PGCODES:
                        raise
                    last_exc = exc
                    if attempt == max_attempts - 1:
                        break
                    sleep_for = base_delay * (2 ** attempt) + random.uniform(0, 0.05)
                    time.sleep(sleep_for)
            raise last_exc
        return wrapper
    return decorator

@retry_on_serialization(max_attempts=3)
def transfer(session, src_id: int, dst_id: int, amount: float):
    # Lock in PK order to prevent the deadlock in the first place
    first, second = sorted([src_id, dst_id])
    with session.begin():
        a = session.get(Account, first, with_for_update=True)
        b = session.get(Account, second, with_for_update=True)
        src, dst = (a, b) if a.id == src_id else (b, a)
        if src.balance < amount:
            raise InsufficientFunds()
        src.balance -= amount
        dst.balance += amount
```


Section D — Real-World DB Patterns

The last three questions cover the plumbing every backend engineer touches by month three: schema migrations, read-replica routing, and the cache or session sidecar (almost always Redis in 2026). The interviewer is checking that you've shipped this stuff, not just read about it — the giveaways are knowing what `alembic revision --autogenerate` misses, how to route reads in SQLAlchemy without a second engine per query, and the idempotency-key shape that turns at-least-once delivery into effectively-once. The fifteenth question — idempotency keys — is the one Razorpay and Stripe-style fintech interviews go deep on.

Q13 · Alembic — autogenerate, manual migrations, and the safe-deploy patterns

Tags: Difficulty: Medium · Asked at: Razorpay, Cred, Postman, Walmart Labs · Topic: migrations

The question (interviewer's verbatim phrasing):

Walk me through how Alembic works with SQLAlchemy. What's a safe rollout for adding a non-null column to a 50M-row table?

The 30-second answer: Alembic compares your `Base.metadata` against the live database and generates a migration script you commit to the repo. `alembic revision --autogenerate -m "add foo"` is the workflow; the generated `upgrade()` and `downgrade()` functions use `op.add_column()`, `op.create_index()`, etc. For a non-null column on a 50M-row table, do three migrations across two deploys: first add the column as nullable with a default expression so writes work without a backfill lock; deploy code that always writes the column; backfill historical rows in chunks; finally drop the default and add `NOT NULL` in a separate migration. The naive single-migration `add_column(..., nullable=False, server_default=...)` takes a full-table rewrite lock and can lock writes for minutes on Postgres.

Step-by-step thought process: 1. Wire-up. `alembic init alembic` scaffolds `alembic.ini` and `alembic/env.py`. Set `target_metadata = Base.metadata` in `env.py` so autogenerate sees your models. Each migration is a Python file in `alembic/versions/` with a `revision`, `down_revision`, `upgrade()`, and `downgrade()`. 2. Workflow. Change a model, run `alembic revision --autogenerate -m "describe change"`, review the generated script, edit if needed (autogenerate misses some things — see edge cases), commit, deploy. The deploy runs `alembic upgrade head` before the new app code starts. 3. The safe migration pattern for breaking schema changes — the "expand-and-contract" or "online migration" pattern: - Migration 1 (deploy A): expand — add the new column as nullable, or with a default; add the new index `CONCURRENTLY`; create the new table. - App code (deploy A): writes both old and new shape so both readers see consistent data. - Migration 2 (offline or chunked): backfill historical rows. - Migration 3 (deploy B): contract — flip the column to `NOT NULL`, drop the old column or index. 4. Postgres-specific traps. `ADD COLUMN ...`

`NOT NULL DEFAULT 'x'` is fast in PG 11+ (stored in catalog, not rewriting rows) but only if the default is a constant; volatile defaults like `now()` still rewrite the table. Adding an index without `CONCURRENTLY` takes an `AccessExclusiveLock` on the whole table — fatal in production. 5. Downgrade is honest. Many migrations are practically irreversible (you can't un-drop a column with the same data). Write `downgrade()` for the cases that matter (column add, index add) and leave a `raise NotImplementedError` for genuinely one-way ones — better than a `pass` that lies.

Edge cases to mention: - Autogenerate does NOT detect: column renames (it sees a drop + add), check constraint changes, type changes that need explicit USING clauses, enum value additions in Postgres. Hand-edit these. - Cross-environment drift — staging schema diverges from production schema after a hand-applied hotfix. Use `alembic current` and compare across envs; reconcile before next migration. - Long-running migrations should set a per-migration `statement_timeout` and `lock_timeout` to fail fast rather than blocking application writes for hours.

What NOT to say: - "I'll just `op.execute` raw DDL." You can, but you lose downgrade safety and metadata sync. Use Alembic ops where possible; fall back to raw DDL only for unsupported operations. - "Run migrations from the app at startup." Race conditions across replicas; instead, run migrations as a deploy step before any pod accepts traffic.

Common follow-up: "Show me the three Alembic migrations for adding a non-null `email_verified` column to a 50M-row users table."

Stand-out line: "Say this and you'll separate yourself: expand-and-contract is the only zero-downtime shape on a hot table — single-migration `ADD COLUMN NOT NULL DEFAULT now()` takes an `AccessExclusiveLock` and ships as an incident, and the fix is three migrations across two deploys, not one."

```
# Migration 1 – expand: add column as nullable with default
def upgrade():
    op.add_column(
        "users",
        sa.Column("email_verified", sa.Boolean(), nullable=True, server_default=sa.false()),
    )

def downgrade():
    op.drop_column("users", "email_verified")

# Application code: every INSERT and UPDATE now writes email_verified
# Backfill – chunked, run as a separate offline script
# UPDATE users SET email_verified = false WHERE email_verified IS NULL AND id BETWEEN :lo AND :hi
# Loop 100k IDs at a time, commit between chunks

# Migration 2 – contract: enforce NOT NULL, drop the catalog default
def upgrade():
    op.alter_column("users", "email_verified", nullable=False, server_default=None)

def downgrade():
    op.alter_column("users", "email_verified", nullable=True, server_default=sa.false())
```

Q14 · Read replicas — routing writes to the primary, reads to replicas

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Swiggy, PhonePe · Topic: read replicas

The question (interviewer's verbatim phrasing):

Your Postgres primary is at 80% CPU. You've added a read replica. How do you route SELECTs in SQLAlchemy to the replica while keeping writes on the primary?

The 30-second answer: Two engines, one routing rule. Create one engine pointing at the primary and one at the replica DSN. Build a custom `Session` with `binds={Model: primary_engine}` or override `get_bind()` to inspect the SQL statement — if it's a `SELECT` and the session is read-only, return the replica engine; otherwise return the primary. The naive "route everything that starts with SELECT" is dangerous because it can read your own uncommitted writes from the replica before replication catches up — guard against this with "sticky to primary for the rest of the request after any write." Replication lag is the real concern; budget for 200–500 ms typically and design endpoints accordingly.

Step-by-step thought process: 1. Set up the two engines. Same connection options, different DSN. Tag the replica engine with `pool_pre_ping=True` because replica restarts during failovers are common. 2. Routing implementation. Subclass `Session` and override `get_bind(mapper, clause)`. Inspect the `clause` — if it's a `Select` or otherwise read-only and no writes have happened in this session yet, return the replica engine. For any `Insert`, `Update`, `Delete`, or after any write in this session, return the primary. 3. The "read-your-writes" hazard. Replication lag means the replica may not yet have your last write. A user POST-ing then GET-ing their own data sees stale state and reloads thinking it failed. Fix: once a write happens in the session, stick to the primary for the rest of the request — set a `session._wrote = True` flag and route accordingly. 4. Endpoint-level explicit routing. Some endpoints want the replica regardless — analytics, dashboards, search. Expose a context manager or query option like `with use_replica(session):` that flips the bind for that block. 5. Failover handling. When the replica goes down, requests should fall back to the primary, not 500. Wrap the replica path with a try/except `OperationalError`, log it, and retry on the primary. Don't make the fallback silent — emit a metric so you can see lag and failures.

Edge cases to mention: - Transactions can only target one engine. A `with session.begin():` block that does a write must not then read from the replica, even in the same Session — the primary is the only consistent view of in-flight writes. - Don't put `SET TRANSACTION READ ONLY` on the primary connection and assume that makes routing safe — the application layer still needs to know which engine to use. - For multi-region setups (replica in another AZ or region), the latency floor on the replica may be higher than on the primary. Measure before you assume "replica is faster because less load."

What NOT to say: - "I'll just use pgpool/pgbouncer's read-write splitting." That works for some shops, but app-level routing is more reliable and gives you per-endpoint control. Don't punt the question to

middleware. - "Replication lag isn't a problem with synchronous replication." Sync replication doubles write latency and doesn't eliminate lag on async standbys — it adds back-pressure rather than removing the lag problem.

Common follow-up: *"Implement the `get_bind` override that routes reads to the replica until the first write, then sticks to the primary."*

Stand-out line: *"Say this and you'll separate yourself: read-your-writes is the load-bearing safety — route to the replica by default, but stick to the primary the moment the session does any write, or the user POSTs and then GETs and thinks the request failed."*

```

from sqlalchemy import create_engine
from sqlalchemy.orm import Session
from sqlalchemy.sql import Insert, Update, Delete

primary_engine = create_engine("postgresql+psycopg://user:pass@primary.db/billing")
replica_engine = create_engine("postgresql+psycopg://user:pass@replica.db/billing",
                                pool_pre_ping=True)

class RoutingSession(Session):
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self._dirty = False          # True after any write in this session

    def get_bind(self, mapper=None, clause=None, **kw):
        # Anything mutating goes to primary and marks the session dirty
        if isinstance(clause, (Insert, Update, Delete)):
            self._dirty = True
            return primary_engine
        # After the first write, read-your-writes pins us to primary
        if self._dirty:
            return primary_engine
        return replica_engine

# Usage
from sqlalchemy.orm import sessionmaker
SessionLocal = sessionmaker(class_=RoutingSession)

def get_orders_then_create(user_id: int):
    with SessionLocal() as session:
        with session.begin():
            # read – goes to replica
            orders = session.execute(
                select(Order).where(Order.user_id == user_id)
            ).scalars().all()
            # write – goes to primary; session now "dirty"
            session.add(Order(user_id=user_id, total=99.0))
            # next read – goes to primary because of read-your-writes
            count = session.execute(
                select(func.count()).select_from(Order).where(Order.user_id == user_id)
            ).scalar()
        return orders, count

```

Q15 · Redis as cache and session store; idempotency keys for exactly-once

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Swiggy, PhonePe, Postman · Topic: Redis + idempotency

The question (interviewer's verbatim phrasing):

Two patterns. First, when do you reach for Redis as a cache or session store in front of Postgres? Second, your payments endpoint must charge exactly once even if the client retries the request — design the idempotency-key flow.

The 30-second answer: Redis as cache: any read that's hot, small, and tolerant of seconds of staleness — feature flags, user profile snapshots, session tokens, rate-limit counters. Pattern: cache-aside with TTL — read from Redis, on miss read from Postgres, write back, return. Redis as session store: same DB-of-record argument when your sessions don't need durable storage (logout-on-restart is acceptable). Idempotency: the client sends an `Idempotency-Key` header (a UUID it generated). The server records `(key, request_hash, response)` in a Postgres table with a unique constraint on `key`. First request: hash the body, insert the key with status `processing`, do the charge inside the same transaction, update with the response, return. Retried request with the same key: read the stored response and return it verbatim — no re-charge. Done right, this turns at-least-once network delivery into effectively-once business semantics.

Step-by-step thought process: 1. Cache-aside, the canonical Redis pattern. Read from Redis with key derivation rule (`user:{id}:profile`); on hit, deserialise and return. On miss, query Postgres, serialise to Redis with `setex(key, ttl, value)`, return. Choose TTL by tolerance: 60s for fast-changing data, 1 hour for slowly-changing reference data. 2. Cache invalidation. Two strategies. TTL-only (cache lives a fixed window, accepts staleness). Write-through invalidation (on any Postgres write that affects a cached row, `DEL` the key in Redis from the same code path). TTL-only is the right default; explicit invalidation is for cases where staleness is unacceptable. 3. Redis as session store. Store the session blob keyed by session ID with a TTL matching the session lifetime; on every authenticated request, `GET` then refresh TTL. Faster than Postgres for high-QPS reads, accepts losing sessions on Redis restart for systems where re-login is fine. 4. Idempotency key flow. Client generates a UUID, sends as `Idempotency-Key` header on the POST. Server has `idempotency_keys(key TEXT PRIMARY KEY, request_hash TEXT, response_body JSONB, status TEXT, created_at TIMESTAMP)`. On request: hash the body, attempt to insert the key with status `processing`; on `UniqueViolation` read the existing row instead. 5. Decision tree from the existing row. If `status = 'completed'` and the body hash matches the request hash, return the stored `response_body` — this is the safe retry case. If the body hash does NOT match, the client used the same key for a different payload — reject with 422 (this catches client bugs). If `status = 'processing'`, the original request is still running — return 409 with a `Retry-After` header. The whole flow is wrapped in a transaction; the charge and the response are stored in the same commit.

Edge cases to mention: - TTL the idempotency-key row (typically 24 hours), not forever. Use a partial index or a periodic cleanup job to keep the table small. - The cache stampede / thundering herd. When a hot key expires, N concurrent requests all miss and all hit Postgres. Mitigate with `SETNX` lock-and-fill, or by recomputing slightly before expiry (probabilistic early refresh). - Idempotency keys are useless without bounded retries on the client. Clients that retry forever with new keys defeat the system — pair idempotency keys with exponential backoff on the client and a finite retry budget.

What NOT to say: - "Redis gives me exactly-once delivery." It does not — Redis is a cache and a key-value store, not a transactional system. Exactly-once comes from the idempotency-key pattern in your DB, not from Redis. - "Use Redis transactions to make the charge atomic." Redis `MULTI/EXEC` doesn't span Postgres; the atomic boundary is the Postgres transaction that contains the charge and the idempotency-row write together.

Common follow-up: *"Show me the SQL table and the FastAPI endpoint that handles the idempotency-key flow."*

Stand-out line: *"Say this and you'll separate yourself: exactly-once doesn't come from Redis — it comes from a Postgres `idempotency_keys` table with a unique constraint plus the side effect committed in the same transaction, which is the Razorpay/Stripe pattern every senior backend engineer can sketch on a whiteboard in 60 seconds."*

```
# Schema
"""
CREATE TABLE idempotency_keys (
    key          TEXT PRIMARY KEY,
    request_hash TEXT NOT NULL,
    status       TEXT NOT NULL,      -- 'processing' | 'completed'
    response_body JSONB,
    created_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_idem_created_at ON idempotency_keys (created_at);
"""

import hashlib, json
from fastapi import Header, HTTPException, FastAPI
from sqlalchemy import text
from sqlalchemy.exc import IntegrityError

app = FastAPI()

def hash_body(body: dict) -> str:
    canonical = json.dumps(body, sort_keys=True, separators=(",", ":"))
    return hashlib.sha256(canonical.encode()).hexdigest()

@app.post("/charges")
def create_charge(
    body: dict,
    idempotency_key: str = Header(..., alias="Idempotency-Key"),
    session=Depends(get_session),
):
    req_hash = hash_body(body)
    with session.begin():
        # Try to claim the key
        try:
            session.execute(
                text("INSERT INTO idempotency_keys (key, request_hash, status) "
                    "VALUES (:k, :h, 'processing')"),
                {"k": idempotency_key, "h": req_hash},
            )
        except IntegrityError:
            # Existing row – read and decide
            session.rollback()
            with session.begin():
                row = session.execute(
                    text("SELECT request_hash, status, response_body "
                        "FROM idempotency_keys WHERE key = :k FOR UPDATE"),
                    {"k": idempotency_key},
                ).mappings().first()
                if row["request_hash"] != req_hash:
                    raise HTTPException(422, "Key reused with different payload")
                if row["status"] == "processing":
                    raise HTTPException(409, "Original request still in flight")
            return row["response_body"] # safe replay
```



```
# Fresh key – actually perform the charge
charge = perform_charge(body, session) # debits account, inserts payment row
response_body = {"id": charge.id, "status": "captured", "amount": charge.amount}
session.execute(
    text("UPDATE idempotency_keys SET status = 'completed', "
         "response_body = :r WHERE key = :k"),
    {"r": json.dumps(response_body), "k": idempotency_key},
)
return response_body
```

That's all 15. The chapter covered SQLAlchemy 2.x Core vs ORM choice, the Session as a unit-of-work workspace, the modern `DeclarativeBase` mapping shape, raw-SQL escape hatches and bulk-load options, pool sizing for both sync and async drivers, the N+1 detection-and-fix toolkit, transaction context managers with explicit isolation levels and savepoints, the deadlock-retry pattern with jitter, Alembic's expand-and-contract migration shape, read-replica routing with read-your-writes safety, and Redis-plus-idempotency-keys as the exactly-once primitive for payments. These are the questions that come up in round two and round three at Indian product companies and GCCs in 2026; the candidate who has shipped this code answers them in 90 seconds each. The candidate who has only read about them stalls on the second sentence.

The next chapter — System-Level Scenarios — turns five of these patterns into multi-step round-three problems: "your Django ORM is N+1 in three places, your asyncpg pool is exhausted, the deadlock retry is double-charging, fix the system." The library closes there; what you've covered up to here is the foundation those scenarios assume.

Chapter 7 — System-Level Scenarios (5 Multi-Step Scenarios)

Python Interview Prep — 2026 Edition · Learn & Excel · File 07 Backend Track Library

This is the round-2 / round-3 chapter of the Backend track — the one where the interviewer stops asking trivia and starts handing you a sick service. The shape of each scenario is the same: someone slides a problem across the table ("our checkout is double-charging customers" / "our worker memory grows by 200MB every hour" / "FastAPI is dropping requests under load"), and you have to **diagnose what is happening**, then **prescribe and write the fix**, and finally **defend the fix against the edge cases the panel will throw at it**. Junior candidates jump to a fix in 10 seconds; senior candidates spend three minutes on the diagnosis and then write the right fix on the first try. The structure below is engineered to give you the second behaviour.

Each of the 5 scenarios runs in the same shape:

- **The scenario** — one paragraph in the interviewer's voice describing the failure, the framing context (Razorpay / Swiggy / Cred etc.), and the symptom on the wire.
- **30-second framing** — what you say while you take the first sip of water. The "I think this is X, here is why I think that, here is what I will check" opener that buys you 30 seconds of thinking and signals seniority.
- **Step-by-step diagnosis** — 3 to 5 numbered steps. What you check, what tool you reach for, what you expect to see, and the read on the data that nails the hypothesis.
- **The fix** — before / after code, with the actual flags / class / method that the framework documents. Production-shaped, not toy.
- **Gotchas** — the 4 to 6 edge cases that bite people who only memorised the surface fix.
- **Stand-out line** — the single sentence that the panel will remember after the loop. Say this and you separate yourself from the candidate who wrote the same fix.

Code is Python 3.12 unless otherwise noted. Django examples target Django 5.x with PostgreSQL 16. FastAPI examples target FastAPI 0.110+ with Pydantic v2. SQLAlchemy examples target SQLAlchemy 2.0 with the async engine where relevant.

Table of Scenarios

- Scenario 1 — The N+1 Query in Your Django ORM
- Scenario 2 — The FastAPI Endpoint Blocking the Event Loop
- Scenario 3 — The Memory Leak in Your Long-Running Celery Worker
- Scenario 4 — The Race Condition on Order Processing
- Scenario 5 — Connection Pool Exhaustion Under Load

Scenario 1 — The N+1 Query in Your Django ORM

Tags: Difficulty: Medium · Asked at: Razorpay, Swiggy, Cred, Postman, Flipkart, Walmart Labs · Topic: Django ORM perf · Round: 2

The scenario

"You're on a Swiggy-style order management service. The merchant dashboard at `/dashboard/orders/` lists today's 500 orders for a restaurant — order ID, customer name, customer phone, and the total. It works fine on staging. In prod, with 500 rows on the page, the endpoint p95 is 4.2 seconds. The merchant ops lead is on Slack asking why the dashboard is

unusable on lunch-hour. You SSH into the box. The DB CPU is fine, network is fine, the Python process is mostly waiting. Find the bug, and show me the fix on a screen-share."

```
# orders/views.py – the current code
from django.shortcuts import render
from .models import Order

def dashboard(request, restaurant_id):
    orders = Order.objects.filter(
        restaurant_id=restaurant_id,
        created_at__date=today(),
    )
    return render(request, "dashboard.html", {"orders": orders})
```

```
{# dashboard.html – the template loop #}
{% for order in orders %}
    <tr>
        <td>{{ order.id }}</td>
        <td>{{ order.customer.name }}</td>
        <td>{{ order.customer.phone }}</td>
        <td>{{ order.total }}</td>
    </tr>
{% endfor %}
```

30-second framing

"DB is idle, Python is waiting, page is slow, and the loop does `order.customer.name` for 500 rows. That smells like an N+1 — one query for the orders, then one extra query per order for the customer foreign key, 501 queries total. I want to confirm that with the queryset's underlying SQL and the query count, then fix it with `select_for_update` — wait, sorry, `select_related` since `customer` is a `ForeignKey` and the join is a one-to-one from order's side."

That opener does three things the panel grades on: it names the failure mode out loud (N+1), it explains *why* you believe it before checking (low DB CPU + waiting Python = many small queries, not one slow one), and it picks the right tool with the right reasoning (`select_related` for FK / OneToOne, not `prefetch_related`). The self-correction mid-sentence is fine — interviewers prefer it to a confident wrong answer.

Step-by-step diagnosis

Step 1 — Confirm the query count with `django-debug-toolbar` or `connection.queries` .

The cleanest non-toolbar check works at the Django shell:

```
from django.db import connection, reset_queries
from django.conf import settings

settings.DEBUG = True  # connection.queries only logs when DEBUG=True
reset_queries()

from orders.models import Order
orders = Order.objects.filter(restaurant_id=42, created_at__date=today())
for order in orders:
    _ = order.customer.name  # force the access the template does

print(f"Total queries: {len(connection.queries)}")
# Total queries: 501
```

501 is the smoking gun — one `SELECT` on `orders`, then 500 `SELECT` statements on `customers`, one per row. If you have `django-debug-toolbar` wired up, the SQL panel will show the same thing with the duplicate-query warning highlighted in red.

Step 2 — Read one of the duplicate queries to confirm the FK pattern.

```
print(connection.queries[1]["sql"])
# 'SELECT "customers"."id", "customers"."name", "customers"."phone", ...
# FROM "customers" WHERE "customers"."id" = 12847'
print(connection.queries[2]["sql"])
# 'SELECT ... FROM "customers" WHERE "customers"."id" = 12848'
```

Each query is a single-row lookup by primary key. That confirms the access pattern: the template hits `order.customer.X`, Django's descriptor sees the FK is not pre-loaded, issues a fresh `SELECT` for that customer. Repeat 500 times.

Step 3 — Identify the relationship type to pick the right loader.

```
print(Order._meta.get_field("customer"))
# <django.db.models.fields.related.ForeignKey: customer>
```

`ForeignKey` (and `OneToOneField`) means the relation is on a single column, joinable in a single SQL `JOIN`, so `select_related` is correct. If it had been a `ManyToManyField` or a reverse `ForeignKey` (`order.items.all()` from the order side, where one order has many items), you would reach for `prefetch_related` instead — separate query, but only one extra query per relation, not N.

Step 4 — Estimate the win before writing the fix.

500-row loop, 501 round-trips, each round-trip $\approx$ 8ms on a hot VPC (1ms network + 7ms query overhead). $501 \times 8 = 4$ seconds. Matches the p95 exactly. One round-trip with a JOIN at $\sim$ 25ms total $\rightarrow$ 175 $\times$ speedup expected. Stating that math out loud is the senior tell.

Step 5 — Verify the JOIN plan is sane before shipping.

```
EXPLAIN ANALYZE
SELECT o.*, c.* FROM orders o
INNER JOIN customers c ON c.id = o.customer_id
WHERE o.restaurant_id = 42 AND o.created_at::date = CURRENT_DATE;
```

If `customers.id` is indexed (it's a PK, so yes) and `orders(restaurant_id, created_at)` has a composite index (check it), the plan is a nested-loop or hash join, both linear in the result size. No surprises.

The fix

```
# orders/views.py – before
def dashboard(request, restaurant_id):
    orders = Order.objects.filter(
        restaurant_id=restaurant_id,
        created_at__date=today(),
    )
    return render(request, "dashboard.html", {"orders": orders})

# orders/views.py – after
def dashboard(request, restaurant_id):
    orders = (
        Order.objects
        .filter(restaurant_id=restaurant_id, created_at__date=today())
        .select_related("customer")          # one JOIN, no per-row queries
        .only(                               # narrow columns to what the template uses
            "id", "total",
            "customer__name", "customer__phone",
        )
    )
    return render(request, "dashboard.html", {"orders": orders})
```

After the change, `len(connection.queries) == 1`. The generated SQL is one `SELECT` with an `INNER JOIN` against `customers`, projecting only the columns the template touches. P95 should drop from 4.2s to $\sim$ 30ms in prod.

If the dashboard *also* iterates `order.items.all()` (one order, many items), stack `prefetch_related`:

```
orders = (
    Order.objects
    .filter(restaurant_id=restaurant_id, created_at__date=today())
    .select_related("customer")
    .prefetch_related("items")      # 1 extra query batching all items in one IN(...)
)
```

`select_related` adds JOINs; `prefetch_related` issues one follow-up query and stitches in Python. Stacking both is the canonical pattern.

Gotchas

- `prefetch_related` is not always cheaper than `select_related`. If the related set is small (≤ 4 rows per parent) and the columns are narrow, a JOIN can beat the second query plus Python-side stitching. Measure; don't religiously prefer one.
- `only()` and `defer()` on the related model use double-underscore syntax — `only("customer__name")`, not `only("customer.name")`. Forgetting that silently selects all columns and waste the saving.
- `prefetch_related` with a slice on the inner queryset doesn't do what you think. `order.items.all()[:3]` inside the template re-runs the query for each order. Use `Prefetch("items", queryset=Item.objects.order_by("-id")[:3])` with `prefetch_related(...)`, or filter at the parent level.
- The template's `{{ order.customer.name }}` is the trigger, not the queryset. If a downstream serialiser, a signal handler, or a Django admin `__str__` touches another relation, the N+1 reappears. Run with `assertNumQueries` in tests.
- `values()` and `values_list()` short-circuit `select_related`. If you've chained `.values("customer__name")`, the JOIN happens but the ORM model objects don't, so subsequent `.customer.phone` triggers a fresh query. Pick one mode — dict-style or model-style — and stick with it.
- The `restaurant_id` and `created_at__date` filter wants a composite index to avoid a full table scan on `orders`. Fixing N+1 with `select_related` won't help if the outer query itself is doing 8M row scans. Run `EXPLAIN` on the post-fix SQL.
- `select_related` with nullable FKs uses `LEFT OUTER JOIN` automatically — fine, but means a missing customer row silently returns `None`, and `order.customer.name` becomes `AttributeError: 'NoneType'`. Guard in template or DB-constrain the FK as `null=False`.

Stand-out line

"Say this and you'll separate yourself from the candidates who just memorised
`select_related` : 'N+1 isn't a Django problem, it's the lazy-loading abstraction trade-off — the ORM gives you `order.customer` as if it's an attribute, which is great for code clarity until

you put it in a loop. The fix isn't a magic method; it's deciding at queryset definition time which relations you'll dereference in the loop, and declaring them. Every senior ORM user develops the reflex of writing `select_related` / `prefetch_related` before they write the loop, not after. The interview question is really testing whether you have that reflex."

That answer reframes the bug as an abstraction trade-off rather than a forgotten method call, names the senior reflex by its true shape (declare relations at queryset time), and shows you understand the ORM's design rather than just its API.

Scenario 2 — The FastAPI Endpoint Blocking the Event Loop

Tags: Difficulty: Medium-Hard · Asked at: Razorpay, Cred, PhonePe, Postman, Microsoft India, Goldman
· Topic: asyncio + FastAPI · Round: 2 / 3

The scenario

"You're maintaining the Razorpay merchant settings API. There's a `GET /merchants/{id}` endpoint defined as `async def` on FastAPI 0.110. It works in dev. In prod, when traffic spikes to ~200 RPS, p99 latency jumps from 80ms to 6 seconds and uvicorn workers start dropping connections. You attach `py-spy` to a worker and see the event loop pinned, almost all of its time spent inside one frame. The frame is your DB call. What's happening, and what is the smallest possible change to fix it without rewriting your entire ORM?"

```
# merchants/api.py — current code
from fastapi import FastAPI
from sqlalchemy.orm import Session
from .db import SessionLocal
from .models import Merchant

app = FastAPI()

@app.get("/merchants/{merchant_id}")
async def get_merchant(merchant_id: int):
    db: Session = SessionLocal()          # sync SQLAlchemy 2.0 sync Session
    try:
        merchant = db.query(Merchant).get(merchant_id)
        return {
            "id": merchant.id,
            "name": merchant.name,
            "kyc_status": merchant.kyc_status,
        }
    finally:
        db.close()
```

30-second framing

" `async def` plus a synchronous DB call — that's the classic FastAPI footgun. When you declare `async def`, FastAPI runs the function on the single-threaded event loop. The sync `db.query(...).get(...)` is a blocking syscall — it doesn't yield, so the entire event loop sits frozen waiting for Postgres to reply. Every other in-flight request — including all the ones doing async I/O — is starved. The fix is one of three, ordered cheapest to most thorough: (1) drop `async def` so FastAPI runs it in a thread-pool worker by default; (2) keep `async def` and wrap the sync call with `asyncio.to_thread`; (3) migrate the route to the async SQLAlchemy engine and `AsyncSession`. I'd pick (1) for the smallest possible change, with a follow-up plan for (3) over the quarter."

This opener nails three things: it names the failure mode in the framework's own vocabulary (event-loop block), it explains the mechanism (sync call doesn't yield), and it ranks fixes by cost, which is the senior signal.

Step-by-step diagnosis

Step 1 — Reproduce the symptom locally with a controlled stall.

Before chasing prod, prove the hypothesis with a minimal repro. Drop in a `time.sleep(1)` where the DB call lives, hit the endpoint 10 times concurrently with `httpx.AsyncClient`, observe sequential latency:


```
@app.get("/test")
async def test():
    import time; time.sleep(1)      # simulates a blocking sync call
    return {"ok": True}
```

```
# client
async with httpx.AsyncClient() as c:
    start = time.perf_counter()
    await asyncio.gather(*[c.get("http://localhost:8000/test") for _ in range(10)])
    print(time.perf_counter() - start)
# 10.04s - they're serialised, not concurrent.
```

10 seconds for 10 calls confirms: the event loop is single-threaded, and one sync call blocks every other coroutine. Now replace the `time.sleep` with `await asyncio.sleep(1)` and see the same 10 calls finish in ~1.03 seconds. The difference is the diagnostic.

Step 2 — Inspect the worker with `py-spy dump` to confirm what's blocking.

In prod, attach to a uvicorn worker PID and dump:

```
py-spy dump --pid 12847
```

You'll see the worker stack ending in `psycpg.connection.execute` or `socket.recv`, with `asyncio.events.run_forever` higher up the stack. That stack shape — event loop calling into a sync syscall — is the smoking gun.

Step 3 — Audit every dependency of the route.

The dependency injection in FastAPI is recursive — if any dependency uses sync I/O, it blocks. Check for sync HTTP libs (`requests`, `urllib3`), sync DB drivers (`psycpg2`, sync `psycpg`, sync `pymysql`), sync cache clients (`redis-py` in non-async mode), sync file I/O (`open(...).read()` of a big file), and CPU-bound work (hashing, JSON parsing of large payloads, image work). Any of them sitting inside an `async def` is the same bug wearing a different jacket.

Step 4 — Quantify the impact of moving to async-correct code.

Before fixing, measure: how many concurrent requests does the endpoint actually see in prod? If it's 50 RPS with a 20ms DB call, the event loop spends 1 full second per second servicing this endpoint alone, leaving 0 headroom for anything else. The bottleneck isn't the DB; it's coroutine-time-on-CPU. Quote that number in the post-mortem.

Step 5 — Pick the fix tier.

Fix	Code change	When to pick
Drop <code>async def</code> to <code>def</code>	1 character	Smallest possible change. FastAPI auto-routes <code>def</code> to a thread pool.
<code>await</code> <code>asyncio.to_thread(sync_fn, ...)</code>	1 line + refactor	When you must stay <code>async def</code> (because you also <code>await</code> other things in the route).
Migrate to <code>AsyncSession</code> + <code>asyncpg</code>	~10 lines per route	The right long-term fix. Worth doing for hot endpoints.

The fix

Fix Tier 1 — Drop `async def` (the 1-character change):

```
# before
@app.get("/merchants/{merchant_id}")
async def get_merchant(merchant_id: int):
    db: Session = SessionLocal()
    ...

# after — just remove async
@app.get("/merchants/{merchant_id}")
def get_merchant(merchant_id: int):
    db: Session = SessionLocal()
    ...
```

FastAPI inspects the route function. If it's `async def`, it runs on the event loop. If it's plain `def`, FastAPI runs it in a thread pool (default size 40 in anyio's `BlockingPortal`). Sync DB calls in plain `def` routes run on a thread, don't block the loop, and concurrent requests run on different threads. This is the cheapest fix and is correct for sync-driver codebases.

Fix Tier 2 — Keep `async def`, offload the sync work explicitly:

```
import asyncio

@app.get("/merchants/{merchant_id}")
async def get_merchant(merchant_id: int):
    def _load(mid: int):
        db: Session = SessionLocal()
        try:
            m = db.query(Merchant).get(mid)
            return {"id": m.id, "name": m.name, "kyc_status": m.kyc_status}
        finally:
            db.close()

    return await asyncio.to_thread(_load, merchant_id)
```

`asyncio.to_thread` runs the callable in the default `ThreadPoolExecutor` and returns an awaitable. The event loop is free for other coroutines while the thread does the sync DB call. Pick this when the route also awaits other things (an async HTTP fan-out, an `await redis.get`) and you'd rather not lose the `async def` shape.

Fix Tier 3 — Async all the way down (the right long-term fix):

```
# db.py
from sqlalchemy.ext.asyncio import create_async_engine, async_sessionmaker, AsyncSession

engine = create_async_engine(
    "postgresql+asyncpg://...", # asyncpg driver, not psycopg2
    pool_size=20, max_overflow=10, pool_pre_ping=True,
)
AsyncSessionLocal = async_sessionmaker(engine, expire_on_commit=False)
```

```
# merchants/api.py
from sqlalchemy import select
from fastapi import Depends

async def get_db() -> AsyncSession:
    async with AsyncSessionLocal() as s:
        yield s

@app.get("/merchants/{merchant_id}")
async def get_merchant(merchant_id: int, db: AsyncSession = Depends(get_db)):
    merchant = await db.scalar(select(Merchant).where(Merchant.id == merchant_id))
    return {"id": merchant.id, "name": merchant.name, "kyc_status": merchant.kyc_status}
```

This uses SQLAlchemy 2.0's async engine with the `asyncpg` driver. The query is a real coroutine; the event loop schedules other work while Postgres is responding. P99 at 200 RPS holds steady; no thread-pool indirection cost.

Gotchas

- `asyncio.to_thread` defaults to a 40-worker thread pool (anyio's default through uvicorn). That's enough for ~40 simultaneous DB calls. Beyond that, requests queue. Size the pool to your DB connection pool, not to your traffic peak.
- Sync `requests.get` inside `async def` is the same bug, even if your DB is async. Audit every blocking call, not just the DB.
- `time.sleep` in an async route is a free-tier interview trap. It blocks the loop. Use `await asyncio.sleep` always.
- CPU-bound work in `async def` blocks the loop too. A 200ms JSON parse on a 5MB payload, a sync `bcrypt.checkpw` against a 12-round hash, an image thumbnail — all of these need `asyncio.to_thread` or a process pool, not just `async def`.
- `AsyncSession.expire_on_commit=False` matters — the default expires attributes after commit, so accessing `merchant.name` after the commit triggers a refresh, which is a coroutine — but if you've returned the dict already, you read stale attributes. Set `expire_on_commit=False` in the session factory.
- `Depends(get_db)` with `yield` is a common-but-subtle bug if you forget the `async with` and the session isn't closed when the request raises. Use the context-manager form religiously.
- Mixing sync and async on the same engine throws `MissingGreenlet` at runtime — SQLAlchemy 2.0 enforces the boundary. Don't pass an `AsyncSession` into a sync helper.

Stand-out line

"Say this and you'll separate yourself from the FastAPI-as-Flask candidate: 'FastAPI is not faster than Flask because of `async def` — it's faster **only when** the route is doing I/O-bound work and the entire call chain yields control. The moment a sync syscall lives inside an `async def`, your event loop becomes single-threaded Flask with worse semantics, because every request is serialised behind the blocking one instead of being distributed across worker processes. The instinct that wins interviews and prod is: any time I write `async def`, I owe the function a proof that everything it touches is awaitable. If I can't write that proof, I drop the `async` and let FastAPI thread-pool it.'"

That answer redefines `async def` from "the FastAPI default" to "a contract I have to honor with my dependencies," and that reframing is what marks a candidate who has actually debugged event-loop blocking in prod.

Scenario 3 — The Memory Leak in Your Long-Running Celery Worker

Tags: Difficulty: Hard · Asked at: Swiggy, Cred, Razorpay, Postman, Walmart Labs, JPMorgan · Topic: memory / Celery / production debugging · Round: 3

The scenario

"You're on Swiggy's order-ingestion stack. There's a Celery worker that processes order-events from a Kafka topic — about 200 messages per second. The worker pod is provisioned with 2 GB of memory. Fresh after deploy, RSS sits at 180 MB. Over 6 hours, RSS climbs steadily to 1.9 GB, then the kernel OOM-killer SIGKILLs the worker, Kubernetes restarts the pod, and the cycle repeats. The traffic is constant — there's no spike. Your manager wants to know: is it a leak, is it just legitimate memory pressure, and either way, what do we ship by EOD?"

```
# tasks.py – current worker code (simplified to the relevant pieces)
from celery import Celery
import logging
from .db import SessionLocal
from .models import Order, OrderEventLog

app = Celery("orders", broker="redis://...")
logger = logging.getLogger(__name__)

ORDER_CACHE = {}      # in-memory cache keyed by order_id
EVENT_HISTORY = []     # tail log for debugging

@app.task
def process_order_event(event: dict):
    db = SessionLocal()
    try:
        order_id = event["order_id"]
        if order_id not in ORDER_CACHE:
            ORDER_CACHE[order_id] = db.query(Order).get(order_id)
        order = ORDER_CACHE[order_id]
        log = OrderEventLog(order_id=order_id, payload=event)
        db.add(log)
        db.commit()
        EVENT_HISTORY.append({"event": event, "order": order, "ts": time.time()})
        logger.info("Processed order %s with event %s", order_id, event)
    except Exception:
        logger.exception("Failed to process %s", event)
```

30-second framing

"Slow RSS growth from 180 MB to 1.9 GB over 6 hours, traffic flat — that's a classic unbounded-cache leak, not a true GC leak. I see three smells in the code before I even run a profiler: (1) `ORDER_CACHE` is a module-level dict that never evicts, (2) `EVENT_HISTORY` is a module-level list that only appends, (3) the SQLAlchemy `Session` is opened but never released to the pool on exception. I'd use `tracemalloc` to confirm which of those three is the dominant allocator, but I'd already plan to ship: bounded `lru_cache` for orders, drop `EVENT_HISTORY` (it's for debugging, not prod), `with SessionLocal() as db` for guaranteed close, and `worker_max_tasks_per_child` as the belt-and-braces."

This opener does the work senior candidates do: it eyeballs the code first (cheap), lists three concrete leak suspects (specific, not generic), and proposes a hierarchy of fixes (immediate code change + process-level safety net).

Step-by-step diagnosis

Step 1 — Confirm "leak" vs "legitimate growth" with `tracemalloc` snapshots.

A real leak shows the *same* allocation site growing without bound across snapshots. Legitimate steady-state pressure shows fluctuating top sites.

```
# tasks.py – instrument at startup (not in prod hot path)
import tracemalloc
tracemalloc.start(25)          # 25 frames of context per allocation

# in the task, after every 10k tasks processed:
@app.task
def memory_snapshot():
    snap = tracemalloc.take_snapshot()
    for stat in snap.statistics("lineno")[:10]:
        print(stat)
```

Run two snapshots, 1 hour apart, then diff:

```
snap1 = tracemalloc.take_snapshot()
# ... wait an hour of traffic ...
snap2 = tracemalloc.take_snapshot()
diff = snap2.compare_to(snap1, "lineno")
for stat in diff[:10]:
    print(stat)
```

In this scenario you'd see something like:

```
tasks.py:11: size=720MiB (+716MiB), count=2_400_000 (+2_390_000)
tasks.py:12: size=480MiB (+478MiB), count=1_200_000 (+1_198_000)
```

Lines 11 and 12 are `ORDER_CACHE` and `EVENT_HISTORY`. Growth is monotonic, concentrated, and proportional to task throughput. That is a leak.

Step 2 — Distinguish heap leaks from descriptor / connection leaks.

If `tracemalloc` shows no obvious heap culprit but RSS still grows, the leak is probably outside Python's tracked heap — leaked sockets, leaked file descriptors, leaked native memory from a C extension. Check:

```
ls -la /proc/$(pgrep -f celery)/fd | wc -l      # FD count over time
cat /proc/$(pgrep -f celery)/status | grep VmRSS
```

If FD count climbs in lockstep with RSS, the `SessionLocal()` not being closed (or the underlying DB connection not being returned to the pool on exception) is leaking sockets. SQLAlchemy's `with engine.connect() as conn:` or `with SessionLocal() as db:` solves this; bare `db = SessionLocal(); ... db.close()` doesn't, because the exception path skips the close.

Step 3 — Look at the top objects with `objgraph`.

```
import objgraph
objgraph.show_growth(limit=10)
# dict      2_400_002      +2_400_000
# Order      400_000      +400_000
# OrderEventLog 1_200_000 +1_200_000
```

`Order` objects accumulating is `ORDER_CACHE`. `OrderEventLog` accumulating is the SQLAlchemy session not being expunged. The dicts are likely the `EVENT_HISTORY` entries. Three sources, all confirmed.

Step 4 — Quote the per-task allocation in the post-mortem.

200 RPS × (~1 KB per cached Order + ~500 B per event log + ~250 B per EVENT_HISTORY entry) ≈ ~350 KB / sec. Over 6 hours = ~7.5 GB. The kernel kills at ~1.9 GB because that's the cgroup limit, but the actual leak rate is much higher. That math is what your senior manager wants in the post-mortem, not a vague "memory grew."

Step 5 — Decide on the eviction policy before writing code.

For `ORDER_CACHE`, the question is: how stale can a cached `Order` get before correctness suffers? If order state can change in the DB and the worker holding the stale object would act on the wrong data, the cache is dangerous as well as leaky — replace with `lru_cache(maxsize=1024)` with a short TTL, or drop entirely and re-query (200 RPS × 1 small PK lookup is fine).

The fix

```
# tasks.py – after
from celery import Celery
from functools import lru_cache
import logging
from .db import SessionLocal
from .models import Order, OrderEventLog

app = Celery("orders", broker="redis://...")
logger = logging.getLogger(__name__)

# 1. Bounded cache with explicit eviction
@lru_cache(maxsize=2048)
def _load_order(order_id: int) -> Order:
    # fresh session per call – read-only, immediately released
    with SessionLocal() as db:
        return db.query(Order).get(order_id)

@app.task(bind=True, max_retries=3)
def process_order_event(self, event: dict):
    order_id = event["order_id"]
    try:
        order = _load_order(order_id)
        # 2. Session as context manager – guaranteed close on exception
        with SessionLocal() as db:
            log = OrderEventLog(order_id=order_id, payload=event)
            db.add(log)
            db.commit()
        # 3. EVENT_HISTORY removed – it was a debug aid, not prod telemetry
        logger.info("Processed order %s", order_id)
    except Exception as exc:
        logger.exception("Failed to process order %s", order_id)
        raise self.retry(exc=exc, countdown=60)
```

And the belt-and-braces in the worker configuration:

```
# celeryconfig.py
worker_max_tasks_per_child = 10_000      # restart worker after N tasks
worker_max_memory_per_child = 800_000    # KB – restart if RSS exceeds 800MB
task_acks_late = True                    # ack only after task completes
task_reject_on_worker_lost = True        # retry tasks lost to OOM
```

`worker_max_tasks_per_child` is the production safety net: even if a future commit introduces a slow leak, the worker recycles before it hits OOM. Pair with `worker_max_memory_per_child` so it triggers on RSS even when traffic is low and the task counter takes too long.

Gotchas

- `lru_cache` on a method (`self.method`) leaks `self` . Each cache entry holds a reference to the instance. Use module-level functions, or `functools.cached_property` if the cache should live on the instance.
- `@lru_cache(maxsize=None)` and `@cache` are unbounded. Same leak with a different word. Always set a number.
- Celery prefork workers run multiple processes; each has its own `ORDER_CACHE` . A 1024-entry cache becomes 8×1024 entries if `--concurrency=8` . Account for it.
- `with SessionLocal() as db:` works in SQLAlchemy 2.0 but not all 1.x versions. If you're stuck on a transition, use `try: ... finally: db.close()` explicitly.
- `logger.info("Processed %s", order)` with an `Order` model object triggers `__repr__` and can hold a transient reference if the logger handler buffers — usually fine, but `repr()` of an unloaded SQLAlchemy object can trigger a query. Log the ID, not the object.
- `tracemalloc.start(25)` itself uses memory — about 50 MB for a busy worker. Turn it off after the diagnostic, don't leave it on in prod.
- Kafka consumers with `auto.offset.reset=earliest` after an OOM re-read every old message. The leak fix is half the work; the offset hygiene is the other half.
- Module-level mutable state in a Celery worker survives between tasks but not between worker forks. That's why module-level dicts can leak inside one worker yet look "clean" to a developer who restarts the worker between tests.

Stand-out line

"Say this and you'll separate yourself from the candidate who just adds `maxsize` to the cache: 'Memory leaks in long-running Python workers almost never look like classic C-style leaks — Python's GC will eventually collect anything not referenced. They look like unbounded data structures held by globally-reachable references: module-level caches without eviction, lists that only append, sessions that hold result rows because the transaction never ended, closures that captured something that captured something. The diagnostic question I always ask is "what root reference is keeping this alive?" — if I can answer that with the GC chain from `gc.get_referrers`, the fix writes itself. The reason `worker_max_tasks_per_child` exists is that this class of bug is impossible to fully prevent in a system this dynamic, so we engineer for recovery as well as correctness.'

That answer signals two things at once: you understand Python's memory model (refcount + tracing GC, not C's malloc/free), and you've designed long-running systems that survive imperfect code — which is what round 3 actually tests.

Scenario 4 — The Race Condition on Order Processing

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Flipkart, Swiggy, Walmart Labs, Goldman · Topic: concurrency / transactions / locking · Round: 3

The scenario

"You're on a Flipkart-style flash-sale checkout. The endpoint is `POST /checkout/`. The flow is: load the SKU's current stock, check `stock > 0`, decrement stock, create the order. In normal traffic it works. On the iPhone-launch sale at 12:00:00 IST, you get 5,000 requests in 800ms for the last 100 units. The DB ends up with `stock = -47`. 47 customers are charged for a phone that doesn't exist. Customer support has a queue, the CFO is asking who designed this, and the launch is in 30 minutes. Two parts: explain how this happened, and write the two canonical Django ORM fixes — pessimistic and optimistic."

```
# checkout/views.py – current buggy code
from django.shortcuts import get_object_or_404
from .models import SKU, Order

def checkout(request):
    sku_id = request.POST["sku_id"]
    qty = int(request.POST["qty"])
    sku = SKU.objects.get(id=sku_id)    # read

    if sku.stock < qty:
        return JsonResponse({"error": "out of stock"}, status=400)

    sku.stock -= qty                    # mutate in Python
    sku.save()                          # write
    order = Order.objects.create(user=request.user, sku=sku, qty=qty)
    return JsonResponse({"order_id": order.id})
```

30-second framing

"This is a textbook read-modify-write race: between the `SELECT`, the Python-side decrement, and the `UPDATE`, the DB row sits unprotected. Two concurrent requests both `SELECT stock=1`, both pass the `if` check, both `UPDATE stock = 0` — only one of them is "right" but both create orders, and at high enough concurrency the stock goes negative. The two canonical fixes in Django are: (1) pessimistic locking — `select_for_update` to take a row lock for the duration of the transaction, so the second request waits; or (2) optimistic / DB-side mutation — do the decrement in a single SQL `UPDATE ... SET stock = stock - %s WHERE stock >= %s`, and check `rowcount == 1`. Pessimistic is heavier but composes

with the rest of the order-creation transaction; optimistic is cheaper and atomic by construction. For a flash sale, I'd lead with optimistic — it doesn't hold locks during the user's checkout latency."

This framing names the race in textbook vocabulary (read-modify-write), describes both canonical fixes by name, and picks one with a stated reason. That sequence is what differentiates someone who's read about `select_for_update` from someone who's chosen between it and the alternative in production.

Step-by-step diagnosis

Step 1 — Sketch the interleaving on paper / whiteboard.

The senior tell here is drawing the timeline before writing code. Two requests, time axis going down:

T	Request A	Request B
1	SELECT stock FROM sku	----
2	----	SELECT stock FROM sku
3		(both see stock=1)
4	if 1 >= 1: pass	if 1 >= 1: pass
5	UPDATE stock = 0	UPDATE stock = 0
6		(both wrote, both succeeded)
7	INSERT order	INSERT order
		(two orders, one unit of inventory)

The bug isn't that the code is wrong in isolation. The bug is that the DB has no way to know, between step 1 and step 5, that another transaction is doing the same operation. Without a lock or a conditional `UPDATE`, the check is meaningless.

Step 2 — Confirm the bug reproduces with a concurrency test.

```
# tests/test_checkout_race.py
import threading, requests

def hit():
    requests.post("http://localhost:8000/checkout/", data={"sku_id": 42, "qty": 1})

# stock seeded at 1
threads = [threading.Thread(target=hit) for _ in range(20)]
for t in threads: t.start()
for t in threads: t.join()

# expected: 1 successful order, 19 errors. observed: 20 orders, stock=-19.
```

If you can repro it locally with 20 threads, the panel will trust your fix. If you can't, your "fix" is theoretical.

Step 3 — Decide which lock model fits the workload.

Model	Cost per request	Holds locks during...	Right for
Pessimistic (<code>select_for_update</code>)	One row lock for the txn duration	The full order-creation transaction (potentially seconds if payment is in the txn)	Multi-row coordination, complex business logic in the same txn
Optimistic (<code>F('stock') - 1</code> with WHERE clause)	No lock; one atomic UPDATE	Just the duration of the UPDATE itself (microseconds)	Single-row decrements, high contention, simple invariants
Serializable isolation	Implicit locking by the engine	The full txn; can deadlock & retry	When the business logic is too complex for either explicit pattern

For a flash sale, optimistic wins because of the 5,000-RPS pressure — pessimistic locking serialises everyone behind the first holder, queueing 5,000 transactions on one row.

Step 4 — Verify the DB will actually atomically apply the conditional UPDATE.

`UPDATE sku SET stock = stock - 1 WHERE id = 42 AND stock >= 1` is atomic at the DB level — Postgres and MySQL both guarantee row-level atomicity for `UPDATE`. The `WHERE stock >= 1` clause is evaluated *during* the row lock the engine takes for the write. That's why this works even without explicit `SELECT FOR UPDATE`: the conditional `UPDATE` is a check-and-set, all in one statement.

Step 5 — Plan the rollback path.

If the UPDATE succeeded (1 row affected) but the payment fails afterwards, you must restock. Wrap the whole flow in a transaction and handle the rollback explicitly — `@transaction.atomic` for Django.

The fix**Fix A — Optimistic / DB-side decrement (preferred for flash sales):**

```
# checkout/views.py – after, optimistic version
from django.db import transaction
from django.db.models import F
from django.http import JsonResponse
from .models import SKU, Order

@transaction.atomic
def checkout(request):
    sku_id = int(request.POST["sku_id"])
    qty = int(request.POST["qty"])

    # 1. Single atomic UPDATE with the inventory check inside the WHERE
    rows_updated = SKU.objects.filter(id=sku_id, stock__gte=qty).update(
        stock=F("stock") - qty
    )
    if rows_updated == 0:
        return JsonResponse({"error": "out of stock"}, status=409)

    # 2. Order creation only happens if step 1 succeeded
    order = Order.objects.create(user=request.user, sku_id=sku_id, qty=qty)
    return JsonResponse({"order_id": order.id})
```

`SKU.objects.filter(...).update(stock=F("stock") - qty)` translates to:

```
UPDATE sku SET stock = stock - 1 WHERE id = 42 AND stock >= 1;
```

If two requests race, Postgres locks the row for each `UPDATE` serially. The first one finds `stock >= 1`, decrements, returns rowcount=1. The second one finds `stock >= 1` is no longer true, doesn't decrement, returns rowcount=0. Your check on `rows_updated` catches it. Stock never goes negative.

Fix B — Pessimistic / `select_for_update` (preferred when the transaction is complex):

```
# checkout/views.py – after, pessimistic version
from django.db import transaction
from django.http import JsonResponse
from .models import SKU, Order

@transaction.atomic
def checkout(request):
    sku_id = int(request.POST["sku_id"])
    qty = int(request.POST["qty"])

    # 1. SELECT ... FOR UPDATE takes a row lock until txn ends
    try:
        sku = SKU.objects.select_for_update().get(id=sku_id)
    except SKU.DoesNotExist:
        return JsonResponse({"error": "sku not found"}, status=404)

    if sku.stock < qty:
        return JsonResponse({"error": "out of stock"}, status=409)

    sku.stock -= qty
    sku.save(update_fields=["stock"])    # narrow write, not the whole row

    order = Order.objects.create(user=request.user, sku=sku, qty=qty)
    return JsonResponse({"order_id": order.id})
```

`select_for_update()` translates to:

```
SELECT ... FROM sku WHERE id = 42 FOR UPDATE;
```

Postgres locks the row immediately. The second request's `SELECT FOR UPDATE` blocks until the first transaction commits or rolls back. By the time the second one wakes, the row reflects the first's decrement; the `if stock < qty` check now correctly returns 409.

`select_for_update()` requires `@transaction.atomic` — Django raises `TransactionManagementError` otherwise. The lock releases on commit or rollback, so wrap *only* the critical section in the atomic — if the payment gateway call is slow (a Razorpay charge), don't hold the lock during it.

`select_for_update(skip_locked=True)` is the variant for queue-style workloads — workers picking up jobs — where you want the second caller to skip the locked row and try the next one rather than wait.

Gotchas

- `F("stock") - qty` returns a Python object representing a DB-side expression, not a number. `sku.stock = F("stock") - 1` then `sku.save()` works, but reading

`sku.stock` afterwards gives you the `CombinedExpression` object, not the new value — call `sku.refresh_from_db()` to re-read.

- `select_for_update` is a no-op on SQLite and silently ignored. Your tests on SQLite will pass; prod on Postgres will be the first time the lock actually fires. Run integration tests on Postgres.
- `select_for_update().get(...)` outside `@transaction.atomic` raises `TransactionManagementError: select_for_update cannot be used outside of a transaction`. Don't try to "be safe" with the lock without the wrapper.
- `@transaction.atomic` on the entire view is fine but holds the DB connection longer than you want if the view also calls a slow external API (payment gateway, fraud check). Scope the atomic narrowly — only the locked region — and do payment outside the lock.
- The optimistic pattern won't work if you need to check multiple conditions in Python before writing (e.g., user's wallet balance, anti-fraud rules, regional restrictions). For that, `select_for_update` is the only way to take all the reads under one consistent lock.
- `update(stock=F('stock') - qty)` doesn't fire `save()` signals. `post_save`, `pre_save`, and any signal handler you have on `SKU` is skipped. If the signal updates a search index, you'll have a drift.
- Replicas don't see the lock. `select_for_update` on a read-replica connection is a programming error — the replica is read-only and rejects the lock. Route checkout to primary explicitly.
- Postgres locks are per-row; the lock is fine. MySQL's InnoDB takes gap locks under `REPEATABLE READ` and can deadlock on `INSERT` patterns you didn't expect. Test under prod-like load.
- `select_for_update` with `nowait=True` is great for "I want to fail fast if someone else has the lock" UX — but is rarely the right default. Most flash-sale flows want the wait so the second user gets the 409, not a 500.

Stand-out line

"Say this and you'll separate yourself from the candidate who just types `select_for_update` :
 'The check-then-act pattern in a multi-process system isn't a Django problem, it's a fundamental concurrency invariant: any condition you check in application code that you then act on against the same data is a TOCTOU bug unless the check and the act are inside the same atomic boundary. Locks are one way to draw that boundary; conditional UPDATES are another, cheaper way; database constraints (a `CHECK (stock >= 0)` column constraint that raises on violation) are a third, even cheaper way that engages the DB engine as the final arbiter. The interview question is testing whether you reach for any boundary by reflex when you see a check-and-act, or only when symptoms appear in prod.'"

That answer reframes the bug as a TOCTOU pattern (time-of-check-to-time-of-use), names the three classes of fix (lock / conditional write / constraint), and surfaces the reflex that distinguishes seniors. It's also the answer that lets you pivot to a follow-up about `CHECK` constraints — "should we just add `CHECK (stock >= 0)` and let the DB reject?" — which is a genuinely better answer for some workloads.

Scenario 5 — Connection Pool Exhaustion Under Load

Tags: Difficulty: Hard · Asked at: Razorpay, Cred, Swiggy, PhonePe, Postman, JPMorgan, Walmart Labs ·
Topic: SQLAlchemy / pool sizing / production · Round: 2 / 3

The scenario

"You run the Cred rewards-redemption service on FastAPI + SQLAlchemy 2.0 + Postgres. P50 latency is 25ms, p99 is 80ms — normal load. At 14:00 IST a promotional email goes out and traffic jumps 8x. Within 90 seconds, requests start returning 500s. Your logs are full of `sqlalchemy.exc.TimeoutError: QueuePool limit of size 10 overflow 10 reached, connection timed out, timeout 30.00`. Postgres itself is healthy — CPU 30%, idle-in-transaction count is climbing. You have one engineer (you) and the CTO is on the call. Triage, immediate fix, and the root cause."

```
# db.py — current setup
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker

engine = create_engine(
    "postgresql+psycopg2://...",
    pool_size=10,
    max_overflow=10,
    pool_timeout=30,
)
SessionLocal = sessionmaker(bind=engine)
```



```
# rewards/api.py — the hot endpoint
from fastapi import FastAPI, Depends
from .db import SessionLocal

app = FastAPI()

def get_db():
    db = SessionLocal()
    return db

@app.post("/redeem/")
def redeem(payload: dict, db = Depends(get_db)):
    # ... business logic, multiple queries ...
    return {"ok": True}
```

30-second framing

" `QueuePool limit reached` after a 30s wait means every connection in the pool of 20 — `pool_size 10 + max_overflow 10` — is checked out by some request, and the 21st request is waiting. Three classes of cause, in the order I'd check them: (1) leaked connections — code path opens a session and never closes it, so the pool monotonically depletes (the `get_db` dependency doesn't yield with cleanup); (2) legitimate saturation — concurrent in-flight requests genuinely exceed the pool size because the queries are slow (a 200ms query × 100 RPS = 20 in-flight = pool full); (3) idle-in-transaction connections — a request started a transaction, then went off to call an external API, holding the connection. The 'idle-in-transaction climbing' fact in the prompt points at (3). The immediate fix is bigger pool + connection timeout + statement timeout so DB doesn't hang on slow queries; the root fix is `Depends(get_db)` with proper `yield` cleanup and not holding a transaction across external calls. "

That opener triages by mechanism: it doesn't jump to "raise the pool size" (the wrong-shaped fix that papers over a leak) and it doesn't jump to "shed traffic" (the right-shaped fix for the wrong cause). The "idle-in-transaction climbing" detail in the prompt is the clue that the interviewer wants you to spot.

Step-by-step diagnosis

Step 1 — Read the error message precisely, then check pool stats.

`QueuePool limit of size 10 overflow 10 reached` means `pool_size + max_overflow = 20` and all 20 are checked out. `timeout 30.00` means SQLAlchemy waited 30 seconds for a connection to free up, then gave up.

SQLAlchemy exposes pool state via `engine.pool.status()` :

```
print(engine.pool.status())
# "Pool size: 10  Connections in pool: 0 Current Overflow: 10 Current Checked out connections: 20"
```

`Connections in pool: 0` + `Checked out: 20` means everything is in use. Either (a) genuine concurrency or (b) leaked.

Step 2 — Distinguish a leak from saturation with `pg_stat_activity`.

```
SELECT pid, state, query_start, state_change, query
FROM pg_stat_activity
WHERE datname = 'cred_rewards'
ORDER BY query_start;
```

What to read: - All connections `active` with recent `query_start` → genuine concurrency. Raise the pool. - Most connections `idle in transaction` with old `state_change` → connection leak / transaction held during external call. Don't raise the pool; that just delays the inevitable. Fix the leak. - Connections `idle` with old `query_start` → checked out from the pool but doing nothing. Almost always a missing `db.close()`.

In our scenario, `state = 'idle in transaction'` with `state_change` 30+ seconds ago is the pattern. The cause: the endpoint opens a transaction (any write triggers an implicit BEGIN), calls out to a payment provider in the middle, and holds the connection during the network round trip.

Step 3 — Audit `Depends(get_db)` for proper cleanup.

```
# the bug – db is returned, never closed
def get_db():
    db = SessionLocal()
    return db
```

This returns a session and never closes it. FastAPI doesn't know to call `db.close()` because there's no `yield`. Every request leaks one connection until the pool is empty.

The correct shape:

```
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

The `yield` makes it a dependency with cleanup; FastAPI runs the `finally` after the response is sent. If the original code looked like the buggy form, that alone explains the exhaustion.

Step 4 — Look at the slow-query log.

```
-- in postgresql.conf
log_min_duration_statement = 100    -- log queries > 100ms
log_lock_waits = on
```

Slow queries hold connections. A 5-second query held by 4 concurrent requests pins 4 of your 20 slots for 5 seconds. Under an 8× spike, four slow queries is enough to choke the pool.

Step 5 — Check application-side timeouts.

Without a statement timeout, a runaway query holds its connection forever:

```
engine = create_engine(
    "postgresql+psycopg2://...",
    connect_args={"options": "-c statement_timeout=5000"}, # 5s server-side timeout
)
```

That ensures any query exceeding 5 seconds is cancelled by Postgres, the connection is returned, and your pool recovers automatically instead of dying.

The fix

Fix 1 — Correct the `Depends(get_db)` to release connections:

```
# before
def get_db():
    db = SessionLocal()
    return db

# after
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

This alone often resolves "the pool depletes over hours even with low traffic" bugs. Required, not optional.

Fix 2 — Size the pool to your real concurrency:

```
# db.py – after
from sqlalchemy import create_engine

engine = create_engine(
    "postgresql+psycopg2://...",
    pool_size=20,                # base pool
    max_overflow=30,             # burst capacity
    pool_timeout=5,              # fail fast, don't queue 30s
    pool_pre_ping=True,          # detect dead connections
    pool_recycle=1800,           # recycle after 30 min to dodge load-balancer TCP timeouts
    connect_args={
        "options": "-c statement_timeout=5000", # 5s server-side cap
        "connect_timeout": 3,
    },
)
```

Sizing rule of thumb: `pool_size = peak_in_flight_request_count_per_worker × 1.2`. For a uvicorn worker handling 50 in-flight requests at peak, `pool_size=50, max_overflow=20`. Multiply by worker count for the total Postgres connection load — make sure Postgres `max_connections` (typically 100 by default) supports it.

Fix 3 — Don't hold a DB transaction across an external API call.

This is the most common root cause of `idle in transaction`. Refactor:

```

# before – holds the connection during the payment call
@app.post("/redeem/")
def redeem(payload: dict, db = Depends(get_db)):
    with db.begin():
        # opens a txn
        user = db.scalar(select(User).where(...))
        balance = user.balance
        # ... 800ms call to payment provider ...
        response = payment_gateway.charge(...)
        user.balance -= response.amount
    return {"ok": True}

# after – separate the external call from the DB transaction
@app.post("/redeem/")
def redeem(payload: dict, db = Depends(get_db)):
    user = db.scalar(select(User).where(...))
    balance = user.balance
    db.close()

    response = payment_gateway.charge(...) # external call, no DB connection held

    db = SessionLocal()
    try:
        with db.begin():
            user = db.scalar(select(User).where(...).with_for_update()) # re-read with lock
            user.balance -= response.amount
        return {"ok": True}
    finally:
        db.close()

```

This is the structural fix. Pool sizing helps; not holding the connection during the external call is the actual cure.

Fix 4 — Consider `asyncpg` + `AsyncSession` for high-fanout endpoints.

If the route does a lot of `await` work and your DB driver is the limiting factor, migrating to the async engine cuts connection-hold time dramatically because the coroutine yields control during DB waits, and a single connection can handle many concurrent requests at the cost of multiplexing — though SQLAlchemy 2.0 with `asyncpg` still uses a pool, the per-request hold time drops to the actual query duration, not the request lifetime.

Gotchas

- `pool_size + max_overflow` is the hard ceiling per engine, per process. If your service runs 8 uvicorn workers each with `pool_size=20, max_overflow=30`, you're consuming $8 \times 50 = 400$ Postgres connections at peak. Verify Postgres `max_connections`. PgBouncer in transaction-pooling mode in front of Postgres is the standard fix.

- `pool_pre_ping=True` adds a `SELECT 1` before every checkout. That's 1ms of overhead per request. Worth it; disable only if your DB is genuinely never restarted and there are no NAT timeouts.
- `pool_recycle` should be shorter than the LB's idle timeout (AWS NLB is 350s, GCP LB is 600s, cloud SQL proxy is varied). Without recycle, you get phantom dead connections that look fine to SQLAlchemy and fail on next use.
- `pool_timeout=30` is the default and is too long for an interactive request. Customers don't wait 30 seconds; the request will hit the upstream gateway timeout first. Drop to 5 or less; let the request return 503 fast.
- `statement_timeout` is set per-session in Postgres. Setting it in `connect_args` applies to all sessions on the connection. Setting `SET statement_timeout = ...` mid-transaction applies only to that transaction.
- **Implicit transactions bite.** SQLAlchemy 2.0 starts a transaction on the first write. If your code does `db.scalar(...)` (read), then 800ms of business logic, then `db.add(...)` (write), then commits — the transaction is implicitly opened on the `db.add`, and you've held the connection from the read through to commit. Use `db.begin()` explicitly to scope it.
- `Depends(get_db)` with `yield` runs the `finally` after the response is sent to the wire. If your response is large and the client is slow, the connection stays checked out during the response stream. For very large responses, get the data, close the db, then return.
- `max_overflow` connections are real Postgres connections; they're closed (not pooled) when returned. Overflow churn means more `pg_hba` auth handshakes, slower under sustained load. Set `pool_size` high enough that you rarely overflow.
- `SQLALCHEMY_ECHO=True` + connection pool issues = a debugging nightmare because the echo locks the GIL. Turn echo off when reproing pool issues.

Stand-out line

"Say this and you'll separate yourself from the candidate who just bumps `pool_size` : 'A connection pool isn't a performance optimisation — it's a resource boundary that translates the application's concurrency assumptions into a contract with the database. Pool exhaustion is almost never "the pool is too small"; it's "the application is holding connections longer than the assumed contract allowed." The correct first question is "what's the median checkout duration of a connection from the pool, and what's the p99?" If p99 is 30 seconds and the pool is 20, the system is engineered for ~0.6 RPS. Bumping the pool to 200 just makes the system fail at a higher traffic ceiling for the same underlying reason. The senior reflex is to look at hold time per request, not pool size — and to eliminate every reason a request would hold a connection across a network call.'"

That answer reframes pool sizing from "knob to turn" to "consequence of a hold-time contract," which is the framing that separates engineers who've operated a database under load from engineers who've only configured one.

Closing notes for the chapter

These five scenarios cover the highest-frequency round 2/3 prompts in Indian product-co Backend interviews in 2026:

Scenario	Frequency at Razorpay / Cred / Swiggy / Flipkart loops	Failure pattern of unprepared candidates
1 — N+1 in Django ORM	Asked in ~80% of Django-stack loops	Jumps to "add <code>select_related</code> " without explaining N+1, can't pick between <code>select_related</code> and <code>prefetch_related</code>
2 — FastAPI event-loop block	Asked in ~70% of FastAPI-stack loops	Doesn't know <code>async def</code> + sync DB is a bug; suggests "more workers"
3 — Memory leak in long-running worker	Asked in ~50% of senior loops	Reaches for "restart the worker periodically" without diagnosing root cause
4 — Race condition on checkout	Asked in ~90% of payments / inventory loops	Knows <code>select_for_update</code> by name but can't choose pessimistic vs optimistic
5 — Connection pool exhaustion	Asked in ~60% of senior loops	Suggests "bigger pool" without checking for connection leaks or held transactions

The pattern across all five is the same: the panel wants you to **diagnose before you prescribe, name the failure mode in its textbook vocabulary** (N+1, event-loop block, unbounded cache, TOCTOU, hold-time contract), and **pick a fix with a stated reason against a stated alternative**. Memorising any one of these scenarios doesn't pass the round; internalising the diagnostic shape does.

When you walk into the room, the actual prompts will be variations — "your Flask app is slow," "your asyncio scraper hangs," "your Kafka consumer keeps OOMing," "your inventory goes negative on Black Friday," "your service starts 502-ing under load" — but the shape of the answer is what you practice here.

End of Chapter 7 — System-Level Scenarios. End of File 07 Backend Track Library.